Temps: 2 hores i 30 minuts

Notes 24 gener tarda **Revisió: 25 gener tarda** **Cada pregunta en un full separat**

1) **(2 punts)** Considereu l'esquema de la base de dades següent:

```
CREATE TABLE clients
(dni char(9),
nomClient char(50) UNIQUE NOT NULL,
ciutatResidencia char(15),
PRIMARY KEY (dni));
-- Hi ha una fila per cada client d'una entitat bancària.

CREATE TABLE comptesBancaris
(numCompte char(16),
dniClient char(9) NOT NULL,
saldoDisponible real,
PRIMARY KEY (numCompte),
FOREIGN KEY (dniClient) REFERENCES clients(dni));
-- Hi ha una fila per cada compte bancari d'una entitat. El saldoDisponible és el saldo
que hi ha en el instant actual en el compte bancari.

CREATE TABLE ingressos
(numCompte char(16),
instantIngres integer NOT NULL,
quantitat integer NOT NULL CHECK (quantitat>0),
PRIMARY KEY (numCompte, instantIngres),
FOREIGN KEY (numCompte) REFERENCES comptesBancaris(numCompte));
-- Hi ha una fila per cada ingrés fet al compte bancari.

CREATE TABLE reintegraments
(numCompte char(16),
instantReintegrament integer NOT NULL,
quantitat integer NOT NULL CHECK (quantitat>0),
PRIMARY KEY (numCompte, instantReintegrament),
FOREIGN KEY (numCompte) REFERENCES comptesBancaris(numCompte));
-- Hi ha una fila per cada reintegrament fet al compte bancari.
```

1.1) Escriviu una sentència SQL per obtenir el nom dels clients que tenen un o més comptes amb saldo disponible negatiu i cap ingrés.

```
SELECT distinct c.nomClient
FROM clients c, comptesBancaris cb
WHERE c.dni=cb.dniClient and
    cb.saldoDisponible < 0 and
    NOT EXISTS (SELECT * FROM ingres i
          WHERE i.numCompte = cb.numCompte)
```

1.2) Escriviu una sentència SQL per obtenir el dni i el nom dels clients que tenen algun compte bancari en el que s'han ingressat més de 50000 euros entre l'instant 1000 i el 2000, i en el que no s'ha fet cap reintegrament en el mateix període.

```
SELECT DISTINCT c.dni, c.nomClient
FROM clients c, comptesBancaris cb, ingressos i
WHERE c.dni=cb.dniClient AND
      cb.numCompte=i.numCompte AND
      i.instantIngres >=1000 AND
      i.instantIngres <= 2000 AND
      NOT EXISTS (SELECT * FROM reintegraments r
                        WHERE r.numCompte = cb.numCompte AND
                              r.instantReintegrament >=1000 AND
                              r.instantReintegrament <= 2000)
GROUP BY c.dni, c.nomClient, cb.numCompte
HAVING sum(i.quantitat) > 50000
```

1.3) Es vol obtenir els saldos dels comptes bancaris a l'instant 1000. Concretament es vol per cada número de compte, el saldo disponible a l'instant 1000. Per fer-ho algú ha implementat aquesta vista:

```
CREATE VIEW saldos1000p (num,saldo) AS
SELECT cb.numCompte, cb.saldoDisponible -(sum(i.quantitat) - sum(r.quantitat))
FROM comptesBancaris cb, ingressos i, reintegraments r
WHERE cb.numCompte=i.numCompte and i.instantIngres>1000 and
      cb.numCompte = r.numCompte and r.instantReintegrament>1000
GROUP BY cb.numCompte, cb.saldoDisponible
```

Dóneu una extensió de les taules de la base de dades i de la vista que demostrin que la implementació és incorrecta. Raoneu la resposta.

clients (dni, nomClient, ciutatResidencia)

10	Anna	Barcelona

comptesBancaris(numCompte, dniClient, saldoDisponible)

111	10	5000

ingressos(numCompte, instantIngres, quantitat)

111	1001	100
111	1002	200

reintegraments(numCompte, instantReintegrament, quantitat)

111	1003	500

El resultat hauria de ser:

saldos1000p (num, saldo)

111	5200

El resultat obtingut en la implementació donada és:

saldos1000p (num, saldo)

111	5700

Degut a que les files que es formen en el where són:

numCompte	dniClient	saldoDisponible	instIngres	quantitat	instReintegrament	quantitat
111	10	5000	1001	100	1003	500
111	10	5000	1002	200	1003	500

Un altre error en la vista que també s'ha considerat com una solució a l'apartat és que un compte no apareix a la vista si no té ingressos o reintegraments després de l'instant 1000.

2) (2 punts) Considereu l'esquema de la base de dades següent:

```
CREATE TABLE Departaments (
     num_dpt int primary key,
     pressupost int not null check (pressupost>0));

CREATE TABLE Empleats (
     num_empl int primary key,
     sou int not null check (sou>=0),
     num_empl_cap int references Empleats,
     num_dpt int not null references Departaments);

CREATE or REPLACE FUNCTION pr_primer() RETURNS trigger AS $$
BEGIN
    UPDATE Empleats SET sou=3000 WHERE sou=new.pressupost;
    RETURN null;
END;
$$LANGUAGE plpgsql;

CREATE TRIGGER primer
AFTER INSERT on Departaments
FOR EACH ROW EXECUTE PROCEDURE pr_primer();

CREATE or REPLACE FUNCTION pr_segon() RETURNS trigger AS $$
BEGIN
    UPDATE departaments SET pressupost=pressupost+500;
    RETURN null;
END;
$$LANGUAGE plpgsql;

CREATE TRIGGER segon
AFTER UPDATE on Empleats
FOR EACH ROW EXECUTE PROCEDURE pr_segon();
```

Es demana:

a. Suposeu que el contingut inicial de la base de dades és el següent:

Departaments(num_dpt,pressupost)	(1,3000)		
Empleats(num_empl, sou, num_empl_cap, num_dpt)	(1,1000,null,1)	(2,2000,1,1)	(3,2000,1,1)

Digueu quin és el contingut final de la base de dades, després de l'execució de la sentència SQL: *INSERT INTO Departaments VALUES (2,2000)*. Justifiqueu breument la resposta, explicant les accions que segueix el SGBD a conseqüència d'aquesta inserció.

b. Repetiu l'apartat a) suposant que se substitueix la sentència SQL dins el procediment *pr_segon* per: *UPDATE departaments SET pressupost=pressupost-1000*. Agafeu com a contingut inicial, de la base de dades, el contingut inicial de l'apartat a).

c. Definiu una asserció en SQL Standard per garantir el compliment de la restricció següent: "Tot empleat que té un cap, ha de tenir un cap que pertany al departament de l'empleat".

d. Expliqueu com implementaríeu l'asserció anterior en PostgreSQL mitjançant disparadors i mitjançant procediments emmagatzemats:

d.1. Per cada disparador cal que indiqueu: l'esdeveniment activador, la taula i el tipus de disparador. En cas de l'esdeveniment UPDATE cal també les columnes rellevants. Per cada disparador, cal també que justifiqueu breument el tipus de disparador escollit.

d.2. Pels procediments emmagatzemats, expliqueu breument la solució proposada.

d.3. Expliqueu un possible avantatge d'implementació de l'asserció mitjançant disparadors, respecte a la seva implementació mitjançant procediments emmagatzemats.

Solució:

a)

Departaments(num_dpt,pressupost)	(1,4000)	(2, 3000)	
Empleats(num_empl, sou, num_empl_cap, num_dpt)	(1,1000,null,1)	(2,3000,1,1)	(3,3000,1,1)

L'execució de l'acció del disparador primer provoca l'activació del segon disparador. L'acció d'aquest segon disparador s'executa dues vegades, una per cada fila de la taula Departaments.

b) El contingut inicial de les taules d'empleats i departaments no es veu modificat per l'execució de la sentència SQL.

Les accions del segon disparador s'han d'executar dues vegades: una per la tupla (1,3000) I una altra per la tupla (2,2000). La primera vegada que s'executa l'acció del segon disparador, no es produeix cap error, però la segona vegada es produeix una violació del check de departaments pressupost >0, donat que la sentència UPDATE departaments SET pressupost=pressupost-1000 deixa el pressupost de la segona tupla a zero. Per tant, quan es produeix aquesta violació, es desfan les accions realitzades per la transacció en curs. És a dir, es desfan les accions realitzades pel segon disparador, pel primer i per la sentència que ha disparat el disparador.

c)
```
CREATE ASSERTION assercio CHECK (
        NOT EXISTS (SELECT *
            FROM empleats e1, empleats e2
            WHERE e1.num_empl_cap=e2.num_empl and
                e2.num_dpt<>e1.num_dpt))
```

Es considera també correcta una solució on s'afegeixi la condició "e1.num_empl_cap is not null". Encara que es tracta d'una condició innecessària, ja que en cas que no es compleixi aquesta condició mai es pot complir la condició "e1.num_empl_cap=e2.num_empl".

d.1) Possible implementació amb disparadors:

Esdeveniments i tipus de disparadors:
INSERT on Empleats BEFORE/FOR EACH ROW
UPDATE of num_dpt on Empleats BEFORE/FOR EACH ROW
UPDATE of num_empl_cap on empleats BEFORE/FOR EACH ROW
Com que l'enunciat no especifica la freqüència amb que es viola aquesta restricció respecte a les restriccions d'integritat de la BD, el tipus de disparador podria ser BEFORE o AFTER. No obstant, recordeu en la recomanació general en aquestes situacions és comprovar la restricció del disparador el més aviat possible.
L'esdeveniment DELETE on Empleats no és rellevant:
- Considerant que en la base de dades quan s'esborra un empleat s'apliqués una política de manteniment d'integritat referencial RESTRICCIÓ, no seria rellevant. El motiu és que aquest esdeveniment no podria arribar mai a violar la restricció, ja que només s'arribaria a fer l'esborrat en cas que l'empleat no fos cap de cap altre empleat.
- Considerant que en la base de dades quan s'esborra un empleat s'apliqués una política de manteniment d'integritat referencial CASCADA, tampoc seria rellevant. El motiu és que en esborrar l'empleat de les claus foranes dels empleats que li eren subordinats, aquests empleats no podrien passar a violar la restricció, ja que serien empleats que no tindrien cap, i per tant a qui no afectaria la restricció.

d.2) Possible implementació amb procediments emmagatzemats:

Per cada procediment que actualitza dades d'empleats afegir la comprovació de la restricció. Els procediments afectats per tant tots aquells que insereixin i/o modifiquin o bé l'atribut num_dpt o el num_empl_cap d'un empleat.

d.3) Sí s'afegeix la restricció a cada procediment que actualitza dades d'empleats, aleshores la restricció queda amagada, distribuïda i replicada a diversos procediments. És difícil de localitzar i modificar.

3) (2,5 punts)

a. Sigui un SGBD sense cap mecanisme de control de concurrència, i suposem que es produeix l'horari següent (R= Read, RU= Read for Update, W= Write; les accions s'han numerat per facilitar fer-hi referència):

Acc#	T1	T2	T3	T4
10			R(E)	
20	RU(A)			
30	W(A)			
40			RU(F)	
50			W(F)	
60		RU(E)		
70				R(A)
80				RU(F)
90				W(F)
100		W(E)		
110		R(B)		
120	R(B)			
130		R(C)		
140				COMMIT
150			R(E)	
160	RU(A)			
170	W(A)			
180			COMMIT	
190		COMMIT		
200	COMMIT			

Contesteu, **argumentant les respostes**, a les preguntes següents:

a.1. Quin és el graf de precedències associat a l'horari donat?

a.2. Quines interferències es produeixen? per cada interferència cal que doneu: nom de la interferència, transaccions i grànuls implicats.

a.3. Quins horaris serials donen resultats equivalents a l'horari proposat?

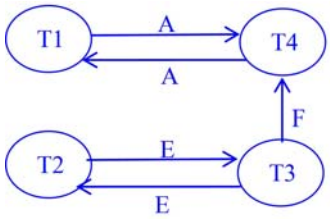
a.4. L'horari proposat, és recuperable? En cas afirmatiu doneu la definició de quan un horari és recuperable. En cas negatiu indiqueu alguna de les transaccions i operacions de l'horari que mostrin que no ho és.

b. Suposeu ara que tenim un mecanisme de control de concurrència basat en reserves S, X i que les transaccions T1, T3 i T4 treballen a un nivell d'aïllament de READ COMMITTED i que T2 treballa amb un nivell d'aïllament de REPEATABLE READ. Contesteu a les preguntes següents:

b.1. Com quedaria l'horari? L'horari ha d'incloure, a més de les peticions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves i l'ordre d'execució de totes aquestes peticions.

b.2. Quins horaris serials hi són equivalents?

a.1) Graf de precedències:



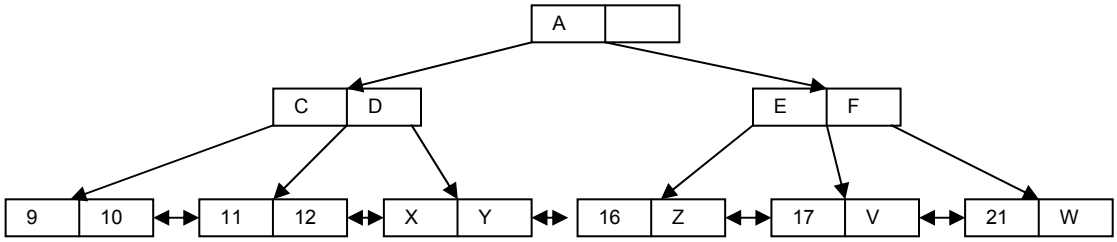
a.2) Entre T1 i T4 sobre el gràNul A es produeix una lectura no confirmada, i entre T2 i T3 sobre el gràNul E es produeix una lectura no repetible.
a.3) No existeix cap horari serial equivalent, donat que existeixen interferències.
a.4) L'horari no és recuperable. Per exemple, T4 recupera dades pendents de confirmació i confirma la seva execució abans que la transacció que ha modificat les dades (aquesta transacció és T1).

b.1) L'horari quedaria de la manera següent:

Acc#	T1	T2	T3	T4
10			L(E,S)	
20			R(E)	
30			U(E)	
40	L(A,X)			
50	RU(A)			
60	W(A)			
70			L(F,X)	
80			RU(F)	
90			W(F)	
100		L(E,X)		
110		RU(E)		
120				L(A,S)
130		W(E)		
140		L(B,S)		
150		R(B)		
160	L(B,S)			
170	R(B)			
180	U(B)			
190		L(C,S)		
200		R(C)		
210	RU(A)		L(E,S)	
220	W(A)			
230		COMMIT(U(E),U(B),U(C))		
240			R(E)	
250			U(E)	
260			COMMIT(U(F))	
270	COMMIT(U(A))			
280				R(A)
290				U(A)
300				L(F,X)
310				RU(F)
320				W(F)
330				COMMIT(U(F))

b.2) No existeix cap horari serial, la interferència de lectura no repetible entre T2 i T3 sobre el gràNul E se segueix produint, donat que T3 treballa amb nivell d'aïllament READ COMMITTED, i per evitar la interferència T3 hauria d'haver treballat amb un nivell d'aïllament de REPEATABLE READ.

4) (1,5 **punts)** Donat l'arbre B+ d'ordre 1 (**d**=1) que correspon a un índex definit sobre una taula T per a un atribut que és clau primària de la taula:



Es demana:

- **a.** Indiqueu quins són els valors possibles de X, Y, Z, V i W.
- **b.** Indiqueu quins són els valors possibles de C i D.
- **c.** Indiqueu quins són els valors possibles de E i F.
- **d.** Indiqueu quins són els valors possibles de A.

Justifiqueu convenientment la resposta de cada apartat.

a) Segons la definició d'arbre B+, els valors en les fulles estan ordenats i no hi ha repetits. Per tant, $12 < X < Y < 16$, i conseqüentment:
X=13, aleshores Y = 14 o 15
X=14, aleshores Y = 15
X=15, aleshores Y no podria tenir cap valor.

Z no pot contenir cap valor ja que hauria de ser $16 < Z < 17$,

V pot ser 18, 19 o 20

W>21

b) Segons la definició de node intern d'un arbre B+, C>10 i C <=11,per tant només pot ser 11. De manera similar, D > 12 i D <=X; però tenint en compte que tots els valors dels nodes interns han d'estar a les fulles D també ha d'estar a les fulles, i per tant ha de ser igual a X.
En resum, C=11 i D=X (notis, doncs, que en realitat D no pot ser < que X)

c) Els valors de E i F, estan determinats, només poden ser 17 i 21. Els argument són similars al cas C.

d) A ha de complir que ha de ser <= que tots els valors de la dreta de l'arbre i a més a més ha d'estar a les fulles. Per tant, necessàriament ha de ser 16.

5) **(2 punts)** Considereu l'esquema de la base de dades format per les taules següents:

```
Actor(nomactor, adreça, telèfon, anynaix, sexe, religió, caché)
                                 on caché és el que cobra un actor per
                                          fer una pel.lícula

Tema(nomtema)
Pel·lícula(nompel, nomtema)          on nomtema referencia Tema
Habilitat(nomactor, nomtema)         on nomtema referencia Tema
                                     on nomactor referencia Actor
Actuar(nomactor, nompel)             on nomactor referencia Actor
                                     on nompel referencia Pel·lícula
```

a. Escriviu una seqüència d'operacions d'àlgebra relacional per obtenir per cadascun dels actors, els noms de totes les pel·lícules on ha actuat però que no eren d'un tema en que l'actor tenia habilitat. Concretament es demana aquesta informació en parelles: nom actor, nom pel·lícula.

```
A = Pel.licula*Habilitat
B = A[nomactor,nompel]
R = Actuar-B
```

b. A quins atributs afecta la Llei orgànica de protecció de dades personals (LOPD), i en quin/s dels seus tres nivells classificaríeu aquests atributs?

Només als atributs de les taules: Actor, Habilitat i Actuar

Nivells de seguretat:

- Nivell bàsic: Nom Actor, Adreça, Telèfon, Any Naixement, Sexe, Habilitat en temes, Actuació en pel·lícules

- Nivell mitjà: Caché

- Nivell alt: Religió

c. Digues si són certes o falses les sentències següents. Justifica la resposta.

i. Un Servidor en un Entorn SQL agrupa un conjunt de Catàlegs, que en PostgreSQL s'anomenen Esquemes.

Fals, ja que els catàlegs en PostgreSQL corresponen a les bases de dades.

ii. L'esquema d'informació és l'esquema per defecte al que es connecta un usuari al fer la connexió amb la base de dades.

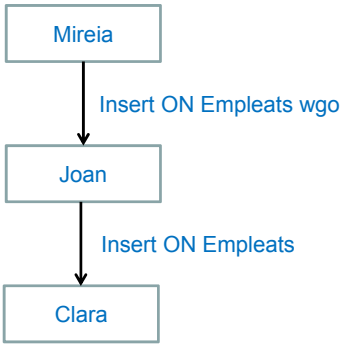
Fals, l'esquema d'informació és un esquema apart dels esquemes definits pels usuari. N´hi ha un per cada catàleg, i conté informació sobre els esquemes definits pels usuaris en el catàleg: noms i atributs de les taules, índexs, restriccions de columna, ...

iii. L'esquema de la base de dades que us hem donat és un esquema conceptual segons l'arquitectura de tres nivells ANSI/SPARC.

Cert.

iv. Donat un cert diagrama d'autoritzacions, l'efecte d'una operació REVOKE sempre es pot anul·lar amb una única operació GRANT.

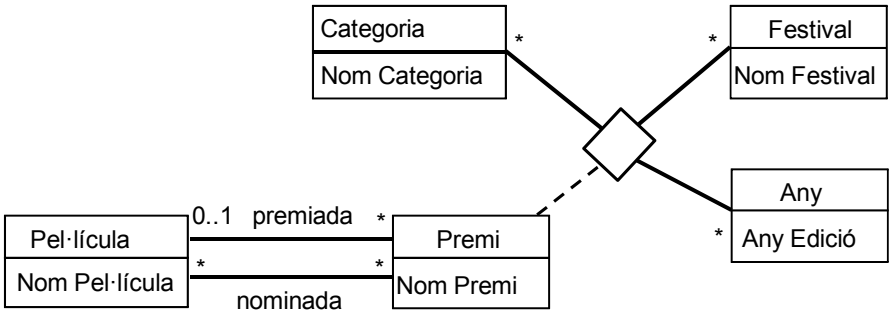
Fals. En el graf d'autoritzacions següent es pot veure que:



Si s'executa l'operació següent: Mireia: REVOKE insert ON Empleats.

Aquesta operació no es pot anul·lar tornant a la situació de partida amb una única operació GRANT.

d. Suposant el model conceptual en UML següent, dóna el disseny lògic que s'obté fent la traducció a model relacional.



Clau externa Categoria: Nom Categoria
Clau externa Festival: Nom Festival
Clau externa Any: Any Edició
Clau externa Pel·lícula: Nom Pel·lícula

Categories (nom_categoria)

Festivals(nom_festival)

Any(any_edició)

Pel·lícules (nom_pel·lícula)

Premis(nom_categoria, nom_festival, any_edició, nom_premi, nom_premiada)

nom_categoria referencia Categories

nom_festival referencia Festivals

any_edició referencia Any

nom_premiada referencia Pel·lícules

Nominades(nom_categoria, nom_festival, any_edició, nom_nominada)

nom_categoria, nom_festival, any_edició referencia Premis

nom_nominada referencia Pel·lícules

També és una solució vàlida si no s'inclou la relació Any (veure transpa 306)

Temps: 3 hores

Notes 26 juny tarda **Revisió: 27 juny tarda** **Cada pregunta en un full separat**

1) (2 punts) Considereu l'esquema de la base de dades següent:

```
create table professors
(dni char(9),
nomProf char(50) unique,
telefon char(15),
sou integer not null check(sou>0),
primary key (dni));

create table despatxos
(modul char(5),
numero char(5),
superficie integer not null check(superficie>12),
primary key (modul,numero));

create table assignacions
(dni char(9),
modul char(5),
numero char(5),
instantInici integer,
instantFi integer,
primary key (dni, modul, numero, instantInici),
foreign key (dni) references professors,
foreign key (modul,numero) references despatxos);
-- assignació d'un professor a un despatx.
-- instantFi te valor null quan una assignacio es encara vigent.
```

1.1) Escriviu una sentència SQL per obtenir la suma del sou dels professors que tenen alguna assignació no vigent a despatxos del mòdul 'Omega' i que tenen 2 o més assignacions a un mateix mòdul però a despatxos diferents.

```
SELECT sum(sou)
FROM professors p
WHERE    EXISTS (SELECT *
                 FROM assignacions a
                 WHERE a.instantFi is not null
                       AND a.dni=p.dni and a.modul = 'Omega')
         AND p.dni IN (SELECT a1.dni
                 FROM assignacions a1, assignacions a2
                  WHERE a1.dni=a2.dni and a1.numero <> a2.numero
                       and  a1.modul=a2.modul);

SELECT sum(sou)
FROM professors p
WHERE    EXISTS (SELECT *
                 FROM assignacions a
                 WHERE a.instantFi is not null
                       AND a.dni=p.dni and a.modul = 'Omega')
         AND EXISTS(SELECT a1.modul
                 FROM assignacions a1
                 WHERE a1.dni = p.dni
                 GROUP BY a1.modul
                 HAVING count(distinct a1.numero)>=2);
```

1.2) Suposeu que volem una sentència SQL per trobar els professors (dni, nomProf) que no tenen cap assignació vigent. Digueu, per cadascuna de les sentències següents **si és o no correcta**. En cas de que no ho sigui **doneu una extensió de la base de dades** que faci que la consulta doni un resultat incorrecte.

a)
```
SELECT p.dni, p.nomProf
FROM professors p
WHERE p.dni NOT IN (SELECT a.dni FROM assignacions a
                    WHERE a.instantFi IS NULL);
```

b)
```
SELECT p.dni, p.nomProf
FROM professors p
WHERE not exists (SELECT *
                  FROM assignacions a, professors p1
                  WHERE a.dni = p1.dni
                        AND a.instantFi IS NULL);
```

c)
```
SELECT p.dni, p.nomProf
FROM professors p, assignacions a
WHERE p.dni <> a.dni AND a.instantFi IS NULL;
```

d)
```
SELECT p.dni, p.nomProf
FROM professors p, assignacions a
WHERE p.dni = a.dni
      AND a.instantFi IS NULL
GROUP BY p.dni, p.nomProf
HAVING count(*) = 0;
```

a) Correcte.

b) Incorrecte. Tal com està la consulta un professor sortirà o no segons si hi ha o no alguna assignació vigent a la taula assignacions. Un professor que no té assignacions vigents ha de sortir, però en aquesta consulta no sortirà quan hi hagi altres professors a la base de dades que si que en tinguin.

professors
1, Joan
2, Martí
assignacions
1, Omega, 124, 1000, null

En Martí hauria de sortir i en Joan no.

No sortiran ni en Martí ni en Joan perquè la subconsulta sempre dona resultats per tots dos.

c) Incorrecte. Que hi hagi assignacions vigents que no són del professor, no vol dir que el professor no tingui assignacions vigents. Un professor que té assignacions vigents, si a la base de dades hi ha assignacions d'altres professors amb assignacions vigents, el primer sortirà a la consulta.

professors
1, Joan
2, Martí
assignacions
1, Omega, 124, 1000, null
2, Omega, 123, 1100, null

Cap dels dos professors hauria de sortir.

Sortiran tots dos perquè cadascun es combina amb l'assignació que no és seva i compleixen la condició del where.

d) Incorrecta. Amb un group by mai no hi ha grups sense files. Els professors que no tenen assignacions vigents, desapareixen en la combinació de taules.

professors
1, Joan
2, Martí
assignacions
1, Omega, 204, 1000, null

En Martí hauria de sortir i en Joan no.

No sortirà cap dels dos. En Martí perquè no pot combinar-se amb cap assignació per tant no pot formar grup, en Joan perquè el seu grup té 1 fila.

1.3) Escriviu una seqüència d'operacions d'àlgebra relacional per obtenir els dni dels professors que han tingut alguna assignació en el mateix despatx que el professor amb dni '123'. Tingueu en compte que en el resultat de la consulta no volem que surti el professor '123'.

B=assignacions(dni='123')
C=B[dni, modul, numero]
D=C{dni ->dnip, modul->modulp, numero ->numerop}
E=assignacions[dni, modul,numero]
F=E[dni<>dnip,modul=modulp,numero=numerop]D
R=F[dni]

2) (2 punts)

2.1) Considereu les taules de la base de dades de l'exercici 1, i les vistes següents definides sobre elles:

```
CREATE VIEW personalactualOmega AS
select  dni, modul, numero
from assignacions
where instantFi IS NULL and modul='Omega';

CREATE VIEW dadespersonalactualOmega (nomProf, modul, numero,
telefon) AS
select  p.nomProf, pa.modul, pa.numero, p.telefon
from professors p, personalactualOmega pa
where p.dni=pa.dni;
```

a) Són actualitzables aquestes vistes segons l'estàndard SQL? Justifiqueu la resposta.

personalactualOmega No, perquè no selecciona tota la clau.
dadespersonalactualOmega No, perquè té join i a més està definida sobre una vista no actualitzable.

b) Considereu la consulta següent:

```
SELECT d1.nomprof, d2.nomprof
FROM dadespersonalactualOmega d1, dadespersonalactualOmega d2
WHERE d1.numero = d2.numero and
   d1.nomprof <> d2.nomprof;
```

b.1) Explica breument què retorna la consulta.

S'obtenen les parelles de professors amb nom diferent que tenen assignacions vigents al mateix despatx de l'edifici Omega.

b.2) Doneu una sentència SQL sobre taules, que compleixi els criteris de qualitat establerts a l'assignatura, que retorni el mateix resultat que la consulta anterior sobre vistes.

```
select p1.nomProf, P2.nomprof
from professors p1, assignacions a1,
         professors p2, assignacions a2
where p1.dni= a1.dni and p2.dni= a2.dni
   and a1.instantFi IS NULL and a2.instantFi IS NULL
   and a1.modul='Omega' and a2.modul='Omega'
   and a1.numero=a2.numero and p1.nomprof<>p2.nomprof;
```

c) Si es pot fer una solució que compleix els criteris de qualitat que permet accedir a les mateixes dades consultant les taules, quines avantatges pot aportar el fer-ho amb vistes?

Independència lògica de dades, simplificació de consultes i programes, i control d'accés.

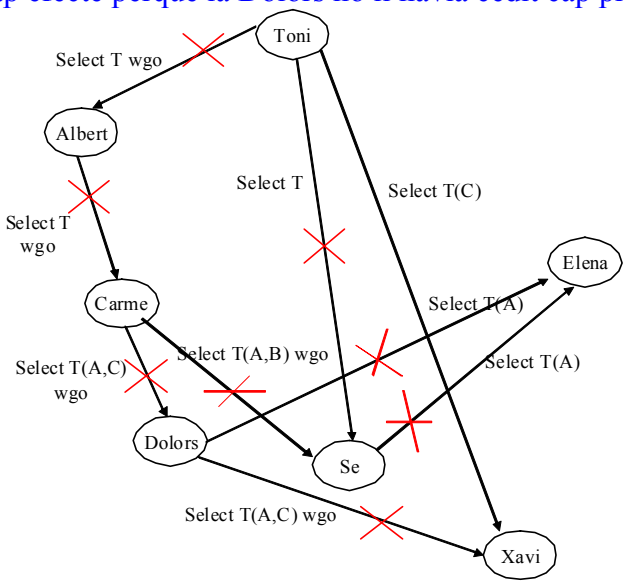
2.2) Considereu la taula professors(dni,nomProf,telefon), propietat d'en Toni. Suposeu també la seqüència de sentències següent relativa a autoritzacions sobre la taula professors. Cada sentència està numerada i s'indica el nom de l'usuari que vol executar-la.

1 - Toni: GRANT SELECT ON professors TO Albert WITH GRANT OPTION
2 - Albert: GRANT SELECT ON professors TO Carme WITH GRANT OPTION
3 - Carme: GRANT SELECT(dni,telefon) ON professors TO Dolors WITH GRANT OPTION
4 - Carme: GRANT SELECT(dni,nomProf) ON professors TO Se WITH GRANT OPTION
5 - Toni: GRANT SELECT ON professors TO Se
6 - Toni: GRANT SELECT(telefon) ON professors TO Xavi
7 - Dolors: GRANT SELECT(dni,telefon) ON professors TO Xavi WITH GRANT OPTION
8 - Se: GRANT SELECT(dni,telefon) ON professors TO Xavi
9 - Dolors: GRANT SELECT(dni) ON professors TO Elena
10 - Se: GRANT SELECT(dni) ON professors TO Elena
11 - Toni: REVOKE SELECT ON professors FROM Se RESTRICT
12 - Carme: REVOKE SELECT(dni,telefon) ON professors FROM Dolors RESTRICT
13 - Dolors: REVOKE SELECT(dni) ON professors FROM Se CASCADE
14 - Albert: REVOKE SELECT ON professors FROM Carme CASCADE
15 - Toni: REVOKE SELECT ON professors FROM Albert RESTRICT

a) Quines d'aquestes sentències, si n'hi ha cap, no s'executaran amb èxit o no tindran cap efecte sobre la base de dades? Raoneu la resposta. Assumirem que les sentències que no s'executin amb èxit, no tindran cap efecte i es continuarà amb la sentència següent.

b) En acabar d'executar les sentències anteriors, quins privilegis tindran els usuaris Xavi i Albert sobre la taula professors?

a) La sentència 8 no tindrà èxit perquè la Se no té els privilegis que vol passar al Xavi.
La sentència 12 no tindrà èxit perquè, a banda de la Dolors també el Xavi perdria part dels privilegis que té en el moment d'executar la sentència (RESTRICT).
La sentència 13 no té cap efecte perquè la Dolors no li havia cedit cap privilegi a la Se.



On T=Professors, A= dni, B=nomProf i C=telefon

La sentència 11 s'executarà amb èxit perquè el revocar el privilegi de la Se no provoca l'eliminació de cap altre privilegi (RESTRICT).

b) Xavi té privilegis de SELECT Professors(telefon).
Albert no té cap privilegi sobre Professors.

c) Definiu els rols "rol-Xavi" i "rol-Albert" de tal manera que aquests rols tinguin els privilegis del Xavi i l'Albert després d'executar les sentències.

```
CREATE ROLE rol-Xavi;
GRANT select(telefon) ON Professors TO rol-Xavi;
CREATE ROLE rol-Albert;
```

3) (2 punts) Donada la taula següent:

```
CREATE TABLE items (item integer primary key,
          name char(25),
          qtt integer,
          preu_total decimal(9,2));
```

i la regla de negoci: "***Una única sentència de modificació (update) no pot augmentar la quantitat total en estoc dels productes en més d'un 50%***", ens demanen implementar amb triggers aquesta regla de negoci. Una possible solució en PostgreSQL seria:

```
a) CREATE TABLE temp(old_qtt integer);

b) CREATE FUNCTION update_items_before()RETURNS trigger AS $$
    BEGIN
    DELETE FROM temp;
    INSERT INTO temp SELECT sum(qtt) FROM items;
    RETURN NULL;
   END $$ LANGUAGE plpgsql;

c) CREATE FUNCTION update_items_after()RETURNS trigger AS $$
    DECLARE
    oldqtt integer default 0;
    newqtt integer default 0;
      BEGIN
    SELECT old_qtt into oldqtt FROM temp;
    SELECT sum(qtt) into newqtt FROM items;
    IF (newqtt>oldqtt*1.5) THEN
      RAISE EXCEPTION 'Violació regla de negoci';
    END IF;
    RETURN NULL;
   END $$ LANGUAGE plpgsql;

d) CREATE TRIGGER regla_negociBS BEFORE UPDATE ON items
   FOR EACH STATEMENT EXECUTE PROCEDURE update_items_before();

e) CREATE TRIGGER regla_negociAS AFTER UPDATE ON items
   FOR EACH STATEMENT EXECUTE PROCEDURE update_items_after();
```

3.1) Expliqueu quin és el principal problema d'aquesta solució, atenent als criteris de qualitat de triggers vistos a classe.

3.2) Quins canvis faríeu a d) i e) per tal superar aquest inconvenient? Per què?

3.3) Codifiqueu la nova funció c) que eliminaria els inconvenients de la solució quan es combines amb els canvis introduïts a 3.2.

3.1) És una solució no incremental.
3.2) Solució incremental exemple 4 de les transparències de Disparadors.
3.3) Solució incremental exemple 4 de les transparències de Disparadors.

4) (2 punts)

4.1) Donada la taula R(a,b) (clau primària subratllada) que conté les files {('a1',1), ('a2',2)} i les dues transaccions següents:

T1: insert into R values ('a3',3); update R set b=b*2;
T2: select * from R; select * from R;

Suposant que el SGBD no implementa cap mecanisme de control de concurrència, digueu quins serien els possibles resultats de les dues sentències *select* de T2, en funció dels possibles ordres d'execució de les transaccions. En cas que alguns d'aquests resultats siguin conseqüència de l'existència d'interferències, digueu quines serien aquestes interferències. Argumenteu breument les vostres respostes.
Considereu que les accions (R, RU, W) corresponents a una mateixa sentència SQL s'executen sempre totes seguides.

4.2) Suposem ara que la transacció T2 s'executa concurrentment amb una transacció T3, en l'ordre que tot seguit s'indica:

T2	**T3**
select * from R	
	select * from R where a='a1'
	update R set b=b*2 where a='a1'
	commit
select * from R	
commit	

Suposant que les dues transaccions treballen amb nivell *de repeatable read*, que l'SGBD fa servir reserves S, X, que el gràNul és la fila, i que la taula R conté les files {('a1',1), ('a2',2)}, contesteu a les preguntes següents:
a) Com quedaria l'horari en termes d'operacions de baix nivell? L'horari ha d'incloure, a més de les peticions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves i l'ordre d'execució de totes aquestes peticions.
b) Quins horaris serials hi són equivalents?

4.3) Ens informen que la BD que conté la taula R, a causa d'un desastre, passa a estar innacesible. En aquest escenari contesteu, argumentant les vostres respostes, a les preguntes següents:

a) Quines propietats de les transaccions es veuen compromeses?
b) Quines fonts d'informació necessita l'SGBD per reconstruir la BD?

4.1) Els resultats obtinguts varien en funció de l'ordre d'execució de les transaccions.
Si no hi ha encavalcament, es poden produir dos possibles resultats, en funció de l'horari serial sota consideració:

T1; T2: les dues *select* retornen el mateix resultat: {('a1',2), ('a2',4), ('a3',6)}. No es produeix cap interferència, donat que es tracta d'una execució serial.
T2; T1: les dues select retornen el mateix resultat: {('a1',1), ('a2',2)}. No es produeix cap interferència, donat que es tracta d'una execució serial.

En el cas d'execucions concurrents, les *selects* poden retornar resultats diferents, com a conseqüència de l'existència d'interferències:

T1	T2	Resultat
	select * from R	{('a1',1), ('a2',2)}
insert into R values('a3', 3)		
	select * from R	{('a1',1), ('a2',2), ('a3',3)}
update R set b=b*2		
commit		
	commit	

S'està produïnt a T2 una interferència de tipus fantasma.

Un altre possible resultat seria:

T1	T2	Resultat
	select * from R	{('a1',1), ('a2',2)}
insert into R values ('a3',3)		
update R set b=b*2		
	select * from R	{('a1',2), ('a2',4), ('a3',6)}
commit		
	commit	

En aquest cas, a T2 s'està produïnt una interferència de lectura no repetible i una interferència de tipus fantasma.

Finalment, el darrer resultat possible seria:

T1	T2	Resultat
insert into R values ('a3',3)		
	select * from R	{('a1',1), ('a2',2), ('a3',3)}
update R set b=b*2		
	select * from R	{('a1',2), ('a2',4), ('a3',6)}
commit		
	commit	

En aquest cas, a T2 s'està produïnt una interferència de lectura no repetible

4.2) L'horari, en termes d'operacions de baix nivell i com a conseqüència d'aplicar reserves S,X amb nivell d'aïllament *repeatable read* sobre gràmuls fila, quedaria tal i com s'indica a continuació:

T2	T3
L(f1,S)	
R(f1)	
L(f2,S)	
R(f2)	
	L(f1,S)
	R(f1)
	L(f1,X)
R(f1)	
R(f2)	
commit (U(f1), U(f2))	
	RU(f1)
	W(f1)
	commit (U(f1))

Les cel·les fosques denoten que T3 ha bloquejat la seva execució. L'horari no conté inferències i té associat com horari serial equivalent T2; T3.
Es considera correcte afegir les lectures de la informació de control (IC), però en cap cas es poden efectuar reserves sobre ella, degut al nivell d'aillament.

4.3) La propietat ACID de les transaccions que es veu vulnerada és la definitivitat (veure pàg 52 del pdf de material complementari), donat que s'estant perdent canvis confirmats sobre la BD.
Les fonts d'informació que necessita l'SGBD per fer la reconstrucció són una còpia de segurat de la BD que contingui un estat correcte de la BD i el dietari (en concret el seu contingut a partir del moment que es va fer la còpia de seguretat), tal i com s'explica a la pàgina 55 del pdf de material complementari.

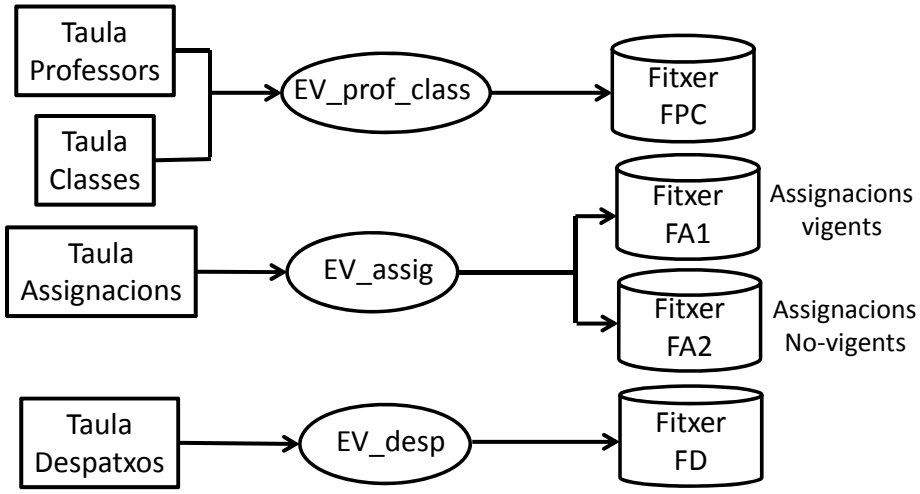
5) (2 punts) Considereu la base de dades de l'exercici 1.

5.1) Doneu un possible model conceptual en UML que generi, quan es fa la traducció a model relacional, les taules anteriors. En el UML hi ha d'haver: classes, atributs, associacions amb les seves multiplicitats, i les claus externes.

5.2) Suposeu ara, que a la base de dades de l'exercici 1 s'hi afegeix la taula *classes*, amb l'esquema següent:

```
classes (aula, horari, grup, dni)
-- on dni és clau forana que referencia professors.
```

a) Digueu a quin tipus correspon cadascun dels espais virtuals indicats a continuació:



b) Com sabeu, els diferents tipus d'espais virtuals poden afavorir certs tipus de consultes. Tenint en compte només els espais virtuals abans identificats, digueu quina de les dues consultes següents es veu afavorida pels espais virtuals anteriors. Justifiqueu breument la resposta.

```
SELECT p.dni,p.nom,c.aula,c.grup,c.horari
FROM professors p, classes c
WHERE p.dni=c.dni;

SELECT d.modul, d.numero, d.superficie
FROM despatxos d, assignacions a
WHERE d.modul=a.modul AND d.numero=a.numero
            AND a.instantFI is NULL and d.modul='Omega';
```

5.3) Suposeu ara que la taula d'assignacions s'emmagatzema emprant un espai virtual de taula i que té aproximadament 10.000 tuples, que estan emmagatzemades en pàgines amb una mitjana de 10 tuples per pàgina. Sabem també que hi ha aproximadament 1000

assignacions amb dni>'444' i que hi ha 300 assignacions al mòdul omega. De les assignacions amb dni>'444' n'hi ha 50 que són al mòdul 'omega'. A més, se sap que s'han definit dos índexs B+ un per dni i un per mòdul. L'arbre B+ per dni és d'ordre d=157, és no agrupat, i té una ocupació de les pàgines de l'índex del 70% en mitjana. L'arbre B+ per mòdul és d'ordre d=227, és no agrupat, i té una ocupació de les pàgines de l'índex del 60% en mitjana.

Donada la consulta següent, estimeu (i justifiqueu breument) el nombre de pàgines (d'índex i de dades) que es llegiran si la consulta es resol emprant els dos índexs amb estratègia d'intersecció de RIDs.

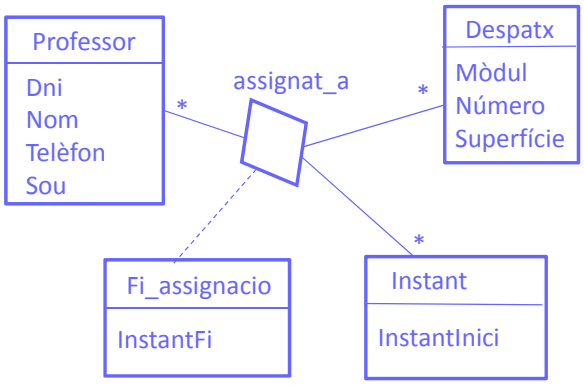
SELECT * FROM assignacions a WHERE a.dni > '444' AND a.modul='Omega'

Solució:

5.1) Clau externa Professor: Dni

Clau externa Despatx: Mòdul, Número

Clau externa Instant: InstantInici



5.2)

a) Espai virtual d'agrupació per professors i classes, espai virtual de taules per despatxos i espai virtual fragmentat per assignacions.

b) La primera consulta es veu afavorida per l'espai d'agrupació donat que es consulten les dades de les dues taules que s'emmagatzemen a l'espai. Cada pàgina de l'espai virtual emmagatzema les files que contenen les dades de cada professor i a continuació les files corresponents a les classes del professor. Per tant, per resoldre la consulta s'accedeix a cada una de les pàgines de l'espai virtual i es van llegint les files de cada pàgina. Com que les dades dels professors i classes s'emmagatzemen juntes el cost de resoldre la consulta és N, sent N el nombre de pàgines que emmagatzemen les dades dels professors més les seves classes.

b) En canvi, la segona consulta no es veu afavorida per l'espai virtual de taula i per l'espai virtual fragmentat donat que es consulten juntes les dades dels despatxos i les assignacions vigents a aquests despatxos. Per resoldre la consulta, cal accedir a cada una de les pàgines de l'espai virtual de taula que emmagatzema les dades dels despatxos, per obtenir les dades dels despatxos situats a l'edifici omega. Per cada despatx, cal recórrer les pàgines del fragment que emmagatzema les assignacions vigents per anar llegint les seves dades. El cost de resoldre la consulta és, N *M sent N el nombre de pàgines que emmagatzemen els despatxos i M el nombre de pàgines que emmagatzemen les assignacions vigents.

5.3)

-1000 assignacions de dni>'444'

-150 assignacions de modul='Omega'

- 50 assignacions que compleixen les dues condicions

Alçada de l'arbre dni = 2

(10000/(157*2*0,7) = 10000/220 = 46 fulles), (46 apuntadors => 1 arrel)

$\lceil \log_{221} 10000 \rceil = 2$

Alçada de l'arbre mòdul = 2

(10000/(227*2*0,6) = 10000/273 = 37 fulles), (37 apuntadors => 1 arrel)

$\lceil \log_{274} 10000 \rceil = 2$

Cost en nombre de pàgines accedides = (ha+Fa) + (hb+Fb) + |RIDa>='444' i RIDb='Omega'|

Cerca de RIDs per dni (ha+Fa):

ha (alçada de l'arbre) = 2

Fa (fulles de l'arbre que cal llegir) = (1000 / 220) -1 = 5 – 1 = 4

Cerca de RIDs per modul (hb+Fb):

ha (alçada de l'arbre) = 2

Fa (fulles de l'arbre que cal llegir) = (300 / 273) -1 = 2 – 1 = 1

Intersecció de RIDs = 50 RIDs

Accés a dades |RIDa>='444' i RIDb='Omega'|:

número de files que compleixen les dues condicions = 50

Cost en nombre de pàgines accedides = (2 + 4) + (2 + 1) + 50 = 59 pàgines.

BD **17.1.2018**

Temps: 3 hores

Notes 31 gener matí **Revisió:** 1 febrer tarda

Cada pregunta s'ha de lliurar en un full separat, excepte la pregunta 1, que s'ha de lliurar els apartats 1.1 i 1.2 en un full, i els 1.3 i 1.4 en un altre

1. (2.5 punts) Suposeu una base de dades amb les taules següents:

```
CREATE TABLE persones
   (nif CHAR(9),
    nom CHAR(30) NOT NULL,
    ciutatRes CHAR(20),
    PRIMARY KEY (nif));
-- ciutatRes és on viu la persona

CREATE TABLE tipus
   (tipusId CHAR(20),
    numMaxInscrits INTEGER,
    PRIMARY KEY (tipusId));

CREATE TABLE activitats
   (tipusId CHAR(20),
    instIni INTEGER,
    descripcio CHAR(30),
    nifP CHAR(9) NOT NULL,
    preu INTEGER,
    ciutat CHAR(20),
    PRIMARY KEY (tipusId, instIni),
    FOREIGN KEY (tipusId) REFERENCES tipus(tipusId),
    FOREIGN KEY (nifP) REFERENCES persones(nif));
    -- el nifP és el de la persona que és professor de l'activitat
    -- ciutat és on es fa l'activitat

CREATE TABLE inscripcions
   (tipusId CHAR(20),
    instIni INTEGER,
    nifI CHAR(9), -- persona inscrita
    instPagament INTEGER,
    PRIMARY KEY (tipusId, instIni, nifI),
    FOREIGN KEY (tipusId, instIni)
            REFERENCES activitats(tipusId,instIni),
    FOREIGN KEY (nifI) REFERENCES persones);
    -- el nifI és el de la persona que s'inscriu a l'activitat
```

Amb les sentències de càrrega de les taules:

```
INSERT INTO persones VALUES ('111', 'Anna', 'Barcelona'), ('222',
'Alicia', 'Sabadell'), ('333', 'Rosa', 'Gava');

INSERT INTO tipus VALUES ('body_pump', 15),('zumba', 20),('aerobic',25);

INSERT INTO activitats VALUES ('body_pump', 212, null, '111', 25,
'Barcelona'), ('zumba', 318, null, '222', 25, 'Sabadell'),('aerobic',
118, null, '222', 20, 'Sabadell');

INSERT INTO inscripcions VALUES ('zumba', 318, '333', 10), ('aerobic',
118, '333', 10);
```

1.1 Per cadascuna de les situacions que es plantegen a continuació indica si és o no possible, tenint en compte les restriccions definides a la base de dades. En cas que no sigui possible indica la restricció que ho evita, en cas que sigui possible dóna els inserts per tal que passi.

a) És possible que hi hagi dues activitats del mateix tipus que comencen al mateix instant impartides per professors diferents?

No, tipusId i instIni formen la clau primària de la taula activitats i, per tant, no és possible que hagi dues tuples amb la mateixa clau.

b) És possible que una persona s'inscrigui en una activitat de la qual ell mateix és professor?

Res ho evita, només cal afegir en els inserts anteriors el següent:

```
INSERT INTO inscripcions VALUES ('aerobic', 118, '222', 10);
```

c) És possible que una persona s'inscrigui en dues activitats del mateix tipus?

Res ho evita, ja que la clau primària és tipusId, instantIni, nifI, es pot repetir el tipus i el nif, i no l'instant. Només cal afegir en els inserts anteriors el següent:

```
INSERT INTO activitats VALUES
('aerobic', 312, null, '111', 25, 'Barcelona'),
INSERT INTO inscripcions VALUES ('aerobic', 312, '333', 10);
```

1.2 Donada la vista:

```
CREATE VIEW nose(identI, nomI, identP, nomP, quantitat) AS
SELECT pI.nif, pI.nom, pP.nif, pP.nom, COUNT(*)
FROM persones PI, persones pP,
     inscripcions i NATURAL INNER JOIN activitats a
WHERE pI.nif = i.nifI AND a.nifP = pP.nif
GROUP BY pI.nif, pP.nif
```

a) Explica breument quines dades s'obtenen quan es fa un select de la vista.

Retorna la quantitat d'inscripcions d'una persona a activitats que fa un professor. Concretament dóna el nif i nom de la persona inscrita, el nif i nom del professor, i la quantitat d'inscripcions de la persona inscrita a activitats del professor.

b) Quin és el contingut de la vista després d'executar els inserts que us donem en l'enunciat?

identi	nomi	identp	nomp	quantitat
333	Rosa	222	Alicia	2

c) És actualitzable? Per què?

No. Motius diversos: múltiples taules, un group by, funció d'agregació.

1.3 Doneu una sentència SQL per obtenir el de les persones que són professors d'alguna activitat en la que no hi ha cap inscrit que visqui en la ciutat en la que es fa l'activitat.

```
SELECT DISTINCT pr.nom
FROM persones pr, activitats a
WHERE  pr.nif = a.nifP and
        NOT EXISTS (SELECT *
                    FROM inscripcions i, persones p
                    WHERE a.tipusId = i.tipusId AND
                          a.instIni = i.instIni AND
                          i.nifI = p.nif AND
                          p.ciutatRes = a.ciutat)
```

1.4 Doneu un conjunt de sentències d'àlgebra relacional que obtenir el nif de les persones que són professors de dues o més activitats que es fan en una mateixa ciutat.

```
A = activitats[tipusId, instantIni, nifP, ciutat]
B = A{tipusId -> tipusId2, instantIni -> instantIni2, nifP -> nifP2, ciutat -> ciutat2}
C = A[nifP = nifP2, ciutat = ciutat2]B
D = C(tipusId <> tipusId2 OR instantIni <> instantIni2)
R = D[nifP]
```

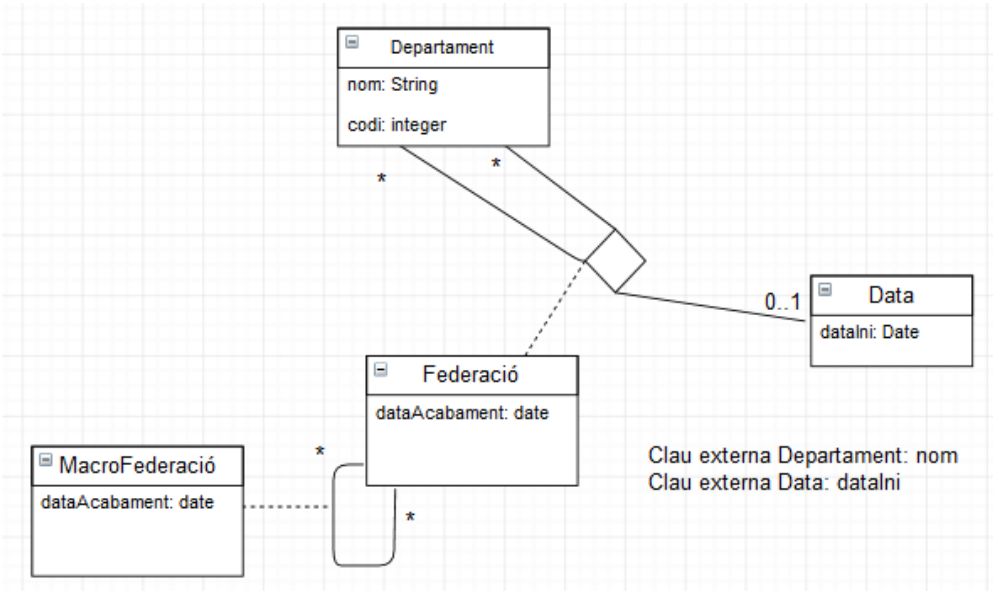
o bé

A = activitats[tipusId, instantIni, nifP, ciutat]
B = A{tipusId -> tipusId2, instantIni -> instantIni2, nifP -> nifP2, ciutat -> ciutat2}
C = A x B
D = C(nifP = nifP2 AND ciutat = ciutat2 AND (tipusId <> tipusId2 OR instantIni <> instantIni2))
R = D[nifP]

o bé

A = activitats[tipusId, instantIni, nifP, ciutat]
B = A{tipusId -> tipusId2, instantIni -> instantIni2, nifP -> nifP2, ciutat -> ciutat2}
C = A [nifP = nifP2 AND ciutat = ciutat2 AND tipusId <> tipusId2] B
D = A [nifP = nifP2 AND ciutat = ciutat2 AND instantIni <> instantIni2] B
E = C u D
R = D[nifP]

2. (2.5 punts) Considerant el disseny conceptual en UML següent:

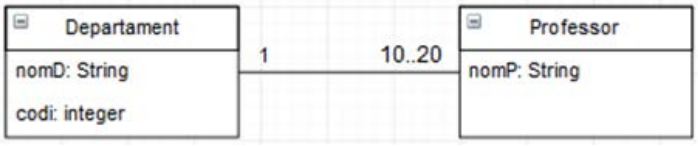


2.1 Doneu disseny lògic que s'obté de traduir el UML anterior a model relacional (cal incloure: taules, atributs, restriccions de clau primària, i restriccions de clau forana).

Departament(nom,codi)
Federació(nomd1,nomd2,dataIni,dataAcabament)
nomd1 references Departament(nom),
nomd2 references Departament(nom)
MacroFederació(nomd1,nomd2,nomd3,nomd4,dataAcabament)
nomd1,nomd2 references Federació(nomd1,nomd2),
nomd3,nomd4 references Federació(nomd1,nomd2),

Tal com s'indica a les transpes les taules corresponents a classes data, hora, instant, no cal posar-les.

2.2 Considereu ara el següent disseny conceptual:



que ha donat lloc a les taules:

```
Departament(nomD, codi)
Professor(nomP, nomD)
    nomD és clau forana que apunta a Departament
    nomD és NOT NULL
```

a) Seria possible expressar en PostgreSQL la cardinalitat 10..20 fent servir alguna restricció de columna o de taula sobre les taules anteriorment definides? En cas de que sigui possible, doneu la sentència. En cas contrari, justifiqueu la resposta.

No seria possible, ja que no es poden tenir subconsultes en els checks

b) Seria possible expressar en SQL estàndard la cardinalitat 10..20 fent servir una asserció. En cas de sigui possible, doneu la sentència. En cas contrari, justifiqueu la resposta.

```
CREATE ASSERTION cardinalitat CHECK(
            NOT EXISTS (SELECT *
                        FROM Professor P
                        GROUP BY P. NOMD
                         HAVING COUNT(*)>20 OR COUNT(*)<10)
          AND NOT EXISTS (SELECT *
                          FROM Departament D
                          WHERE NOT EXISTS (select * from professors p
                                            Where p.nomd=d.nomd)))
```

O bé

```
CREATE ASSERTION cardinalitat CHECK(
            NOT EXISTS (SELECT *
                        FROM Departament D
                            WHERE (SELECT COUNT(*)
                                    FROM professor
                                    WHERE p.nomd=d.nomd) > 20
                                OR ((SELECT COUNT(*)
                                    FROM professor
                                    WHERE p.nomd=d.nomd) < 10)
```

c) Doneu els triggers necessaris sobre Professor per tal de assegurar el compliment de les cardinalitats. Podeu fer servir la plantilla de triggers de PostgreSQL:

```
CREATE TRIGGER nom [BEFORE/AFTER] [UPDATE (of column)/DELETE/INSERT] ON taula
    [FOR EACH ROW/FOR EACH STATEMENT] execute procedure nomp();
CREATE FUNCTION nomp() RETURNS trigger AS $$
    BEGIN
    END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER insercio BEFORE INSERT ON professor
FOR EACH ROW execute procedure insercio();
CREATE or replace FUNCTION insercio() RETURNS trigger AS $$
BEGIN
   IF (select count(*) from professor where nomd=new.nomd) =20 THEN
     RAISE EXCEPTION 'error ins';
   END IF;
   RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER supressió BEFORE DELETE ON professor
FOR EACH ROW execute procedure supressió();
CREATE or replace FUNCTION supressió() RETURNS trigger AS $$
BEGIN
   IF (select count(*) from professor where nomd=old.nomd) =10 THEN
     RAISE EXCEPTION 'error del'; END IF;
   RETURN OLD;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER modificació BEFORE UPDATE of nomD ON professor
FOR EACH ROW execute procedure modificació();
CREATE or replace FUNCTION modificació() RETURNS trigger AS $$
BEGIN
 IF (new.nomd<>old.nomd) THEN
      IF (select count(*) from professor where nomd=new.nomd) =20 THEN
```

```
            RAISE EXCEPTION 'error mod'; END IF;
        IF (select count(*) from professor where nomd=old.nomd) =10 THEN
           RAISE EXCEPTION 'error mod'; END IF;
     END IF;
    RETURN NEW;
    END; $$ LANGUAGE plpgsql;
```

3. **(2.5 punts)** Considera la base de dades i la transacció següent:

Esquema	*Extensió*
oficines(numOficina, adreça) comptes(numOficina, numCompte, saldo) {numOficina} referencia oficines	Oficines 12 Barcelona 24 Vic comptes 12 450 60 24 670 200

T1
UPDATE comptes SET saldo = saldo + 50 WHERE numOficina = 12 AND numCompte = 450;
UPDATE comptes SET saldo = saldo - 50 WHERE numOficina = 24 AND numCompte = 670;
commit;

3.1 Interferències de lectura no repetible

a) Escriu un horari en que participi la transacció T1 i on hi hagi únicament una interferència de lectura no repetible. En el nou horari s'ha d'afegir una transacció T2 que només pot utilitzar sentències `select`.

T1	*T2*
UPDATE comptes SET saldo = saldo + 50 WHERE numOficina = 12 AND numCompte = 450;	
	SELECT * FROM comptes WHERE numOficina = 24 AND numCompte = 670;
UPDATE comptes SET saldo = saldo - 50 WHERE numOficina = 24 AND numCompte = 670;	
	SELECT * FROM comptes WHERE numOficina = 24 AND numCompte = 670;
commit;	
	commit;

És una solució correcta qualsevol T2 on hi hagi dos selects que cadascun consulti una única fila (o bé 12, 450 o bé 24, 670), i que aquesta fila sigui la mateixa. Els dos selects han d'estar sempre abans i després de l'update del mateix gränul que consulten. Els selects han de consultar com a mínim l'atribut saldo del compte.

b) Indica quin és el nivell d'aïllament mínim que evita aquest tipus d'interferències.

Repeatable Read

3.2 Interferències d'anàlisi inconsistent

a) Escriu un horari en que participi la transacció T1 i on hi hagi únicament una interferència d'anàlisi inconsistent. En el nou horari s'ha d'afegir una transacció T2 que només pot utilitzar sentències `select`.

T1	*T2*
UPDATE comptes SET saldo = saldo + 50 WHERE numOficina = 12 AND numCompte = 450;	

	`SELECT * FROM comptes WHERE numOficina = 12 AND numCompte = 450;`
	`SELECT * FROM comptes WHERE numOficina = 24 AND numCompte = 670;`
`UPDATE comptes SET saldo = saldo - 50 WHERE numOficina = 24 AND numCompte = 670;`	
`commit;`	
	`commit;`

És una solució correcta qualsevol T2 on hi hagi dos selects que consultin cadascun una fila de la taula (o bé 12, 450 o bé 24, 670). Els dos selects poden estar tots dos entremig dels dos updates (en qualsevol ordre), o poden estar un abans i un després dels updates (en qualsevol ordre).

És també una solució correcta una T2 on hi hagi un únic select que consulti les dues files de la taula, o totes les files de la taula (que són les dues files). En aquest cas el select ha d'estar necessàriament entremig dels dos updates.

Els selects han de consultar com a mínim l'atribut saldo del compte.

b) Indica quin és el nivell d'aïllament mínim que evita aquest tipus d'interferències.

Repeatable Read

3.3 Interferències de fantasmes

a) Escriu un horari en que participi la transacció T1 i on hi hagi únicament una interferència de fantasmes. En el nou horari s'ha d'afegir una transacció T2 que només pot utilitzar sentències `select`.

T1	***T2***
`UPDATE comptes SET saldo = saldo + 50 WHERE numOficina = 12 AND numCompte = 450;`	
	`SELECT * FROM comptes WHERE saldo<200;`
`UPDATE comptes SET saldo = saldo - 50 WHERE numOficina = 24 AND numCompte = 670;`	
	`SELECT count(*) FROM comptes WHERE saldo<200;`
`commit;`	
	`commit;`

En cas de posar una T2 amb dos selects que consultin els comptes amb un saldo superior a 70 al voltant del primer update, a part de un fantasma hi hauria un anàlisi inconsistent.

En el cas de posar una T2 amb dos selects un abans dels dos updates i l'altre després a part d'un fantasma es pot produir una interferència de lectura no repetible.

b) Tradueix l'horari a operacions R, RU i W tenint en compte que el gràNul és la fila.

Suposant que la fila 12, 450 és el gràNul a, i que la fila 24, 670 és el gràNul b.

	T1	T2
40	RU(a)	
50	W(a)	
60	RU(IC)	
70	W(IC)	
80		R(IC)
90		R(a)
110	RU(b)	
120	W(b)	

130	RU(IC)	
140	W(IC)	
150		R(IC)
160		R(a)
179		R(b)
190	c	
200		c

c) Indica quin és el nivell d'aïllament mínim que evita aquest tipus d'interferències.

SERIALIZABLE

d) Dona l'horari de l'apartat b amb les reserves i les esperes incorporades segons el nivell d'aïllament indicat.

	T1	T2
40	L(a,X)	
50	RU(a)	
60	W(a)	
70	L(IC,X)	
80	RU(IC)	
90	W(IC)	
100		L(IC,S)
120	L(b,X)	
140	RU(b)	
150	W(b)	
160	RU(IC)	
170	W(IC)	
180	c+U(a,b,IC)	
190		R(IC)
200		L(a,S)
210		R(a)
220		L(b,S)
230		R(b)
240		c+U(a,b,IC)

4. (2.5 punts) Considerant la base de dades de l'exercici anterior. Suposa que a la taula oficines hi ha 3.000 oficines, i que a la taula comptes hi ha 15.000.000 comptes.

4.1 Indica si com a resultat de cadascuna de les consultes anteriors es poden obtenir o no resultats repetits. En cas que sí, dóna un contingut de les taules que faci que s'obtinguin resultats repetits. En cas que no, explica perquè no pot passar.

a) SELECT COUNT(*) FROM comptes WHERE saldo>200 GROUP BY numOficina;

b) SELECT numCompte FROM comptes WHERE numOficina=555;

c) SELECT numOficina FROM oficines NATURAL INNER JOIN comptes;

4.2 Suposa que sobre la taula comptes hi ha únicament definit un índex agrupat (arbre B+) multi-atribut definit pels atributs numOficina, numCompte. La d de l'arbre és 70, i està ple al 80%. Suposa que cada oficina té una mitjana de 5.000 comptes, i que el factor de bloqueig de les pàgines de dades és 100. Explica quina és la manera òptima de resoldre cadascuna de les consultes següents. Calcula els cost de cada consulta, indicant el que significa cada factor que intervé en el càlcul.

a) SELECT * FROM comptes ORDER BY numOficina, numCompte;

b) SELECT * FROM comptes WHERE numOficina=350 AND numCompte = 18;

c) SELECT numOficina, numCompte FROM comptes WHERE numOficina=480;

4.3 Suposa que tenim un índex no agrupat (arbre B+) definit sobre l'atribut saldo que té 50000 nodes fulla, i que l'índex està ple als 70%.

a) Indica quina és la d de l'arbre. Explica com l'has obtingut.

b) Explica on surten els RIDs en un índex arbre B+. Indica quants RIDs hi haurà en total en els nodes de l'índex indicat. Explica com has fet els càlculs.

4.1.a Hi haurà resultats repetits si dos oficines tenen un mateix número de comptes amb saldo superior a 200.

oficines (1, 'Barcelona), (2, 'Vic)

comptes (1, 1, 250), (1, 2, 350), (2, 1, 450), (2, 2, 500)

en aquest cas el resultat seria (2), (2)

4.1.b No hi pot haver repetits perquè surt una fila per cada compte de l'oficina 555, i en l'oficina 555 no hi pot haver números de compte repetits, tenint en compte la clau primària de la taula comptes

4.1.c Hi haurà repetits en cas que hi hagi una oficina amb més d'un compte.

oficines (1, 'Barcelona), (2, 'Vic)

comptes (1, 1, 250), (1, 2, 350), (2, 1, 450), (2, 2, 500)

en aquest cas el resultat seria (1), (1), (2), (2)

4.2.a Com que es tracta d'un índex agrupat en que les dades estan ordenades com l'índex, i l'accés seqüencial per valor és en el mateix ordre de l'ordenació n'hi haurà prou en accedir al fitxer de dades per resoldre la consulta, així doncs els cost serà número de files de la taula, dividides pel factor de bloqueig de les pàgines de dades.

cost = 15000000 files/100 files/pàgina = 150000 pàgines

4.2.b En aquest cas la consulta és un accés directe per valor que només retorna una única fila, ja que s'accedeix per la clau primària, així doncs els cost serà el cost de baixar per l'arbre més un accés més a la pàgina de dades que conté la fila.

valors per node 70 x 2 x 0.8 = 112

alçada de l'arbre = $\log_{113}15000000 = 3.5 \Rightarrow 4$

cost = 4 + 1

4.2.c En aquest cas la consulta és un accés seqüencial per valor a tots els comptes d'una oficina. Cal tenir en compte que només es mostra els números dels comptes. Cal també tenir en compte que aquests números estan al propi índex, no cal accedir a les dades per buscar aquests números. Així doncs els cost serà el cost de baixar per l'arbre més un accés a les pàgines fulles de l'índex que contenen números de compte de l'oficina.

alçada de l'arbre 4

número de nodes fulla de l'índex que conté números de compte d'oficines

5000 comptes/oficina / 112 valors/node = 45 nodes = 45 pàgines

cal restar 1 perquè la primera fulla es compta dues vegades

cost = 4 + 45 -1

4.3.a 15000000 valors en els nodes fulla

50000 nodes fulla

15000000 valors / 50000 nodes fulla = 300 valors per node

nodes plens al 70%

2 x d x 0.7 = 300, d= 300/1.4, d= 214

4.3.b RIDs <u>només</u> n'hi han als nodes fulla d'un arbre B+, i n'hi ha tants com valors hi ha als nodes fulla. Com que als nodes fulla hi ha d'haver 15000000 valors, hi haurà 15000000 de RIDs.

BD **28.6.2018**

Temps: 3 hores

Notes 4 juliol Revisió: 5 juliol matí

Cada pregunta s'ha de lliurar en un full separat, excepte la pregunta 1, que s'ha de lliurar els apartats 1.1 i 1.2 en un full, i els 1.3 i 1.4 en un altre

1. (2.5 punts) Suposeu una base de dades amb les taules següents:

```
create table departaments
(codiDept integer,
nomDept char(50) unique,
telefonDept char(15),
primary key (codiDept));

create table professors
(dni char(50),
nomProf char(50) unique,
telefonProf char(15),
sou integer not null check(sou>0),
dniCoord char(50),
codiDept integer not null,
primary key (dni),
foreign key (dnicoord) references professors,
foreign key (codiDept) references departaments);
-- dniCoord correspon al dni del coordinador del professor
-- codiDept correspon al departament al que pertany el professor

create table despatxos
(modul char(5),
numero char(5),
superficie integer not null check(superficie>12),
codiDept integer,
primary key (modul,numero),
foreign key (codiDept) references departaments);
-- codiDept correspon al departament al que pertany el despatx

create table assignacions
(dni char(50),
modul char(5),
numero char(5),
instantInici integer,
instantFi integer,
primary key (dni, modul, numero, instantInici),
foreign key (dni) references professors,
foreign key (modul,numero) references despatxos);
-- instantFi te valor null quan una assignacio es encara vigent.
```

1.1 Doneu una sentència SQL per obtenir els departaments tals que totes les assignacions als seus despatxos han sigut assignacions de professors del departament. En la consulta han d'aparèixer també els departaments pels que no hi ha hagut cap assignació a despatxos del departament.

```
select dpt.codiDept
from departaments dpt
where not exists (select *
                  from assignacions a, despatxos d, professors p
                  where a.modul = d.modul and
                        a.numero = d.numero and
                        d.codiDept = dpt.codiDept and
                        a.dni = p.dni and
                        p.codidept != dpt.codiDept)
```

1.2 Escriviu les sentències SQL per tal que l'usuari X (el propietari de les 4 taules) permeti que es puguin fer les operacions següents sobre les taules anteriors:

a) L'usuari A pot modificar tots els atributs de la taula assignacions i de la taula professors, i també modificar el departament dels despatxos. Pot passar aquests privilegis a altres usuaris.

X: GRANT UPDATE ON assignacions to A WITH GRANT OPTION ;
GRANT UPDATE ON professors to A WITH GRANT OPTION ;
GRANT UPDATE(codiDept) ON despatxos to A WITH GRANT OPTION;

b) L'usuari A permet que l'usuari C pugui modificar el departament dels despatxos.

A: GRANT UPDATE(codiDept) ON despatxos to C;

c) L'usuari X permet que l'usuari C modifiqui la taula despatxos. L'usuari C pot passar aquests privilegis a altres usuaris.

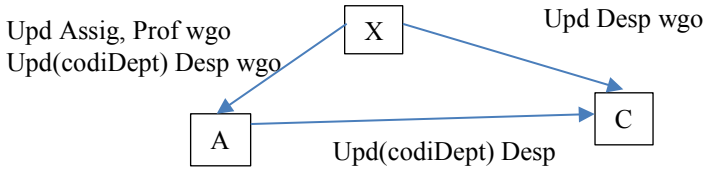
X: GRANT UPDATE ON despatxos to C WITH GRANT OPTION;

d) L'usuari X desautoritza l'usuari A per modificar l'atribut codiDept de la taula despatxos en mode RESTRICT.

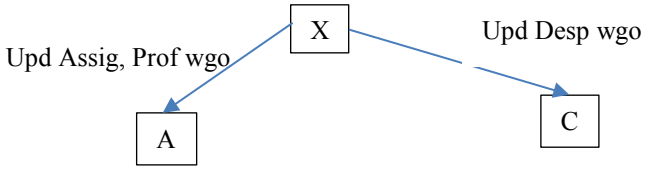
X: REVOKE UPDATE(codiDept) ON despatxos TO A RESTRICT.

e) Dibuixa el diagrama d'autoritzacions després d'executar les sentències de a), b), c) i abans de d) i el diagrama d'autoritzacions resultant després de la sentència d).

Abans del REVOKE



Després del REVOKE



1.3 Doneu una seqüencia d'operacions d'àlgebra relacional per obtenir el codi i nom dels departaments que han tingut o tenen professors assignats a despatxos d'un altre departament.

```
A = assignacions * professors
B = despatxos{modul -> mod, numero -> num, codiDept → cd}
C = A[modul = mod, numero = num, codiDept <> cd]B
D = C[codiDept]
E = D*departaments
F = E[codiDept, nomDept]
```

1.4 Suposant que la quantitat de departaments és DP, la quantitat de professors és P, la quantitat de despatxos és DX, i la quantitat d'assignacions és A, digueu i justifiqueu quin serà l'esquema i la cardinalitat mínima i màxima de la relació R que s'obté de cadascuna de les seqüències d'operacions d'àlgebra relacional següents:

a) R = departaments*professors

R(codiDept, nomDept, telefonDept, dni, nomProf, telefonProf, sou, dniCoord)

Card mínima = 0 Tots els professors tenen el codiDept a NULL o no n'hi ha cap

Card màxima = P Tots els professors tenen codiDept diferent de NULL

b) A = professors{codiDept → cd}

R = departaments[codiDept * cd]A

R(codiDept, nomDept, telefonDept, dni, nomProf, telefonProf, sou, dniCoord)

Card mínima = 0 Tots els professors tenen el codiDept a NULL o no n'hi ha cap

Card màxima = P Tots els professors tenen codiDept diferent de NULL

c) A = professors{codiDept → cd}

R = departaments[codiDept < cd]A

R(codiDept, nomDept, telefonDept, dni, nomProf, telefonProf, sou, cd)

Card mínima = 0 Tots els professors estan al mateix departament o no hi ha cap

Card màxima = (DP-1)*P Tots els professors estan assignats al departament més gran i ha altres departaments

2. (2.5 punts) Considereu la base de dades de l'exercici 1.

2.1 Doneu el disseny conceptual en UML tal que al traduir a model relacional donaria com a resultat l'esquema de la base de dades de l'exercici 1.

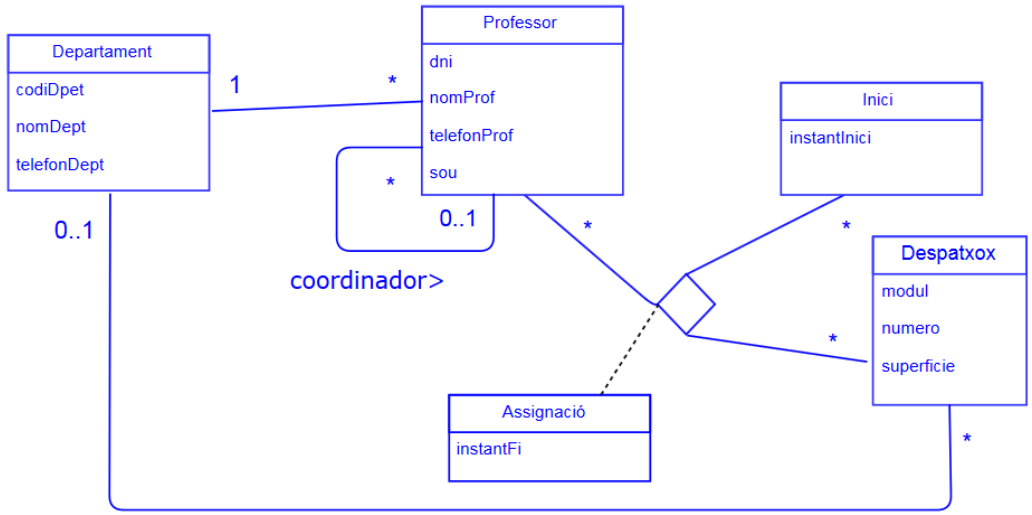
2.2 Definiu una asserció en SQL Standard per garantir el compliment de la restricció següent: "No existeix cap departament amb més de 5 professors que tinguin un coordinador que no és del departament".

2.3 Indiqueu els disparadors que caldria per implementar l'asserció definida a l'apartat 2.2 en PostgreSQL.

Per a cada disparador cal que indiqueu: l'esdeveniment activador, la taula, el tipus de disparador, i la justificació del tipus de disparador escollit. En cas que l'esdeveniment activador sigui un UPDATE cal també explicitar els atributs rellevants.

Tingueu en compte que les restriccions d'integritat definides a la BD (primary key, foreign key,...) es violen amb menys freqüència que la restricció comprovada per aquests disparadors.

2.1)



2.2)

```
CREATE ASSERTION assercioSol1 CHECK (not exists(select p.codiDept
            from professors p,professors p1
            where p.profCoord=p1.dni and
                  p.codiDept<>p1.codiDept
            group by p.codiDept
            having count(*)>5)
```

Es considera també correcta una solució on s'afegeixi la condició "p.dni.coord is not null". Encara que es tracta d'una condició innecessària, ja que en cas que no es compleixi aquesta condició mai es pot complir la condició "p.dnicoord=p1.dni".

La solució següent també funciona, encara que té una subconsulta innecessària.

```
CREATE ASSERTION assercioSol2 CHECK (not extists(select d.codiDept
            from departaments d
            where 5 < (select count(distinct p1.dni)
                        from professors p, professors p1
                        where p.codiDept = d.codiDept and
                              p.profCoord=p1.dni and
                              p.codiDept<>p1.codiDept))
```

2.3)

Professors	INSERT BEFORE FOR EACH ROW	UPDATE, CodiDept BEFORE FOR EACH ROW	UPDATE, dniCoord BEFORE FOR EACH ROW

Com que l'enunciat indica que la restricció es viola amb menys freqüencia que les restriccions d'integritat de la BD, el tipus de disparador ha de ser BEFORE.
L'esdeveniment DELETE on Professors no és rellevant:
- Considerant que en la base de dades quan s'esborra un professor s'apliqués una política de manteniment d'integritat referencial RESTRICCIÓ, no seria rellevant. El motiu és que aquest esdeveniment no podria arribar mai a violar la restricció, ja que només s'arribaria a fer l'esborrat en cas que el professor no fos cap de cap altre professor.
- Considerant que en la base de dades quan s'esborra un professor s'apliqués una política de manteniment d'integritat referencial CASCADA, tampoc seria rellevant. El motiu és que en esborrar el professor de les claus foranes dels professors que li eren coordinats, aquests professors no podrien passar a violar la restricció, ja que serien professors que no tindrien coordinador, i per tant a qui no afectaria la restricció.

3. (2.5 punts) Considereu les taules R(A,B) i S(C). Suposeu que les taules estan buides inicialment i que el gränul de concurrència és la tupla. Per a cadascun de les parelles de transaccions següents determineu tots els possibles resultats [X,Y], on X és el resultat del primer count de T1 i Y el resultat del segon.

Per a cada resultat és imprescindible justificar la resposta en base als nivells d'aïllament de les transaccions, i les potencials esperes degudes a locks. Justifiqueu també perquè no hi ha cap altre resultat possible a part dels que proposeu.

1.1 T1:

```
Set Transaction Isolation Level Read Committed;
Select count(*) From R;
Select count(*) From S;
Commit;
```

T2:

```
Set Transaction Isolation Level Serializable;
Insert Into R Values (1,2);
Insert Into S Values (3);
Commit;
```

[1,1] T1 s'executa després de T2. No hi ha esperes.
[0,0] T2 s'executa després de T1. No hi ha esperes.
[0,1] Tota la transacció T2 s'executa entre el primer i el segon count de T1. Això és possible ja que el mode read Committed allibera immediatament després de la lectura.
Cap altre resultat és possible ja que el nivell read committed farà que T1 esperi fins al commit de T2 abans de llegir les tuples insertades de T2.

1.2 T1:

```
Set Transaction Isolation Level Read Committed;
Select count(*) From R;
Select count(*) From S;
Commit;
```

T2:

```
Set Transaction Isolation Level Serializable;
Insert Into R Values (1,2);
Insert Into R Values (3,4);
Commit;
```

[0,0] Tot t2 s'executa després de t1
[2,0] Tot t1 s'executa després de t2
Cap altre resultat és possible ja que d'una banda el nivell read committed farà que T1 esperi fins al commit de T2 abans de llegir la tuple insertada de T2.
A més, T2 no afecta al segon count.

1.3 T1:

```
Set Transaction Isolation Level Repeatable Read;
Select count(*) From R;
Select count(*) From R;
Commit;
```

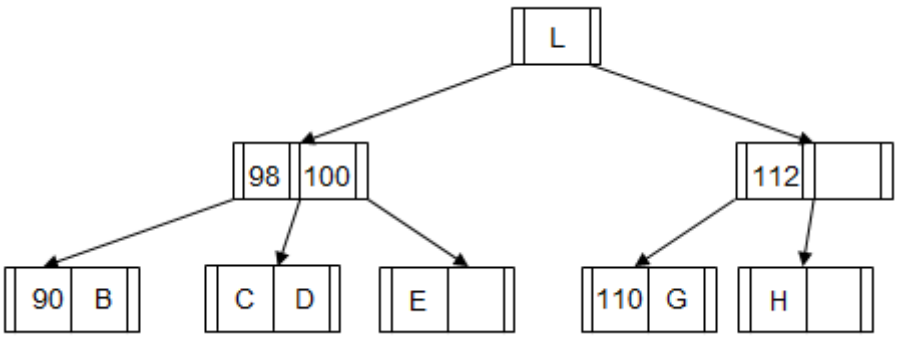
T2:

```
Set Transaction Isolation Level Serializable;
Insert Into R Values (1,2);
Commit;
```

[1,1] T1 s'executa després de T2.
[0,0] T2 s'executa després de T1.
[0,1] Tota la transacció T2 s'executa entre el primer i el segon count de T1. Això és possible ja que el mode repeteable read no reserva la informació de control, i per tant T2 no queda bloquejada. Es produeix un fantasma.

4. (2.5 punts)

4.1 Donat l'arbre B+ següent, d'ordre d=1, i definit per a l'atribut a de la taula T(a,b), indiqueu quins són els valors que poden prendre B, C, D, E, G, H, L.



Cal recordar que tal com s'ha explicat durant el curs:
- Al node arrel i nodes interns NO hi pot haver valors que NO estiguin als nodes fulla. És a dir tots els valors que estan a nodes arrel o interns han d'estar necessàriament a nodes fulla.
- L'apuntador de l'esquerra porta a valors més petits, l'apuntador de la dreta porta a valors superiors o iguals.

B pot no tenir valor, o un valor entre 91 i 97.
C ha de tenir valor 98
D pot no tenir valor, o valor 99.
E ha de tenir valor 100.
G pot no tenir valor, o valor 111.
H ha de tenir valor 112
L ha de tenir valor 110

4.2 Considereu la taula assignacions de la base de dades de la pregunta 1. Suposeu que la taula té 100.000 de files, i un factor de bloqueig de 10 files per pàgina. Suposeu també que: hi ha 18 assignacions del professor 123 a la taula assignacions, de les quals només 2 són assignacions vigents (instantFi null); hi ha 3000 assignacions al mòdul A en despatxos amb número superior a 100, de les quals només 450 són assignacions vigents.

Expliqueu com es resol i mostreu com es calcula el cost de la consulta següent en els casos indicats a sota la consulta.

```
select * from assignacions
where modul = 'A'
      and numero >= '100'
      and dni = '123'
      and instantFi is null
```

a) En cas de que no s'usi cap índex.

b) En cas que s'utilitzi un índex arbre B+ agrupat multiatribut definit per als atributs modul, numero amb 3 nivells, una d de 146 i una ocupació del 80%.

c) En cas que s'utilitzi un índex arbre B+ no agrupat definit per l'atribut dni amb 2 nivells, una d de 292 i una ocupació del 70%.

a) Si no hi ha índex, no hi ha més remei que accedir a totes les pàgines de dades.

(100.000 files):(10 files/pàg)= 10.000 pàg

b) Al ser agrupat el SGBD baixa per l'índex i després va directament a les pàgines de dades (que estaran ordenades igual que l'índex pel motiu de estar definit com a índex agrupat).

A través de l'índex l'únic que es pot trobar són les files del modul 'A' i numero és 100, però no se sap res de la resta de condicions, que caldrà comprovar quan es llegeixi les files en el fitxer de dades. Recordeu que en l'índex només hi ha els valors, que en aquest cas són parelles (modul, numero).

Hi ha 3000 files a la taula assignacions que compleixen les condicions modul = 'A' and numero >= '100'.

3 pàg (baixar per l'índex)
+ $\lceil$(3000 files):(10 files/pàg)$\rceil$ = 303 pàgines

c) Al ser NO agrupat el SGBD baixa per l'índex i després ha d'usar les fulles de l'índex per trobar les assignacions del mateix dni '123'. Per cada valor '123' en les fulles de l'índex el SGBD usa el RID corresponent per trobar la fila al fitxer de dades.

A través de l'índex l'únic que es pot trobar són les files del dni '123', però no se sap res de la resta de condicions, que caldrà comprovar quan es llegeixi les files en el fitxer de dades. Recordeu que en l'índex només hi ha els valors, que en aquest cas són dni.

Hi ha 3000 files a la taula assignacions que compleixen la condició associada als atributs modul, numero.

2 pàg (baixar per l'índex)
+ $\lceil$(18 valors):$\lceil$(292*2*0,7)$\rceil\rceil$ - 1
+ 18 pàgines = 20 pàgines

El -1 és per no comptar dues vegades el primer node fulla de l'arbre (al baixar per l'arbre i al comptar els nodes fulla).

1.5 Sabent que la taula professors de la base de dades de la pregunta 1 té 6 atributs. Indica quants índexs (arbres B+) agrupats i quants índexs no agrupats, sobre un únic atribut, poden existir de manera simultània per a aquesta taula. Justifica la resposta.

Una taula només pot tenir definit un índex agrupat. El motiu és que quan un índex és agrupat les pàgines de dades tenen les files ordenades com l'índex. Per tant, és impossible tenir les files ordenades a l'hora físicament de dues maneres diferents.

Hi poden haver, doncs, índex no agrupats definits per a la resta dels atributs, ja que pels índexs no agrupats no es hi ha un ordre físic establert de les files de la taula.

Temps: 3 hores

1. (2 punts) Suposeu una base de dades amb les taules següents:

```
CREATE TABLE EquipsFutbol(
  nomEquip char(30),
  localitat char(50),
  nomEstadi char(100) UNIQUE NOT NULL,
PRIMARY KEY(nomEquip));

CREATE TABLE Partits(
  nomEquipLocal char(30),
  dataPartit date,
  nomEquipVisitant char(30) NOT NULL,
  golsEquipL integer NOT NULL CHECK(golsEquipL >=0),
  golsEquipV integer NOT NULL CHECK(golsEquipV >=0),
PRIMARY KEY (nomEquipLocal,dataPartit),
FOREIGN KEY (nomEquipLocal) REFERENCES EquipsFutbol,
FOREIGN KEY (nomEquipVisitant) REFERENCES EquipsFutbol,
UNIQUE (nomEquipVisitant, dataPartit),
CHECK(nomEquipLocal <> nomEquipVisitant));

CREATE TABLE Jugadors(
  dniJugador char(9),
  nom char(50) NOT NULL,
  nomEquip char(30) NOT NULL,
PRIMARY KEY (dniJugador),
FOREIGN KEY (nomEquip) REFERENCES EquipsFutbol);

CREATE TABLE Alineacions(
  nomEquipLocal char(30),
  dataPartit date,
  dniJugador char(9),
  gols integer NOT NULL CHECK(gols>=0),
  numTargetes integer NOT NULL CHECK(numTargetes>=0),
PRIMARY KEY (nomEquipLocal,dataPartit,dniJugador),
FOREIGN KEY (nomEquipLocal,dataPartit) REFERENCES Partits,
FOREIGN KEY (dniJugador) REFERENCES Jugadors);
```

1.1 Escriviu una sentència SQL per obtenir els equips que no han perdut cap partit jugat a fora i que han guanyat tots els partits jugats a casa. Es diu que un equip juga un partit a casa quan és l'equip local del partit, i que un equip juga a fora quan és l'equip visitant del partit. Per cadascun dels equips que surtin en el resultat de la consulta es vol el nom de l'equip i la quantitat total de gols que ha fet en els partits jugats a casa.

```
SELECT p.nomEquipLocal, sum(golsEquipL)
FROM partits p
WHERE NOT EXISTS(SELECT *
                 FROM partits pV
                 WHERE p.nomEquipLocal = pV.nomEquipVisitant
                      AND pV.golsEquipV<=pV.golsEquipL)
      AND NOT EXISTS (SELECT *
                 FROM partits pL
                 WHERE p.nomEquipLocal = pL.nomEquipLocal
                      AND pL.golsEquipL<=pL.golsEquipV)
GROUP BY p.nomEquipLocal
```

1.2 Suposeu que volem una sentència SQL per trobar els jugadors (dniJugador, nom) que no han estat en cap alineació. Digueu, per cadascuna de les sentències següents **si dóna el resultat correcte o incorrecte**.

- En cas de que el resultat que doni sigui **correcte,** digueu si hi ha alguna part de la consulta que no passa els criteris de qualitat de l'assignatura, i quina és aquesta part.
- En cas que el resultat que doni sigui **incorrecte**, donar un contingut de les taules, el resultat que dóna la consulta tal com està per aquest contingut i el resultat que hauria de donar si fos correcte.

```
a) SELECT j.dniJugador, j.nom
   FROM jugadors j
   WHERE j.dniJugador NOT IN (SELECT a.dniJugador
                     FROM alineacions a, jugadors ju
                     WHERE a.dniJugador=ju.dniJugador);

b) SELECT DISTINCT j.dniJugador, j.nom
   FROM jugadors j
   WHERE NOT EXISTS (SELECT * FROM alineacions a
                      WHERE a.dniJugador=j.dniJugador);

c) SELECT DISTINCT j.dniJugador, j.nom
   FROM jugadors j, alineacions a
   WHERE j.dniJugador = a.dniJugador and
         j.dniJugador NOT IN (SELECT al.dniJugador
                              FROM alineacions al);
```

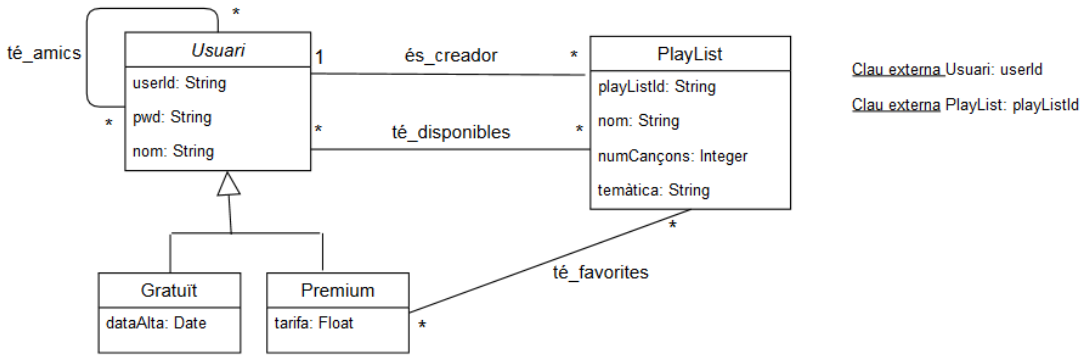
a) Correcte. Taula innecessària jugadors ju.
b) Correcte. DISTINCT innecessari.
c) Incorrecte. En una base de dades amb només una fila a la taula jugadors, la fila hauria de sortir en el resultat de la consulta, i en canvi la consulta tal com està no dóna cap resultat.

1.3 Escriviu una seqüència d'operacions d'àlgebra relacional per obtenir els dni dels jugadors que han tingut alguna alineació en un partit on també ha estat alineat el jugador amb dni '999'. Tingueu en compte que en el resultat de la consulta no volem que surti el jugador '999'.

B=alineacions(dniJugador='999')
C=B[dniJugador, nomEquipLocal,dataPartit]
D=C{dniJugador ->dniJ, nomEquipLocal ->nomE,dataPartit –>dataP}
E=alineacions[dniJugador, nomEquipLocal,dataPartit]
F=E[dniJugador<>dniJ,nomEquipLocal=nomE,dataPartit =dataP]D
R=F[dniJugador]

2. (2 punts)

2.1 Doneu la traducció a model relacional del diagrama de classes UML següent. Cal incloure: taules, atributs, claus primàries, claus foranes, i restriccions NOT NULL i UNIQUE derivades del diagrama.



2.2 Tenint en compte l'esquema de la base de dades que heu obtingut a l'apartat 2.1. Indiqueu si es pot implementar, en PostgreSQL, cadascuna de les restriccions textuals següents **com a restricció de columna o de taula.**

- En cas que es pugui, indica quina serà la restricció i en quina taula es definirà.
- En cas que no es pugui, explica perquè.

RI1 – Un usuari no es pot tenir com a amic a sí mateix.

RI2 – Un usuari només pot crear un màxim de 20 playlists.

RI3 – Les playlists favorites d'un usuari premium han de ser playlists que l'usuari tingui disponibles.

RI4 – No poden existir dues playlists amb el mateix nom i temàtica.

RI5 – El número de cançons d'una playlist ha de tenir valor i aquest valor ha de ser superior a 0.

2.3 Suposeu per aquest exercici una base de dades que té una taula amb els moviments realitzats amb les targetes de crèdit d'una entitat bancària. La taula `moviments` és propietat de l'usuari A. A continuació podeu veure l'esquema de la taula i un exemple del seu contingut.

moviments(idTargeta, instant, botiga, import)

'1111222244445555'	7777	'Zerka'	35
'2222333344445555'	8888	'Sous'	900
'2222333344445555'	9999	'Tnac'	330

a) Escriu les sentències SQL necessàries per tal que l'usuari B pugui consultar i modificar únicament els moviments que tenen un import inferior a 1000.

b) Un cop l'usuari B tingui l'accés que li heu donat a l'apartat anterior, justifiqueu si podrà o no executar la sentència SQL següent:

```
UPDATE moviments
SET import = import + 200
WHERE idTargeta = '2222333344445555' and instant = 8888;
```

2.1 usuari(userId, pwd, nom)

gratuït(userId,dataAlta)
{userId} referencia usuari

premium(userId,tarifa)
{userId} referencia usuari

playlist(playListId,nom,numCançons,tematica,userIdCreador)
{userIdCreador} referencia usuari NOT NULL

disponibles(userId,playListId)
{userId} referencia usuari
{playListId} referencia playList

favorites(userId,playListId)
{userId} referencia premium
{playListId} referencia playList

amics(userId, userIdAmic)
{userId} referencia usuari
{userIdAmic} referencia usuari

2.2 RI1. check (userId <> userIdAmic), taula amics.

RI2. En SQL Standard es podria posar com un check amb una expresió SQL que comptés el número de playlist creades. En PostgreSQL no es pot perquè és un check que afecta a vàries taules.

RI3. {userId, playListId} references disponibles, taula favorites

RI4. UNIQUE(nom, tematica), taula playlist

RI5. NOT NULL CHECK (numCançons>0), taula playlist

2.3

a)
A: create view movs1000 as select * from moviments
where import > 1000

A: grant select, update on movs1000 to B

b) No la pot executar perquè no té privilegis sobre la taula moviments, només sobre la vista.
Si fos la vista podria o no depenent de si la vista s'hagués definit com a "with check option".

3. (2 punts) Considereu una base de dades amb les taules següents:

```
departaments(idD, nomD, pressupost)
empleats(numE, nomE, sou, cap, idD)
     {cap} references empleats(numE)
     {idD} references departaments(idD)
edificis(nomEd, adreça)
assignacions(nomEd, idD, planta)
     {nomEd} references edificis(nomEd)
     {idD} references departaments(idD)
```

3.1 Suposeu que cadascuna de les `regles` que s'indiquen a continuació es vol implementar mitjançant triggers. Indiqueu i justifiqueu per cada regla quins triggers caldria definir, tenint en compte els criteris de qualitat establerts a l'assignatura. Per cada trigger cal indicar:

- esdeveniment que l'activa (cal indicar esdeveniment, taula i atributs rellevants)
- before/after
- row/statement

R1 – No hi pot haver cap empleat que tingui un sou superior al sou del seu cap. En cas que aquesta regla no es compleixi cal que salti una excepció.

R2 – El pressupost d'un departament s'ha de mantenir igual a la suma dels sous dels empleats del departament.

R3 – Només es pot afegir empleats si la suma dels pressupostos de tots els departaments és superior a 100.000. En cas que aquesta regla no es compleixi cal que salti una excepció.

3.2 A les taules `empleats` i `assignacions` es vol que quan s'esborri un departament s'apliquin les polítiques següents:

- taula empleats

```
FOREIGN KEY (idD) REFERENCES departaments(idD)
ON DELETE SET NULL
```

- taula assignacions

```
FOREIGN KEY (idD) REFERENCES departaments(idD)
ON DELETE CASCADE
```

Suposem que en comptes d'indicar aquestes polítiques al crear les taules, es vol implementar aquestes polítiques en el procediment emmagatzemat `tancaDepartament` que serveix per esborrar un departament que es passa com a paràmetre. A més, el procediment ha de llançar l'excepció "Departament no existeix" si el departament passat com a paràmetre no està a la base de dades. Completa la implementació del procediment seguint els criteris de qualitat establerts a l'assignatura.

```
CREATE FUNCTION tancaDepartament(prof departaments.idD%type)
RETURNS void AS $$
...
END;
$$LANGUAGE plpgsql;
```

3.1

	insert	update	delete
RI1	taula empleats before/row	atribut cap taula empleats before/row atribut sou taula empleats before/row	-
RI2	taula empleats after/row	atribut sou o idD taula empleats after/row atribut pressupost (*) taula departaments after/row	delete empleats after/row
RI3	insert empleats before/statement	-	-

(*) Cal tenir en compte que en cas de update de pressupost a la taula departaments s'hauria de definir com propagar el canvi a la taula empleats.

3.2

```
create function tancaDepartament(dept departaments.id%type)
returns void as $$
begin
     update empleats set idD = null where idD = dept;
     delete from assignacions where idD = dept;
     delete from departaments where idD = dept;
     if not found then
          raise exception '%', 'Departament no existeix'; end if;
     return;
end;
$$language plpgsql;
```

4. (2 punts)

4.1 Disposeu de 3 operacions: R(A), RU(A) i W(A).

a) Doneu un horari amb 2 transaccions que facin servir les operacions anteriors que necessiteu (una operació pot aparèixer més d'una vegada si és necessari), i on hi hagi una única interferència d'ACTUALITZACIÓ PERDUDA.

b) Doneu la resta d'horaris possibles que presentin la mateixa interferència i que facin servir el mínim nombre d'operacions donades (igualment, una operació pot aparèixer més d'una vegada si és necessari).

c) Justifica que l'horari que has proposat en l'apartat a) no té un horari serial equivalent.

4.2 Suposeu ara l'horari següent:

T1	T2	T3	T4
		OPX	
	R(A)		
			RU(B)
			RU(C)
RU(A)			
		R (F)	
	R(B)		
	R(C)		
W(A)			
R(A)			
		OPY	
			W(C)
			W(B)
			commit
commit			
	commit		
		commit	

a) Sense tenir en compte OPX i OPY, doneu el graf de precedències corresponent a l'horari donat. En cas que l'horari tingui interferències, digues quines són (nom de la interferència, i transaccions i grànuls implicats).

b) Tingueu en compte que OPX i OPY poden ser operacions simples (R, RU, W) o complexes (RU+W). Doneu tots els parells de valors de OPX i OPY que generin una interferència entre T1 i T3. Indiqueu quina interferència es produeix per a cada parell donat? Justifiqueu breument perquè es produeix cada interferència.

c) Doneu l'horari amb les reserves i les esperes incorporades segons el nivell d'aïllament REPETEABLE READ. Per a aquest horari suposeu que OPX = R(A) i OPY = R(A).

4.1.a

T1	T2
RU(A)	
	RU(A)
W(A)	
	W(A)
commit	
	commit

4.1.b

T1	T2
	RU(A)
RU(A)	
W(A)	
	W(A)
commit	
	commit

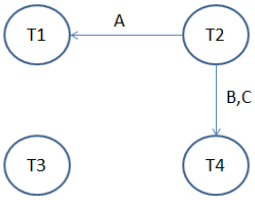
T1	T2
	RU(A)
RU(A)	
	W(A)
W(A)	
commit	
	commit

T1	T2
RU(A)	
	RU(A)
	W(A)
W(A)	
commit	
	commit

4.1.c

No hi ha cap horari serial equivalent perquè hi ha un cicle al graf de precedències que correspon a una interferència de actualització perduda.

4.2.a.



El graf de precedències no té cicles i per tant, podem assegurar que no hi ha cap interferència.

4.2.b.

- OPX = R(A), OPY = R(A), lectura no repetible
- OPX = R(A), OPY = RU(A)+W(A), lectura no repetible
- OPX = RU(A), OPY=W(A), actualització perduda
- OPX = RU(A)+W(A), OPY = R(A), lectura no repetible
- OPX = RU(A)+W(A), OPY = RU(A)+W(A), lectura no confirmada

4.2.c

T1	T2	T3	T4
		L(A,S)	
		R(A)	
	L(A,S)		
	R(A)		
			L(B,X)
			RU(B)
			L(C,X)
			RU(C)
L(A,X)			
		L(F,S)	
		R(F)	
	L(B,S)		
		R(A)	
			W(B)
			c+U(B,C)
	R(B)		
	L(C,S)		
	R(C)		
	c+U(A,B,C)		
		c+U(A,F)	
RU(A)			
W(A)			
R(A)			
c+U(A)			

5. **(2 punts)** Suposeu la taula `logs` següent, amb registres d'accés a un servei web:

IP	usuari	data	acció	durada
10.0.0.2	Joan	22/12/2018 11:30	accés	40
10.0.16.1	Carla	22/12/2018 11:40	compra	20
10.0.14.1	Jordi	22/12/2018 11:41	venda	22
10.0.1.2	Pere	23/12/2018 16:50	accés	10

10.0.0.4	Maria	23/12/2018 17:00	consulta	95
10.0.2.1	Carla	23/12/2018 17:14	venda	21
10.0.0.2	Pere	28/12/2018 09:00	compra	42
10.0.0.2	Pere	28/12/2018 09:01	compra	15
10.0.0.2	Pere	28/12/2018 09:02	compra	41
10.0.2.2	Joan	28/12/2018 20:00	consulta	32
10.0.3.2	Maria	30/12/2018 14:16	accés	33

5.1 Creeu un arbre B+ d'alçada 2 (h=2) i d'ordre 2 (d=2) sobre l'atribut `durada`, amb un 75% d'ocupació.

5.2 La taula logs ha crescut al llarg de l'any, i el servei web disposa ara de 300 usuaris únics, on cadascun d'ells ha fet en mitjana 50 accessos, 50 compres, 50 vendes, i 50 consultes. Les files de la taula es distribueixen uniformement entre 800 pàgines. Considereu que s'ha definit un índex agrupat multiatribut pels atributs `acció,usuari` d'ordre 100, ple al 75%. Per cadascuna de les consultes següents:

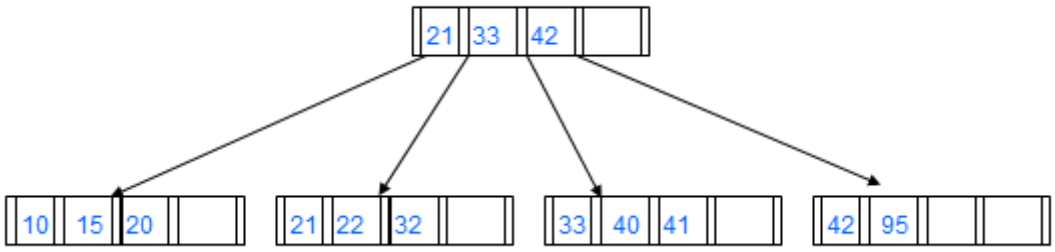
- Expliqueu com es resoldrà la consulta.
- Indiqueu el cost de la consulta en número de pàgines d'índex i número de pàgines de dades, mostrant els càlculs que feu.

a) SELECT * FROM log WHERE usuari = 'maria'

b) SELECT DISTINCT(usuari) FROM log WHERE accio = 'venda'

SOLUCIÓ

5.1



5.2.a

L'índex no afavoreix la consulta, pel que les dades s'obtindran amb un sequential access.

Cost accés índex = 0

Cost accés dades = 800 pàgines de dades

5.2.b

L'índex acció, usuari permet obtenir el resultat accedint únicament a l'índex:

n = d * 2 * ocupació = (100 * 2 * 75) / 100 = 150 valors/node

$h = \log_{151} 60000 = 2.1 \Rightarrow 3$ nivells

50 vendes * 300 usuaris = 15000 vendes

Cost = 3 + 15000 / 150 – 1 = 3 + 100 – 1 = 102 pàgines d'index.

Temps: 3 h

1. (3 punts) Considereu l'esquema de la base de dades següent:

```
create table producte
(idProducte char(9),
nom char(20),
mida char(20),
preu integer check(preu>0),
primary key (idProducte),
unique (nom,mida));

create table domicili
(numTelf char(9),
nomCarrer char(20),
numCarrer integer check(numCarrer>0),
pis char(2),
porta char(2),
ciutat char(20),
primary key (numTelf));

create table comanda
(numComanda integer check(numComanda>0),
instantFeta integer not null check(instantFeta>0),
instantServida integer check(instantServida>0),
numTelf char(9),
import integer,
primary key (numComanda),
foreign key (numTelf) references domicili,
check (instantServida>instantFeta));

create table producteComprat
(numComanda integer,
idProducte char(9),
quantitat integer check(quantitat>0),
primary key(idProducte, numComanda),
foreign key (idProducte) references producte ,
foreign key (numComanda) references comanda);
// Hi ha una fila a la taula per cada producte comprat
// en una comanda.
```

1.1. Doneu una sentència SQL per obtenir els productes comprats que no s'han demanat en comandes servides després de l'instant 333.

```
select pc.idproducte
from comanda c natural inner join producteComprat pc
group by pc.idproducte
having max(c.instantServida) <=333

select distinct pc.idProducte
from productescomprats pc
where not exists (select * from productescomprats pc2, comandes c
        where pc2. idProducte = pc.idProducte and
            pc2.numComanda = c.numComanda    and
            c.instantServida >333)
```

1.2. Escriu una asserció en SQL que asseguri que l'import de les comandes sigui igual a la suma de l'import de cadascun dels productes comprats en la comanda. Cal tenir en compte que l'import d'un producte comprat en una comanda és el resultat de multiplicar el preu del producte per la quantitat del producte que s'ha comprat en la comanda.

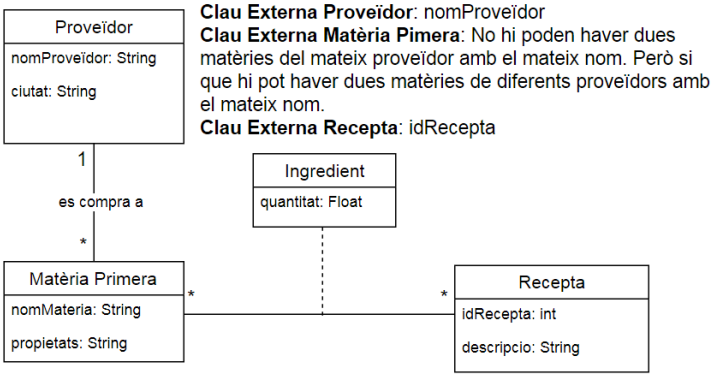
```
create assertion control_import check
(not exists (select *
            from comanda c
            where c.import != (select sum(p.preu*pc.quantitat)
                               from producteComprat pc natural inner join producte p
                               where pc.numComanda = c.numComanda))

create assertion control_import check
(not exists (select *
            from comanda c natural inner join
                        producteComprat pc natural inner join producte p
            group by c.numComanda
            having c.import != sum(p.preu*pc.quantitat))
```

1.3. Escriu una seqüència d'operacions d'àlgebra relacional per obtenir les comandes en les que s'ha comprat un únic producte.

```
A= productecomprat[numComanda, idProducte]
B = A{numComanda -> nc, idProducte ->idp}
C = A[numComanda = nc, idProducte != idp]B
D= C[numComanda] /* pedidos con más de un producto */
E = A[numComanda] /* todos los pedidos */
F = E – D              /* pedidos con 1 solo producto */
```

1.4. Suposem el diagrama UML següent. Feu la traducció de UML a model relacional, indicant taules, atributs, claus primàries, claus foranes i restriccions de UNIQUE i NOT NULL que es deriven del diagrama.



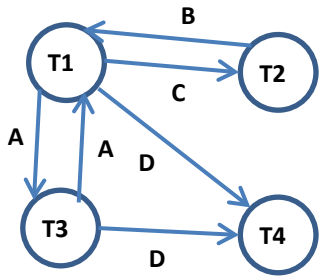

```
proveidors(nomProveidor, ciutat)
materiesPrimeres(nomProveidor, nomMateria, propietats)
        {nomProveidor} referencia proveidors
receptes(idRecepta, descripcio)
ingredients(nomProveidor, nomMateria, idRecepta, quantitat)
        {nomProveidor,nomMateria} referencia materiesPrimeres
        {idRecepta} referencia receptes
```

2. (2.5 punts)

2.1. Donat un SGBD sense cap mecanisme de control de concurrència, suposem que es produeix l'horari següent (les accions s'han numerat per facilitar la seva referència):

#Acc	T1	T2	T3	T4
10		RU(B)		
20		W(B)		
30			R(D)	
40	RUA)			
50	R(D)			
60	RU(B)			
70	W(B)			
80			RU(E)	
90			W(E)	
100			RU(A)	
110			W(A)	
120			RU(F)	
130				RU(D)
140				W(D)
145	W(A)			
150			W(F)	
160	RU(C)			
170	W(C)			
180		RU(C)		
190		W(C)		
200				COMMIT
210		COMMIT		
220			COMMIT	
230	COMMIT			

3.1-a) Dibuixeu el graf de precedències associat a l'horari.



3.1-b) Es produeixen interferències? En cas de resposta negativa, argumenteu breument la vostra resposta. En cas de resposta positiva digueu quina/es interferències es produeixen (cal donar el nom de la interferència, gràNul i transaccions implicades).

Es produeix una actualització perduda entre T1 i T3 sobre el gràNul A.
Anàlisi Inconsistent entre T1,T2

3.1-c) Quins horaris serials hi són equivalents? Justificar la resposta.

No existeix cap horari serial equivalent, atès que hi ha interferències.

3.1-d) És recuperable aquest horari? Justificar la resposta.

No, ja que per exemple T2 llegeix C i confirma abans que T1 que també ha llegit i escrit C

2.2. Suposem que s'incorporen tècniques de reserva. Les transaccions estan definides com SET TRANSACTION ISOLATION LEVEL REPEATABLE READ.

- Donar l'horari resultant d'aplicar les reserves per aquest nivell d'aïllament. El nou horari ha d'incloure, a més de les operacions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves (LOCK, UNLOCK), i l'ordre d'execució de totes aquestes operacions.
- En cas que en algun instant de l'horari no es pugui executar cap operació, no continueu i justifiqueu el motiu.
- Si hi ha alguna interferència en l'horari resultat digues quina/es és/són i entre quines transaccions es produeix/en. Altrament, digues quin és l'horari serial equivalent.

#Acc	T1	T2	T3	T4
10		L(B,X)		
20		RU(B)		
30		W(B)		
40			L(D,S)	
50			R(D)	
60	L(A,X)			
70	RU(A)			
80	L(D,S)			
90	R(D)			
100	L(B,X)			
110			L(E,X)	
120			RU (E)	
130			W(E)	
140			L(A,X)	
170				L(D,X)
250		L(C,X)		
260		RU(C)		
270		W(C)		
280		c+u		
290	RU(B)			
300	W(B)			
310	W(A)			
320	L(C,X)			
330	RU(C)			
340	W(C)			
350	c+u			
360			RU(A)	
370			W(A)	
380			L(F,X)	
390			RU(F)	
400			W(F)	
410			c+u	
420				RU(D)
430				W(D)
440				c+u

Horari serial equivalent: t2;t1;t3;t4

3. **(2.5 punts)** Suposeu la base de dades de l'exercici 1. Suposeu que hi ha definit el procediment emmagatzemat `nouProducteComprat` que s'invoca per cadascun dels productes comprats en una comanda, passant com a paràmetre el número de comanda, el producte comprat i la quantitat del producte comprat en la comanda. En la mateixa base de dades hi ha definit el disparador `disparadorBDComandes`.

```
10    create or replace function nouProducteComprat(numCom integer, numProd
11    char(9), qtt int)
12    returns void language plpgsql as $$
13    begin
14    if((not exists(select * from comanda
15                where numComanda = numCom) or
16      (not exists(select * from producte
17                where idproducte = numProd)))) then
18                raise exception 'La comanda o el producte no existeixen';
19    elseif(exists(select * from producteComprat
20                where idproducte = numProd and numComanda = numCom)) then
21                raise exception 'El producte comprat ja es a la comanda';
22    else insert into productecomprat values(numCom,numProd,qtt);
23    end if;
24    return;
25    exception
26          when raise_exception then raise exception '%',SQLERRM;
27          when others then raise exception 'Error inern';
28    end; $$

29    create or replace function procDisparadorBDComandes()
30    returns trigger language plpgsql as $$
31    declare varNumComanda integer;
32           varImport integer;
33    begin
34    for varNumComanda in select numComanda from comanda
35    loop
36          varImport := (select sum(p.preu*pc.quantitat)
37                               from producte p, productecomprat pc
38                               where p.idproducte = pc.idProducte and
39                                     pc.numcomanda = varNumComanda);
40          update comanda
41          set import = varImport
42          where numComanda = varNumComanda;
43    end loop;
44    return null;
45    end; $$

46    create trigger disparadorBDComandes
47    after insert on producteComprat
48    for each row execute procedure procDisparadorBDComandes();
```

3.1. Partint del contingut de la base de dades que s'indica a continuació. Digueu quin és el contingut de les taules de la base de dades després de l'execució de cadascuna de les seqüències de sentències següents:

a)
```
select * from nouProducteComprat(333,'300',1);
select * from nouProducteComprat(333,'200',3);
```

b)
```
select * from nouProducteComprat(222,'100',1);
select * from nouProducteComprat(222,'300',1);
```

taula comanda

numComanda	instantFeta	instantServida	numTelf	import
111	1	10	123	8
222	11	20	123	12
333	21	30	123	0

taula producte

idProducte	nom	mida	preu
100	dodotus	gran	5
200	kloonex	petit	3
300	tollolets	normal	6

taula domicili

numTelf	nomCarrer	numCarrer	pis	porta	ciutat
123	Pont de Baix	24	1	2	Manlleu

taula producteComprat

numComanda	idProducte	quantitat
111	100	1
111	200	1
222	300	2

SOLUCIÓ

3.1.a) Les taules que canvien són:

taula comanda

numComanda	instantFeta	instantServida	numTelf	import
111	1	10	123	8
222	11	20	123	12
333	21	30	123	15

taula producteComprats

numComanda	idProducte	quantitat
111	100	1
111	200	1
222	300	2
333	300	1
333	200	3

3.1.b) Les taules que canvien són les següents. La segona sentència fa saltar una excepció perquè intenta afegir un producte comprat que ja està a la taula.

taula comanda

numComanda	instantFeta	instantServida	numTelf	import
111	1	10	123	8
222	11	20	123	17
333	21	30	123	0

taula producteComprats

numComanda	idProducte	quantitat
111	100	1
111	200	1
222	300	2
222	100	1

Es considera correcta una solució on diguin que l'estat de la base de dades és el mateix que l'inicial en cas que les dues sentències de l'apartat b formen una transacció.

3.2. Trobeu les sentències del procediment emmagatzemat que no compleixen els criteris de qualitat treballats a l'assignatura. Per cada sentència/es que no compleixin els criteris de qualitat:

- Indiqueu el número/s de la/es sentències que no compleixen el criteri
- Indiqueu quin és el criteri de qualitat que no compleixen
- Justifiqueu perquè no el compleixen
- Doneu una nova versió del procediment emmagatzemat `nouProducteComprat` que sí que compleixi el criteri

14 a 18: selects innecessaris perquè l'excepció es pot capturar mitjançant la violació de la condició de foreign key

19 a 21: select innecessari perquè l'excepció es pot capturar mitjançant la violació de la restricció de unique/primary key

```
create function nouProducteComprat(numCom integer, numProd char(9), qtt int)
returns void language plpgsql as $$
begin   insert into producteComprat values(numCom, numProd, qtt);
return;
exception when foreign_key_violation then
            raise exception 'La comanda o el producte no existeixen';
          when unique_violation then
            raise exception 'El producte comprat ja es a la comanda';
          when others then raise exception 'Error intern';
end; $$
```

3.3. El tipus del disparador que s'ha usat en la implementació que us donem és el millor segons els criteris de qualitat treballats a l'assignatura, però la implementació del procediment `procDisparadorBDComandes` que activa el disparador no és la millor perquè és una implementació NO incremental. Doneu una implementació alternativa del procediment que faci el mateix però que sigui incremental.

```
create or replace function incrementPreuComanda()
returns trigger language plpgsql as $$
    declare pr integer;
    begin
        pr := (select preu from producte
               where idProducte = NEW.idProducte);
        update comanda
        set import = import + NEW.quantitat*pr
        where numComanda = NEW.numComanda;
    return null;
end; $$
```

4. (2 punts) Suposem la taula `comanda` de l'exercici 1.

`comanda(numComanda, instantFeta, instantServida, numTelf, import)`

Hi ha 5.000.000 files a la taula `comanda` distribuïdes uniformement en 50.000 pàgines de dades. Les pàgines tenen una mida de 4096 bytes. S'ha decidit crear dos índexs, implementats com un arbre B+, sobre la taula.

- Un índex agrupat definit sobre l'atribut `numComanda`. L'atribut `numComanda` ocupa 6 bytes. Els apuntadors a nodes de l'índex ocupen 3 bytes. Els apuntadors a files de la taula ocupen 4 bytes.
- Un índex no agrupat definit sobre els atributs `numTelf`, `instantFeta`. L'ordre d de l'índex és 146. L'arbre està ple al 75%. L'arbre té 3 nivells. Les comandes de la base de dades s'han fet des de 20.000 telèfons diferents, i cada telèfon ha fet una mitjana de 250 comandes.

Mostra en detall com has fet els càlculs que es demanen a continuació:

4.1. La consulta següent no es veu afavorida per usar cap dels dos índexs definits. Calcula el cost (número de pàgines accedides) de resoldre-la.

- select * from comanda where import = 50

4.2. Índex per numComanda.

4.1-a) Calcula l'ordre d òptim de l'índex

4.1-b) Suposant l'índex té una ocupació del 80%, i suposant l'ordre que has trobat en l'apartat anterior:

- Calcula quants valors hi haurà a cada node de l'arbre.
- Calcula quantes pàgines ocuparan els nodes fulla de l'arbre.
- Calcula quants nivells tindrà l'arbre B+.

4.1-c) Suposant les dades calculades en els apartats anteriors, calcula el cost (número de pàgines accedides) de resoldre la consulta següent usant l'índex

- select * from comanda where numComanda = 55555

4.3. Índex per numTelf, instantFeta. Calcula el cost (número de pàgines accedides) de resoldre la consulta següent usant aquest índex.

- select * from comanda where numTelf = '123'

SOLUCIÓ:

4.1) com que no usem índex, s'ha de fer accés seqüencial a totes les pàgines de dades per tant el cost és 50.000

4.2.a)

4096 >= 6*2d+(3*(2d+1)) => d<= 227

4096 >= 6*2d + 4*2d + 2*3 => d<= 204

d òptima 204

4.2.b) n = 2*204*0,8 = 327

num pàgines = num nodes fulla

5000000 valors / 327 valors/nodesfulla = 15291 pàgines

$\log_{328}500000 = 2,66$ => 3 nivells

4.2.c) Com que numComanda és PK, només busquem una fila

h + 1 = 3 + 1

4.3) num de valors per node = 2*146 * 0,75 = 219

busquem en mitjana 250 comandes

h + F + D(numTelf = '123') = 3 + (250/219 -1) + 250 = 254

Temps: 3 h
Notes 21 gener tarda **Revisió: 21 gener tarda**
Cada pregunta en un full separat

1) (2.5 punts) Considereu l'esquema de la base de dades següent:

```
CREATE TABLE EquipsFutbol(
nomEquip char(30),
localitat char(50),
nomEstadi char(100) UNIQUE NOT NULL,
totalSalaris real not null check(totalSalaris>=0),
PRIMARY KEY(nomEquip));

CREATE TABLE Partits(
nomEquipLocal char(30),
dataPartit date,
nomEquipVisitant char(30),
golsEquipL integer NOT NULL CHECK(golsEquipL >=0),
golsEquipV integer NOT NULL CHECK(golsEquipV >=0),
PRIMARY KEY (nomEquipLocal,dataPartit,nomEquipVisitant),
FOREIGN KEY (nomEquipLocal) REFERENCES EquipsFutbol,
FOREIGN KEY (nomEquipVisitant) REFERENCES EquipsFutbol,
CHECK(nomEquipLocal <> nomEquipVisitant));

CREATE TABLE Jugadors(
dniJugador char(9),
nom char(50) NOT NULL,
nomEquip char(30) NOT NULL,
salari real not null check(salari>=0),
PRIMARY KEY (dniJugador),
FOREIGN KEY (nomEquip) REFERENCES EquipsFutbol);
```

- nomEquip és l'equip on juga el jugador

```
CREATE TABLE Alineacions(
nomEquipLocal char(30),
dataPartit date,
nomEquipVisitant char(30),
dniJugador char(9),
gols integer NOT NULL CHECK(gols>=0),
numTargetes integer NOT NULL CHECK(numTargetes>=0),
PRIMARY KEY (nomEquipLocal,dataPartit,
               nomEquipVisitant, dniJugador),
FOREIGN KEY (nomEquipLocal,dataPartit,nomEquipVisitant)
                          REFERENCES Partits,
FOREIGN KEY (dniJugador) REFERENCES Jugadors);
```

- el jugador dniJugador ha estat alineat al partit idenficat per nomEquipLocal, dataPartit, nomEquipVisitant
- els gols i targetes són les que han posat al jugador durant el partit.

1.1) Escriviu una sentència SQL per obtenir els equips de futbol que han jugat, com a equip local, en partits contra més de 3 equips diferents i que, a la vegada, en aquests equips locals, no hi juga cap jugador que tingui targetes. En el resultat de la consulta hi ha d'haver el nom dels equips locals i la quantitat de partits que han jugat a casa amb equips diferents.

```
Select p.nomequiplocal, count(distinct p.nomequipvisitant)
From partits p
Where not exists (select *
                  From jugadors j, alineacions a
                  Where j.nomEquip=p.nomEquipLocal and
                        j.dniJugador=a.dniJugador and
                        a.numtargetes>0)
Group by p.nomequiplocal
Having count(distinct p.nomequipvisitant) >3;
```

1.2) Vistes.

a. Definiu una vista amb els partits empatats (és a dir, els que tant l'equip local com el visitant han fet el mateix número de gols). La vista ha de incloure tots els atributs de la taula partits i la clausula WITH CHECK OPTION.

```
CREATE VIEW partitsEmpatats AS
      SELECT *
      FROM Partits p
      WHERE p.golsEquipL=p.golsEquipV
      WITH CHECK OPTION;
```

b. Doneu una sentència update de la vista, que inclogui la modificació de l'atribut golsEquipL, i que NO violi la restricció WITH CHECK OPTION.

```
update partitsEmpatats
set golsEquipL = golsEquipL+2,
    golsEquipV = golsEquipLV+2
```

c. Doneu una sentència update de la vista, que inclogui la modificació de l'atribut golsEquipL, i que SI que violi la restricció WITH CHECK OPTION.

```
update partitsEmpatats
set golsEquipL = golsEquipL+2
```

1.3) La cardinalitat de les taules de la base de dades és respectivament EF, P, J, A (amb EF,P,J,A > 0). Digueu quina serà la cardinalitat de la relació R per cadascuna de les seqüències d'operacions d'àlgebra relacional següents. En cas de no poder dir exactament quina serà dóna una cardinalitat mínima i una màxima. Justifiqueu la resposta.

```
a. R = Partits * Alineacions

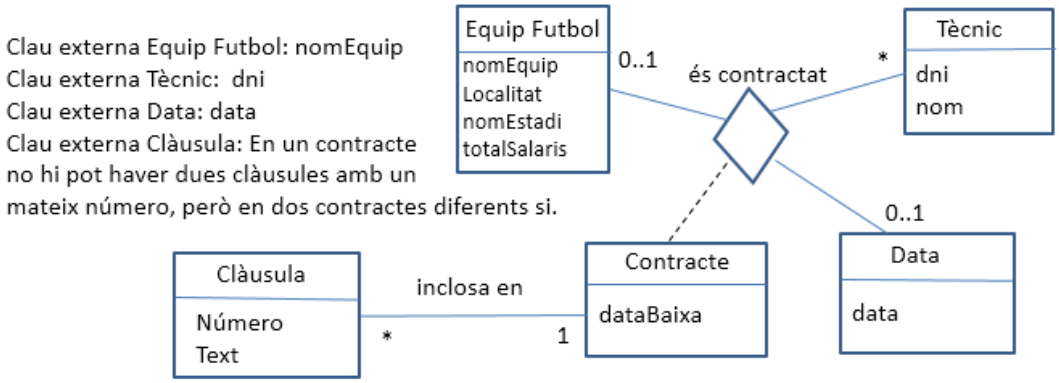
b. R= Partits x Alineacions

c. A= Partits[golsEquipPL]
   B= Partits[golsEquipPV]
   R= A U B

(iv) A = Alineacions[nomEquipLocal,dataPartit,nomEquipVisitant]
     B = Partits[nomEquipLocal,dataPartit,nomEquipVisitant]
     R = A ∩ B
```

i. card(R)= A
ii. card(R)= P x A
iii. card(R)màxima = 2P, card(R) mínima = 1
iv. card(R)màxima P, card(R) mínima = 1

1.4) Suposeu el model conceptual en UML següent. Doneu el disseny lògic que s'obté fent la traducció a model relacional. Cal incloure, taules, atributs, claus primàries, claus foranies i restriccions NOT NULL i UNIQUE que siguin necessàries.




```
equipsFutbol(nomEquip, localitat, nomEstadi, totalSalaris)
tècnics(dni, nom)
contractes(dni, dataAlta, nomEquip, dataBaixa)
      {dni} referencia tècnics
      {nomEquip} referencia equipsFutbol NOT NULL
      UNIQUE(nomEquip, dataAlta)
       --és també correcte posar com a PK nomEquip, dataAlta
       --i posar com a UNIQUE dni, dataAlta
clàusules(dni, dataAlta, número, text)
      {dni, dataAlta} referencia contractes
```

2) **(2.5 punts)** Suposeu la base de dades de l'exercici 1. Suposeu que a la base de dades s'afegeixen les taules següents. S'inclou a continuació algunes de les files que contenen les taules.

```
t_files(nomTaula, quantesFiles)
            -- conté per cada taula quantes files hi ha a la taula,
            -- en cas que no n'hi hagi cap contindrà un 0
    EquipsFutbol 22
    Jugadors    420

t_sentencies(nomSentencia, nomTaula, quantesSentencies)
            -- conté per cada taula i per cada tipus de sentència, quantes vegades s'ha
                  executat una sentència del tipus sobre la taula
            --en cas que no n'hi hagi cap, contindrà un 0
        insert          EquipsFutbol      2
        insert          Jugadors         10
        insert          Partits           6
        delete          Partits           1
        insert          Alineacions      80
        update          Alineacions      10
```

2.1) Considereu les restriccions següents:

a. Es mantingui correcte el valor de la columna *quantesFiles* de fila *Jugadors* de la taula *t_files*.
b. Es mantingui correcte el valor de la columna *quantesSentencies* de les files on apareix *Jugadors* a la taula *t_sentencies*.
c. Es mantingui correcte la columna *totalSalaris* de la taula *EquipsFutbol*, que ha de correspondre a la suma dels salaris dels jugadors de l'equip.

Indiqueu quins són els triggers que cal definir sobre la taula *Jugadors*. Per cada trigger cal indicar:

- Esdeveniment (Insert, Delete, Update – en cas d'update indicar atribut/s rellevants),
- Before/After
- Row/Statement

insert	after	statement
delete	after	statement
update	after	statement
insert	after	Row
delete	after	Row
update of nomEquip, salaris	after	Row

2.2) Explicar breument el què caldrà que facin els procediments a definir per al/s trigger/s *delete*. En cas de que no hagin estat necessaris triggers *delete*, explicar perquè no fan falta.

- After, statement
  - Incrementa en 1 la columna quantesSentencies de la a fila 'delete', 'jugadors' de la taula t_sentencies
- After, row
  - Decrementa en 1 la columna quantesFiles de la fila 'jugadors' de la taula t_files
  - Decrementa l'atribut salarisTotals de la taula EquipsFutbol, de l'equip OLD.nomEquip amb el valor de OLD.salari

2.3) Implementar els procediments necessaris per al/s trigger/s *update*. En cas de que no hagin estat necessaris triggers *update*, explicar perquè no fan falta.

```
CREATE FUNCTION upJugadorsStatement() RETURNS trigger AS $$
BEGIN
     UPDATE t_sentencies
     SET quantesSentencies = quantesSentencies + 1
     WHERE nomSentencia='update' and
         nomTaula='jugadors';
     RETURN NULL;
END; $$ LANGUAGE plpgsql;

CREATE FUNCTION upJugadorsRow() RETURNS trigger AS $$
BEGIN
     UPDATE equipsFutbol
     SET totalSalaris = totalSalaris + NEW.salari
     WHERE nomEquip=NEW.nomEquip;
     UPDATE equipsFutbol
     SET totalSalaris = totalSalaris -OLD.salari
     WHERE nomEquip=OLD.nomEquip;
     RETURN NEW;
END; $$ LANGUAGE plpgsql;
```

3) **(2.5 punts)** S'ha creat una taula empleats(numEmpl, nom, sou, ciutat, categoria). La taula empleats té 100.000 tuples i els valors de numEmpl estan repartits uniformement entre 1 i 100.000. El factor de bloqueig del fitxer de dades és de 20 files per pàgina. Es defineixen dos índexs sobre la taula empleats:

- un índex B+ agrupat per l'atribut `numEmpl`, amb ordre d=60 i amb nodes plens al 80% de la seva capacitat.
- un índex B+ no agrupat per l'atribut `sou`, amb ordre d=80 i amb nodes plens al 75% de la seva capacitat.

3.1) Expliqueu quina diferència hi ha entre un índex agrupat i un índex no agrupat.

3.2) Digueu quants índexs agrupats hi pot haver definits com a màxim sobre una taula. Justifiqueu la resposta.

3.3) Es vol obtenir totes les files que compleixen les condicions `numEmpl>95.000 AND sou>2000`. Sabent que hi ha 400 files que compleixen sou>2000, i hi ha 100 files que compleixen les dues condicions. Mostreu com calculeu el cost de la consulta en cadascun dels casos següents.

a. Emprant recorregut seqüencial de la taula *empleats*.
b. Emprant l'índex B+ agrupat per *numEmpl*.
c. Emprant l'índex B+ no agrupat per *sou*.

3.4) Digueu si hi ha algun índex que es pogués afegir a la taula empleats i que millorés l'eficiència de la consulta. Justifiqueu la resposta en funció dels costos.

3.1. Veure transparències assignatura
3.2. Veure transparències assignatura
3.3.

a. Recorregut seqüencial = 100.000 files / 20 files-pàgina = 5000 pàgines
b. índex numEmpl
   núm valors per node = 2d*ocupació = 2*60*0,8 = 96 valors
   núm apuntadors per node = 96+1 = 97 apuntadors
   cost = h + D
   h = alçada de l'arbre = $\log_{97}100.000 = 2.51 \Rightarrow 3$
   D = número de pàgines de dades =
   5000 files compleixen la condició / 20 files-pàgina = 250
   cost = 3 + 250 = 253 pàgines
c. índex sou
   núm valors per node = 2d*ocupació = 2*80*0,75 = 120 valors
   núm apuntadors per node = 120+1 = 121 apuntadors
   cost = h + (F -1) + D
   h = alçada de l'arbre = $\log_{121}100.000 = 2.4 \Rightarrow 3$
   F = número de pàgines, nodes fulla = 400 valors sou>2000 / 120 valors =
   4 pàgines, nodes fulla
   D = número de pàgines de dades = 400 pàgines en el cas pitjor, una per fila.
   cost = 3 + (4 - 1) + 400 = 406 pàgines

3.4.

Un índex no agrupat, multiatribut, concretament pels atributs sou, numEmpl.
Hauria de ser no agrupat, perquè ja n'hi ha un d'agrupat.
El cost al ser no agrupat hauria de ser h+F+D

- h=3
  - Tindrà entre 3 o 4 nivells segons el tamany del valor.
  - Com que el tamany del valor serà més gran que en els altres dos arbres, la d serà més petita (no hi cabran tants valors per node). I per tant, podria ser que l'arbre passés a ser de 4 nivells.
  - Considerarem 3 nivells.
- F = 400 / n
  - hi ha 400 valors que cal considerar, ja que es fa accés a les fulles de l'arbre a partir del valor "2.000, 95.000", i cal buscar valors superiors a 2.000 i superiors també a 95.000. Cal entendre que els 100 valors que compleixen les dues condicions no estaran seguits. Només cal pensar en el valor "2.100, 85.000". Aquest valor estarà entre els 400 però no serà un dels 100 que ens interessen. I després d'aquest si que trobarem valors que tornen a complir les condicions com el valor "2.100, 96.000".
  - Per altra banda el valor n el desconeixem. Desconeixem la d, encara que sabem que serà inferior a la dels altres arbres, i desconeixem com estarà de ple el arbre. Posem-nos en un cas dolent de d=30 i 60% d'ocupació. Aleshores n = 30*2*0,7 = 42 valors/node.
- D = 100 en el cas pitjor.

El cost seria 3 + (10 -1) + 100 = 112, considerablement millor que els altres dos índexos.

4) (2.5 punts) Donades les dues taules següents:

```
Clients(numClient, nom, instantNaixement, població, saldoTotal)
Comptes(numCompte, titular, vigent, saldo)
    {titular} referencia Clients
```

El contingut en un cert instant de les taules és:

numClient	nom	instantNaixement	poblacio	saldoTotal
1	Anna Puig	422	Barcelona	28.900
2	Rosa Carrasco	145	Barcelona	33.250

numCompte	titular	vigent	saldo
11	1	N	28.000
21	1	S	-1.500
31	1	S	2.400
42	2	S	13.250
52	2	N	20.000

4.1) Suposeu que el grànul és la fila, que no existeix cap mecanisme per al control de la concurrència i que es produeix l'horari següent (les operacions s'han numerat per facilitar la seva referència).

#op	T1	T2	T3
1	`select saldoTotal from clients where numClient =1`		
2		`update clients set saldoTotal= saldoTotal * 1.1 where numClient = 1`	
3			`select saldoTotal from clients where numClient = 1`
4	`select max(saldoTotal) from clients`		
5			`select * from comptes where titular = 2`
6		`update comptes set titular = 2 where titular = 1 and vigent = 'N'`	
7			`select saldo from comptes where titular =2`
8	`insert into clients values(3, 'Marta López', 323, 'Barcelona', 50.000)`		
9			`commit`
10		`Commit`	
11	`commit`		

a. Digueu si existeixen interferències en l'horari. Si la resposta és afirmativa, doneu el nom de la/es interferència/es, grànul i transaccions implicades.
b. Digueu quins horaris serials donen resultats equivalents a l'horari proposat.
c. Indiqueu, en cas que hi hagi interferències, quin és el nivell mínim d'aïllament necessari per evitar cadascuna d'elles.
d. Digueu si l'horari proposat és recuperable. Justifiqueu breument la resposta.

a. Sí, hi ha interferències.
- Entre T1 i T2 una lectura no repetible a T1 sobre la taula clients, granul 1. L'update de T2 entre els 2 select de T1 canvia el contingut de la taula.
- Entre T2 i T3 hi ha un fantasma que produeix T2 canviant el titular d'una fila de la taula comptes. Quan T3 fa el segon select sobre aquesta taula, el contingut ha canviat i apareix una nova fila. Granul, el fantasma és el compte 11 i la informació de control IC.

b. Com que hi ha interferències, és impossible que hi hagi un horari serial que doni un resultat equivalent.
c. Per evitar la lectura no repetible entre T1 i T2 necessitem nivell REPETEABLE READ. Per evitar el fantasma SERIALIZABLE.
d. No, no és recuperable. Per exemple, T3 fa lectures sobre clients i comptes i fa el commit abans que T2 que és la que escriu sobre clients i comptes. Si T2, en comptes de fer un commit, fes un rollback, T3 hauria confirmat la lectura d'uns continguts que no són veritat.

4.2) Suposeu que el gràNul és la fila, que tenim un mecanisme per al control de la concurrència basat en reserves S, X i que la transacció T1 treballa a nivell d'aïllament de READ COMMITTED i T2 i T3 a nivell SERIALITZABLE.

a. Doneu l'horari que ha d'incloure, a més de les operacions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves (LOCK, UNLOCK), i l'ordre d'execució de totes aquestes operacions. Quan més d'una transacció espera per un mateix gràNul, considereu que la política de la cua és FIFO (First In, First Out).
b. Indiqueu els horaris serials equivalents, en cas que n'hi hagi. Si no n'hi ha, justifiqueu la resposta.

T1	T2	T3
L(client,1,S)		
R(client,1)		
U(client,1)		
	L(client,1,X)	
	RU(client,1)	
	W(client,1)	
		L(client,1,S)
L(client,1,S)		
	L(compte,11,X)	
	RU(compte,11)	
	W(compte,11)	
	L(IC,X)	
	RU(IC)	
	W(IC)	
	C+U((client,1), (compte,11))	
		R(client,1)
R(client,1)		
U(client,1)		
L(client,2,S)		
R(client,2)		
U(client,2)		
		L(IC,S)
		R(IC)
		L(comptes,42,S)
		R(comptes,42)
		L(comptes,52,S)
		R(comptes,52)
		L(comptes,11,S)
		R(comptes,11)
		R(comptes,42)
		R(comptes,52)
		R(comptes,11)
L(client,3,X)		
RU(client,3)		
W(client,3)		
		C+U((client,1), (comptes,42), (comptes,52), (comptes,11))
C+U(client,3)		

b. No hi ha horaris serials equivalents perquè hi continua havent la interferència de lectura no repetible

BD-GEI - Examen parcial **5.11.2020**

Temps: 90 minuts

Notes 23 novembre

Cada pregunta s'ha de lliurar en un full separat

Suposeu una base de dades amb les taules següents:

```
CREATE TABLE EquipsFutbol(
nomEquip char(30),
localitat char(50),
nomEstadi char(100) UNIQUE NOT NULL,
pressupost real NOT NULL check(pressupost>=0),
PRIMARY KEY(nomEquip));

CREATE TABLE Partits(
idPartit integer,
nomEquipLocal char(30) NOT NULL,
dataPartit date NOT NULL,
nomEquipVisitant char(30) NOT NULL,
golsEquipL integer NOT NULL CHECK(golsEquipL >=0),
golsEquipV integer NOT NULL CHECK(golsEquipV >=0),
PRIMARY KEY (idPartit),
FOREIGN KEY (nomEquipLocal) REFERENCES EquipsFutbol,
FOREIGN KEY (nomEquipVisitant) REFERENCES EquipsFutbol,
UNIQUE(nomEquipLocal,nomEquipVisitant));

CREATE TABLE Jugadors(
dniJugador char(9),
nom char(50) NOT NULL,
nomEquip char(30) NOT NULL,
salari real NOT NULL check(salari>=0),
PRIMARY KEY (dniJugador),
FOREIGN KEY (nomEquip) REFERENCES EquipsFutbol);
```

- nomEquip és l'equip **on està contractat** el jugador

```
CREATE TABLE Alineacions(
idPartit integer,
dniJugador char(9),
gols integer NOT NULL CHECK(gols>=0),
numTargetes integer NOT NULL CHECK(numTargetes>=0),
PRIMARY KEY (idPartit, dniJugador),
FOREIGN KEY (idPartit) REFERENCES Partits,
FOREIGN KEY (dniJugador) REFERENCES Jugadors);
```

- el jugador dniJugador **ha estat alineat** al partit idenficat per idPartit
- els gols i targetes són les que han posat al jugador durant el partit.

1. **(1 punt)** Per cadascuna de les restriccions d'integritat següents.

- En cas que es pugui implementar com a restricció d'integritat de columna o de taula, en una sentència CREATE: Digueu la taula on s'haurien de definir, i la restricció d'integritat de columna o de taula en SQL.
- Si no es pot: Expliqueu breument perquè no es pot.

1.1 Un equip de futbol no pot jugar un partit contra ell mateix.

1.2 El número de gols d'un equip local en un partit ha de ser igual a la suma dels gols dels seus jugadors que han sigut alineats al partit.

1.3 Un equip de futbol no pot jugar dos partits en una mateixa data.

- RI1 – Si que es pot
  - A la taula partits
  - CHECK (nomEquipLocal<>nomEquipVisitant)
- RI2 – No es pot implementar
  - Perquè és una restricció que té a veure amb files de més d'una taula.
- RI3 – Es pot implementar parcialment. Amb els dos UNIQUE següents s'assegura que un equip de futbol no pot jugar com a local o com a visitant dos partits a la mateixa data. Però si que podria jugar un partit com a local i un com a visitant en una mateixa data.
  - A la taula partits
  - UNIQUE(nomEquipLocal,dataPartit), UNIQUE(nomEquipVisitant,dataPartit)

2. **(3 punts)** Doneu una sentència SQL per obtenir els equips de futbol (nomEquip) que no han jugat mai com a equip visitant en un partit empatat. A més es vol que un equip només surti si el salari total dels jugadors contractats a l'equip és més gran que la mitjana del pressupost de tots els equips.

```
select e.nomEquip
from equipsFutbol e natural inner join jugadors j
where not exists (select *
            from partits p
            where e.nomEquip = p.nomEquipVisitant and
                  p.golsEquipL=p.golsEquipV)
group by e.nomEquip
having sum(j.salari) > (select avg(ef.pressupost)
                  from equipsfutbol ef);
```

3. **(1.5 punt)** Doneu una seqüencia d'operacions en àlgebra relacional per obtenir els jugadors alineats en l'equip local que ha jugat el partit amb idPartit 333. Per cada jugador cal que surti el DNIJugador, el nom del jugador i el seu nom d'equip.

```
Una possible solució:
 A = partits(idPartit=333)
 B = A*alineacions
 C = B{DNIJugador -> DNIJugadorA}
 D = C[nomEquipLocal=nomEquip,DNIJugadorA=DNIJugador]jugadors
 R = D[DNIJugador, nom, nomEquip]
```

4. **(1 punt)** Doneu una sentència assertion per assegurar que en un partit no s'ha alineat més de 30 jugadors.

```
CREATE ASSERTION numeroJugadorsPartit CHECK
    (NOT EXISTS (SELECT p.idPartit
                 FROM alineacions a
                 GROUP BY p.idPartit
                 HAVING COUNT(*)>30));
```

5. **(1 punt)** Suposeu la vista següent:

```
CREATE VIEW massaCar (nomEquip, pressupost, costJugadors)AS
    SELECT e.nomEquip, e.pressupost, sum(j.salari)
    FROM EquipsFutbol e NATURAL INNER JOIN Jugadors j
    WHERE e.localitat in ('Barcelona','Madrid')
    GROUP BY e.nomEquip;
```

5.1 Doneu un contingut de les taules de la base de dades que tingui com a mínim 3 files a la taula jugadors i que faci que quan es consulti la vista el resultat sigui el següent:

massaCar	nomEquip	pressupost	costJugadors
	Barça	50M	70M

equipsFutbol	nomEquip	localitat	estadi
	Barça	BCN	CN

jugadors	DNIJugador	nom	nomEquip	salari
	111	Ansu	Barça	10M
	222	Messi	Barça	40M
	333	Piqué	Barça	20M

5.2 Justifiqueu si la vista massaCar és o no actualitzable.

```
No, per exemple perquè hi ha join o sum.
```

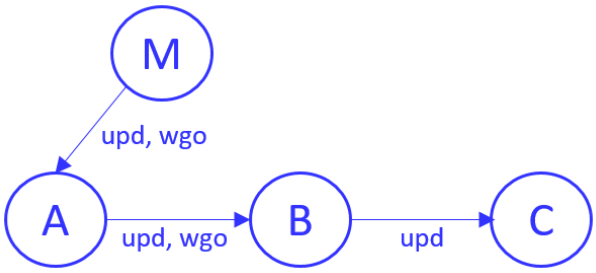
6. **(1 punt)** Suposeu que el propietari de les taules de la base de dades és M.

6.1 Doneu el diagrama d'autoritzacions, després de que s'executin les sentències següents:

M: GRANT update ON partits TO A WITH GRANT OPTION;

A: GRANT update ON partits TO B WITH GRANT OPTION;

B: GRANT update ON partits TO C;



6.2 Suposeu la situació donada pel diagrama d'autoritzacions anterior. Doneu una sentència UPDATE de la taula partits per la qual l'usuari A no tingui suficients permisos.

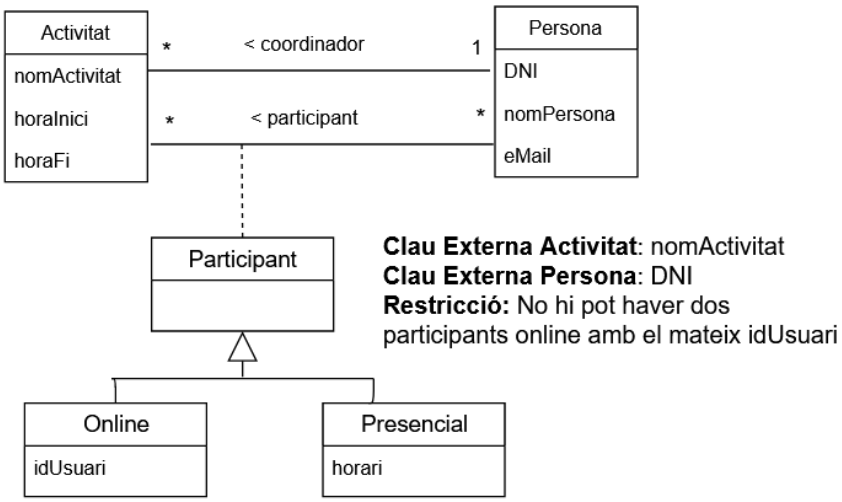
Qualsevol sentència UPDATE que per modificar necessiti consultar algun atribut de la taula partits o de qualsevol altre taula.

6.3 Suposeu la situació donada pel diagrama d'autoritzacions anterior. Doneu el nou diagrama d'autoritzacions un cop executada la sentència següent. Justifiqueu breument la resposta.

M: REVOKE GRANT OPTION FOR update FROM partits TO A RESTRICT;

El diagrama d'autoritzacions no canvia perquè la sentència no es pot executar. El motiu és que en ser RESTRICT no es pot executar perquè no només estaria perdent privilegis A, sinó també B i C. Ja que B i C no estan rebent els privilegis per cap altre camí.

7. **(1.5 punts)** Suposeu el model conceptual en UML següent. Doneu el disseny lògic que s'obté fent la traducció a model relacional. Cal incloure, taules, atributs, claus primàries, claus foranies i restriccions NOT NULL i UNIQUE que siguin necessàries.



persones(DNI, nomPersona, email)
activitats(NomActivitat, horaInici, horaFi, DNICoordinador)
{DNICoordinador} referencia persones
NOT NULL DNICoordinador
participants(DNI, nomActivitat)
{DNI} referencia persones
{nomActivitat} referencia activitats
participantsOnline(DNI, nomActivitat, idUsuari)
{DNI, nomActivitat} referencia participants
UNIQUE(idUsuari)
participantsPresencials(DNI, nomActivitat, horari)
{DNI, nomActivitat} referencia participants

BD-GEI - Examen final **15.1.2021**

Temps: 2 hores

Notes 27 gener Revisió: 28 gener **Cada pregunta s'ha de lliurar en un full separat**

1. **(1 punt)** Donades les tres taules següents:

```
professors(dni, nomProf, telefon, sou)
despatxos(modul, numero, superficie)
assignacions(dni, modul, numero, instantInici, instantFi)
      {dni} referencia professors
      {modul,numero} referencia despatxos
```

Considereu el següent fragment de codi Java que implementa, utilitzant JDBC, l'assignació de tots els professors que tenen un sou superior al valor de la variable sou al despatx identificat per les variables modul i numero, amb instancInici = 100 i instanFi = null. En cas que algun dels profesors ja estigui assignat al despatx s'acaba el programa mostrant un missatge d'error.

```
try {
 /* Tenim ja una connexió c establerta amb la base de dades */
 int sou = 1200;
 String modul = "omega";
 String numero = "118";

 Statement st1 = c.createStatement();
 ResultSet rs1 = st1.executeQuery("select dni from professors where sou < " + sou);


 while (rs1.next()) {
   String dniProf = rs1.getString(1);

   // Consultem si el professor ja ha estat assignat
   Statement st2 = c.createStatement();
   ResultSet rs2 = st2.executeQuery("select * from assignacions"
                    + " where dni='" + dniProf + "'"
                    + " and modul='" + modul + "'"
                    + " and numero='" + numero + "'");

   // Si rs2 retorna resultat, aleshores el professor ja està assignat
   if (rs2.next()) {
         System.out.println("Error: Un dels professors ja està assignat");
         c.rollback();
         break;    /* surt del bucle  */
   }
   // En cas contrari, procedim a crear l'assignació
   else {
         Statement st3 = c.createStatement();
         st3.executeUpdate("insert into assignacions values ("
                          + "'" + dniProf + "',"
                          + "'" + modul + "',"
                          + "'" + numero + "',"
                          + "100, null)");
         st3.close();
    }
    rs2.close(); st2.close();
    }
 st1.close(); rs1.close();c.commit(); c.close();
 }
catch (ClassNotFoundException ce) {
         System.out.println ("Error al carregar el driver");}
catch (SQLException se) {
         System.out.println ("Excepcio: "+ se.getMessage());}
```

Donat aquest fragment de codi, identifiqueu quins criteris de qualitat no es compleixen. Per cadascun d'ells, expliqueu perquè no es compleix, i expliqueu amb detall quins canvis serien necessaris en la codificació per tal de satisfer aquests criteris.

SOLUCIÓ:

No es compleixen els següents criteris de qualitat:

- 1 - Execució de sentència SQL innecessària. La comprovació feta per st2 sobre l'existència de l'assignació és innecessària, ja que aquesta comprovació es pot fer simplement mitjançant l'execució de l'operació insert, delegant la responsabilitat al propi gestor de bases de dades, i capturant/gestionant la SQLException com correspongui.
- 2 - Utilització inadequada de Statement. Tant st2 com st3 utilitzen Statement, però aquestes operacions s'executen per a cada valor dniProf obtingut a la consulta st1. Per tant, el més adequat seria utilitzar PreparedStatement establint com a variable de les operacions el valor dniProf. En el cas de st2 , i en relació al criteri de qualitat 1, aquest criteri es solucionaria també simplement suprimint aquesta comprovació.
- 3 – Execució de sentència SQL innecessària. Es podria també implementar el programa amb una única sentència SQL feta com a un Statement que faci un insert amb una subconsulta que determina els professors que s'ha d'inserir. En cas que algun dels professors ja està assignat, es genera el missatge quan es captura l'excepció afegida segons el criteri de qualitat 1.

2. **(2 punts)** Considereu una base de dades amb les taules següents:

```
cantants (nom, pais, anyRetir)
albums (titolAlbum, nomCantant, anyPublicacio, preu)
         {nomCantant} referencia cantants
critiques (titolAlbum, nomCantant, font, puntuacio)
         {titolAlbum, nomCantant} referencia albums
estadístiques (nomCantant, millorPuntuació)
         {nomCantant} referencia cantants
```

Indiqueu quins són els triggers necessaris (tenint en compte els criteris de qualitat establerts a l'assignatura) a definir per tal que:

- **RI1.** Es generi una excepció en cas que un esdeveniment faci que no es compleixi la següent restricció: Un cantant no pot publicar nous àlbums en els anys posteriors a l'any de retir.

- **RI2.** Es mantingui el valor de l'atribut *millorPuntuació*. Aquest atribut ha de tenir com a valor, la puntuació més alta que ha rebut un cantant en els àlbums que ha publicat.
- **RI3.** Es generi una excepció en cas que un esdeveniment faci que no es compleixi la següent restricció: No es poden afegir crítiques si la base de dades conté menys de 10 cantants.

Per cada trigger cal indicar:

- Taula per a la que es defineix
- Esdeveniment que l'activa (INSERT, DELETE, UPDATE - en cas d'UPDATE indicar atribut/s rellevants)
- BEFORE/AFTER (justificar la resposta)
- ROW/STATEMENT (justificar la resposta)

SOLUCIÓ

	Trigger RI1	Trigger RI1	Trigger RI2	Trigger RI3
Taula	cantants	albums	critiques	critiques
Esdeveniments	UPDATE any_retir	INSERT UPDATE anyPublicacio	INSERT, DELETE UPDATE puntuacio	INSERT
before/after	BEFORE (1)	BEFORE (1)	AFTER (3)	BEFORE (1)
row/statement	ROW (2)	ROW (2)	ROW (4)	STATEMENT (5)

(1) Si fossin AFTER, en cas de violar-se la restricció caldria desfer l'esdeveniment sobre la taula cantants/albums/critiques
(2) Al ser ROW poden implementar-se de manera incremental, fent les comprovacions de la restricció només per a les files inserides/modificades
(3) Si fossin BEFORE, en cas que els esdeveniments no s'acabessin executant caldria desfer els canvis de l'atribut *millorPuntuació* fets.
(4) Al ser ROW pot implementar-se de manera incremental, només aplicant canvis a l'atribut millorPuntuació dels cantants als que se'ls ha fet canvis en les crítiques.
(5) La restricció no té a veure amb quines files s'insereixen ni amb quantes files s'insereixen, per tant amb un STATEMENT n'hi ha prou.

3. **(2 punts)** Suposeu la base de dades següent, que s'ha creat en un SGBD sense cap mecanisme de control de concurrència. Considereu que el gránul és la fila.

```
autors(   codiAutor,  nom,        telefon,   poblacio)
          ca1         Autor1      111        Pobl1
          ca2         Autor2      222        Pobl2

obres(    idObra,     codiAutor,  titol,     anyEdicio)
          ob1         ca1         Bon dia    2020
          ob2         ca2         Hola       2020
          ob3         ca1         Bona nit   2020
```

Suposeu també les sentències SQL següents:

s1	UPDATE obres SET codiAutor = 'ca1' WHERE idObra= 'ob2';
s2	DELETE FROM obres WHERE idObra='ob1';
s3	INSERT INTO obres VALUES('ob4','ca1','Adeu',2020);
s4	SELECT * FROM obres WHERE codiAutor='ca1';
sX és un codi que us pot permetre fer referència a les sentències	

3.1 Doneu un horari on intervinguin dues transaccions, que poden executar una o més de les sentències SQL donades, i que presenti una interferència d'ACTULITZACIÓ PERDUDA. Una sentència es pot usar únicament una vegada en l'horari, i s'ha d'incloure a l'horari el mínim número de sentències. En cas que no sigui possible formar un horari amb els requisits indicats, justifiqueu perquè no.

3.2 Doneu un horari on intervinguin dues transaccions, que poden executar una o més de les sentències SQL donades, i que presenti una interferència de FANTASMA. S'ha d'incloure a l'horari el mínim número de sentències, però una sentència es pot usar més d'una vegada. En cas que no sigui possible formar un horari amb els requisits indicats, justifiqueu perquè no.

SOLUCIO

3.1) Si volem actualització perduda, les dues transaccions han de fer una actualització (RU,W) sobre el mateix gränul. Les tres operacions d'actualització proposades treballen sobre gränuls diferents així que NO és possible construir un horari amb aquesta interferència.

3.2) Si volem que es produeixi una interferència de tipus FANTASMA, una de les dues transaccions ha de 'canviar' el contingut de la BD. Per exemple, una transacció ha de fer l'UPDATE o l'INSERT (amb una instrucció n'hi ha prou, hem de fer servir el mínim nombre d'instruccions) i l'altra transacció ha de fer dues lectures semblants però forçant que donin resultats diferents. Per tant, ficarem l'UPDATE/INSERT en mig de les dues lectures:

T1	T2
SELECT * FROM obres WHERE codiAutor='ca1';	
	INSERT INTO obres values('ob4','ca1','Adeu',2020,'g1'); -- o bé UPDATE obres SET codiAutor = 'ca1' WHERE idObra= 'ob2';
SELECT * FROM obres WHERE codiAutor='ca1';	
	commit
commit	

4. **(2 punts)** Considereu l'horari següent:

	T1	T2
1	R(A)	
2		RU(A)
3		W(A)
4		RU(B)
5		W(B)
6	RU(B)	
7	W(B)	
8	COMMIT	
9		COMMIT

Suposeu que s'incorporen tècniques de control de concurrència amb reserves.

4.1 En cas que les transaccions treballin en nivell d'aïllament READ UNCOMMITTED.

a) Doneu l'horari aplicant reserves corresponents al nivell d'aïllament (*)

b) Doneu el graf de precedències de l'horari resultant d'aplicar reserves

c) En cas que hi hagi alguna interferència en l'horari resultant digueu quina/es són, i el/s gränul/s implicat/s.

4.2 En cas que les transaccions treballin en nivell d'aïllament REPETEABLE READ.

a) Doneu l'horari aplicant reserves corresponents al nivell d'aïllament (*)

b) Doneu el graf de precedències de l'horari resultant d'aplicar reserves

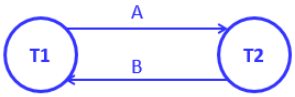
c) En cas que hi hagi alguna interferència en l'horari resultant digueu quina/es són, i el/s gränul/s implicat/s.

(*) En els horaris heu d'incloure, a més de les operacions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves (LOCK, UNLOCK), i l'ordre d'execució de totes aquestes operacions. Quan més d'una transacció espera per un mateix gränul, considereu que la política de la cua és FIFO (First In, First Out).

SOLUCIÓ

4.1)

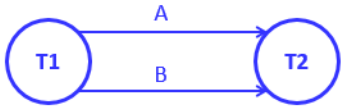
	T1	T2
1	R(A)	
2		LOCK(A,X)
3		RU(A)
4		W(A)
5		LOCK(B,X)
6		RU(B)
7		W(B)
8	LOCK(B,X)	
9	...	COMMIT+UNLOCKS (A,B)
10	RU(B)	
11	W(B)	
12	COMMIT+UNLOCK (B)	



Hi ha una interferència d'anàlisi inconsistent amb gränuls A,B

4.2)

	T1	T2
1	LOCK(A,S)	
2	R(A)	
3		LOCK(A,X)
4	LOCK(B,X)	...
5	RU(B)	...
6	W(B)	...
7	COMMIT+UNLOCKS (A,B)	...
8		RU(A)
9		W(A)
10		LOCK(B,X)
11		RU(B)
12		W(B)
13		COMMIT+UNLOCKS (A,B)



No hi ha cap interferència

5. **(2 punts)** Suposeu la taula *albums*:

```
albums (titolAlbum, nomCantant, anyPublicacio, preu)
```

Aquesta taula té 2.000.000 files, i el factor de bloqueig, de les pàgines de dades on s'emmagatzemen aquestes files, és de 40 files per pàgina.

Sabem que el cantant `'Juli Catedrals'` ha fet 500 àlbums. També sabem que la condició `titolAlbum > 'xxx'` la compleixen 8.000 àlbums que han fet uns 1.000 cantants diferents.

Considereu la consulta següent.

```
SELECT *
FROM albums
WHERE nomcantant = 'Juli Catedrals'
     AND titolAlbum > 'xxx';
```

Doneu el cost en número de pàgines d'índex i número de pàgines de dades a les que cal accedir per resoldre la consulta:

5.1 Usant un índex agrupat definit sobre l'atribut nomCantant, amb ordre d=24, ple al 80%.

5.2 Usant un índex no agrupat definit sobre l'atribut titolAlbum, amb ordre d=16, ple al 70%

SOLUCIÓ:

5.1) Número pàgines d'índex + Número pàgines de dades = 4 + 13 = 17 pàgines

Número de pàgines d'índex = nivells de l'arbre = $\lceil \log_{40} 2.000.000 \rceil = 4$

número de valors per node = $\lceil 2d*0,8 \rceil = \lceil 2*24*0,8 \rceil = 39$ valors / node

número d'apuntadors per node = 39 + 1 = 40 apuntadors / node

Número de pàgines de dades = $\lceil$ (albums(nomcantant = 'Juli Catedrals') / factor de bloqueig $\rceil$

$= \lceil 500 / 40 \rceil = 13$

albums(nomcantant = 'Juli Catedrals') = 500

factor de bloqueig = 40

5.2) Número pàgines d'índex + Número pàgines de dades = 352 + 8.000 = 8.352 pàgines

Número de pàgines d'índex = nivells de l'arbre

$+ \lceil$ (albums(titolAlbum > 'xxx') / num valors/node $\rceil$ -1

$= \lceil \log_{24} 2.000.000 \rceil + \lceil 8.000 / 23 \rceil - 1 = 5 + 347 = 352$ pàgines

número de valors per node = $\lceil 2d*0,7 \rceil = \lceil 2*16*0,7 \rceil = 23$ valors / node

número d'apuntadors per node = 23 + 1 = 24 apuntadors / node

Número de pàgines de dades = albums(titolAlbum > 'xxx') = 8.000

6. **(1 punt)** Considereu un índex arbre B+ amb els següents paràmetres: el valor pel que es crea l'índex té V bytes, la mida dels nodes és B bytes, els RID tenen T bytes, i els apuntadors a nodes de l'índex tenen A bytes.

6.1 Sigui nf la quantitat màxima de valors que un node del nivell fulla pot contenir. Expresseu nf en funció dels paràmetres V, B, T, A

6.2 Suposa ara que 2d = 100.

a) Quin és el número màxim de files que el arbre B+ pot apuntar si té 3 nivells?

b) Quants nodes es necessiten per emmagatzemar l'índex en aquest cas?

SOLUCIÓ:

6.1) nf = $\lfloor (B-2A) / (V + T) \rfloor$

6.2.a) files = 101*101*100 = 1,020,100

6.2.b) nodes = 1+ 101 + 101*101 =10303

BD-GEI **19.4.2021**

Temps: 90 minuts

Notes 6 maig **Cada pregunta s'ha de lliurar en un full separat**

Suposeu una base de dades amb les taules següents:

```
create table departaments(
codiDept integer,
nomDept char(50) unique not null,
pressupost integer not null check(pressupost>=0),
primary key (codiDept));

create table professors(
idProf integer,
nomProf char(50) not null,
codiDept integer not null,
sou real,
primary key (idProf),
foreign key (codiDept) references departaments);
-- codiDept és del departament en el que treballa el professor

create table assignatures(
idAssig char(5),
nomAssig char(50) not null,
codiDept integer not null,
profResponsable integer not null,
primary key (idAssig),
foreign key (codiDept) references departaments,
foreign key (profResponsable) references professors);
-- codiDept és del departament al que està assignada l'assignatura
-- profResponsable és el professor responsable de l'assignatura

create table assignacions(
idProf integer,
idAssig char(5),
quadrimestre char(6),
horesAssig integer not null check (horesAssig>0),
primary key (idProf, idAssig, quadrimestre),
foreign key (idProf) references professors,
foreign key (idAssig) references assignatures);
-- hi ha una fila si el professor idProf ha donat o dóna
-- classes de l'assignatura idAssig en el quadrimestre
```

1. (1.75 punts) Considereu la base de dades donada a l'inici de l'examen. Doneu una sentència SQL per obtenir les assignatures (nomAssig) per a les que no hi ha cap assignació d'un professor de més de 3 hores (horesAssig) en el quadrimestre '2020-2'.

```
SELECT DISTINCT a.nomAssig
FROM assignatures a
WHERE NOT EXISTS (SELECT *
                  FROM assignacions ass
                  WHERE ass.idAssig = a.idAssig AND
                        ass.quadrimestre = '2020-2' AND
                        ass.horesAssig > 3);
```

2. (1.75 punt) Considereu la base de dades donada a l'inici de l'examen. Doneu una seqüència d'operacions d'àlgebra relacional per obtenir l'identificador i el nom dels professors que han estat o estan assignats amb menys de 5 hores de docència (horesAssig) a assignatures d'un departament que no és el seu.

```
A = assignacions(hores<5)
B = A * professors
C = assignatures{codiDept -> cd, idAssig → ia}
D = B[idAssig = ia, codiDept <> cd]C
E = D[idProf,nomProf]
```

3. (3 punts) Considereu la base de dades donada a l'inici de l'examen.

3.1 Per cadascuna de les restriccions següents:

- Si es pot implementar com a restricció en la definició de les taules, doneu el codi SQL que cal afegir als CREATE TABLE de la BD per a implementar-la, i en quina taula s'ha d'afegir.
- Si no es pot, explicar per què.

a) Un professor no pot ser responsable de més d'una assignatura.

Afegim un UNIQUE(profResponsable) a la taula assignatures.

b) No s'ha de poder executar sentències UPDATE de l'atribut pressupost de la taula departaments.

No es pot implementar com a restricció de taula, caldria fer un grant

c) Un professor ha de tenir sou positiu, a no ser que estigui de baixa, cosa que s'indicarà quan l'atribut sou tingui valor NULL.

Afegim la condició CHECK (sou > 0 OR sou IS NULL) a la taula professors

3.2 Donar una sentència de creació d'una asserció per tal que a la base de dades es compleixi que: "Tots els professors assignats a una assignatura en el mateix quadrimestre, han de ser del mateix departament."

```
CREATE ASSERTION totsMateixDept
   CHECK (NOT EXISTS (SELECT a.idAssig, a.quadrimestre
           FROM assignacions a NATURAL JOIN professors p
           GROUP BY a.idAssig, a.quadrimestre
           HAVING COUNT (DISTINCT p.codiDept)>1))
```

```
CREATE ASSERTION totsMateixDept
   CHECK (NOT EXISTS (SELECT *
           FROM assignacions a1 NATURAL JOIN professors p1,
             assignacions a2 NATURAL JOIN professors p2
           WHERE a1.idAssig = a2.idAssig AND
                 a1.quadrimestre = a2.quadrimestre AND
                 p1.codiDept <> p2.codiDept))
```

4. (2 punts)

4.1 Considereu la vista següent definida sobre la base de dades donada a l'inici de l'examen.

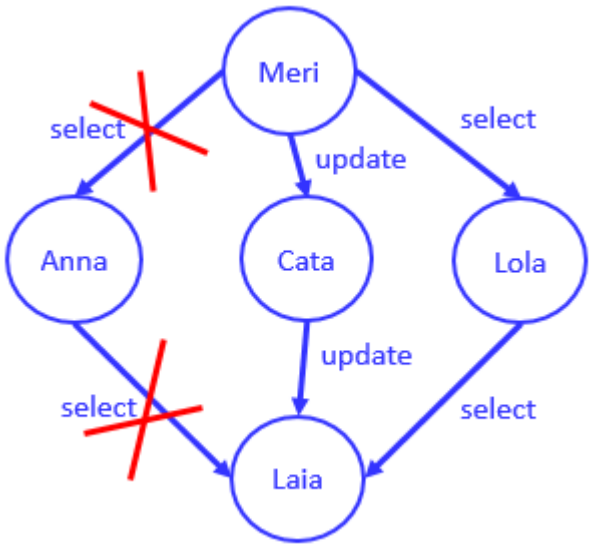
- En cas que la vista sigui actualitzable, doneu una sentència INSERT/DELETE/UPDATE que doni una excepció per al check de la vista.
- En cas que la vista no sigui actualitzable, expliqueu el motiu pel qual no ho és.

```
create view professorsDept5 as
   select p.idProf, p.nomProf
   from professors p
   where p.codiDept = 5
   with check option;
```

La vista no és actualitzable perquè, encara que està definida sobre una única taula, no inclou tots els atributs NOT NULL de la taula que no tenen valor per defecte. En concret, perquè no inclou l'atribut codiDept.

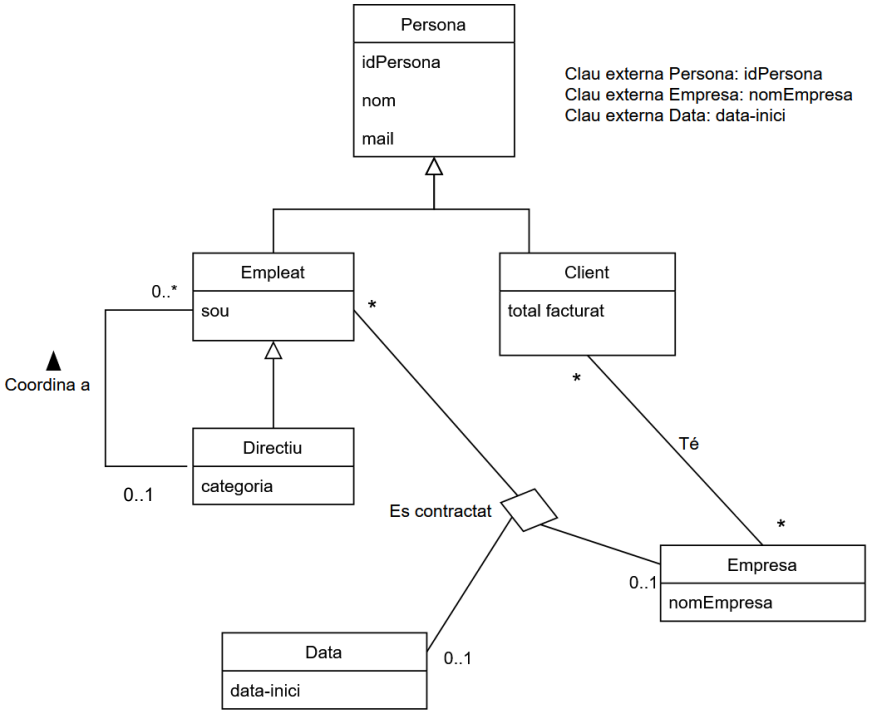
4.2 Suposem que les taules de la base de dades són propietat de la Meri. Considereu les sentències GRANT i REVOKE següents. Doneu els diagrames d'autoritzacions corresponents a l'instant abans d'executar les sentències REVOKE, i a l'instant després d'executar les sentències REVOKE. Justifica el diagrama d'autoritzacions de després dels REVOKE.

```
Meri: GRANT select ON professors TO Anna WITH GRANT OPTION;
Meri: GRANT update ON professors TO Cata WITH GRANT OPTION;
Meri: GRANT select ON professors TO Lola WITH GRANT OPTION;
Anna: GRANT select ON professors TO Laia;
Cata: GRANT update ON professors TO Laia;
Lola: GRANT select ON professors TO Laia;
Meri: REVOKE select ON professors FROM Anna RESTRICT;
Meri: REVOKE update ON professors FROM Cata RESTRICT;
```



El REVOKE de la Meri a l'Anna s'executa, perquè ningú més, a part de l'Anna perd cap privilegi. En canvi, el REVOKE de la Meri a la Cata no s'executa amb èxit, perquè a part de la Cata, la Laia perdria el privilegi.

5. (1.5 punts) Suposeu el model conceptual en UML següent. Doneu el disseny lògic que s'obté fent la traducció a model relacional. Cal incloure, taules, atributs, claus primàries, claus foranes i restriccions NOT NULL i UNIQUE que siguin necessàries.



Persona(idPersona,nom,mail)

Empleat(idPersona, sou, directiu)

{idPersona} referencia Persona

{directiu} referencia Directiu

Directiu(idPersona,categoria)

{idPersona} referencia Empleat

Client(idPersona, totalfacturat)

{idPersona} referencia Persona

Empresa(nomEmpresa)

Data(data-inici)

Te(idPersona,nomEmpresa)

{idPersona} referencia Client

{nomEmpresa} referencia Empresa

EsContratctat(idPersona,nomEmpresa,data-inici)

[data-inici} referencia Data NOT NULL

UNIQUE (idPersona,data-inici)

{idPersona} referencia Empleat

{nomEmpresa} referencia Empresa

Temps: 2,5 hores

Notes 22 juny Revisió: 28 juny **Cada pregunta s'ha de lliurar en un full separat**

1. **(1 punt)** Donades les base de dades següent:

```
create table professors
(dni char(50),
nomProf char(50) unique,
telefon char(15),
primary key (dni));

create table despatxos
(modul char(5),
numero char(5),
superficie integer not null check(superficie <=25),
primary key (modul,numero));

create table assignacions
(dni char(50),
modul char(5),
numero char(5),
instantInici integer,
instantFi integer,
primary key (dni, modul, numero, instantInici),
foreign key (dni) references professors,
foreign key (modul,numero) references despatxos,
check (instantInici < instantFi));
```

Considereu el següent fragment de codi Java/JDBC. Implementeu el cos del **while** per tal que s'incrementi en 5 unitats la superfície dels despatxos de cadascun dels mòduls obtinguts en la variable varModul. Definiu també els Statements o PreparedStatements necessaris on creieu convenient. Cal que el programa tregui un missatge d'excepció si la superfície d'algun despatx passa a ser superior a 25. En cas que no hi hagi cap excepció el programa indicarà el número de despatxos modificats de cada mòdul.

```
try {
 /* Tenim ja una connexió c establerta amb la base de dades */
 /* Tenim un variable in que permet llegir el que escriu l'usuari */

 System.out.print("Escriu el nom d'un mòdul: ");
 String varModul = in.nextLine();

 while (!varModul.equals("acabar")) {

     //codi a implementar

     System.out.print("Escriu el nom d'un mòdul: ");
     varModul = in.nextLine();
     }
 c.commit();
 }
catch (ClassNotFoundException ce) {
        System.out.println ("Error al carregar el driver");}
catch (SQLException se) {
        System.out.println ("Excepcio: "+ se.getMessage());}
```

Recordeu que podeu usar els mètodes/codis d'excepció de JDBC estudiats a classe:

Statements

- Statement createStatement();
- ResultSet executeQuery(String sql);
- int executeUpdate(String sql);

PreparedStatement

- PreparedStatement prepareStatement(String sql);
- ResultSet executeQuery();
- int executeUpdate();
- void setXXX(int posicioParametre, XXX valor);

ResultSet

- boolean next();
- XXX getXXX(String nomColumna)

SQLException

- // 23502 -not_null_violation, 23503 -foreign_key_violation
- // 23505 -unique_violation, 23514 -check_violation
- String getSQLState();

SOLUCIÓ

```
try {
    /* Tenim ja una connexió c establerta amb la base de dades */

    System.out.println("Escriu el nom d'un mòdul: ");
    String varModul = System.console().readLine();

    PreparedStatement ps = c.prepareStatement("update despatxos d "+
                                    "set superficie = superficie + 5 "+
                                    "where d.modul = ?");


    while (!varModul.equals("acabar")) {

        //codi a implementar
        ps.setString(1,varModul);
        int numFilesActualitzades = ps.executeUpdate();
        System.out.println (varModul+"  "+numFilesActualitzades);
        System.out.println("Escriu el nom d'un mòdul: ");
        varModul = System.console().readLine();
        }
    c.commit();
    }
   catch (ClassNotFoundException ce) {
             System.out.println ("Error al carregar el driver");}
   catch (SQLException se) {
            if(se.getSQLState().equals("23514"))
                  System.out.println("La superficie no pot ser més gran que 25");
            else
                  System.out.println ("Excepcio: "+ se.getMessage());}
```

2. (1 punt) Suposeu la base de dades següent:

```
empleats1 (numEmp1, nomEmp1, ciutatEmp1);
empleats2 (numEmp2, nomEmp2, ciutatEmp2);
```

Suposeu que es vol implementar la restricció següent mitjançant disparadors:

Els valors de l'atribut `ciutatEmp1` de la taula `empleats1` han d'estar inclosos en els valors de l'atribut `ciutatEmp2` de la taula `empleats2`

La idea és llançar una excepció en cas que s'intenti executar una sentència que violi la restricció.

2.1 Digueu quins són els esdeveniments rellevants (taula/es i esdeveniment/s).

empleats1	Insert	
empleats1	Update	ciutatEmp1
empleats2	Delete	
empleats2	Update	ciutatEmpl2

2.2 Digueu i justifiqueu de quin tipus han de ser el disparadors (ROW / STATEMENT, BEFORE/AFTER).

Tots els disparadors han de ser before, for each row.

Before perquè com que es tracta de que salti una excepció quan es violi una restricció d'integritat, com més aviat es descarti que la restricció es viola millor, per tal d'evitar fer feina innecessària.

For each row perquè és millor una solució incremental. Suposant que a cada taula hi ha una gran quantitat d'empleats, i que els esdeveniments afecten a poques files, una solució incremental sempre comportarà menys accessos a la base de dades, perquè només requereix fer comprovacions per a les tuples que s'han inserit/esborrat/modificat.

2.3 Implementeu un dels disparadors que hagueu identificat per a la taula empleats1. En cas de que no n'hi hagi cap justifiqueu perquè. Podeu fer servir la plantilla següent.

```
CREATE FUNCTION comprovarCiutat() RETURNS trigger AS $$
    BEGIN
     //implementació
    END;
$$LANGUAGE plpgsql;

CREATE TRIGGER ex3_3
    BEFORE/AFTER <esdeveniment>
    FOR EACH ROW/STATEMENT EXECUTE PROCEDURE comprovarCiutat();
```

```
CREATE FUNCTION comprovarCiutEmpl1() RETURNS trigger AS $$
    BEGIN
     IF  not exists(select* from empleats2
         where  ciutatEmp2=new. ciutatEmp1) THEN
         RAISE EXCEPTION 'Error en inserir o modificar empleats1';
     END IF;
     RETURN NEW;
    END;
$$LANGUAGE plpgsql;

CREATE TRIGGER ex3_3
    BEFORE INSERT OR UPDATE of ciutatEmp1 ON empleats1
    FOR EACH ROW EXECUTE PROCEDURE comprovarCiutEmpl1();
```

3. **(2 punt)** Considereu la base de dades de l'exercici 1 amb el contingut següent:

Professors	dni	nomProf	telefon	sou
	111	Joana	97743121	10000
	222	Martí	93818903	3000
	333	Lourdes	97261201	50000

Despatxos	modul	numero	superficie
	C6	101	13
	Omega	300	8
	Omega	301	8

Assignacions	dni	modul	numero	instantIni	instantFi
	333	C6	101	5	9
	111	C6	101	3	NULL
	222	Omega	300	10	NULL

Considereu també el procediment emmagatzemat següent:

```
CREATE TYPE despatx AS (d_modul char(5), d_numero char(5));

CREATE FUNCTION assignaProfessor(dni_p char(50), modul_d char(5),
                                 numero_d char(5), inici_a integer)
            RETURNS SETOF despatx AS $$
DECLARE desp despatx;
BEGIN
  IF (NOT EXISTS(SELECT * FROM Professors WHERE dni=dni_p)
       OR NOT EXISTS(SELECT * FROM Despatxos
                     WHERE modul=modul_d AND numero=numero_d)) THEN
       RAISE EXCEPTION 'El professor o el despatx no existeix';
     ELSIF (EXISTS (SELECT * FROM Assignacions a
                    WHERE a.dni=dni_p AND a.instantFi IS NULL)) THEN
          UPDATE Assignacions SET instantFi = inici_a - 1
          WHERE dni=dni_p AND instantFi IS NULL;
   END IF;
   INSERT INTO Assignacions VALUES (dni_p,modul_d,numero_d,inici_a,NULL);
   FOR desp IN SELECT d.modul,d.numero
                  FROM Professors p NATURAL JOIN Assignacions a
                                    NATURAL JOIN Despatxos d
               WHERE a.instantFi IS NULL
               GROUP BY d.modul, d.numero
               HAVING COUNT(*) >= 2
    LOOP
      RETURN NEXT desp;
    END LOOP;
   EXCEPTION
       WHEN raise_exception THEN raise exception '%',sqlerrm;
 END;
 $$LANGUAGE plpgsql;
```

3.1 Indiqueu i justifiqueu quin és l'estat de la base de dades i el valor de retorn després de l'execució de la consulta: `SELECT * FROM assignaProfessor('111','Omega','300',15);`

Les taules Professors i Despatxos no es modifiquen, el contingut de la taula Assignacions és:

Dni	modul	numero	instantIni	instantFi
333	C6	101	5	9
111	C6	101	3	14
222	Omega	300	10	NULL
111	Omega	300	15	NULL

El procediment ha finalitzat totes les assignacions actives del professor amb dni_p (posant com a instant fi el d'inici menys 1), i n'ha creat una de nova per al despatx donat com a paràmetre. L'execució de la consulta retorna:

d_modul	d_numero
Omega	300

els quals corresponen als despatxos compartits (més d'un professor assignat) amb assignacions actives (instantFi amb valor NULL).

3.2 Digueu quins són els criteris de qualitat que no es compleixen en la implementació del procediment donat i com caldria canviar el codi per tal que es complíssin.

- Accessos innecessari a la BD.
  - El primer IF amb els seus SELECTs no cal. Per tant els SELECTs són innecessaris perquè l'excepció es pot capturar mitjançant la violació de la condició de foreign key que saltarà en fer el INSERT de l'assignació nova.
  - El ELEIF amb els seu SELECT no calen. El UPDATE ja només modifica les assignacions amb instantFi NULL si és que n'hi ha alguna.
- TAULES+JOINs innecessàries
  - En el SELECT que serveix per buscar el resultat que ha de donar el procediment, les taules Professors i Desspatxos no són necessàries.

4. (3 punts) Transaccions:

4.1 Suposeu ara l'horari següent.

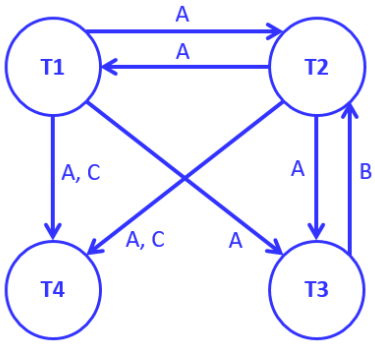
a) Doneu el graf de precedències per aquest horari.

b) En cas que hi hagi alguna interferència en l'horari resultant digueu quina/es són, les transaccions i el/s gränul/s implicat/s.

c) És serialitzable? En cas afirmatiu doneu el horari serial equivalent. En cas negatiu perquè no.

	T1	T2	T3	T4
10			R(B)	
20		RU(A)		
30	RU(A)			
40		W(A)		
50		R(C)		
60		R(F)		
70	W(A)			
80				R(A)
90		RU(B)		
100	R(C)			
110				RU(C)
120		W(B)		
130			R(A)	
140				W(C)
150	commit			
160		commit		
170			commit	
180				commit

a) Graf de precedències.



b) Hi ha una actualització perduda entre T1 i T2 i un anàlisi inconsistent entre T2 i T3
c) No és serialitzable perquè hi ha interferències.

4.2 En cas que les transaccions treballin en nivell d'aïllament READ COMMITTED. Doneu l'horari un cop aplicades les reserves corresponents al nivell d'aïllament. En l'horari heu d'incloure, a més de les operacions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves (LOCK, UNLOCK), i l'ordre d'execució de totes aquestes operacions. Quan més d'una transacció espera per un mateix gràNul, considereu que la política de la cua és FIFO (First In, First Out).

T1	T2	T3	T4
		L(B,S)	
		R(B)	
		U(B)	
	L(A,X)		
	RU(A)		
L(A,X)			
	W(A)		
	L(C,S)		
	R(C)		
	U(C)		
	L(F,S)		
	R(F)		
	U(F)		
			L(A,S)
	L(B,X)		
	RU(B)		
	W(B)		
		L(A,S)	
	C+U(A,B)		
RU(A)			
W(A)			
L(C,S)			
R(C)			
U(C)			
C+U(A)			
			R(A)
			U(A)
			L(C,X)
			RU(C)
		R(A)	
		U(A)	
			W(C)
		C	
			C+U(C)

4.3 Considereu ara un SGBD sense cap mecanisme de control de concurrència, on el gràNul és la fila. I considereu l'horari següent que té una interferència d'anàlisi inconsistent. Expliqueu la interferència d'anàlisi inconsistent en base a l'horari i al contingut de la base de dades donat en l'exercici 3.

	T1	T2
1	select * from despatxos where modul='Omega' and numero=300;	
2		update despatxos set superficie = superficie + 8 where modul='Omega' and numero=300;
3		select * from despatxos where modul='Omega' and numero=301;
4	update despatxos set superficie = superficie + 5 where modul='Omega' and numero=301;	
5	Commit	
6		Commit

La interferència d'anàlisi inconsistent té a veure amb la visió que dues transaccionstenen d'un conjunt de dades. En aquest horari el conjunt de dades són les files 'Omega', 300 i 'Omega', 301.

En aquest horari:

- T1 llegeix les files amb valors 'Omega', 300, 8 i 'Omega', 301, 8.
- T2 llegeix les files amb valors 'Omega', 300, 8 i 'Omega', 301, 8

L'horari no és correcte perquè no dóna el mateix resultat que cap dels dos horaris serial.

En cas de l'horari seria T1;T2:

- T1 llegeix les files amb valors 'Omega', 300, 8 i 'Omega', 301, 8
- T2 llegeix les files amb valors 'Omega', 300, 8 i 'Omega', 301, 13

En cas de l'horari seria T2;T1:

- T2 llegeix les files amb valors 'Omega', 300, 8 i 'Omega', 301, 8
- T1 llegeix les files amb valors 'Omega', 300, 16 i 'Omega', 301, 8

5. **(3 punt)** Considereu la taula R(a,b,c,d). Aquesta taula té 100.000 files, i el factor de bloqueig, de les pàgines de dades on s'emmagatzemen aquestes files, és de 10 files per pàgina.

Considereu la consulta següent.

```
SELECT *
FROM R
WHERE b=5 and c >=50
```

Se sap que hi ha 70 files que compleixen la condició b=5, 100 files la condició c>=50 i només 1 fila les dues condicions alhora.

IMPORTANT: En tots els apartats cal detallar tots els càlculs que feu per calcular els costos.

5.1 Suposant que <u>només podem definir un únic índex, arbre B+, per un únic atribut</u>, i que aquest índex tindrà una ocupació del 80% i un ordre d=50, Quin tipus d´índex (agrupat o no agrupat) escolliríeu i per quin atribut hauria d'estar definit per tal de fer la consulta el més eficient possible? Justifiqueu les respostes en base els costos de totes les opcions possibles.

SOLUCIÓ:

Les opcions possibles són definir un únic índex agrupat o no agrupat pels atributs b o c, i aplicar la resta de condicions a mida que es van obtenint les files de les pàgines de dades. Qualsevol altre índex no augmentaria l'eficiència de la consulta.

Índex agrupat R.b: Cost = h +D = 3 +7 = 10

Valors per node = $\lceil 2d*0,8 \rceil = \lceil 2*50*0,8 \rceil$ = 80 valors / node

h = $\lceil \log_{81} 100000 \rceil$ = 3

D= $\lceil$ card(R.b=5) / 10 $\rceil = \lceil 70 / 10 \rceil$ = 7

Índex no agrupat R.b: Cost = h + F + card(R.b=5) = 3 +0 + 70 = 73

h = $\lceil \log_{81} 100000 \rceil$ = 3

F= $\lceil 70 / 80 \rceil$ - 1= 0

card(R.b=5) = 70

Índex agrupat R.c: Cost = h +D = 3 +10 = 13

Valors per node = $\lceil 2d*0,8 \rceil = \lceil 2*50*0,8 \rceil$ = 80 valors / node

h = $\lceil \log_{81} 100000 \rceil$ = 3

D= $\lceil$ card(R.c>=50) / 10 $\rceil = \lceil 100 / 10 \rceil$ = 10

Índex no agrupat R.c: Cost = h + F + card(R.c > 50) = 3 +1 + 100 = 104

h = $\lceil \log_{81} 100000 \rceil$ = 3

F= $\lceil 100/ 80 \rceil$ - 1= 1

card(R.c > =50) = 100

La millor opció serà definir un índex agrupat per l'atribut R.b, amb un cost de 10 accessos a disc.

5.2 Suposant ara que en lloc d'un únic índex per un únic atribut, en podem definir 2 també per un únic atribut i del mateix tipus que abans (una ocupació del 80% i un ordre d=50) quin seria el cost fent servir l'estratègia de intersecció de RIDs?

Cost índex per R.b = h + F = 3

Cost índex per R.c = h + F = 4

Cost per acedir a dades = Card(R.b=5 and R.c >50) = 1

El cost seria 8 accesos a disc

5.3 Suposant ara que podem definit un índex no agrupat multiatribut pels atributs b, c. Quin seria el cost fent servir aquest índex en cas de tenir ordre d=25 i ocupació 80%?

el cost seria: h +F +Card(R.b=5 and R.c >=50) = 4 + 0 +1 = 5

Valors per node = $\lceil 2d*0,8 \rceil = \lceil 2*25*0,8 \rceil$ = 40 valors / node

h = $\lceil \log_{41} 100000 \rceil$ = 4

F = F= $\lceil 1/ 80 \rceil$ - 1= 0

Temps: 90 minuts

Notes 25 novembre

Cada pregunta s'ha de lliurar en un full separat

Suposeu una base de dades amb les taules següents:

```
CREATE TABLE EquipsFutbol(
nomEquip char(30),
localitat char(50),
nomEstadi char(100) UNIQUE NOT NULL,
pressupost real NOT NULL check(pressupost>=0),
PRIMARY KEY(nomEquip));

CREATE TABLE Partits(
idPartit integer,
nomEquipLocal char(30) NOT NULL,
dataPartit date NOT NULL,
nomEquipVisitant char(30) NOT NULL,
golsEquipL integer NOT NULL CHECK(golsEquipL >=0),
golsEquipV integer NOT NULL CHECK(golsEquipV >=0),
PRIMARY KEY (idPartit),
FOREIGN KEY (nomEquipLocal) REFERENCES EquipsFutbol,
FOREIGN KEY (nomEquipVisitant) REFERENCES EquipsFutbol,
UNIQUE(nomEquipLocal,nomEquipVisitant));

CREATE TABLE Jugadors(
dniJugador char(9),
nom char(50) NOT NULL,
nomEquip char(30) NOT NULL,
salari real NOT NULL check(salari>=0),
PRIMARY KEY (dniJugador),
FOREIGN KEY (nomEquip) REFERENCES EquipsFutbol);
```

- nomEquip és l'equip **on està contractat** el jugador

```
CREATE TABLE Alineacions(
idPartit integer,
dniJugador char(9),
gols integer NOT NULL CHECK(gols>=0),
numTargetes integer NOT NULL CHECK(numTargetes>=0),
PRIMARY KEY (idPartit, dniJugador),
FOREIGN KEY (idPartit) REFERENCES Partits,
FOREIGN KEY (dniJugador) REFERENCES Jugadors);
```

- el jugador dniJugador **ha estat alineat** al partit idenficat per idPartit
- els gols i targetes són les que han posat al jugador durant el partit.

1. **(2.5 punts)** Doneu una sentència SQL per obtenir els partits en els que ha guanyat un equip visitant, i en els que, a més, no s'ha alineat cap jugador a l'equip visitant que tingui un salari inferior a la mitjana dels salaris de tots els jugadors de la base de dades. Es vol que en el resultat hi hagi, la data del partit, el nom de l'equip local, el nom de l'equip visitant i l'estadi on es van jugar. Considereu que els partits es juguen a l'estadi de l'equip local.

```
select p.dataPartit, p.nomEquipLocal,
          p.nomEquipVisitant, e.nomEstadi
from partits p, equipsfutbol e
where p.golsEquipV > p.golsEquipL
     and p.nomEquipLocal = e.nomEquip
     and not exists (select *
                from alineacions a, jugadors j
                where a.dniJugador = j.dniJugador
                     and a.idPartit = p.idPartit
                     and j.nomEquip = p.nomEquipVisitant
                     and j.salari < (select avg(j2.salari)
                          from jugadors j2));
```

2. **(1.5 punt)** Doneu una asserció que no permeti que els equips de futbol tinguin un pressupost inferior a la suma dels salaris dels seus jugadors.

```
CREATE ASSERTION sostreSalarial CHECK (
NOT EXISTS (
     SELECT ef.nomEquip
     FROM EquipsFutbol ef NATURAL INNER JOIN Jugadors j
     GROUP BY ef.nomEquip
     HAVING ef.pressupost < sum(j.salari)))
```

3. **(1 punt)** Suposeu la vista `pichichi` que té per objectiu mostrar la classificació dels golejadors del campionat.

```
CREATE VIEW pichichi (dniJugador, totalGols) AS
        SELECT dniJugador, SUM(gols)
        FROM alineacions
        GROUP BY dniJugador;
```

Abans de l'inici del campionat es van insertar tots els equips i tots els jugadors de cada equip a les taules corresponents. Per registrar el primer partit es van fer els següents inserts:

```
INSERT INTO PARTITS VALUES (1,'FCB','2021/08/15','RSO',4,2);
INSERT INTO ALINEACIONS VALUES (1,'33333333C', 3,0);
INSERT INTO ALINEACIONS VALUES (1,'11111111A', 0,0);
INSERT INTO ALINEACIONS VALUES (1,'22222222B', 2,0);
```

a) Quin és el resultat d'executar la sentència següent, un cop executats els inserts:

```
SELECT * FROM pichichi ORDER BY totalGols DESC
```

b) És actualitzable la vista? Per què?

c) Modifiqueu la definició de la vista per tal que mostri només els jugadors que han marcat en total més de 2 gols.

a)

dniJugador	totalGols
'33333333C'	3
'22222222B'	2
'11111111A'	0

b) No és actualitzable perquè té funcions d'agregació.

c) Cal afegir a la definició el HAVING SUM(gols) > 2

4. (1 punt) Suposeu que la `Laia` és la propietària de la taula `jugadors`.

a) Doneu el diagrama d'autoritzacions que hi haurà després de l'execució de les sentències següents:

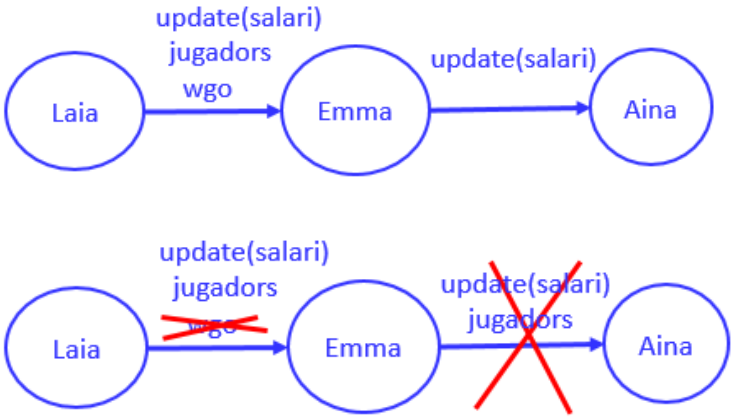
```
Laia: GRANT update(salari) ON jugadors TO Emma WITH GRANT OPTION
Emma: GRANT update(salari) ON jugadors TO Aina
```

b) Partint del diagrama d'autoritzacions resultant de l'apartat anterior, doneu el diagrama d'autoritzacions després de l'execució de la sentència següent:

```
Laia: REVOKE GRANT OPTION FOR update(salari) ON jugadors FROM Emma CASCADE
```

c) Indiqueu quins privilegis mínims li farien falta al `Josep` per tal de poder executar la sentència següent:

```
UPDATE jugadors SET salari= salari +100
WHERE NOT EXISTS (SELECT a.idPartit FROM alineacions a
                  WHERE a.dniJugador=jugadors.dniJugador
                        AND a.gols=0);
```



c) Hauria de tenir privilegis de:

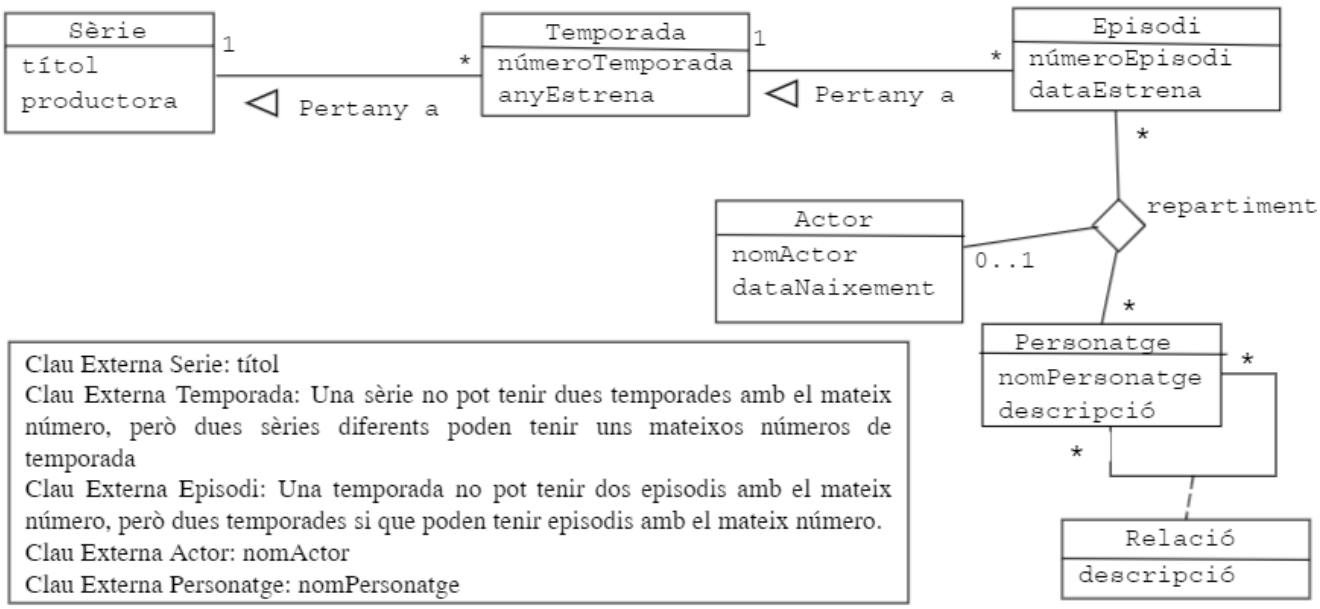
select(dniJugador, salari), update(salari) – jugadors

select(dniJugador, gols, idPartit) – alineacions

5. (1.5 punt) Doneu una seqüencia d'operacions en àlgebra relacional per obtenir els noms dels equips que han marcat fora de casa (han fet algun gol com a visitants) en un únic partit.

```
A=partits(golsEquipV>0)
B=A[nomEquipVisitant, idPartit]
C=B{nomEquipVisitant->nomEquipVisitant2, idPartit->idPartit2}
D=B[nomEquipVisitant=nomEquipVisitant2, idPartit<>idPartit2]C
E=D[nomEquipVisitant]
F=A[nomEquipVisitant]
R=F-E
```

6. (1.5 punts) Suposeu el model conceptual en UML següent. Doneu el disseny lògic que s'obté fent la traducció a model relacional. Cal incloure, taules, atributs, claus primàries, claus foràanies i restriccions NOT NULL i UNIQUE que siguin necessàries.



sèries(títol, productora)
temporades(títol, númeroTemporada, anyEstrena)
{títol} referencia sèries
episodis(títol, númeroTemporada, númeroEpisodi, dataEstrena)
{títol, númeroTemporada} referencia temporades
actors(nomActor, dataNaixement)
personatges(nomPersonatge, descripció)
repartiments(títol, númeroTemporada, númeroEpisodi, nomPersonatge, nomActor)
{títol, númeroTemporada, númeroEpisodi} referencia episodis
{nomPersonatge} referencia personatges
{nomActor} referencia actors NOT NULL
relacions(nomPersonatge, nomPersonatgeRelacionat, descripció)
{nomPersonatge} referencia personatges
{nomPersonatgeRelacionat} referencia personatges

7. (1 punt)

7.1 L'actualització incorrecta de dades redundants, una avaria en un disc on estan els fitxers d'una BD o transaccions que no poden acabar per un tall de subministrament elèctric són exemples de situacions que comprometen la fiabilitat d'un SGBD. Expliqueu breument un dels mecanismes que tingui un SGBD per garantir l'objectiu de fiabilitat.

7.2 Un altre objectiu fonamental dels SGBDs és que permetin a molts usuaris accedir concurrentment a una BD. Expliqueu breument algun problema que es pot produir com a conseqüència de l'accés simultani de molts usuaris a una BD (en cas de no disposar dels mecanismes que ofereixen els SGBD per evitar aquest problema).

Veure material del tema 1 de l'assignatura

1. (1 punt) Donades les taules següents.

```
assignatures(idAssignatura,nomAssignatura,credits)

estudiants(idEstudiant,nomEstudiant,username)

assignaturaMatriculada(idEstudiant,idAssignatura,quadrimestre,nota)
   {idEstudiant} referencia estudiants
   {idAssignatura} referencia assignatures
```

Considereu el següent fragment de codi Java/JDBC. El programa té dos parts. En la primera part es mostra en quantes assignatures han estat matriculats cadascun dels estudiants indicats en la variable *ids*. En la segona part s'esborra de la base de dades una assignatura identificada en la variable *id* si no hi ha cap estudiant que s'hi hagi matriculat; si a l'assignatura s'hi ha matriculat algun estudiant, aleshores es dóna un missatge d'error.

```
01  /* Tenim ja una connexió c establerta amb la base de dades */
02
03  /* consultar assignatures */
04  ResultSet rs = null;
05  int[] ids = {1, 2, 3};
06  PreparedStatement ps = c.prepareStatement("select distinct idAssignatura "+
07                                                "from assignaturaMatriculada "+
08                                                "where idEstudiant = ?");
09  for(int i=0; i<ids.length;i++){
10     ps.setInt(1,ids[i]);
11     rs = ps.executeQuery();
12     int quantesAssignatures=0;
13     while(rs.next()){quantesAssignatures = quantesAssignatures + 1;}
14     System.out.println(ids[i] + " ha estat matriculat en "
15                               + quantesAssignatures + " assignatures!");
16     }
17
18  /* esborrar assignatura */
19  int id = 4;
20  Statement stCons = c.createStatement();
21  Statement stDel = c.createStatement();
22  rs = stCons.executeQuery("select count(*) as quants "+
23                          "from assignaturaMatriculada "+
24                          "where idAssignatura = "+id);
25  if (rs.next() && rs.getInt("quants")>0)
26      {System.out.println("Error: no es pot esborrar assignatura "+
27                           "perquè hi ha estudiants matriculats a"+id);}
28    else {stDel.executeUpdate("delete from assignatures where idAssignatura ="+id);}
29
30 /* Commit i desconnexio de la base de dades */
```

Donat aquest fragment de codi, identifiqueu quins dels criteris de qualitat estudiats a l'assignatura no es compleixen. Per cadascun d'ells:

- Expliqueu perquè no es compleix
- Expliqueu en detall quins canvis seria necessari fer en el codi donat per tal de que es complexi.

Solució

- Primera part - Part del codi és redundant
  - No és necessari el WHILE que calcula la quantitat d'assignatures matriculades d'un estudiant. Cal canviar el SELECT per buscar directament aquest total en el mateix SELECT.
  - El nou SELECT, que substitueix l'existent en línies 05..07:

```
select count(distinct idAssignatura) as total
from students_marks where student_id=?
```

  - S'eliminarà el WHILE de la línia 12, i la línia 11 canviarà de la manera indicada a continuació:

```
rs.next();
int quantesAssignatures = rs.getInt("total");
```

- Segona part – Select innecessari
  - Es pot saber si hi ha l'error simplement executant el DELETE, i recullint en cas que es produeixi l'error de clau forana. Per tant, no es necessari fer el SELECT, i s'ha de recollir l'excepció amb un CATCH.
  - S'eliminarà les línies de codi 19, 21-23 i 24
  - El DELETE de la línia 27 es farà just després de crear el Statement de la línia 20

```
stDel.executeUpdate("delete from assignatures where idAssignatura ="+id);
```

  - Les línies 18, 19 i la nova línia amb el DELETE estaràn en try, i s'afegirà un catch que contidrà les línies 25 i 25 dins de un IF.

```
catch (SQLException se)
        { if (se.getSQLState().equals("23503")
          {System.out.print("Error: ....  ");}
```

2. (2.5 punts) Considereu la base de dades següent.

```
tipusProductes(idTipus, nomTipus, preuMaxim)

productes(nomProducte, idTipus, preu)
    {idTipus} references tipusProductes

comandes(numComanda, import)

productesComanda(numComanda, nomProducte, quantitat)
    {numComanda} references comandes
    {nomProducte} references productes
```

2.1 Suposeu que les taules de la base de dades estan inicialment buides i que executem una crida al següent procediment emmagatzemat *afegir_producte()*. Quin és l'estat de la taula *productes* després de l'execució? Justifiqueu la resposta.

```
CREATE or replace FUNCTION afegir_producte() RETURNS SETOF char(20)
AS $$
    DECLARE nom char(20);
    BEGIN
        FOR nom IN SELECT nomProducte FROM productes
            LOOP RETURN NEXT nom;
            END LOOP;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'Error inesperat';
        END IF;
        INSERT INTO tipusProductes VALUES (1, 'aperitius', 10);
        INSERT INTO productes VALUES ('patates', 1, 4);
    RETURN;
END; $$LANGUAGE plpgsql;
```

En no haver-hi files a productes, el programa no entra en el FOR i per tant FOUND val fals. S'entra a l'IF i salta l'excepció, aturant l'execució del procediment. No s'arriba a fer l'insert i per tant la taula segueix estant buida.

2.2 Suposeu que cadascuna de les regles que s'indiquen a continuació es vol implementar mitjançant triggers. Indiqueu i justifiqueu, per a cada regla, quins triggers caldria definir, tenint en compte els criteris de qualitat de l'assignatura. Per cada trigger cal indicar:

- Esdeveniment que l'activa (cal indicar esdeveniment, taula i atributs rellevants)
- *Before* o *After* (justifiqueu la resposta)
- *Row* o *Statement* (justifiqueu la resposta)

a) REGLA1: Hi ha d'haver una fila a la taula `logs(tipusEsdeveniment, usuari, instant)` per cada vegada que un usuari de la base de dades faci insercions o modificacions de comandes. Es vol mantenir amb triggers el contingut de la taula `logs`.

b) REGLA2: L'`import` d'una comanda ha de ser igual a la suma dels resultats de multiplicar el `preu` de cada producte comprat en la comanda per la `quantitat` d'aquest producte que s'ha comprat. Es vol mantenir correcte el valor de l'atribut `import` quan quedi afectat per esdeveniments sobre la taula `productes`.

c) REGLA3: No pot existir cap producte a la base dades amb un `preu` superior al `preuMaxim` del tipus del producte. Es vol generar una excepció en cas que no es compleixi aquesta regla.

Regla 1

esdeveniments:
- INSERT comandes
- UPDATE comandes

before/after: AFTER. Per no fer feina innecessària en cas que es violi alguna RI de la BD
row/statement: STATEMENT. només ens importa l'esdeveniment i no les files

Regla 2

esdeveniments:
- UPDATE of preu on productes

before/after: AFTER. Per no fer feina innecessària en cas que es violi alguna RI de la BD
row/statement: ROW. Es pot fer statement però implicaria recalcular tots els imports de totes les comandes de la base de dades -> millor incremental

Regla 3

esdeveniment:

- UPDATE of idTipus,preu on productes
- UPDATE preuMaxim on tipusProductes
- INSERT on productes

before/after: BEFORE. Ja que és millor comprovar el més aviat possible el que es compleixen les restriccions d'integritat.

row/statement: ROW. Es pot fer statement però implicaria verificar que es compleix la restricció per tots els productes de la base de dades -> millor incremental

2.3 Implementeu un trigger que mantingui correcte el valor de l'atribut `import` de la taula `comandes` quan s'executi una sentència INSERT sobre la taula `productesComanda`. Tingueu en compte la REGLA2 de l'apartat anterior que defineix com es calcula l'import d'una comanda.

Podeu fer servir la plantilla de triggers de PostgreSQL següent:

```
CREATE TRIGGER nomTrigger
[BEFORE/AFTER] [UPDATE (of column)/DELETE/INSERT]ON taula
[FOR EACH ROW/FOR EACH STATEMENT] execute procedure nomp();

CREATE FUNCTION nomp() RETURNS trigger AS $$
    DECLARE
        ...
    BEGIN
        ...
    END;
$$ LANGUAGE plpgsql;
```

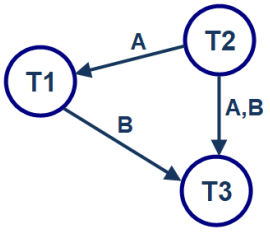
```
create function actualitza_import() returns trigger as $$
     declare
          preu integer;
     begin
          select p.preu into preu from productes p
          where nomProducte = new.nomProducte;

          update comandes
          set import = (import + (preu * new.quantitat))
          where numComanda = new.numComanda;

          return null;
        end;
$$ language plpgsql;

create trigger mante_import after insert on productesComanda
for each row execute procedure actualitza_import();
```

3. (2 punt) Donat el graf de precedències següent:



3.1 Digueu per cadascuna de les interferències que es produeixen: entre quines transaccions es produeixen, i els grànuls implicats.

No es produeix cap interferència donat que no hi ha cap cicle.

3.2 Doneu un horari que inclogui les mínimes operacions (R, RU, W, COMMIT) possibles i que es correspongui amb el graf de precedències.

Una possible solució és:

	T1	T2	T3
10		RU(A)	
20		W(A)	
30	R(A)		
40			R(A)
50	R(B)		
60			RU(B)
70		R(B)	
80			W(B)
90		commit	
100	commit		
110			commit

3.3 Digueu si l'horari que heu donat en l'apartat anterior és o no recuperable. Justifiqueu la resposta.

L'horari donat en la solució anterior és recuperable. Veure transparència.

3.4 Modifiqueu l'horari que heu donat, afegint una única operació (R,RU,W), per tal que s'afegeixi una única interferència de lectura no repetible entre T2 i T3. Si no és possible, justifiqueu la resposta.

Una possible solució és:

	T1	T2	T3
			R(A)
10		RU(A)	
20		W(A)	
30	R(A)		
40			R(A)
50	R(B)		
60			RU(B)
70		R(B)	
80			W(B)
90		commit	
100	commit		
110			commit

4. (2 punt) Donat un SGBD sense cap mecanisme de control de concurrència, suposem que es produeix l'horari següent:

Temps	T1	T2	T3
10		RU(B)	
20		W(B)	
40	RU(A)		
50	W(A)		
60	R(D)		
70	R(B)		
80			R(E)
100			RU(A)
110			W(A)
120			RU(F)
130		R(F)	
140		RU(D)	
150		W(D)	
160			W(F)
170	RU(F)		
180	W(F)		
190		COMMIT	
200			COMMIT
210	COMMIT		

4.1 Suposeu que s'incorpora un mecanisme per al control de la concurrència basat en reserves S, X, on les transaccions T1 i T3 treballa amb SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED, i T2 amb SET TRANSACTION ISOLATION LEVEL READ COMMITTED. Doneu l'horari aplicant reserves que ha d'incloure, a més de les operacions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves (LOCK, UNLOCK), i l'ordre d'execució de totes aquestes operacions. Quan més d'una transacció esperi per un mateix gràNul, considereu que la política de la cua és FIFO (First In, First Out). En cas que cap acció de l'horari pugui executar-se en un moment donat, no continueu i justifiqueu el motiu.

T1	T2	T3
	LOCK(B,X)	
	RU(B)	
	W(B)	
LOCK(A,X)		
RU(A)		
W(A)		
R(D)		
R(B)		
		R(E)
		LOCK(A,X)
	L(F,S)	
	R(F)	
	U(F)	
	LOCK(D,X)	
	RU(D)	
	W(D)	
LOCK(F,X)		
RU(F)		
W(F)		
	COMMIT+U(B,D)	
COMMIT+U(A,F)		
		RU(A)
		W(A)
		LOCK(F,X)
		RU(F)
		W(F)
		COMMIT+U(A,F)

4.2 Suposeu que s'incorpora un mecanisme per al control de la concurrència basat en reserves S, X, on les transaccions T1, T2 i T3 treballen amb SET TRANSACTION ISOLATION LEVEL SERIALIZABLE. Doneu l'horari aplicant reserves que ha d'incloure, a més de les operacions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves (LOCK, UNLOCK), i l'ordre d'execució de totes aquestes operacions. Quan més d'una transacció esperi per un mateix gränul, considereu que la política de la cua és FIFO (First In, First Out). En cas que cap acció de l'horari pugui executar-se en un moment donat, no continueu i justifiqueu el motiu.

T1	T2	T3
	LOCK(B,X)	
	RU(B)	
	W(B)	
LOCK(A,X)		
RU(A)		
W(A)		
LOCK(D,S)		
R(D)		
LOCK(B,S)		
		LOCK(E,S)
		R(E)
		LOCK(A,X)
	LOCK(F,S)	
	R(F)	
	LOCK(D,X)	

5. (2.5 punts) Considereu la taula T(a,b). La taula T té 40000 tuples i està emmagatzemada en un fitxer on a cada pàgina hi caben en promig 10 files.

5.1 En la taula no hi ha cap índex definit. Doneu el cost de la consulta següent i mostreu el detall de com es calcula.

```
SELECT * FROM T where a = valor
```

5.2 Es defineix ara un índex B+ no agrupat per l'atribut `a` de T. L'índex té d=50 i els nodes estan plens al 70% d'ocupació. Doneu el cost de la consulta de l'apartat anterior i mostreu el detall de com es calcula.

5.3 Es defineix ara un índex B+ no agrupat per l'atribut `b` de T. L'índex té d=50 i els nodes estan plens al 70% d'ocupació. Doneu el cost de la consulta següent i mostreu el detall de com es calcula.

```
SELECT SUM(b) FROM T
```

5.4 Suposeu ara que només hi ha un índex B+ agrupat per l'atribut `a` de T. L'índex té d=50 i els nodes estan plens al 70% d'ocupació. No es defineix cap índex sobre l'atribut `b`. Suposeu que els valors de l'atribut `a` estan distribuïts uniformement entre 1 i 40000. Doneu el cost de la consulta següent i mostreu el detall de com es calcula.

```
SELECT * FROM T WHERE a>=33000 AND b<=1000
```

Solució

5.1) Com que no hi ha cap índex definit a T i el fitxer no està ordenat, caldrà fer un recorregut seqüencial del fitxer de dades.

Card(T)=40000

f=factor de bloqueig del fitxer de dades=10

Cost = $\lceil Card(T) / f \rceil = \lceil 40000 / 10 \rceil$ = 4000 accessos a pàgines de dades.

5.2) Com que hi ha un índex definit per a l'atribut a, es baixarà per l'arbre buscant el valor (a=valor), i després un cop localitzat s'accedirà a la única fila (clau primària) que pot tenir aquest valor.

Al tenir una d=50, i ocupació del 70%, el número de valors per node és de $\lceil 2*50*0,7 \rceil$, és a dir, 70 valors per node, i 71 apuntadors en nodes interns.

Al baixar per l'arbre s'accedirà a tantes pàgines com nivells (h) té l'arbre.

$h=\lceil \log_{71} 40000 \rceil = 3$

El cost de localitzar la fila on es compleix a=valor, tenint en compte que "a" és clau primària, és:

Cost = h + 1 = 4

5.3) Caldrà accedir a totes les fulles de l'arbre per obtenir els valors de b i anar-los sumant. No és necessari accedir a les pàgines de dades, perquè els valors de b estan a l'índex. Per tant, el cost de resoldre la consulta és:

h és la mateixa que per l'índex sobre l'atribut a, ja que la d i la ocupació són les mateixes.

h = 3

F = nombre de fulles que cal llegir.

F = $\lceil 40000/70 \rceil$= 572

Cost = h + F - 1 = 3 +572 -1 =3 + 572 -1 =575, és resta 1 perquè hi ha una fulla que s'ha comptat dues vegades.

5.4) Com que només tenim un índex agrupat per l'atribut "a". Cal baixar per l'índex buscant el valor 33000. Tenint en compte que l'índex és agrupat, després es va a les pàgines de dades per buscar les files on es compleixi a>=33000. En accedir a les files dins de les pàgines de dades es podrà saber per cada fila si es compleix la condició b<=1000.

h és la mateixa que per l'índex sobre l'atribut a, ja que la d i la ocupació són les mateixes.

h = 3

D = nombre de pàgines de dades que cal llegir

D = $\lceil 7001 / 10 \rceil$ = 701

Cost = h + D = 3 + 701 = 704

Temps: 2 hores

Notes 20 d'abril

Revisió 21 d'abril (a les 13h, presencial)

Cada pregunta s'ha de lliurar en un full separat

Suposeu una base de dades amb les taules següents:

```
create table departaments(
codiDept integer,
nomDept char(50) unique not null,
pressupost integer not null check(pressupost>=0),
primary key (codiDept));

create table professors(
idProf integer,
nomProf char(50) not null,
codiDept integer not null,
sou real,
primary key (idProf),
foreign key (codiDept) references departaments);
-- codiDept és del departament en el que treballa el professor

create table assignatures(
idAssig char(5),
nomAssig char(50) not null,
codiDept integer not null,
profResponsable integer not null,
primary key (idAssig),
foreign key (codiDept) references departaments,
foreign key (profResponsable) references professors);
-- codiDept és del departament al que pertany l'assignatura
-- profResponsable és el professor responsable de l'assignatura

create table assignacions(
idProf integer,
idAssig char(5),
quadrimestre char(6),
horesAssig integer not null check (horesAssig>0),
primary key (idProf, idAssig, quadrimestre),
foreign key (idProf) references professors,
foreign key (idAssig) references assignatures);
-- hi ha una fila si el professor idProf ha impartit o imparteix
-- classes de l'assignatura idAssig en el quadrimestre
```

1. **(2 punts)** Considereu la base de dades donada a l'inici de l'examen. Doneu una sentència SQL per obtenir noms de les assignatures que o bé són l'única assignatura del departament al que pertanyen, o bé no han estat mai impartides pels seus professors responsables.

```
select distinct nomassig
from assignatures a
where not exists (
        select * from assignatures a2
        where a2.idAssig<>a.idAssig and
              a2.codiDpt=a.codiDpt)
    or not exists (
        select * from assignacions aP
        where aP.idAssig = a.idAssig and
              aP.idProf = a.profResponsable)
```

2. (2.5 punts) Tenint en compte l'esquema de la base de dades donat a l'inici de l'examen.

2.1. Per cadascuna de les situacions que es plantegen a continuació indiqueu si és o no possible que succeeixi.

- En cas que no sigui possible, indiqueu quina restricció ho evita.
- En cas que sigui possible, doneu un conjunt d' inserts per tal que passi (parteix de la BD buida).

a) És possible que existeixin dues o més assignatures amb el mateix professor responsable?

b) És possible que existeixin dues assignacions d'un mateix professor, a una mateixa assignatura en un mateix quadrimestre?

c) És possible que quan s'esborri algun professor que és responsable d'alguna assignatura, es posi a null el valor de l'atribut profResponsable per les assignatures afectades per l'esborrat?

2.2. Doneu una sentència de creació d'una asserció que garanteixi que cap professor que té un sou inferior a 1500 euros, doni classes a més de tres assignatures diferents en un mateix quadrimestre.

SOLUCIÓ

2.1

a) Sí que és possible que passi.

```
insert into departaments values (1,'ESSI', 10000);
insert into professors values (11,'Pep',1,1000);
insert into assignatures  values ('bd','bases de dades',1,11);
insert into assignatures  values ('so','sistemes opertius',1,11);
```

b) No és possible que passi, la PK d'assignacions ho evita.

c) No és possible que passi, l'atribut ProfResponsable té restricció not null.

2.2

```
CREATE ASSERTION maxim_assig CHECK (
NOT EXISTS (SELECT a.idprof, a.quadrimestre
        FROM assignacions a, professors p
        WHERE a.idprof=p.idprof and p.sou<1500
        GROUP BY a.idprof,a.quadrimestre
        HAVING count (*)>3));
```

3. **(1 punts)** Considereu les taules R, S, i T, i la vista Tot. Com es pot observar, d'acord amb els criteris vistos a classe, la vista Tot no és actualitzable i per aquest motiu els SGBD impediran que s'hi faci qualsevol operació d'actualització, tot i que en realitat podria ser que algunes operacions sí que es poguessin fer sense ambigüitat.

```
create table R(c integer primary key, d integer);
create table S(b integer primary key, c integer references R);
create table T(a integer primary key, b integer references S);

create view tot(a1,a2,a3,a4,a5,a6) as select T.a,T.b,S.b,S.c,R.c,R.d
                                      from T natural inner join S
                                             natural inner join R;
```

3.1 Indiqueu una operació (Delete o Update) sobre la vista Tot i un contingut de les taules R, S, i T que mostrin que la operació NO es pot realitzar sense ambigüitat. En cas de que no sigui possible, justifiqueu la resposta.

Una possible solució: Si tenim R(3,4) S(2,3) T(1,2) , la vista tindrà la tupla tot(1,2,2,3,3,4). Un Delete d'aquesta tupla no es podria fer de manera no ambigua ja que hi ha múltiples solucions.

3.2 Indiqueu una operació (Delete o Update) sobre la vista Tot i un contingut de les taules R, S, i T que mostrin que la operació es pot realitzar sense ambigüitat. En cas de que no sigui possible, justifiqueu la resposta.

Una possible solució: Si tenim R(3,4) S(2,3) T(1,2), la vista tindrà la tupla tot(1,2,2,3,3,4). I una operació d'update del valor 4 d'aquesta tupla es podria realitzar sense cap problema.

4. (2.5 punts)

4.1 Doneu una seqüència d'operacions d'àlgebra relacional per obtenir el codi i nom dels departaments que no tenen cap professor que hagi impartit classes el quadrimestre '1819q2'

```
A = assignacions(quadrimestre='1819q2')
B = A*professors
C = B[codiDept]
D = departaments[codiDept]
E = D-C
F = E * departaments
R = F[codiDept,nomDept]
```

4.2 Tenint en compte l'esquema de la base de dades de l'inici de l'examen i suposant que tenim:

a) **3 departaments,** amb codis de departament 1, 2, i 3.

b) **25 professors,** 10 del departament 1, i 15 professors del departament 2.

c) **5 assignatures** que pertanyen al departament 1, i el seu professor responsable és del departament 2.

Indiqueu quantes files s'obtindran en el resultat de les consultes següents. Justifiqueu la resposta. Si no podeu dir el nombre exacte, indiqueu un mínim i un màxim.

1) R = Assignatures * Departaments

   5 files: 1 per cada assignatura (assignada al departament 1).

2) P = Professors { codiDept -> codiDeptProf }

   R = Assignatures [profResponsable = idProf, codiDept = codiDeptProf] P

   0 files: perquè el codiDept de la assignatura (sempre 1) mai és igual al codiDept del professor responsable (sempre 2)

3) Q = Professors * Departaments

   R = Q[codiDept,nomDept]

   2 files: perquè en fer la join, només queden 25 files dels professors dels departaments 1 i 2, quan projectem només queden les dades dels 2 departaments de la taula professors

5. (2 punts)

5.1 Considerant la base de dades donada a l'inici de l'examen. Per cada una de les restriccions següents:

a) Si es pot implementar com a restricció en la definició de taules, doneu el codi SQL que cal afegir a la sentència CREATE TABLE de la BD per implementar-la, indicant en quina taula s'ha d'afegir.

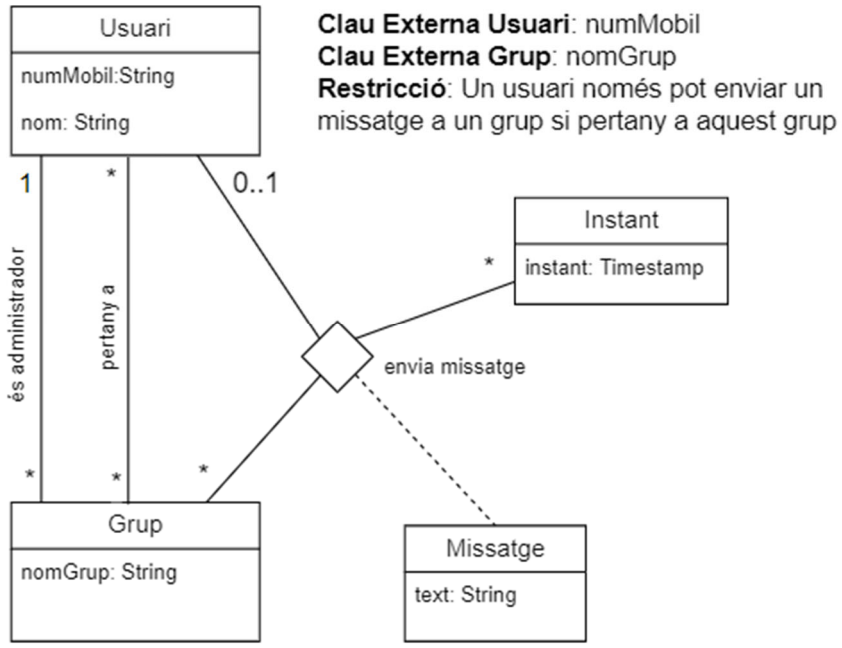
b) Si no es pot, expliqueu per què, i indiqueu com es podria implementar.

R1. El departament al que està assignada una assignatura ha de coincidir amb el departament del professor responsable d'aquella assignatura.

R2. Un professor no pot impartir més de 6 hores en una mateixa assignatura en un mateix quadrimestre.
R3. Quan s'esborren departaments que tenen assignatures o professors assignats, també caldrà esborrar les assignatures o professors d'aquests departaments. També caldrà esborrar les assignacions d'aquests professors a aquestes assignatures.

- R1: No es pot implementar com a restricció de taula o columna, perquè el codi de departament d'un professor i el departament al que està assignada una assignatura estan a taules diferents.
    - En SQL estàndard R1 es podria implementar amb una asserció.
- R2: Sí que és pot implementar com a restricció de columna a la columna horesAssig.
    - Afegir a la taula assignacions CHECK (horesAssig<=6)
- R3: Si que és pot implementar. Cal indicar que la política per mantenir la integritat referencial en cas d'esborrats és en cascada.
    - Afegir a Assignatures, foreign key (codiDept) references departaments ON DELETE CASCADE
    - Afegir a Professors, foreign key (codiDept) references departaments ON DELETE CASCADE
    - Afegir a Assignacions, foreign key (idProf) references professors ON DELETE CASCADE i foreign key (idAssig) references assignatures ON DELETE CASCADE

5.2 Suposeu el model conceptual en UML següent. Doneu el disseny lògic que s'obté fent la traducció a model relacional. Cal incloure, taules, atributs, claus primàries, claus foranes i restriccions NOT NULL i UNIQUE que siguin necessàries.



Usuari (numMobil, nom)

Grup (nomGrup, administrador)

{administrador} referencia Usuari NOT NULL

Pertinença(nomGrup, numMobil)

{nomGrup} referencia Grup {numMobil} referencia Usuari

Missatge (instant, nomGrup, numMobil, text)

{numMobil] referencia Usuari NOT NULL

{nomGrup,numMobil} referencia Pertinença /* per implementar la restricció*/

text NOT NULL

Temps: 2,5 hores

Notes 20 juny - Revisió d'examen: 21 juny (presencial)

Cada pregunta s'ha de lliurar en un full separat

1. (1 punts) Donada la base de dades següent:

```
create table professors
(dni char(50),
nomProf char(50) unique,
sou integer,
primary key (dni));

create table despatxos
(modul char(5),
numero char(5),
superficie integer not null check(superficie <=25),
primary key (modul,numero));

create table assignacions
(dni char(50),
modul char(5),
numero char(5),
instantInici integer,
instantFi integer,
primary key (dni, modul, numero, instantInici),
foreign key (dni) references professors,
foreign key (modul,numero) references despatxos,
check (instantInici < instantFi));
/*instantFI té valor nul quan una assignació és vigent */
```

Considereu el següent fragment de codi Java/JDBC. Implementeu el cos del programa per tal que per cada professor de la base de dades que tingui alguna assignació vigent al despatx número 124 del mòdul 'Omega', es decrementi el sou del professor en 20 euros. Cal que el programa tregui un missatge d'error si no hi ha cap professor que compleixi la condició indicada.

```
try {
 /* Tenim ja una connexió c establerta amb la base de dades */

 //codi a implementar

 c.commit();
 }
catch (ClassNotFoundException ce) {
        System.out.println ("Error al carregar el driver");}
catch (SQLException se) {
        System.out.println ("Excepcio: "+ se.getMessage());}
```

Recordeu que podeu usar els mètodes/codis d'excepció de JDBC estudiats a classe:

Statements

- Statement createStatement();
- ResultSet executeQuery(String sql);
- int executeUpdate(String sql);

PreparedStatement

- PreparedStatement prepareStatement(String sql);
- ResultSet executeQuery();
- int executeUpdate();
- void setXXX(int posicioParametre, XXX valor);

ResultSet

- boolean next();
- XXX getXXX(String nomColumna)

SQLException

- // 23502 -not_null_violation
- // 23503 -foreign_key_violation
- // 23505 -unique_violation
- // 23514 -check_violation
- String getSQLState();

Solució

```
try {
 /* Tenim ja una connexió c establerta amb la base de dades */
  Statement stDel = c.createStatement();
  int numDcr = stDel.executeUpdate("update professors p "+
                 "set sou = sou - 20 "+
                 "where exists (select * from assignacions a "+
                               "where a.dni = p.dni and a.modul='Omega' "+
                               "and a.numero='124' "+
                                "and a.instantFi is null)");
  if (numDcr==0)
     {System.out.println("Error");}
     else {System.out.println("S'han decrementat ="+numD);}

 c.commit();}
 catch (ClassNotFoundException ce)
        {System.out.println     ("Error     al     carregar     el     driver");}
 catch (SQLException se)
        {System.out.println ("Excepcio: "+ se.getMessage());}
```

2. (3.5 punts) Donades les transaccions següents (per cada transacció s'indica les operacions que vol fer i l'ordre en que la transacció realitza aquestes operacions):

T1: RU(A), W(A), RU(A), W(A), Commit

T2: R(A), RU(B), W(B), Commit

T3: R(B), R(C), R(B), Commit

Es demana:

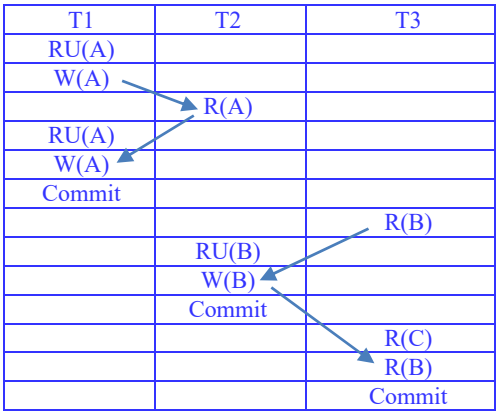
2.1 Proposeu un horari que inclogui les tres transaccions i que tingui dues interferències. Cal indicar el nom de les interferències, els grànuls implicats i les transaccions implicades.

2.2 Proposeu un horari, que inclogui encavalcaments entre les tres transaccions, i que tingui els dos horaris serials equivalents següents: T2;T1;T3 i T2; T3;T1.

2.3 Considereu ara només T1 i T2 i suposeu que tenim control de concurrència basat en reserves. Per cadascun dels quatre nivells d'aïllament, proposeu un horari amb aquestes dues transaccions de tal manera que es produeixi una interferència. En cas que no sigui possible produir la interferència, expliqueu breument perquè. L'horari ha d'incloure, a més de les peticions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves i l'ordre d'execució de totes aquestes peticions. També ha d'incloure, en cas que es produeixin, les esperes de les transaccions.

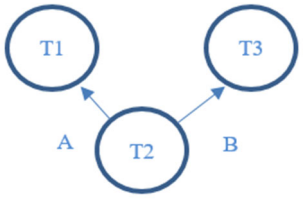
2.4 Considereu ara només T2 i suposeu que tenim control de concurrència basat en reserves. Per al nivell d'aïllament serialitzable proposeu un horari format per T2 i una altra transacció de tal manera que es produeixi una abraçada mortal. L'horari ha d'incloure, a més de les peticions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves i l'ordre d'execució de totes aquestes peticions. També ha d'incloure, en cas que es produeixin, les esperes de les transaccions. Doneu també el graf d'espera just després de que es produeixi l'abraçada mortal.

2.1.



Es produeix una lectura no confirmada entre T1 i T2 pel gränul A i una lectura no repetible entre T2 i T3 pel gränul B.

2.2. Per tenir els dos horaris serials equivalents demanats cal tenir el següent graf de precedències.



Per tant, cal generar un horari on les operacions no commutables entre T2 i T1 segueixin l'ordre següent: primer s'executen les de T2 i després les de T1. Entre T2 i T3 s'ha de produir un cas similar. Evidentment, l'horari ha d'estar lliure d'interferències.

T1	T2	T3
	R(A)	
RU(A)		
W(A)		
RU(A)		
W(A)		
Commit		
	RU(B)	
	W(B)	
		R(B)
		R(C)
	Commit	
		R(B)
		Commit

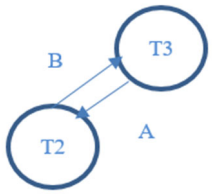
2.3. En el nivell Read Uncommitted no es fan reserves per lectura (modalitat S) i es fan reserves en modalitat X fins a l'acabament de la transacció. Per generar la interferència de lectura no confirmada hem d'assegurar-nos que la lectura del gränul A que fa T2 es faci entre les dues escriptures que fa T1.

T1	T2
Lock (A,X)	
RU(A)	
W(A)	
	R(A)
RU(A)	
W(A)	
Commit (+ unlock (A))	
	Lock (B,X)
	RU(B)
	W(B)
	Commit (+ unlock (B))

Aplicant les reserves del nivell Read Committed s'evita la lectura no confirmada. Per tan, no és possible produir un horari amb aquesta interferència.
Amb la resta de nivells d'aïllament també s'evita aquesta interferència. Per tant, no és possible generar un horari amb aquesta interferència.

2.4. A l'horari següent es pot veure que a partir de l'instant t=8 les dues transaccions estan suspeses sense possibilitat de continuar. T2 està esperant per reservar B (aquest gränul el té reservat T3) i T3 està esperant per reservar A (aquest gränul el té reservat T2).

Temps	T2	T3
1	Lock(A,S)	
2	R(A)	
3		
4		Lock (B,S)
5		R(B)
6	Lock(B,X)	
7		Lock (A,X)
8		



Graf d'espera en temps=8:

3. **(2 punts)** Considereu la base de dades següent.

```
competicions(idCompetició, nomCompetició)
equips(nomEquip, ciutat)
atletes(idAtleta, nomAtleta, dataNaixement, nomEquip)
    {nomEquip} referencia equips
arribadaAtletes(idCompetició, idAtleta, temps)
    {idCompetició} referencia competicions
    {idAtleta} referencia atletes
    -- el valor de l'atribut temps correspon al temps que
    ha trigat l'atleta a arribar a la meta en la competició
arribadaEquips(idCompetició, nomEquip, numAtletes)
    {idCompetició} referencia competicions
    {nomEquip} referencia equips
```

3.1 Es vol mantenir amb disparadors el contingut de la taula *arribadaEquips*.

- Hi ha d'haver una fila en aquesta taula per una competició i equip a partir de que es registra l'arribada a la meta del primer atleta de l'equip que participa en la competició.
- L'atribut *numAtletes* de la taula *arribadaEquips* ha de tenir per valor el número d'atletes d'un equip pels que s'ha registrat la seva arribada a la meta per una determinada competició.

Implementeu el(s) disparador(s) necessari(s) per al cas de l'esdeveniment d'inserció de files a la taula *arribadaAtletes*. Podeu fer servir la plantilla de triggers de PostgreSQL inclosa.

3.2 Suposeu ara que a la base de dades, en lloc d'haver-hi la taula *arribadaEquips*, hi ha la taula

```
ranquing(idCompetició, idAtleta, posicio, temps)
    {idCompetició} referencia competicions
    {idAtleta} referencia atletes
```

Es vol mantenir amb disparadors el contingut de la taula *ranquing*.

- Hi ha d'haver una fila en aquesta taula per cada atleta pel que s'ha registrat la seva arridada a la meta en una competició.
- El valor de l'atribut *temps* correspon al temps que ha trigat l'atleta a arribar a la meta en la competició.
- El valor de l'atribut *posicio* correspon al lloc on l'atleta ha quedat en la competició. La posició es calcula en funció del temps que han trigat els atletes a arribar a la meta. Dos atletes NO poden trigar exactament el mateix temps en arribar a la meta.

Implementeu el(s) disparador(s) necessari(s) per al cas de l'esdeveniment d'inserció de files a la taula *arribadaAtletes*. Aquestes insercions poden NO fer-se en l'ordre d'arribada a la meta dels atletes. Podeu fer servir la plantilla de triggers de PostgreSQL inclosa.

```
CREATE TRIGGER nomTrigger
[BEFORE/AFTER] [UPDATE (of column)/DELETE/INSERT] ON taula
[FOR EACH ROW/FOR EACH STATEMENT] execute procedure nomp();

CREATE FUNCTION nomp() RETURNS trigger AS $$
    DECLARE
        ...
    BEGIN
        ...
    END;
$$ LANGUAGE plpgsql;
```

2.1

```
CREATE or replace FUNCTION fn_novaArribadaAtleta_21() RETURNS trigger AS $$
   DECLARE
     nomEquipAtleta equips.nomEquip%type;
   begin
        -- INSERTAR O ACTUALITZAR LA TAULA D'arribadaEquips

        -- Cal recuperar el nom de l'equip de l'atleta
        select nomEquip into nomEquipAtleta
        from atletes
        where idatleta = new.idatleta;

        -- Si tenim valor fem UPDATE incrementant en 1 el numAtletes,
        -- sinó fem INSERT amb 1 atleta
        update arribadaEquips set numAtletes = numAtletes+1
        where idCompeticio = new.idCompeticio
              and nomEquip = nomEquipAtleta;

        if not found then
            insert into arribadaEquips
               values (new.idCompeticio, nomEquipAtleta, 1);
        end if;
       return null;
   END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_novaArribadaAtleta_21
AFTER INSERT ON arribadaatletes
      FOR EACH ROW execute procedure fn_novaArribadaAtleta_21();
```

2.2

```
CREATE or replace FUNCTION fn_novaArribadaAtleta_22() RETURNS trigger AS $$
   DECLARE
     maxpos ranquing.posicio%type;

   begin
        -- INSERTAR O ACTUALITZAR A LA TAULA DE ranquing
        select max(r.posicio) into maxpos
        from ranquing r
        where r.idcompeticio = new.idcompeticio and r.temps <= new.temps;

        -- no hi ha temps inferior o igual, la posició ha de ser 1
        if maxpos is null then
          insert into ranquing
           values (new.idCompeticio, new.idAtleta, 1, new.temps);
        else
          --temps màxim inferior, la posició ha de ser màxima +1
          insert into ranquing
            values (new.idCompeticio, new.idAtleta, maxPos+1,new.temps);
        end if;

        -- desplaçem el ranquing dels arribats en temps superiors
        update ranquing set posicio = posicio+1
        where idcompeticio = new.idCompeticio and temps > new.temps;

        return null;
   END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_novaArribadaAtleta_22
AFTER INSERT ON arribadaatletes
      FOR EACH ROW execute procedure fn_novaArribadaAtleta_22();
```

4. (3.5 punts)

4.1 Suposeu les dues situacions següents per a la base de dades de l'exercici 3:

- Les taules *equips* i *atletes* estan cadascuna en un espai virtual de taula.
- Les taules *equips* i *atletes* estan en un espai virtual d'agrupació.

a) Expliqueu quines diferències hi ha a nivell físic entre les dues situacions anteriors.

b) Doneu un exemple de consulta SQL que es vegi afavorida per cadascuna de les situacions anteriors.

4.2 Suposeu ara que la taula *atletes* està en un espai virtual de taula i té esquema:

atletes(idAtleta, nomAtleta, dataNaixement, nomEquip)

A la taula hi ha 120.000 atletes, i les pàgines de dades tenen un factor de bloqueig de 20. Hi ha 1000 atletes amb una data de naixement anterior a l'any 2000. Sobre la taula hi ha definits:

- un índex arbre B+ agrupat per *idAtleta* d'ordre d=255 i ple al 75%.
- un índex arbre B+ per *dataNaixement*. Es tracta d'un índex no agrupat amb ordre d=146, i ple al 80%.

a) Expliqueu com el SGBD resoldrà la consulta següent, doneu el cost en número de pàgines accedides, i mostreu el detall de com es calcula aquest cost.

SELECT * FROM atletes WHERE idAtleta = 34567 OR nomAtleta = 'Joan Pi'

b) Expliqueu com el SGBD resoldrà la consulta següent, doneu el cost en número de pàgines accedides, i mostreu el detall de com es calcula aquest cost.

SELECT * FROM atletes WHERE dataNaixement <= '31-12-1999'

4.1.a) En el primer cas, cada taula estarà emmagatzemada en un fitxer, les pàgines de cada fitxer contindran files únicament de la taula corresponent.

En el segon cas, les dues taules estaran emmagatzemades en el mateix fitxer, les pàgines de la taula estaran combinades les files de les dues taules tenint en compte el nom de l'equip (és a dir, hi haurà una fila corresponent a un equip, seguida de les files corresponents als atletes de l'equip, i a continuació un altre equip,...).

4.1.b) En el primer cas,

SELECT * FROM equips

En el segon cas,

SELECT * FROM equips natural inner join atletes WHERE nomEquip='BCN'

4.2.a) En aquest cas l'índex definit sobre idAtleta no ajuda en res a la resolució de la consulta, i per aquest motiu el SGBD farà una accés seqüencial a les pàgines de dades

cost = 120.000 files / 20 files per pàgina = 6.000 pàgines

4.2.b) En aquest cas s'usarà l'índex per dataNaixement, baixant per l'arbre B+, recorrent les fulles corresponents a valors on dataNaixement <= '31-12-1999', i per cada valor s'usarà el RID per buscar la fila corresponent al valor.

El cost de resoldre la consulta usant el índex seria:

n = ⌈2*d*ocupació⌉ = ⌈2*146*0,8⌉ = ⌈ 233,6 ⌉ = 234 valors per node

ap = n+1 = 234+1 = 235 apuntadors per node intern

h = ⌈ log235 120.000 ⌉ = ⌈ 2,14 ⌉ = 3 nodes = 3 pàgines

F = ⌈ valors a llegir en els nodes fulla / 234 valors per node⌉ =

⌈ 1000/234 nodes fulla⌉ = ⌈ 4,27 ⌉ = 5 pàgines

D = tantes pàgines com número de files que compleixen la condició

dataNaixement <= '31-12-1999' = 1000 pàgines

cost = 3+(5-1)+1000 = 1007 pàgines

BD-GEI **2.11.2022**

Temps: 2 hores

Notes 21 de novembre

Revisió 24 de novembre (a les 18h, presencial)

Cada pregunta s'ha de lliurar en un full separat

Suposeu una base de dades amb les taules següents:

```
create table departaments(
codiDept integer,
nomDept char(50) unique not null,
pressupost integer not null check(pressupost>=0),
primary key (codiDept));

create table professors(
idProf integer,
nomProf char(50) not null,
codiDept integer not null,
sou real,
ciutatRes char(100) not null,
primary key (idProf),
foreign key (codiDept) references departaments);
-- codiDept és del departament en el que treballa el professor

create table assignatures(
idAssig char(5),
nomAssig char(50) not null,
codiDept integer not null,
profResponsable integer not null,
primary key (idAssig),
foreign key (codiDept) references departaments,
foreign key (profResponsable) references professors);
-- codiDept és del departament al que pertany l'assignatura
-- profResponsable és el professor responsable de l'assignatura

create table assignacions(
idProf integer,
idAssig char(5),
quadrimestre char(6),
horesAssig integer not null check (horesAssig>0),
primary key (idProf, idAssig, quadrimestre),
foreign key (idProf) references professors,
foreign key (idAssig) references assignatures);
-- hi ha una fila si el professor idProf ha impartit o imparteix
-- classes de l'assignatura idAssig en el quadrimestre
```

1. (2,5 punts) Considereu la base de dades donada a l'inici de l'examen.

1.1 Doneu una sentència SQL per obtenir els noms dels professors que en el quadrimestre Q122 estan assignats a assignatures que al mateix quadrimestre Q122 tenen assignats professors de departaments diferents.

```
select distinct p.nomProf
from assignacions a, professors p, professors p1, assignacions
a1
where p.idprof=a.idprof and a.quadrimestre='Q122' and
      p1.idprof=a1.idprof and a1.idAssig=a.idAssig and
      a1.quadrimestre='Q122' and
      p1.codiDept<>p.codiDept
```

1.2 Doneu una sentència SQL per obtenir els codis dels departaments on algun dels seus professors no és responsable d'assignatura.

```
select distinct p.codidept
from professors p
where not exists (select *
                  from assignatures a
                  where p.idprof=a.profresponsable);
```

2. (2 punts) Donada la següent definició de vista:

```
CREATE VIEW AssignacionsAssignatura AS
      SELECT idAssig, quadrimestre, horesAssig
      FROM Assignacions
      WHERE quadrimestre = 'Q122'
WITH CHECK OPTION;
```

2.1 Digueu si la vista és o no actualitzable segons l'estàndard SQL. Raoneu la resposta**.**

SOLUCIÓ:

Per ser actualitzable cal que la vista:

- Faci SELECT sobre una única relació sense agregats ni distinct → OK
- No faci agrupacions → OK
- Els atributs seleccionats incloguin tots els atributs amb restricció NOT NULL → NO OK, donat que falta idProf que explícitament no és NOT NULL, però ho és implícitament pel fet de formar part de la clau primària.

Per tant NO ÉS ACTUALITZABLE

2.2 Suposeu que l'usuari A és el propietari de la taula `professors` i que s'executa la seqüència de sentències següent:

```
1- A: GRANT Insert, Select, Update ON Professors TO B WITH GRANT OPTION
2- B: GRANT Insert, Select, Update ON Professors TO C
3- A: GRANT Insert, Select, Update ON Professors TO D WITH GRANT OPTION
4- D: GRANT Update ON Professors TO C
5- A: GRANT Insert, Select, Update ON Professors TO E
6- A: REVOKE Insert, Select, Update ON Professors  FROM B CASCADE
7- A: REVOKE Insert, Select, Update ON Professors FROM D RESTRICT
8- E: GRANT Select, Update ON Professors TO C
```

2.2.1. Doneu el diagrama d'autoritzacions resultant després d'executar les sentències anteriors.

2.2.2. Suposat les autoritzacions representades en el diagrama anterior, digueu si l'usuari C podrà executar la sentència següent. Raoneu la resposta.

```
UPDATE Professors
SET sou = 1000
WHERE sou < 1000;
```

2.2.3. Suposant les autoritzacions representades en el diagrama anterior, digueu si l'usuari E podrà executar la sentència de l'apartat 2.2.2. Raoneu la resposta

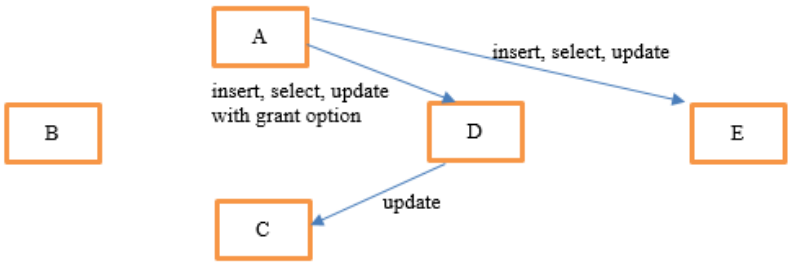
2.2.4. Suposeu ara que la sentència 7 queda substituïda per la que es dóna a continuació:

```
7- A: REVOKE GRANT OPTION ON Professors FROM D CASCADE
```

Digueu si amb aquest canvi l'usuari C podrà executar la sentència de l'apartat 2.2.2.

- En cas que pugui, doneu el nou diagrama d'autoritzacions tenint en compte la sentència substituïda, i raoneu la resposta.
- En cas que no pugui, indiqueu les sentències GRANT mínimes necessàries per a que l'usuari C la pugui executar.

2.2.1 El diagrama és:



2.2.2 L'usuari C no pot executar la sentència UPDATE perquè no té autorització a llegir el sou, i el necessita per la verificar la condició. La sentència 8 no té cap efecte perquè E no té autorització per atorgar permisos.

2.2.3 L'usuari E pot executar la sentència UPDATE perquè té permisos per SELECT i UPDATE.

2.2.4 L'usuari C no pot executar la sentència UPDATE perquè en substituir la sentència 7, el nou REVOKE fa que a l'usuari C li falta el privilegi de SELECT i, a més a més, la de UPDATE. L'únic usuari que li pot donar permisos és l'usuari A, per tant la sentència GRANT que faria falta:

```
A: GRANT Update, Select ON Professors TO C
```

3. (2 punts) Doneu una seqüència d'operacions en àlgebra relacional per obtenir l'identificador i nom dels professors que tenen un sou superior a 3000 i són responsables d'assignatures que han tingut 2 ó més professors diferents assignats en quadrimestres diferents.

```
A = assignacions[idProf, idAssig, quadrimestre]
B = A{idProf->idProfB, idAssig->idAssigB, quadrimestre->quadrimestreB}
C = A[idAssig = idAssigB, idProf <> idProfB,
      quadrimestre <> quadrimestreB]B
D = assignatures * C   // no fa join per professor, perquè els atributs
                       // tenen noms diferents
E = D[profResponsable]
F = professors(sou > 3000)
G = E[profResponsable = idProf]F
R = G[idProf, nomProf]
```

4. (2 punts)

4.1 Per cada una de les restriccions següents:

a) R1. Tots els professors que tenen assignacions a una mateixa assignatura han de pertànyer al mateix departament.

b) R2. Un professor només podrà consultar les seves assignacions.

c) R3. Al quadrimestre de primavera del curs 2022-2033 (quadrimestre='Q222') els professors han d'impartir entre 4 i 6 hores de docència per cada assignatura a la que estiguin assignats.

- Si es pot implementar com a restricció en la definició de taules, doneu el codi SQL que cal afegir a la sentència CREATE TABLE de la BD per implementar-la, indicant en quina taula s'ha d'afegir.
- Si no es pot, expliqueu per què, i indiqueu com es podria implementar.

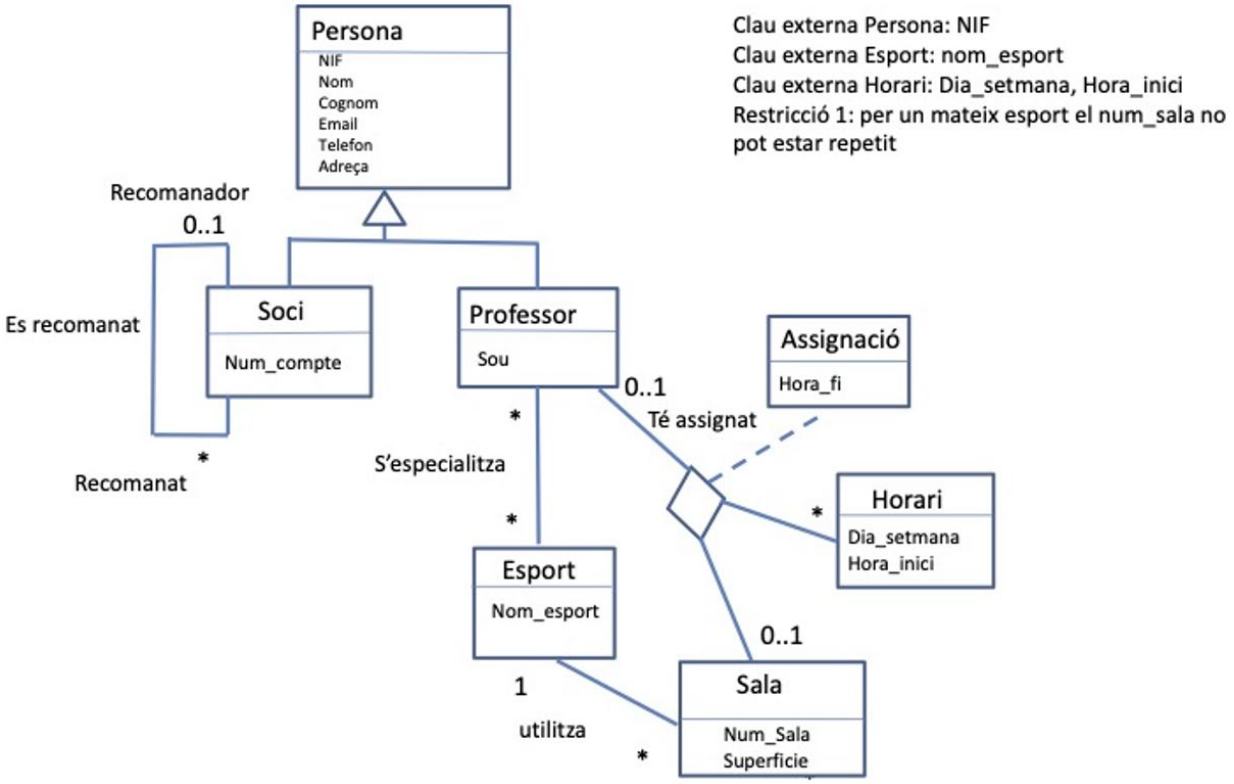
4.2 Doneu una asserció que comprovi que tots els professors de fora de Barcelona fan en total menys de 20 hores de classe per quadrimestre.

2.1

- R1. No es pot implementar com a restricció de taula o columna perquè el departament al que pertanyen els professors està a una altra taula. En SQL estàndard es podria fer una asserció i en Postgres aquesta restricció s'implementaria amb disparadors.
- R2.No es pot implementar com a restricció de taula o columna. S'haurien de crear vistes per a que cada professor només pugui consultar les seves assignacions i després l'administrador atorgar privilegis de consulta sobre la vista corresponent a cada professor.
- R3. Sí que es podria implementar com a restricció de taula. A la taula assignacions hauríem d'afegir un CHECK (quadrimestre<>'Q222' or (quadrimestre='Q222' and hores>=4 and hores<=6))

2.2 – CREATE ASSERTION sous_valids CHECK (
NOT EXISTS (SELECT *
FROM professors p natural inner join assignacions a
WHERE p.ciutatRes <> 'Barcelona'
GROUP BY a.idprof, a.quadrimestre
HAVING SUM(a.horesAssig) > 20))

5. (1.5 punts) Suposeu el model conceptual en UML següent. Doneu el disseny lògic que s'obté fent la traducció a model relacional. Cal incloure, taules, atributs, claus primàries, claus foranes i restriccions NOT NULL i UNIQUE que siguin necessàries.



persona(nif, nom, cognom, email, adreça, telèfon)

soci(nif, num_compte, nif_soci_recomanador)

{nif} referencia persona

{nif_soci_recomanador} referencia soci

professor(nif, sou)

{nif} referencia persona

esport(nom esport)

especialització (nif professor,nom esport)

{nif_professor} referencia professor

{nom_esport} referencia esport

horari(dia setmana,hora inici)

sala(nom esport,num sala, superficie)

{nom_esport} referencia esport

assignacions(dia setmana,hora inici,nom esport,num sala,
nif_professor,hora_fi)

{dia_setmana,hora_inici} referencia horari

{nom_esport,num_sala} referencia Sala

{nif_professor} referencia Professor

{nif_professor} not null

Unique (dia_setmana,hora_inici, nif_professor)

Temps: 3 hores

Notes 25 de gener. Revisió d'examen: 26 de gener a la tarda (presencial)

Cada pregunta s'ha de lliurar en un full separat

1. (1 punts) Donada la base de dades següent:

```
artistes (nom, genere, anyDebut, anyRetirada)
albums(titol, anyAlbum, artista)
     {artista} referencia artistes
```

Considereu el codi Java/JDBC següent que esborra artistes que compleixen certes condicions:

```
try {
   PreparedStatement s, s2;
   ResultSet r, r2;
   int num = 0;
   // c és la connexió a la BD
   // Seleccionem els artistes que es van retirar abans del 1960
   s = c.prepareStatement("select distinct nom from artistes "+
                          "where anyRetirada < 1960");
   r = s.executeQuery();
   // Preparem un statement s2 per comprovar si un artista té àlbums
   s2 = c.prepareStatement("select * from albums where artista = ?");
   while (r.next()) {
      // Per cada artista resultant de s, executem s2 per comprovar si té àlbums
      s2.setString(1, r.getString("nom"));
      r2 = s2.executeQuery();
      if (r2.next()) {
          // Té àlbums, no l'esborrem, fem rollback i acabem
          System.out.println("Error: Algun dels artistes a esborrar té àlbums");
          c.rollback();
          num = 0;
          break; }
      else {
          // No té àlbums, executem s3 per a esborrar-lo i seguim
          Statement s3 = c.createStatement();
          num = s3.executeUpdate("delete from artistes where nom = '" +
                                 r.getString("nom") + "'"); }
   }
   if (num > 0) System.out.println("Esborrats els artistes antics i sense àlbums");
   // TANCAMENT DE LES ESTRUCTURES, COMMIT I DESCONNEXIÓ
}
catch (SQLException se) {
      System.out.println("Excepció: " + se.getSQLState() + ":" + se.getMessage());
}
```

Proposeu una implementació alternativa a aquest fragment de codi, de manera **que tingui el mateix comportament**, i **que compleixi els criteris de qualitat** de l'assignatura. **Justifiqueu** els canvis que heu fet.

Recordeu que podeu usar els mètodes/codis d'excepció de JDBC estudiats a classe i que estan inclosos a continuació:

Statement

- `Statement createStatement();`
- `ResultSet executeQuery(String sql);`
- `int executeUpdate(String sql);`

PreparedStatement

- `PreparedStatement prepareStatement(String sql);`
- `ResultSet executeQuery();`
- `int executeUpdate();`
- `void setXXX(int posicio, XXX valor);`

ResultSet

- `boolean next();`
- `XXX getXXX(String nomColumna)`

SQLException

- `// 23502 -not_null_violation`
- `// 23503 -foreign_key_violation`
- `// 23505 -unique_violation`
- `// 23514 -check_violation`
- `String getSQLState();`

SOLUCIÓ:

Una solució que compleix tots els criteris seria:

```
try {
      // c és la connexió a la BD
      Statement s = c.createStatement();
      // Intentem esborrar els artistes que es van retirar abans del 1960
      // Si algún d'ells té un àlbum a la BD, saltarà una excepció de clau forana
      int num = s.executeUpdate("delete from artistes where anyRetirada < 1960");
      if (num>0) System.out.println("Els artistes antics i sense àlbums han estat
esborrats ");
      // TANCAMENT DE LES ESTRUCTURES, COMMIT I DESCONNEXIÓ
}
catch (SQLException se) {
      if(se.getSQLState().equals("23503"))
            System.out.println("Error: Algun dels artistes a esborrar té àlbums");
         else {
            System.out.println("Excepció: " + se.getSQLState() + ":"
                                          + se.getMessage());
      }
}
```

El que s'ha fet és:

- Eliminar el primer select (PreparedStatement s) perquè és innecessari: es pot afegir com a condició del delete. A més, té un distinct innecessari, i s'hauria de fer amb un Statement.
- Eliminar el segon select (PreparedStatement s2) perquè és innecessari: es pot detectar l'error capturant una excepció de violació de clau forana produïda pel delete.
- Com que es pot fer tot amb una sentència de delete que s'executarà una sola vegada (ja no tenim cap bucle), hem utilitzat un Statement. A la solució inicial, l'opció correcta hauria estat un PreparedStatement (si el bucle fos necessari).

2.(3 punts) Suposeu una base de dades amb les taules i contingut següents.

```
articles (idArticle, nom,    marca,      stock)
            (42,    'ps5', 'sony',      1)
            (43,    'ps4', 'nintendo',  100)

cistelles (idClient, idArticle, quantitat)
             (31,     42,         null)
             (55,     42,         null)
```

Considereu l'horari que apareix a continuació.

T1	T2	T3
	select * from articles where marca = 'sony';	
		select * from articles where idArticle = 42;
	update cistelles set quantitat = 1 where idClient = 31 and idArticle = 42;	
		update cistelles set quantitat = 1 where idClient = 55 and idArticle = 42;
	update articles set stock = stock - 1 where idArticle = 42;	
		select * from articles where idArticle = 42;
		commit
update articles set marca = 'sony';		
commit		
	abort	

2.1 Tenint en compte que el gränul és la fila (per identificar un gränul, utilitzeu la clau primària de la fila). Es demana:

a) Doneu l'horari en termes d'operacions de baix nivell sobre gränuls (no poseu operacions sobre el gränul IC, a no ser que hagueu vist que hi ha alguna interferència Fantasma a l'horari donat).

b) En cas que hi hagi interferències a l'horari, indiqueu, per cada interferència: nom de la interferència, transaccions implicades, gränuls afectats, i el nivell mínim d'aïllament per evitar-la. En cas que no n'hi hagi, indiqueu els horaris serials equivalents.

2.2 Suposeu ara que T1 treballa amb nivell d'aïllament REPEATEABLE READ, i T2 i T3 treballen amb READ COMMITTED.

a) Doneu l'horari anterior aplicant els nivell d'aïllament indicats. L'horari ha d'incloure, a més de les operacions que executen les transaccions (R, RU, W, COMMIT, ABORT), les operacions de petició i alliberament de reserves (LOCK, UNLOCK), i l'ordre d'execució de totes aquestes operacions. Quan més d'una transacció espera per un mateix gränul, considereu que la política de la cua és FIFO (First In, First Out).

b) Doneu el graf d'espera just abans de l'abort de T2.

2.3 En cas que hi hagi interferències a l'horari donat a l'apartat 2.2, cal que indiqueu per cada interferència: nom de la interferència, transaccions implicades, grànuls afectats. En cas que no n'hi hagi, indiqueu els horaris serials equivalents.

SOLUCIÓ 2.1

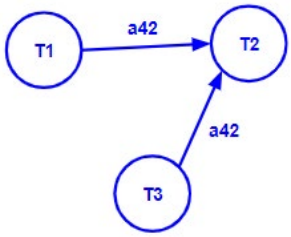
T1	T2	T3
	R(a42)	
		R(a42)
	RU(c3142)	
	W(c3142)	
		RU(c5542)
		W(c5542)
	RU(a42)	
	W(a42)	
		R(a42)
		commit
RU(a42)		
W(a42)		
RU(a43)		
W(a43)		
commit		
	abort	

Sí, existeix tres interferències:

- Entre T1 i T2 lectura no confirmada sobre a42. Nivell d'aïllament: Read Committed.
- Entre T2 i T3 lectura no repetible sobre a42. Nivell d'aïllament: Repeteable Read.
- Entre T2 i T3 lectura no confirmada sobre a42. Nivell d'aïllament: Read Committed.

SOLUCIÓ 2.2

T1 Repeateable Read	T2 Read Committed	T3 Read Committed
	L(a42, S)	
	R(a42)	
	U(a42)	
		L(a42, S)
		R(a42)
		U(a42)
	L(c3142, X)	
	RU(c3142)	
	W(c3142)	
		L(c5542, X)
		RU(c5542)
		W(c5542)
	L(a42, X)	
	RU(a42)	
	W(a42)	
		L(a42, S)
LOCK(a42, X)		
	ABORT+U(c3142, a42)	
		R(a42)
		U(a42)
		COMMIT+U(c5542, a42)
RU(a42)		
W(a42)		
L(a43, X)		
RU(a43)		
W(a43)		
COMMIT+U(a42, a43)		



SOLUCIÓ 2.3

No hi ha cap interferència.

- Les de lectura no confirmada s'han evitat pels nivells d'aïllament usats en les transaccions.
- La de lectura no repetible no s'evita amb Read Committed, però s'evita circumstancialment degut a l'Abort.

Els horaris serials equivalents són: T3;T1

3.(3 punts) Considereu la taula `Empleats(id, nom, ciutat)`. Hi ha un índex arbre B+ agrupat per `id` i dos índexs arbre B+ no agrupats, un per `nom` i un per `ciutat`. La taula té 100.000 tuples, i els id dels empleats estan repartits uniformement entre 1 i 100.000. Hi ha 1.000 empleats de Barcelona, i d'aquests 200 tenen el id més gran que 20.000. Els índexs són tots d'ordre 50, estan ocupats en mitjana al 80%, i a cada pàgina de dades hi ha 5 tuples.

3.1 Donada la consulta següent, calculeu (mostreu i justifiqueu clarament els càlculs) el cost en nombre de pàgines (d'índex i de dades) que es llegiran si la consulta es resol:

a) usant l'índex per `id`
b) usant l'índex per `ciutat`
c) usant els 2 índexs alhora

```
SELECT *
FROM Empleats e
WHERE e.id > 20000 AND e.ciutat ='Barcelona'
```

3.2 Considereu ara la consulta següent. Quin hauria de ser el valor de X per tal de que la consulta resolta usant l'índex per `id` tingui un cost menor que resolent la consulta amb un recorregut seqüencial? Mostreu i justifiqueu clarament els càlculs que heu fet per obtenir el valor de X.

```
SELECT *
FROM Empleats e
WHERE e.id > X AND e.ciutat ='Barcelona'
```

SOLUCIÓ:

3.1)

a) $n = 2*d*\text{ocupació} = 2*50*0,8 = 80$ valors per node
$h = \lceil \log_{81} 100000 \rceil = 3$
$D = 80000/5 = 16000$

Cost total = 3 + 16000

b)

n = 2*d*ocupació = 2*50*0,8 = 80 valors per node
h = ⌈ $\log_{81}$ 100000 ⌉ = 3
F = ⌈ valors a llegir en els nodes fulla / 80⌉ = ⌈ 1000/80 ⌉ nodes fulla = 13

Cost total = 3 + 12+ 1000 = 1015 pàgines

c)

n = 2*d*ocupació = 2*50*0,8 = 80 valors per node
h = ⌈ $\log_{81}$ 100000 ⌉ = 3
F = id a llegir en els nodes fulla / 80 = 80000/80 nodes fulla = 1000

Cost índex per id = h + (F-1) = 3 + 999 = 1002
Cost índex per ciutat = h + (F-1) = 3 + 12 = 15 (aquesta F s'ha calculat a 3.1.b)

Cost total: 15 + 1002 +200 pàgines

3.2)

Cost Rec. Seq: 100000/5 = 20000 pàgines

Índex per id: 3 + ⌈ (100000- X)/5 ⌉

$$3 + \lceil (100000 - X)/5 \rceil < 20000$$

Qualsevol X més gran que 20 farà que la consulta amb índex tingui un cost menor (es considera també correcta la solució X més gran que 15 a la que s'arriba si no es considera l'arrodoniment).

4.(3 punts) Considereu una base de dades amb les taules següents. Per simplificar, suposeu que les dates estan implementades com a `integer`.

```
clients (dni, nom, telèfon, mail)
tipusHabitacions (idHotel, idTipus, descripció)
        -- hi ha una fila a la taula per cada tipus d'habitació
        que hi ha en un hotel
reserves (dni, dataIni, dataFi, idHotel, idTipus)
        {idHotel, idTipus} referencia tipusHabitacions
        {dni} referencia clients
        {dataFi} is not null
        -- les reserves es fan d'un tipus d'habitació d'un hotel
        des d'una dataIni fins a la dataFi. Per exemple hi pot
        haver una reserva d'una habitació doble a l'hotel Ritz
        entre les dates 4 i 8 feta pel client amb dni 333.
        -- per simplificar suposarem que les dates estan
        implementades com un número enter. Per exemple, una
```

```
        reserva entre les dates 4 i 8, ocuparà una habitació en
        les dates 4,5,6,7 i 8.
ocupacioHotels (idHotel, idTipus, data, reservades)
        {idHotel, idTipus} referencia tipusHabitacions
        check (reservades>=1)
        -- l'atribut reservades indica quantes habitacions del
        tipus idTipus de l'hotel idHotel estan reservades en una
        data concreta.
        -- hi ha una fila a la taula ocupacioHotels quan hi ha
        una o més reserves d'habitacions d'un tipus en un hotel
        i una data. Per exemple, hi haurà una fila si en la data
        7 hi ha 5 habitacions dobles reservades a l'hotel Ritz.
        Però no hi haurà cap fila corresponent a l'hotel Ritz,
        habitació individual, i data 10, si per aquest hotel,
        tipus d'habitació i data no hi ha cap habitació
        reservada.
```

Implementeu amb disparadors el comportament següent que **s'ha de fer complir quan s'insereix una reserva a la taula reserves**:

- No hi pot haver dues reserves solapades en dates fetes per un mateix client. En cas que no es compleixi caldrà generar una excepció. Dues reserves estan solapades si en els intervals entre la seva data d'inici i data fi, s'inclou una mateixa data (per exemple, la reserva ('dni1', 10, 20, 'hotel1', 'individual') està solapada amb la reserva ('dni1', 18, 25, 'hotel2', 'individual') concretament en les dates 18, 19 i 20).
- Hi ha d'haver una fila a la taula ocupacióHotels per un hotel, tipus d'habitació i data a partir de que hi hagi hi ha una o més reserves fetes d'una habitació del tipus, a l'hotel, en la data.
- El valor de l'atribut derivat *reservades*, de la taula *ocupacioHotels*, ha de correspondre a la quantitat d'habitacions del tipus que hi ha reservades en l'hotel en aquella data.

Podeu fer servir la plantilla de triggers següent.

```
CREATE OR REPLACE FUNCTION actualitza_lloger() RETURNS TRIGGER AS $$
DECLARE
  ... ...
BEGIN
  ... ...
EXCEPTION
  WHEN { raise_exception | unique_violation | foreign_key_violation | check_violation | others }
  THEN
   SELECT texte INTO missatge FROM missatgesExcepcions WHERE num=?;
```

```
    RAISE EXCEPTION '%', missatge;
   ...
END;
$$ LANGUAGE PLPGSQL;

CREATE TRIGGER disparador {BEFORE | AFTER} {esdeveniment[OR ...]} ON taula
FOR [EACH] {ROW | STATEMENT} EXECUTE PROCEDURE actualitza_lloger();
```

SOLUCIÓ:

```
CREATE OR REPLACE FUNCTION novaReservaBef() RETURNS TRIGGER AS $$
BEGIN
    if (exists(select * from reserves
               where new.dni = dni and
                    ((new.dataIni >=dataini and new.dataIni<=dataFi)
                  or (new.dataFi >=dataIni and new.dataFi<=dataFi)
                  or (new.dataIni <=dataIni and new.dataFi>=dataFi))))
      then raise exception 'reserva solapada';
     end if;
     return new;
END;
$$ LANGUAGE PLPGSQL;

CREATE TRIGGER disparadorBef BEFORE INSERT ON reserves
FOR EACH ROW EXECUTE PROCEDURE novaReservaBef();

CREATE OR REPLACE FUNCTION novaReservaAft() RETURNS TRIGGER AS $$
DECLARE
    dataA integer;
BEGIN
    for dataA in new.dataIni..new.dataFi
      loop
            update ocupacioHotels
            set ocupades = ocupades + 1
            where idHotel = new.idHotel
                  and idTipus = new.idTipus
                  and data = dataA;
            if not found then
                  insert into ocupacioHotels
                  values (new.idHotel,new.idTipus,dataA,1);
            end if;
      end loop;
    return null;
END;
$$ LANGUAGE PLPGSQL;

CREATE TRIGGER disparadorAft AFTER INSERT ON reserves
FOR EACH ROW EXECUTE PROCEDURE novaReservaAft();
```

BD **24.4.2023**

Temps: 2 hores

Notes 9 maig tarda **Revisió: 10 maig al mat** **Cada pregunta en un full separat**

Considereu l'esquema de la base de dades següent:

```
create table professors
(dni char(9),
nomProf char(50) unique,
telefon char(15),
sou integer not null check(sou>0),
primary key (dni));

create table despatxos
(modul char(5),
numero char(5),
superficie integer not null check(superficie>12),
primary key (modul,numero));

create table assignacions
(dni char(9),
modul char(5),
numero char(5),
instantInici integer,
instantFi integer,
primary key (dni, modul, numero, instantInici),
foreign key (dni) references professors,
foreign key (modul,numero) references despatxos);
-- assignació d'un professor a un despatx en un periode que va des de
l'instantiInici a l'instantFi
-- instantFi te valor null quan una assignacio es encara vigent.
```

1. **(2,5 punts)** Escriviu una sentència SQL per obtenir els despatxos que només han tingut un únic professor assignat amb un sou superior a 1000 i que, a més, el mateix professor ha estat assignat al despatx en 3 o més períodes diferents.
La sentència ha de mostrar l'identificador del despatx, el nom del professor i la quantitat de vegades que el professor ha estat assignat al despatx.

```
select a.modul, a.numero, p.nomProf, count(*) as numVegades
from assignacions a join professors p on a.dni = p.dni
where p.sou > 1000 and
not exists (
           select *
           from assignacions a2 join professors p2 on a2.dni = p2.dni
           where p2.sou > 1000 and
                 a2.modul = a.modul and a2.numero = a.numero and
                 a2.dni != a.dni
           )
group by a.modul, a.numero, p.nomProf
having count(*) >= 3
```

2. **(2,5 punts)** Escriviu una seqüència d'operacions d'àlgebra relacional per obtenir el dni dels professors que tenen dues o més assignacions en despatxos que estan al mateix mòdul però tenen diferent número, o no tenen assignacions a despatxos més grans de 15 metres quadrats.

```
A = assignacions[dni, modul, numero]
B = assignacions[dni, modul, numero]
C = B{ dni-> dni2, modul-> modul2, numero->numero2}
D = C[dni = dni2, modul=modul2, numero !=numero2]A
E=D[dni]
F=despatxos(superfície>15)
G=F*assignacions
H=G[dni]
I=professors[dni]
J=I-H
R=E_u_J
```

3. **(2,5 punts)**

3.1 Definir en SQL estàndard una asserció per garantir que no hi ha cap mòdul que tingui 5 o més assignacions d'un mateix professor a despatxos diferents.

```
CREATE ASSERTION no-mes-de-dos CHECK(
       NOT EXISTS(SELECT a.modul
             FROM assignacions a
             GROUP BY a.dni, a.modul
             HAVING count(distinct a.numero)>=5));
```

3.2 Considereu les vistes i la consulta SQL següents:

```
CREATE VIEW personalActualOmega AS
  SELECT dni, modul, numero
  FROM assignacions
  WHERE instantFi IS NULL AND modul='Omega';

CREATE VIEW dadesPersonalActualOmega (nomProf, modul, numero,
telefon) AS
  SELECT  p.nomProf, pa.modul, pa.numero, p.telefon
  FROM professors p, personalactualOmega pa
  WHERE p.dni=pa.dni;

SELECT d1.nomprof, d2.nomprof
FROM dadesPersonalActualOmega d1,
            dadesPersonalActualOmega d2
WHERE d1.modul=d2.modul AND
      d1.numero = d2.numero AND
      d1.nomprof <> d2.nomprof;
```

a) Explica breument què retorna la consulta.

S'obtenen les parelles de professors que tenen assignacions vigents a un mateix despatx del mòdul Omega.

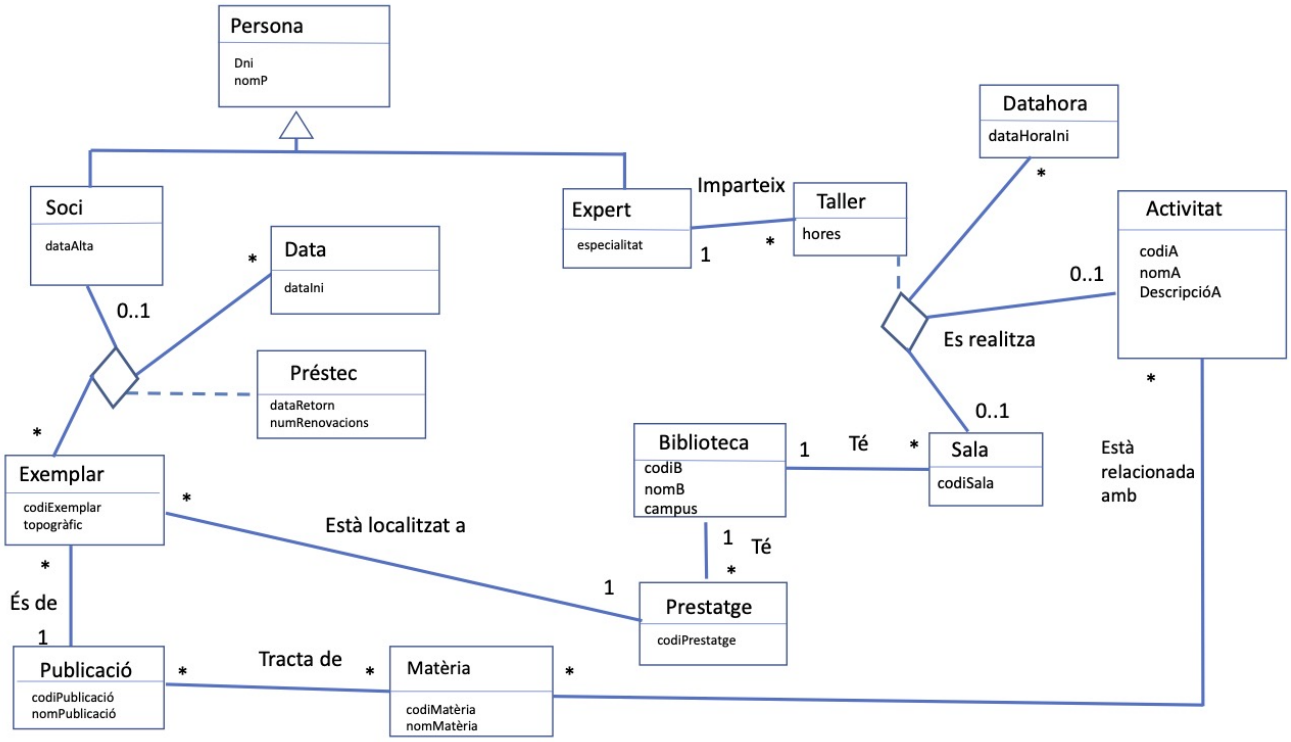
b) Doneu una sentència SQL definida sobre les taules de la base de dades, que compleixi els criteris de qualitat establerts a l'assignatura, i que retorni el mateix resultat que la consulta donada.

```
SELECT p1.nomProf, P2.nomprof
FROM professors p1, assignacions a1,
     professors p2, assignacions a2
WHERE p1.dni= a1.dni and p2.dni= a2.dni AND
     a1.instantFi IS NULL and a2.instantFi IS NULL AND
     a1.modul='Omega' and a2.modul='Omega' AND
     a1.numero=a2.numero and p1.nomprof<>p2.nomprof;
```

c) Expliqueu breument quins avantatges pot aportar el fet de fer definir la consulta sobre les vistes, en lloc de definir-la sobre les taules bàsiques.

Independència lògica de dades, simplificació de consultes i programes.

4. (2,5 punts) Donat el model conceptual següent en UML:



Clau externa **Biblioteca:** codiB
Clau externa **Sala:** No poden existir dues sales amb el mateix codiSala a la mateixa biblioteca, però sí a biblioteques diferents.
Clau externa **Prestatge:** No poden existir dos prestatges amb el mateix codiPrestatge a una mateixa biblioteca, però sí a biblioteques diferents.
Clau externa **Publicació:** codiPublicació
Clau externa **Exemplar:** codiExemplar
Clau externa **Matèria:** codiMatèria
Clau externa **Persona:** dni
Clau externa **Data:** dataIni
Clau externa **DataHora:** dataHoraIni
Clau externa **Activitat:** codiA

Es demana fer la traducció a model relacional donant el disseny lògic de la base de dades resultant. Concretament cal indicar:

- taules
- atributs
- claus primàries
- claus foranes
- restriccions NOT NULL per a les claus foranes que ho requereixin
- restriccions UNIQUE en cas d'existir claus alternatives

Solució:

```
Biblioteca (codiB, nomB, campus)
Sala(codiSala,codiB)
    {codiB} Referencia Biblioteca
Prestatge(codiPrestatge, codiB)
    {codiB} Refernencia Biblioteca
Publicació(codiPublicació, numPublicació)
Exemplar(codiE, topogràfic, codiPublicació, codiB,codiPrestatge)
    {codiPublicació} Referencia Publicació NOT NULL
    {codiB,codiPrestatge} Referencia Prestatge NOT NULL
Matèria(codiMatèria, nomMatèria)
TractaDe(codiPublicació,codiMatèria)
    {codiPublicació} Referencia Publicació
    {codiMatèria} Referencia Matèria
Persona(dni,nomP)
Soci(dni, dataAlta)
    {dni} Referencia Persona
Expert(dni, especialitat)
    {dni} Referencia Persona
Prèstec(codiE,dniSoci,dataIni,dataRetorn,numRenovacions)
    {codiE} Referencia Exemplar
    {dniSoci}Referencia Soci NOT NULL
Activitat(codiA,nomA,descripcióA)
RelacionadaAmb(codiA,codiMateria)
    {codiA} Referencia Activitat
    {codiMatèria} Referencia Matèria
RealitzaTaller(codiB,codiSala,codiA,dataHoraIni,hores, dniExpert)
    {codiB,codiSala} Referencia Sala
    {codiA} Referencia Activitat NOT NULL
    (dataHoraIni,codiA) UNIQUE NOT NULL
    {dniExpert} Referencia Expert NOT NULL
```

Temps: 3 hores

Notes 27 de juny. Revisió d'examen: 29 de juny a la tarda (presencial)

Cada pregunta s'ha de lliurar en un full separat

1. (1 punts) Donada la base de dades següent:

```
create table professors
(dni char(50),
nomProf char(50) unique,
sou integer check (sou<=2000),
primary key (dni));

create table despatxos
(modul char(5),
numero char(5),
superficie integer not null check(superficie <=25),
primary key (modul,numero));

create table assignacions
(dni char(50),
modul char(5),
numero char(5),
instantInici integer,
instantFi integer,
primary key (dni, modul, numero, instantInici),
foreign key (dni) references professors,
foreign key (modul,numero) references despatxos,
check (instantInici < instantFi));
```

Considereu el següent fragment de codi Java/JDBC. Implementeu el cos del programa per tal que per cada professor de la base de dades que tingui assignacions a un despatx amb una superfície més gran que 20 que no sigui del mòdul Omega, s'augmenti el seu sou en 100. Cal que el programa tregui un missatge d'excepció si el sou d'algun professor passa a ser superior a 2000. En cas que no hi hagi cap excepció el programa indicarà el número de professors modificats i si no se n'ha modificat cap, es mostrarà un text que ho indiqui.

```
try {
 /* Tenim ja una connexió c establerta amb la base de dades */

 //codi a implementar

 c.commit();
 }
catch (ClassNotFoundException ce) {
        System.out.println ("Error al carregar el driver");}
catch (SQLException se) {
        System.out.println ("Excepcio: "+ se.getMessage());}
```

Recordeu que podeu usar els mètodes/codis d'excepció de JDBC estudiats a classe:

Statement

- `Statement createStatement();`
- `ResultSet executeQuery(String sql);`
- `int executeUpdate(String sql);`

PreparedStatement

- `PreparedStatement prepareStatement(String sql);`
- `ResultSet executeQuery();`
- `int executeUpdate();`
- `void setXXX(int posicio, XXX valor);`

ResultSet

- `boolean next();`
- `XXX getXXX(String nomColumna)`

SQLException

- `// 23502 -not_null_violation`
- `// 23503 -foreign_key_violation`
- `// 23505 -unique_violation`
- `// 23514 -check_violation`
- `String getSQLState();`

SOLUCIÓ:

```
try {
    /* Tenim ja una connexió c establerta amb la base de dades */
    Statement st = c.createStatement();
    int numP = st.executeUpdate("UPDATE professors "+
      "SET sou = sou + 100 "+
      "WHERE exists (SELECT *"+
            "FROM assignacions a, despatxos d "+
            "WHERE a.dni = professors.dni "+
                 "AND a.modul = d.modul AND a.numero = d.numero "+
                 "AND d.superficie > 20 AND d.modul != 'Omega')");
    if (numP==0)
      {System.out.println("Error");}
    else
      {System.out.println("S'han incrementat "+numP)+" sous;}
    c.commit();
    }
catch (ClassNotFoundException ce) {
    System.out.println ("Error al carregar el driver");}
catch (SQLException se) {
   if(se.getSQLState().equals("23514"))
        System.out.println("La sou no pot ser més gran de 2000");
    else
        System.out.println ("Excepcio: "+ se.getMessage());} }
```

2. (3 punts)

2.1 Considereu la transacció següent: T1: RU(A);W(A);R(A);COMMIT;

a) Justifiqueu si pot existir un horari on intervingui només aquesta transacció i sigui NO serialitzable

b) Justifiqueu si pot existir un horari on intervingui només aquesta transacció i sigui NO recuperable

SOLUCIÓ

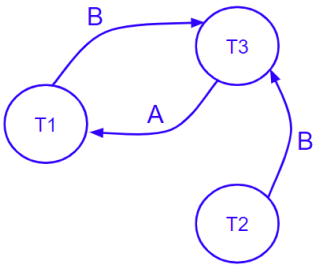
No poden existir. Veure definicions de serialitzable i recuperable en les transparències.

2.2 Considereu l'horari següent:

a) Indiqueu si és serialitzable, justificant breument la resposta a partir del graf de precedències corresponent a l'horari.

b) Indiqueu si es produeix alguna interferència. En cas afirmatiu, digueu quines són, sobre quins gràNul i quin és el nivell mínim d'aïllament que podria evitar cadascuna d'elles. En cas negatiu, justifiqueu la resposta.

Temps	T1	T2	T3
10		R(E)	
20	RU(A)		
30			
40			
50			R(A)
60	R(B)		
70			RU(B)
80	W(A)		
90		R(B)	
100			W(B)
110		RU(C)	
120		W(C)	
130		COMMIT	
140	RU(A)		
150	W(A)		
160	COMMIT		
170			
180			COMMIT
190			



a) No és serialitzable atès que es produeix un cicle en el graf de precedències.

b) Es produeix una interferència:

- Entre T1 i T3 es produeix una interferència d'anàlisi inconsistent sobre els gràNuls A i B (op. 50, 80, 100, 160). El nivell mínim d'aïllament que l'evitaria seria REPETEABLE READ.

2.3 Suposeu ara que sobre l'horari de l'apartat anterior, s'incorpora un mecanisme de control de concurrència basat en reserves S, X i que les transaccions T1 i T3 treballen a un nivell d'aillament REPEATABLE READ i T2 ho fa a un nivell d'aïllament READ COMMITED.

a) Doneu l'horari resultant

b) Doneu els horaris serials equivalents a l'horari resultant? Si no n'hi ha cap, justifiqueu la resposta.

Temps	T1	T2	T3
10		LOCK(E,S)	
20		R(E)	
30		UNLOCK(E)	
40	LOCK(A,X)		
50	RU(A)		
60			
70			LOCK(A,S)
80	LOCK(B,S)		
90	R(B)		
100	W(A)		
110		LOCK(B,S)	
120		R(B)	
130		UNLOCK(B)	
140		LOCK(C,X)	
150		RU(C)	
170		W(C)	
180		COMMIT + U(C)	
190	RU(A)		
200	W(A)		
210	COMMIT + U(A,B)		
220			R(A)
230			LOCK(B,X)
240			RU(B)
260			W(B)
270			COMMIT + U(A,B)

b) Hi ha dos horaris serials equivalents: T1;T2;T3 i T2;T1;T3

2.4 Suposeu l'horari resultant de l'apartat anterior. Considereu si és o no possible afegir una transacció T4 que provoqui una lectura no confirmada sobre el gràNul A amb la transacció T1. Si es pot produir la interferència, indiqueu el nivell d'aïllament de la nova transacció T4 i doneu l'horari resultant. Si no és possible justifiqueu la resposta.

L'únic nivell d'aïllament possible per T4 per tal que hi hagi una interferència de lectura no confirmada seria el READ UNCOMMITTED.

Temps	T1	T2	T3	T4
10		L(E,S)		
20		R(E)		
30		U(E)		
40	L(A,X)			
50	RU(A)			
60				
70			L(A,S)	
80	L(B,S)			
90	R(B)			
100	W(A)			
				R(A)
				C
110		L(B,S)		
120		R(B)		
130		U(B)		
140		L(C,X)		
150		RU(C)		
170		W(C)		
180		C + U(C)		
190	RU(A)			
200	W(A)			
210	C + U(A,B)			
La resta de files ja no canvien i no són rellevants per a la lectura no confirmada				

3. (3 punts) Considereu l'esquema de la base de dades següent:

```
CREATE TABLE lots
       (idLot INT primary key,
       preuLot REAL,
       quantsProductes INT,
       check (preuLot < 100 or quantsProductes > 3));

CREATE TABLE productes
       (idProd INT primary key,
       preuProd REAL NOT NULL,
       idLot INT NOT NULL references lots);

CREATE FUNCTION incrPreuPrBef() RETURNS trigger AS $$
 BEGIN
   if (TG_OP = 'INSERT' and NEW.preuprod<50) then
            NEW.preuprod := NEW.preuprod+5;
            RETURN NEW;
   end if;
   RETURN NULL;
 END; $$ LANGUAGE plpgsql;

 CREATE FUNCTION incrPreuPrAft() RETURNS trigger AS $$
 BEGIN
   if (TG_OP = 'INSERT') then
       update lots
       set preuLot = preuLot + NEW.preuProd,
            quantsProductes = quantsProductes + 1
       where idLot = NEW.idLot;
  else
       update lots
       set preuLot = preuLot + (NEW.preuProd-OLD.preuProd)
       where idLot = NEW.idLot;
   end if;
 RETURN NULL;
 END; $$ LANGUAGE plpgsql;

 CREATE TRIGGER incrPreuTrBef
 BEFORE INSERT OR UPDATE OF preuProd ON productes
 FOR EACH ROW EXECUTE procedure incrPreuPrBef();

 CREATE TRIGGER incrPreuTrAft
 AFTER INSERT OR UPDATE OF preuProd ON productes
 FOR EACH ROW EXECUTE procedure incrPreuPrAft();
```

3.1 Suposeu que s'executen una a una les sentències SQL següents (en cada apartat - a i b - partint de la base de dades buida). Indiqueu quin serà el resultat de les sentències ***select * from lots; select * from productes;*** en el cas de ser afegides i executades just després de cadascun dels inserts de cadascun dels apartats. Justifiqueu breument la resposta, explicant les accions que fa el SGBD a conseqüència de cada inserció.

a)
```
insert into lots values (1,0,0);
insert into productes values(1,30,1);
insert into productes values(2,50,1);
insert into productes values(3,40,1);
insert into productes values(4,35,1);
```

b)
```
begin transaction;
insert into lots values (1,0,0);
insert into productes values(1,30,1);
insert into productes values(2,50,1);
insert into productes values(3,40,1);
insert into productes values(4,35,1);
commit;
```

3.2 Doneu una implementació equivalent al trigger after anterior, amb la mateixa funcionalitat, i que sigui For Each Statement. Justifiqueu quina de les dues maneres d'implementar el trigger s'adequa més als criteris de qualitat de l'assignatura.

3.1.a

Sentència que s'executa	Lots	Productes	Accions del SGBD
insert into lots values (1,0,0);	(1,0,0)		Només s'executa el insert
insert into productes values(1,30,1);	(1,35,1)	(1,35,1)	El trigger before canvia el preu de producte que s'insereix. S'executa l'insert del producte. El trigger after causa que es modifiqui el preu del lot al que pertany el producte.
insert into productes values(2,50,1);	(1,35,1)	(1,35,1)	El trigger before retorna null, i per tant no s'acaba fent la inserció del producte ni tampoc s'executa el trigger after amb la modificació del preu del lot
insert into productes values(3,40,1);	(1,80,2)	(1,35,1) (3,45,1)	El trigger before canvia el preu de producte que s'insereix. S'executa l'insert del producte. El trigger after causa que es modifiqui el preu del lot al que pertany el producte.
insert into productes values(4,35,1);	(1,80,2)	(1,35,1) (3,45,1)	El trigger before canvia el preu de producte que s'insereix. S'executa l'insert del producte. El trigger after causa que es modifiqui el preu En modificar el preu del lot es dóna error de c perquè la fila 1 de lots tindrà el preu més gra no tindrà més de 3 productes.

3.1.b

Sentència que s'executa	Lots	Productes	Accions del SGBD
begin transaction; insert into lots values (1,0,0);	(1,0,0)		S'inicia la transacció i només s'executa el insert de lot.
insert into productes values(1,30,1);	(1,35,1)	(1,35,1)	El trigger before canvia el preu de producte que s'insereix. S'executa l'insert del producte. El trigger after causa que es modifiqui el preu al que pertany el producte.
insert into productes values(2,50,1);	(1,35,2)	(1,35,1)	El trigger before retorna null, i per tant no s'acaba fent la inserció del producte ni tampoc s'executa el trigger after amb la modificació del preu del lot
insert into productes values(3,40,1);	(1,80,2)	(1,35,1) (3,45,1)	El trigger before canvia el preu de producte que s'insereix. S'executa l'insert del producte. El trigger after causa que es modifiqui el preu del lot al que pertany el producte
insert into productes values(3,40,1);			El trigger before canvia el preu de producte que s'insereix. S'executa l'insert del producte. El trigger after causa que es modifiqui el preu del lot, En modificar el preu del lot es dóna error de check perquè la fila 1 de lots tindrà el preu més gran que 100 i no tindrà més de 3 productes. Al ser dins d'una transacció es torna a l'estat abans de l'inici de la transacció.

3.2

```
CREATE FUNCTION incrPreuPrAft() RETURNS trigger AS $$
 BEGIN
       update lots
       set quantsProductes = select count(*)
                             from productes p
                             where p.idLot = lots.idLot,
            preuLot = select sum(p.preuProd)
                      from productes p
                      where p.idLot = lots.idLot;
       RETURN NULL;
 END; $$ LANGUAGE plpgsql;

 CREATE TRIGGER incrPreuTr
 AFTER INSERT OR UPDATE OF preuProd ON productes
 FOR EACH STATEMENT EXECUTE procedure incrPreuPrAft();
```

- Aquesta solució no és incremental, ja que ha de modificar el preu i la quantitat de productes **de tots els lots**, per tant és pitjor que la For Each Row que només modifica el preu dels lots que tenen productes que han canviat de preu.

4. (3 punts) Considereu la taula `R(a,b,c,d)`. L'atribut `a` és clau primària i sobre l'atribut `c` hi ha definida una restricció unique. La taula té 500.000 tuples. Hi ha un índex definit sobre l'atribut `a`, d'ordre 125, 80% ocupació en mitjana, i a cada pàgina de dades hi ha 5 tuples.

Mostreu clarament com heu fet els càlculs que es demanen als apartats següents.

4.1 Quina és la quantitat de pàgines necessàries per emmagatzemar l'índex i la taula?

Taula: 500000/5 = 100000 pàgines de dades

Índex: 2d* 0.8 = 200

fulles: 500000/200 = 2500

Nivell 1: 2500/201 = 13

Nivell 2: 13/201 = 1

2514 pagines d'índex

4.2 Suposeu que hi ha 4000 tuples que compleixen la condició de select següent. Quin seria el cost de resoldre la consulta si l'índex fos no agrupat? I si fos agrupat?

```
SELECT *
FROM R
WHERE a >= 800000
```

Cost no agrupat = h + (F-1) + D

h = $\lceil \log_{201} 500000 \rceil = 3$

F – 1= 4000/200 -1 = 19

Cost = 3 + 19+ 4000

Cost agrupat = h + D

D= 4000/5 = 800

Cost= 3 + 800

4.3 Considereu ara la següent expressió SQL, i calculeu el cost en els dos casos anteriors, és a dir, si l'índex fos agrupat o no agrupat. Justifiqueu la resposta.

```
Exists (SELECT a
        FROM R
        WHERE a >= 800000 )
```

No agrupat : h =3

Agrupat: h = 3

No és necessari accedir a la taula per resoldre la consulta, n'hi ha prou amb saber si hi ha algun valor que compleix la condició.

4.4 Considereu ara la següent expressió SQL:

```
Exists (SELECT a,c
        FROM R
        WHERE a >= 800000 and c = 10)
```

I suposeu que hi ha un índex agrupat per l'atribut `a`, un índex no agrupat per l'atribut `c`, que tots dos índexs tenen ordre d=125 i 80% d'ocupació.

Quin mètode usaríeu per resoldre la consulta? Justifiqueu la resposta mostrant que el mètode escollit és el de menor cost.

El mètode millor seria accedir per l'índex de c, recuperar la única tupla que compliria la condició (hi ha una restricció UNIQUE per aquest atribut) i accedir al fitxer de dades per comprovar la condició sobre l'atribut a.

- $Cost_c$: h +1

Els altres mètodes tindrien sempre un cost igual o superior. En el cas pitjor:

- índex per a = h + D = h + 4000 /5
- intersecció RIDs = h_a + 19 + h_b + (1-1)

Temps: 2 hores

Notes 21 novembre tarda **Revisió: 22 novembre tarda** **Cada pregunta en un full separat**

1. **(2,5 punts)** Considereu l'esquema de la base de dades següent:

```
CREATE TABLE usuaris
(nomUsuari char(20),
presentacio char(500),
tarifa char(7) check(tarifa IN ('premium','free')) NOT NULL,
reproduccions integer check(reproduccions >=0) NOT NULL,
PRIMARY KEY (nomUsuari));

CREATE TABLE seguidors
(nomSeguit char(20),
nomSeguidor char(20),
silenciat integer CHECK (silenciat IN (0,1)) DEFAULT 1,
PRIMARY KEY (nomSeguit,nomSeguidor),
FOREIGN KEY (nomSeguit) REFERENCES usuaris,
FOREIGN KEY (nomSeguidor) REFERENCES usuaris);
-- un seguidor està silenciat si l'atribut té valor 0

CREATE TABLE playlist
(idList integer,
nomList char(100) NOT NULL,
tema char(50),
creador char(20) NOT NULL,
PRIMARY KEY (idList),
FOREIGN KEY(creador) REFERENCES usuaris);

CREATE TABLE cançons
(idCanço integer,
titol char(100),
cantant char(100),
numTotalReproduccions integer CHECK (numTotalReproduccions>=0),
PRIMARY KEY (idCanço));

CREATE TABLE cançonsAPlaylist
(idList integer,
idCanço integer,
numReproduccions integer CHECK (numReproduccions>=0),
PRIMARY KEY (idList,idCanço),
FOREIGN KEY (idList) REFERENCES playlist,
FOREIGN KEY (idcanço) REFERENCES cançons);
```

1.1. Escriviu una sentència SQL per obtenir l'identificador i el nom de les playlist que tinguin més de cinc cançons d'un mateix cantant i que el seu creador no tingui cap seguidor silenciat.

```
select distinct p.idlist, p.nomlist
from playlist p natural inner join cançonsAPlaylist cp
                natural inner join cançons c
where not exists(select * from seguidors s
                 where s.nomSeguit=p.creador and
                       s.silenciat=0)
group by p.idlist, c.cantant
having count(*)>5;
```

1.2. Escriviu una sentència SQL per obtenir el nom dels usuaris que segueixen a algun usuari que no els segueix a ells.

```
select distinct s.nomSeguidor
from seguidors s
where not exists (select *
                  from seguidors s2
                  where s.nomSeguidor=s2.nomSeguit
                     and s.nomSeguit=s2.nomSeguidor);
```

2. **(2 punts)** Doneu una asserció que assegurar que tots els usuaris amb mes de 100 seguidors tenen una tarifa 'premium'.

```
CREATE ASSERTION usuaris_premium CHECK (
       NOT EXISTS (SELECT s.nomSeguit
       FROM usuaris u, seguidors s
       WHERE s.nomSeguit = u.nomUsuari and
             u.tarifa <> 'premium'
       GROUP BY s.nomSeguit
       HAVING COUNT(*) > 100));
```

3. **(2 punts)** Considereu la BD del principi de l'examen i tingueu en compte que el contingut de la taula usuaris és el següent:

Usuaris			
nomUsuari	presentació	Tarifa	reproduccions
@toni	Vull anar de viatge	Premium	45000
@maria	Avui fa sol	Premium	200000
@dolors	Anem a caminar	Premium	50000
@carme	No se què tinc	Free	5000
@xavier	Quina calor	Premium	80000

Suposeu ara que a la base de dades hi ha definides també les dues vistes següents:

```
create view usPremium
as select * from usuaris where tarifa = 'premium';

create view usPremiumInfluencers
as select * from usPremium where reproduccions >= 50000;
```

3.1. Doneu el resultat dels selects següents:

```
select nomUsuari from usPremium;

select nomUsuari from usPremiumInfluencers;
```

usPremium
@toni
@maria
@dolors
@xavier

usPremiumInfluencers
@maria
@dolors
@xavier

3.2. Repetiu l'apartat 3.1 després de l'execució de les sentències de cadascun dels apartats següents (sempre partint de l'estat de la taula usuaris donat a l'inici de l'exercici). Justifiqueu el resultat.

A.
```
insert into usPremiumInfluencers values
       ('@anna','Marxa','premium',40000);
```

B.
```
update usPremiumInfluencers
set reproduccions=reproduccions-5000, tarifa='free';
```

Solució: Les dues vistes són actualitzables (veure transparències).

Apartat A: En no haver-hi check option la inserció es fan encara que no compleixi les condicions de les vistes.

usPremium
@toni
@maria
@dolors
@xavier
@anna

usPremiumInfluencers
@maria
@dolors
@xavier

Apartat B: La modificació es fa de les files que estan a la vista usPremiumInfluencers.

usPremium
@toni

usPremiumInfluencers

3.3. Suposeu ara que la vista usPremium està definida amb WITH CHECK OPTION. Repetiu l'apartat 3.2. Justifiqueu el resultat.

Apartat A: No hi ha cap canvi respecte a l'apartat 3.2 perquè la inserció compleix el check option de la vista usPremium.

usPremium
@toni
@maria
@dolors
@xavier
@anna

usPremiumInfluencers
@maria
@dolors
@xavier

Apartat B: L'execució dóna una excepció degut a que el Update no compleix la condició del check option de la vista usPremium.

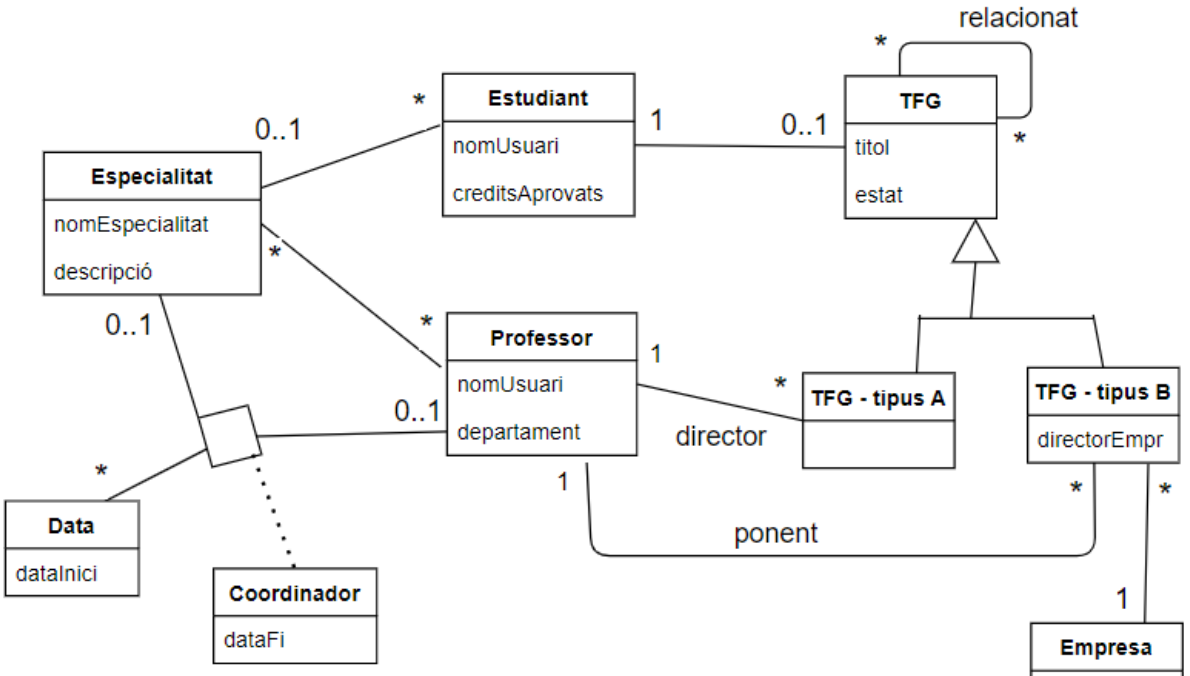
usPremium
@toni
@maria
@dolors
@xavier

usPremiumInfluencers
@maria
@dolors
@xavier

4. **(2 punts)** Considereu la BD del principi de l'examen. Doneu una seqüència d'operacions d'àlgebra relacional per obtenir el nom i la tarifa dels usuaris que no són creadors de cap playList, i que són seguidors d'algun usuari que si que és creador d'alguna playList i que ha fet més de 50000 reproduccions.

```
A = usuaris(reproduccions > 50000)
B = A[nomUsuari=creador]playlist
C = B[nomUsuari=nomSeguit]seguidors
D = C[nomSeguidor]              /* usuaris seguidors d'un usuari que ha fet
                                /* més de 50000 i que es creador d'una playlist
E = playList[creador]
F = D{nomSeguidor -> nomUsuari}
G = E{creador -> nomUsuari}
H = F-G
I = H*usuaris
R = I[nomUsuari,tarifa]
```

5. **(1,5 punts)** Donat el model conceptual següent en UML:



Clau externa Especialitat: nomEspecialitat
Clau externa Data: dataInici
Clau externa Estudiant: nomUsuari
Clau externa Professor: nomUsuari
Clau externa TFG: títol
Clua externa Empresa: nomEmpresa

Es demana fer la traducció a model relacional donant el disseny lògic de la base de dades resultant. Concretament cal indicar:

- taules
- atributs
- claus primàries
- claus foranes
- restriccions NOT NULL per a les claus foranes que ho requereixin
- restriccions UNIQUE en cas d'existir claus alternatives

```
especialitats(nomEspecialitat, descripció)
estudiants(nomUsuari, creditsAprovats, especialitat)
      {especialitat} referencia especialitats
professors(nomUsuari, departament)
coordinadors(nomEspecialitat, dataInici, nomUsuariProf, dataFi)
      {nomUsuariProf, data Inici} UNIQUE
      {nomUsuariProf} NOT NULL
      {nomEspecialitat} referencia especialitats
      {nomUsuariProf} referencia professors
professorsEspecialitat(nomEspecialitat, nomUsuariProf)
      {nomEspecialitat} referencia especialitats
      {nomUsuariProf} referencia professors
```

```
TFGs(títol, estat, nomUsuariEst)
      {nomUsuariEst} UNIQUE NOT NULL
      {nomUsuariEst} referencia estudiants
TFGrelacionats(títolTFG, títolTFGrelacionat)
      {títolTFG} referencia TFGs
      {títolTFGrelacionat} referencia TFGs
TFGtipusA(títol, director)
      {títol} referencia TFGs
      {director} referencia professors NOT NULL
empreses(nomEmpresa)
TFGtipusB(títol, director, ponent, nomEmpresa)
      {títol} referencia TFGs
      {ponent} referencia professors NOT NULL
      {nomEmpresa} referencia empreses NOT NULL
```

Es considera vàlida la solució on la traducció de la ternària és la següent:

```
coordinadors(nomEspecialitat, dataInici, nomUsuariProf, dataFi)
      {nomEspecialitat, data Inici} UNIQUE
      {nomEspecialitat} NOT NULL
      {nomEspecialitat} referencia especialitats
      {nomUsuariProf} referencia professors
```

Es considera vàlida la solució on la traducció inclou la taula dates. Part que canvia:

```
dates(dataInici)
coordinadors(nomEspecialitat, dataInici, nomUsuariProf, dataFi)
      {nomUsuariProf, data Inici} UNIQUE
      {nomUsuariProf} NOT NULL
      {nomEspecialitat} referencia especialitats
      {nomUsuariProf} referencia professors
      {data Inici} referencia dates
```

Es considera vàlida la solució on la traducció de les taules estudiants i TFGs és la següent:

```
TFGs(títol, estat)
estudiants(nomUsuari, creditsAprovats, especialitat,títol)
     {especialitat} referencia especialitats
     {títol} referencia TFGs UNIQUE
```

Temps: 3 hores

Notes 24 de gener a la tarda. Revisió d'examen: 25 de gener a la tarda (presencial)

Cada pregunta s'ha de lliurar en un full separat

1. (1 punts) Donada la base de dades següent on tots els atributs són de tipus enter:

```
Professors(idProf, sumVal, nResp, nQüest)
        CHECK(nQüest <= 5)
```

-- *sumVal* suma del valor de les respostes de tots els qüestionaris del profesor.
-- *nResp* quantes respostes en total hi ha dels qüestionaris del profesor.
-- nQüest quants qüestionaris s'han passat en total pel profesor.

```
Qüestionaris(idQüest, assignatura, grup, idProf)
        {idProf} referencia Professors

Respostes(idQüest, estudiant, valor)
        {idQüest} referencia Qüestionaris
```

Aquesta base de dades guarda a les taules `Qüestionaris` i `Respostes` la valoració que els estudiants han fet dels professors.

Un qüestionari es considera **vàlid** si té una quantitat de respostes superior a un determinat límit inferior.

Doneu els fragments que falten al programa que es dóna a continuació de manera que:

- El primer fragment deixi a cada fila del ResultSet *rs,* l'identificador del professor, la suma de valoracions i la quantitat de respostes de cada qüestionari **vàlid**.
- El segon fragment actualitzi la taula de professors a partir de la informació que hi ha a *rs*.
- Si un professor passa a tenir més de 5 qüestionaris, no es fa cap canvi a la base de dades per cap professor i es mostra el missatge "Un professor no pot tenir més de 5 qüestionaris".
- Si no hi ha cap qüestionari vàlid, es mostra el missatge "No hi ha qüestionaris vàlids".

```
try {
   // c és una variable que conté una connexió a la BD
   // limInf és una variable que conté el límit inferior dels
   qüestionaris vàlids
   String q = "SELECT idProf, SUM(valor) AS sv, COUNT(*) AS cnt " +
              "FROM Questionaris NATURAL JOIN Respostes " +
              "GROUP BY idQuest, idProf " +
              "HAVING COUNT(*) > ";
   ResultSet rs;
   //PRIMER FRAGMENT A IMPLEMENTAR
   //SEGON FRAGMENT A IMPLEMENTAR
```

Recordeu que podeu usar els mètodes/codis d'excepció de JDBC estudiats a classe i que estan inclosos a continuació:

Statement

- `Statement createStatement();`
- `ResultSet executeQuery(String sql);`
- `int executeUpdate(String sql);`

PreparedStatement

- `PreparedStatement prepareStatement(String sql);`
- `ResultSet executeQuery();`
- `int executeUpdate();`
- `void setXXX(int posicio, XXX valor);`

ResultSet

- `boolean next();`
- `XXX getXXX(String nomColumna)`

SQLException

- `// 23502 -not_null_violation`
- `// 23503 -foreign_key_violation`
- `// 23505 -unique_violation`
- `// 23514 -check_violation`
- `String getSQLState();`

SOLUCIÓ

```
//PRIMER FRAGMENT
Statement s = c.createStatement();
rs = s.executeQuery(q + limInf);

//SEGON FRAGMENT
PreparedStatement ps = c.prepareStatement("UPDATE professors " +
        "SET sumval=sumval+?, "+
        "nResp=nResp+?, nQuest=nQuest+1 WHERE idProf=?;");
int n;
for(n = 0; rs.next(); n++) {
        ps.setInt(1, rs.getInt("sv")); ps.setInt(2, rs.getInt("cnt"));
        ps.setInt(3, rs.getInt("idProf")); ps.executeUpdate();
}
if (n == 0) System.out.println ("No hi ha qüestionaris vàlids.");

catch (SQLException se) {
  if(se.getSQLState().equals("23514"))
     System.out.println("Un professor no pot tenir "+
                              "més de 5 qüestionaris");
}
```

2.(3 punts) Donada la taula professors(dni, nomProf, telèfon, sou) que conté les files {(111,'Marcel', 911, 1500), (222, 'Julia', 922, 1600)}.

2.1 Suposeu els horaris següents que s'executen en un SGBD que no implementa cap mecanisme de control de concurrència i on el gràNul és la fila.

Digueu **per cada horari**:

- Quin seria el resultat després d'executar **cadascuna** de les sentències de consulta i quin el contingut de la taula després d'executar **cadascuna** de les sentències d'actualització.
- Quines interferències es produeixen, i entre quins gràNuls. Justifiqueu les respostes.

a)

	T1	T2
a1	select * from professors where sou < 1600	
a2		update professors set sou=sou -50 where sou >=1600
a3	select * from professors where sou < 1600	
a4	COMMIT	
a5		COMMIT

b)

	T1	T2
b1	select * from professors where sou < 1600	
b2		update professors set sou=sou + 50 where sou < 1600
b3	select * from professors where sou < 1600	
b4	COMMIT	
b5		COMMIT

c)

	T1	T2
c1	select * from professors where sou < 1600	
c2		update professors set sou=sou -50 where sou >= 1600
c3		update professors set sou=sou + 50 where sou < 1600
c4	select * from professors where sou < 1600	
c5	COMMIT	
c6		COMMIT

Solució:

a)

	T1	T2	
a1	select * from professors where sou<1600		(111,'Marcel', 911, 1500)
a2		update professors set sou=sou-50 where sou>=1600	(111,'Marcel', 911, 1500) (222, 'Julia', 922, 1550)
a3	select * from professors where sou<1600		(111,'Marcel', 911, 1500) (222, 'Julia', 922, 1550)
a4	COMMIT		
a5		COMMIT	

Interferència de fantasmes. El gràNul és la fila amb identificador 222, Júlia. La Júlia apareix al segon select i no havia aparegut en el primer. No hi ha horaris serials equivalents perquè és un horari amb una interferència.

b)

	T1	T2	
b1	select * from professors where sou<1600		(111,'Marcel', 911, 1500)
b2		update professors set sou=sou+50 where sou<1600	(111,'Marcel', 911, 1550) (222, 'Julia', 922, 1600)
b3	select * from professors where sou<1600		(111,'Marcel', 911, 1550)
b4	COMMIT		
b5		COMMIT	

Interferència de lectura no repetible. El gràNul és la fila amb identificador 111, Marcel. El Marcel surt en els dos selects però el seu sou ha canviat de l'un a l'altre. No hi ha horaris serials equivalents perquè és un horari amb una interferència.

c)

	T1	T2	
c1	select * from professors where sou<1600		(111,'Marcel', 911, 1500)
c2		update professors set sou=sou-50 where sou>=1600	(111,'Marcel', 911, 1500) (222, 'Julia', 922, 1550)
c3		update professors set sou=sou+50 where sou<1600	(111,'Marcel', 911, 1550) (222, 'Julia', 922, 1600)
c4	select * from professors where sou<1600		(111,'Marcel', 911, 1550)
c5	COMMIT		
c6		COMMIT	

Interferència de lectura no repetible. El gràNul és la fila amb identificador 111, Marcel. Encara que els updates modifiquen els dos gràNuls, l'únic gràNul afectat és el Marcel, que surt en els dos selects però el seu sou ha canviat de l'un a l'altre. No hi ha horaris serials equivalents perquè és un horari amb una interferència.

2.2 Doneu tres horaris entre dues transaccions, on una d'elles sigui la transacció donada a continuació (que utilitza un nivell d'aïllament REPEATABLE READ).

Tx: LOCK(B,S),R(B),LOCK(A,X),RU(A),W(A),COMMIT+UNLOCK(A,B)

Les condicions que ha de complir cada horari són les següents:

a) Horari amb una única interferència de lectura no repetible, i on la transacció afegida segueixi un nivell d'aïllament READ COMMITTED. En cas de no ser possible un aquest horari justifiqueu perquè.

b) Horari amb una única interferència d'actualització perduda, i on la transacció afegida segueixi un nivell d'aïllament READ UNCOMMITTED. En cas de no ser possible aquest horari justifiqueu perquè.

c) Horari on en un mateix instant hi hagi un mateix gränul reservat per les dues transaccions, i on la transacció afegida segueixi un nivell d'aïllament REPEATABLE READ. En cas de no ser possible un aquest horari justifiqueu perquè.

Els horaris han d'incloure les operacions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves (LOCK, UNLOCK), les esperes que siguin necessàries.

a)

Tx	**Ty**
L(B,S)	
R(B)	
	L(A,S)
	R(A)
	U(A)
L(A,X)	
RU(A)	
W(A)	
C+U(A,B)	
	L(A,S)
	R(A)
	U(A)
	C

b) No és possible un horari com el demanat, degut a que les dues transaccions tenen un nivell d'aïllament que evita les interferències d'actualització perduda.

b)

Tx	**Ty**
L(B,S)	
R(B)	
	L(B,S)
	R(B)
L(A,X)	
RU(A)	
W(A)	
C+U(A,B)	
	C+U(B)

3. (3 punts) Considereu la taula `professors(dni, nomProf, telèfon, sou)` que té 10.000 tuples. El factor de bloqueig de les pàgines de dades és de 10 i el *sou* és un enter distribuït uniformement de 1001 a 6000. Hi ha un índex arbre B+ no agrupat definit sobre la columna *sou*. L'ocupació de l'arbre és del 80%, i l'ordre de l'arbre és d=100.

3.1 Determineu la quantitat de pàgines necessàries per a emmagatzemar l'índex. Justifiqueu la resposta.

[2*d*ocupació] = [2 * 100 * 0.8] = **160 valors per node de l'índex**

número de valors per node + 1 = 160 + 1 = 161 apuntadors per node de l'índex

Nivell fulla: [10.000 valors que s'han de guardar a les fulles de l'índex / 160 valors per node] = 63 nodes => 63 apuntadors al nivell anterior

Nodes interns: [63 apuntadors / 161 apuntadors per node] = 1 node => 1 node arrel

Número de pàgines necessàries per emmagatzemar l'índex: 63+1 = 64

3.2 Indiqueu quina quantitat de pàgines (d'índex i de dades) cal accedir per resoldre la consulta següent mitjançant l'índex. Justifiqueu la resposta.

```
SELECT * FROM Professors
WHERE sou > 1850 AND sou <= 1900
```

Formula: (h + F) + |R(sou>1850 AND sou<=1900)|

h - alçada de l'arbre

F - nodes fulles addicionals (no comptar el node que està inclòs a l'alçada de l'arbre)

|R(sou>1850 AND sou<=1900)| - número de valors que compleixen la condició de WHERE

per cada valor hi ha 2 files (10.000 files / 5000 valors)

per tant, hi haurà 100 files que compleixin la condició (50 valors entre 1850 i 1900 x 2 professors cada valor)

h = $\lceil \log_{161} 10.000 \rceil$ = 2 (ja ho teníem abans també)

F = [100 / 160 valors per node fulla] -1 = 0

|R(sou>1850 AND sou<=1900)| = 100

Resultat = 2 + 0 + 100 = 102 pàgines

3.3 Suposeu ara que també tenim la taula `assignatures(codi, nomAssig, dniCoordinador, matr)`, on `dniCoordinador` és clau forana que referencia `professors`. Considereu ara que les taules `professors` i `assignatures` estan en un espai virtual d'agrupació, i que les tuples de les dues taules tenen la mateixa mida.

Suposant que cada professor és coordinador en mitjana de 3 assignatures, determineu el cost de la consulta següent, en cas de no disposar de cap índex.

```
SELECT p.dni, p.nomProf, a.codi, a.nomAssig,
FROM professors p, assignatures a
WHERE p.dni = a.dniCoordinador
```

Hi ha 10.000 professors, 3 assignatures per professor => 40.000 tuples d'igual mida (el factor de bloqueig serà el mateix).
Al no haver-hi índex caldrà fer un accés seqüencial a les pàgines de dades.
Cost = 40.000 files / 10 files per pàgina = 4000 pàgines.

4.(1 punt) Considereu la base de dades següent.

```
CREATE TABLE usuaris(
          id integer primary key);

CREATE TABLE estudiants(
          id integer primary key references usuaris);

CREATE TABLE log(
          id_inserit integer primary key,
          username char(25),
          hora_insercio timestamp);

CREATE FUNCTION inserir_usuari() RETURNS trigger AS $$
  BEGIN
    IF (NOT EXISTS (SELECT * FROM usuaris WHERE id=NEW.id))
    THEN INSERT INTO usuaris VALUES (NEW.id);
    ELSE INSERT INTO log
         VALUES (NEW.id,current_user,current_date);
    END IF;
    RETURN NULL;
END; $$ LANGUAGE plpgsql;

CREATE FUNCTION inserir_log() RETURNS trigger AS $$
  BEGIN
    insert into log values (NEW.id,current_user,current_date);
  RETURN NULL;
END;$$ LANGUAGE plpgsql;

CREATE TRIGGER estudiants_son_usuaris
AFTER INSERT ON estudiants
FOR EACH ROW EXECUTE PROCEDURE inserir_usuari();

CREATE TRIGGER actualitzar_log_usuaris
AFTER INSERT ON usuaris
FOR EACH ROW EXECUTE PROCEDURE inserir_log();
```

4.1 Suposeu que s'executen una a una les sentències SQL següents (en cada apartat, – a i b – a partir de la base de dades buida). Indiqueu quin serà el contingut de cada una de les 3 taules quan s'hagin executat cada un dels dos conjunts d'instruccions. Justifiqueu breument la resposta.

a)
1. insert into usuaris values (1);
2. insert into usuaris values (2);
3. insert into estudiants values (2);

b)
1. begin transaction;
2. insert into usuaris values (1);
3. insert into estudiants values (2);
4. commit;

Solució:

a) Usuaris: (1), (2)
Log: (1, nom_usuari1, hora1), (2, nom_usuari2, hora2)
Estudiants: -

A 3 falla la inserció al log que fa el trigger, per violació de clau primària. Es desfà el que ha fet aquesta instrucció.

b) Usuaris: -
Log: -
Estudiants: -

A 3 falla la inserció a estudiant per violació de clau forana. El trigger l'hauria d'arreglar, però al ser AFTER la restricció es comprova primer. Com que hi ha una violació de restricció, es desfà tot el que ha fet la transacció.

5.(2 punts) Considereu l'esquema de la base de dades següent:

```
equips(idEquip, nombreEquip, ciutatEquip);

partits(idPartit, idEqLocal, idEqVisitant, golsLocal, golsVisitant)
      {idEqLocal} REFERENCES equips
      {idEqVisitant} REFERENCES equips
```

-- *golsLocal* quants gols ha marcat l'equip local al partit. Inicialment té valor 0.

-- *golsVisitant* quants gols ha marcat l'equip visitant al partit. Inicialment té valor 0.

```
jugadors(idJugador, nom, golsMarcats, idEquip)
      {idEquip} REFERENCES equips
```

-- *golsMarcats* quants gols en total ha marcat el jugador en els partits que ha jugat. Inicialment té valor 0.

-- *idEquip* indica l'equip en el que juga el jugador.

```
gols(idPartit, idJugador, minut)
      {idPartit} REFERENCES partits
      {idJugador} REFERENCES jugadors
```

-- hi ha una fila per cada gol marcat per un jugador en un partit

La vostra tasca és implementar amb disparadors el manteniment del valor dels atributs *golsLocal*, *golsVisitant*, i *golsMarcats* **quan hi hagi insercions de files a la taula `gols`**, i generar una excepció en cas que el minut del gol inserit tingui un valor inferior a 0 o superior a 90 donant el missatge "Minut incorrecte".

Podeu fer servir la plantilla de triggers següent.

```
CREATE OR REPLACE FUNCTION nom_funcio() RETURNS TRIGGER AS $$
DECLARE
  ... ...
BEGIN
  ... ...
EXCEPTION
  WHEN { raise_exception | unique_violation | foreign_key_violation | check_violation | others }
  THEN
   RAISE EXCEPTION 'missatge excepció';
  ...
END;
$$ LANGUAGE PLPGSQL;

CREATE TRIGGER disparador {BEFORE | AFTER}{esdeveniment[OR ...]} ON taula
FOR [EACH] {ROW | STATEMENT} EXECUTE PROCEDURE nom_funcio();
```

SOLUCIÓ:

```
CREATE OR REPLACE FUNCTION actualitza_gols1() RETURNS TRIGGER AS $$
BEGIN
    if (new.minut<0 or new.minut>90)
    then raise exception 'minut incorrecte';
    end if;
    return new;
END;
$$ LANGUAGE PLPGSQL;

CREATE OR REPLACE FUNCTION actualitza_gols2() RETURNS TRIGGER AS $$
declare idEquipQueHaMarcat integer;
begin
     UPDATE jugadors j
     SET golsMarcats = golsMarcats + 1
     WHERE j.idJugador= new.idJugador;

     select idEquip into idEquipQueHaMarcat
     from jugadors j where j.idJugador=new.idjugador;
```

```
     update partits p
     set golsLocal = golsLocal+1
     where p.idPartit = new.idpartit and
          p.idEqLocal = idEquipQueHaMarcat;

     update partits p
     set golsVisitant = golsVisitant+1
     where p.idPartit = new.idpartit and
          p.idEqVisitant = idEquipQueHaMarcat;

return null;
END;
$$ LANGUAGE PLPGSQL;

CREATE TRIGGER disparador1 BEFORE INSERT ON gols
FOR EACH ROW EXECUTE PROCEDURE actualitza_gols1();

CREATE TRIGGER disparador2 AFTER INSERT ON gols
FOR EACH ROW EXECUTE PROCEDURE actualitza_gols2();
```

BD **5.4.2024**

Temps: 2 hores

Notes 19 abril tarda **Revisió: 22 abril tarda** **Cada pregunta en un full separat**

1. **(3 punts)** Considereu l'esquema de la base de dades següent:

```
CREATE TABLE usuaris
(nomUsuari char(20),
presentacio char(500),
tarifa char(7) check(tarifa IN ('premium','free')) NOT NULL,
reproduccions integer check(reproduccions >=0) NOT NULL,
PRIMARY KEY (nomUsuari));

CREATE TABLE seguidors
(nomSeguit char(20),
nomSeguidor char(20),
silenciat integer CHECK (silenciat IN (0,1)) DEFAULT 0,
PRIMARY KEY (nomSeguit,nomSeguidor),
FOREIGN KEY (nomSeguit) REFERENCES usuaris,
FOREIGN KEY (nomSeguidor) REFERENCES usuaris);
-- tingueu en compte que tot usuari té com a mínim un seguidor
-- silenciat té valor 1 quan nomSeguit ha silenciat nomSeguidor

CREATE TABLE playlist
(idList integer,
nomList char(100) NOT NULL,
tematica char(50),
creador char(20) NOT NULL,
PRIMARY KEY (idList),
FOREIGN KEY(creador) REFERENCES usuaris);

CREATE TABLE cançons
(idCanço integer,
titol char(100) UNIQUE,
cantant char(100),
numTotalReproduccions integer CHECK (numTotalReproduccions>=0),
PRIMARY KEY (idCanço));

CREATE TABLE cançonsAPlaylist
(idList integer,
idCanço integer,
numReproduccions integer CHECK (numReproduccions>=0),
PRIMARY KEY (idList,idCanço),
FOREIGN KEY (idList) REFERENCES playlist,
FOREIGN KEY (idcanço) REFERENCES cançons);
```

1.1. Doneu una sentència SQL per obtenir els usuaris que han silenciat a algun seguidor i que a més aquests usuaris no han creat cap playlist on hi hagi alguna cançó de la cantant 'Joana'. Es vol obtenir el nom i la presentació d'aquests usuaris.

```
SELECT DISTINCT u.nomUsuari, u.presentacio
FROM usuaris u, seguidors s
WHERE u.nomUsuari = s.nomSeguit
     AND s.silenciat = 1
     AND NOT EXISTS (SELECT *
          FROM playlist p, cançonsAPLayList cp, cançons c
          WHERE p.creador = u.idusuari
               AND p.idlist = cp.idlist
               AND cp.idcanço = c.idcanço
               AND c.cantant = 'Joana'))
```

1.2. Suposeu que la quantitat de files que hi ha a cadascuna de les taules de la base de dades anterior és respectivament: U, S, P, C, i CP (siguent totes elles quantitats superiors a 0). Digueu quina és la **quantitat de files** mínima i màxima que s'obtindran com a resultat de l'execució de cadascuna de les consultes següents. Justifiqueu la resposta.

a) SELECT *
FROM usuaris u, playlist p
WHERE u.nomUsuari = p.creador

b) SELECT count(*)
FROM usuaris u, seguidors s
WHERE u.nomUsuari = s.nomSeguit

SOLUCIÓ:

a) Com que la foreign key de playlist cap a usuaris (columna creador) no pot tenir valors nuls, aleshores hi haurà un resultat per cada playlist. La quantitat serà: min = max = P.

b) Una sentència que compta amb un COUNT, sense cap group by, dóna sempre un i només un resultat. La quantitat serà: min = max = 1.

2. (2.5 punts) Considereu la base de dades del principi de l'examen.

2.1. Doneu una asserció per assegurar que un usuari té tarifa 'premium' **si i només si** té més de 100 seguidors.

```
CREATE ASSERTION usuaris_premium CHECK (
NOT EXISTS (SELECT u.nomUsuari
            FROM usuaris u, seguidors s
            WHERE u.nomUsuari = s.nomSeguit
            GROUP BY u.nomUsuari
            HAVING (u.tarifa <> 'premium' AND COUNT(*) > 100)
                 OR
                 (u.tarifa = 'premium' AND COUNT(*) <= 100))
```

2.2.

a) Doneu una vista de nom *nomListRepetit* amb només un atribut *nomList* per obtenir els noms de les llistes de reproducció que estan repetits a la taula playlist. No es pot fer servir funcions d'agregació.

b) Definiu una assertion sobre la vista anterior, per assegurar que no hi ha noms de llistes repetits a la taula Playlist.

c) Proposeu una manera de, utilitzant les restriccions del model relacional, fer el mateix sense necessitat de definir ni la vista ni l'assertion.

```
CREATE VIEW nomListRepetit AS SELECT DISTINCT p.nomList
                      FROM playList p, playList p2
                      WHERE p.idList<>p2.idList
                           AND p.nomList=p2.nomlist)

CREATE ASSERTION noNomListRepetit
            CHECK (NOT EXISTS (SELECT *
                      FROM nomListRepetit)

Es podria fer amb un UNIQUE a la columna nomList de la taula
playList.
```

3. **(2 punts)** Considereu la BD del principi de l'examen. Doneu una seqüència d'operacions d'àlgebra relacional per tal mostrar el nom i la presentació dels usuaris que hagin creat dos o més playlists amb la mateixa temàtica i que no estiguin silenciats per cap dels usuaris dels que són seguidors.

A=seguidors(silenciat=1)

B=A[nomSeguidor]

C=playlist{idList->idList2, creador->creador2, tematica->tematica2}

D=playlist[idList!=idList2, creador=creador2, tematica=tematica2]C

E= D[creador]

F=E{creador->nomUsuari}

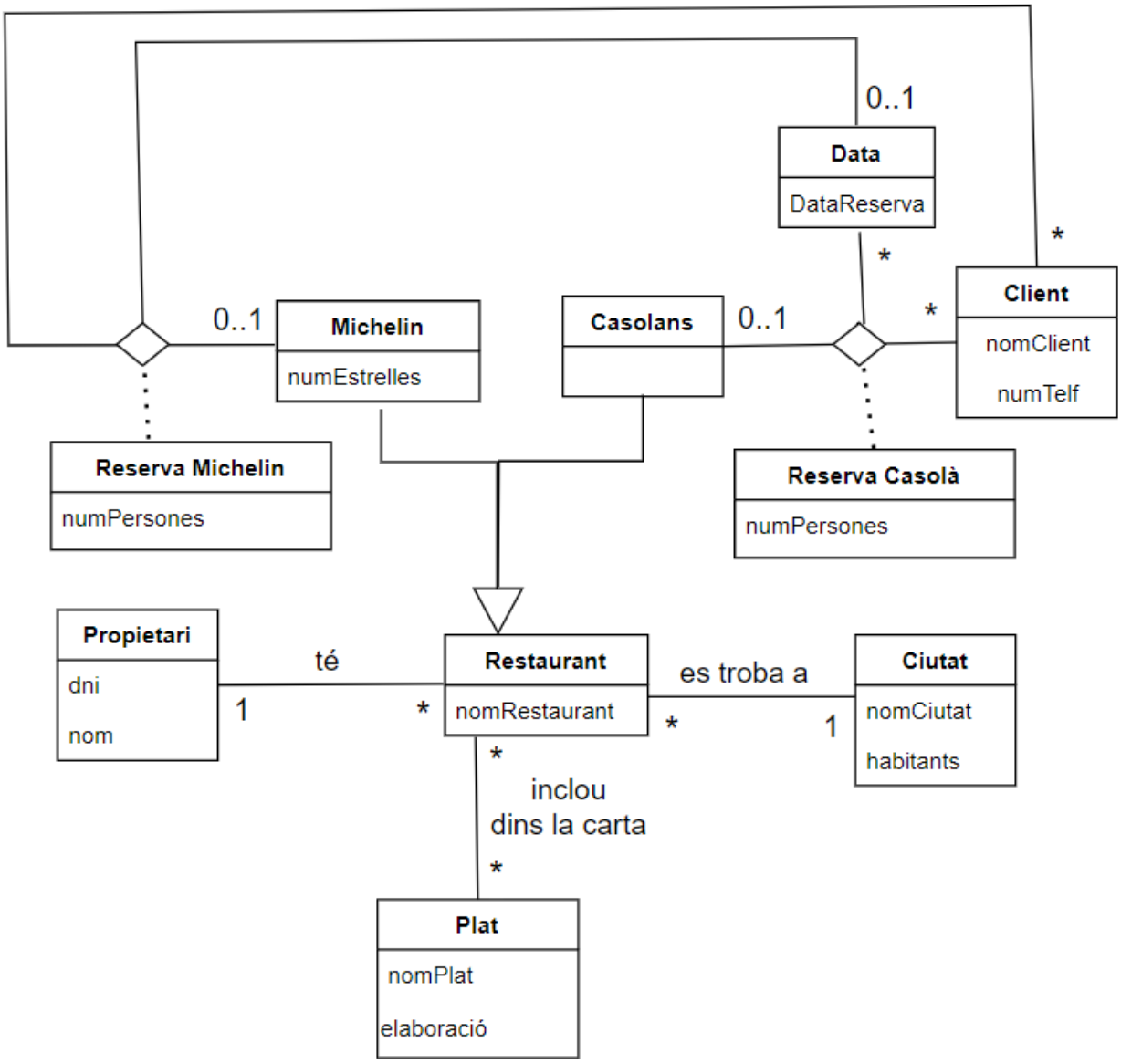
G=B{nomSeguidor->nomUsuari}

H=F-G

I=H*usuaris

J=I[nomUsuari, presentació]

4. **(2.5 punts)** Donat el model conceptual següent en UML:



I les restriccions textuals següents:

- **Clau externa Client**: nomClient
- **Clau externa Data**: dataReserva
- **Clau externa Restaurant**: No poden existir restaurants amb el mateix nom en la mateixa ciutat, però sí a ciutats diferents
- **Clau externa Ciutat**: nomCiutat
- **Clau externa Propietari**: dni
- **Clau externa Plat**: nomPlat

Es demana fer la traducció a model relacional donant el disseny lògic de la base de dades resultant. Concretament cal indicar:

- taules
- atributs
- claus primàries
- claus foranes
- restriccions NOT NULL per a les claus foranes que ho requereixin

- restriccions UNIQUE en cas d'existir claus alternatives

SOLUCIÓ:

Ciutats (nomCiutat, habitants)
Propietaris (dni, nom)
Restaurants (nomRestaurant, nomCiutat, dniPropietari)
    {nomCiutat} referencia Ciutats
    {dniPropietari} referencia Propietaris NOT NULL
Casolans(nomRestaurant, nomCiutat)
    {nomRestaurant, nomCiutat} referencia Restaurants
Michelin(nomRestaurant, nomCiutat, numEstrelles)
    {nomRestaurant, nomCiutat} referencia Restaurants
Clients(nomClient, numTelf)
ReservaCasolans(nomClient, dataReserva, nomRestaurant, nomCiutat,
                numPersones)
    {nomRestaurant, nomCiutat} referencia Casolans NOT NULL
    {nomClient} referencia Clients
ReservaMichelin(nomClient, dataReserva, nomRestaurant, nomCiutat,
                    numPersones)
    {nomRestaurant, nomCiutat} referencia Michelin NOT NULL
    {nomClient} referencia Clients
    UNIQUE(nomClient,nomRestaurant,nomCiutat)
Plats(nomPlat, elaboracio)
Carta(nomRestaurant, nomCiutat, nomPlat)
    {nomRestaurant, nomCiutat} referencia Restaurants
    {nomPlat} referencia Plats

- s'admet també la solució alternativa on la clau primària de la taula ReservaMichelin la formen {nomClient, nomRestaurant, nomCiutat} i la clau alternativa (UNIQUE) la formen {nomClient, dataReserva}. En aquest cas no cal indicar el NOT NULL de {nomResturant, nomCiutat} donat que forma part de la clau primària, però caldrà indicar el NOT NULL per {dataReserva}

- S’admet també una solució on hi hagi una taula Dates com a taula, en aquest cas caldria posar la definició de clau forana des de les taules ReservesCasolanes i ReservesMichelin.

Temps: 3 hores

Notes 21 de juny tarda **Revisió: 25 juny tarda** **Cada pregunta en un full separat**

1. **(1.5 punts)** Considereu l'esquema de la base de dades següent:

```
CREATE TABLE usuaris
(nomUsuari char(20) PRIMARY KEY,
presentacio char(500),
tarifa char(7) check(tarifa IN ('premium','free')) NOT NULL,
reproduccions integer check(reproduccions >=0 AND
                            reproduccions <=200000) NOT NULL);

CREATE TABLE seguidors
(nomSeguit char(20),
nomSeguidor char(20),
silenciat integer CHECK (silenciat IN (0,1)) DEFAULT 0,
PRIMARY KEY (nomSeguit,nomSeguidor),
FOREIGN KEY (nomSeguit) REFERENCES usuaris,
FOREIGN KEY (nomSeguidor) REFERENCES usuaris);
-- tingueu en compte que tot usuari té com a mínim un seguidor
-- silenciat té valor 1 quan nomSeguit ha silenciat nomSeguidor

CREATE TABLE playlist
(idList integer PRIMARY KEY,
nomList char(100) NOT NULL,
tematica char(50),
creador char(20) NOT NULL,
FOREIGN KEY(creador) REFERENCES usuaris);

CREATE TABLE cançons
(idCanço integer PRIMARY KEY,
titol char(100) UNIQUE,
cantant char(100),
numTotalReproduccions integer CHECK (numTotalReproduccions>=0));

CREATE TABLE cançonsAPlaylist
(idList integer,
idCanço integer,
numReproduccions integer CHECK (numReproduccions>=0),
PRIMARY KEY (idList,idCanço),
FOREIGN KEY (idList) REFERENCES playlist,
FOREIGN KEY (idcanço) REFERENCES cançons);
-- hi ha una fila per cada cançó inclosa en una playlist, el
numReproduccions són les que hi ha hagut de la cançó a la playlist
```

```
try {
  /* Tenim ja una connexió c establerta amb la base de dades */
  /* La variable idList conté un únic enter, corresponent a una playlist */

  PreparedStatement ps1 = c.prepareStatement("SELECT idCanço, numReproduccions "+
                                             "FROM cançonsAPlaylist "+
                                             "WHERE idList=?;");
  ps1.setInt(1, idList);
  ResultSet rs = ps1.executeQuery();
  PreparedStatement ps2 = c.prepareStatement("DELETE FROM cançonsAPlaylist " +
                                             "WHERE idList=? AND idCanço=?;");
  ps2.setInt(1, idList);
  PreparedStatement ps3 = c.prepareStatement("UPDATE cançons c "+
                                             "SET numTotalReproduccions=("+
                                                  "SELECT SUM(numReproduccions) "+
                                                  "FROM cançonsAPlaylist cp "+
                                                  "WHERE cp.idCanço=c.idCanço) "+
                                             "WHERE c.idCanço=?;");

  System.out.println("Les cançons que eliminem de la playlist son:");

  while (rs.next()) {
      System.out.println(rs.getString("idCanço")+" tenia "+
                rs.getString("numReproduccions")+" reproduccions a la playlist");
      ps2.setInt(2, rs.getInt("idCanço"));
      x = ps2.executeUpdate();
      if (x==0) { System.out.println("La Playlist no té cançons"); }
      ps3.setInt(1, rs.getInt("idCanço"));
      y = ps3.executeUpdate();
      if (y==0) { System.out.println("La cançó no existeix"); }
      }
  ps1.close(); ps2.close(); ps3.close();
  c.commit();
  }
catch (SQLException se) {
  System.out.println ("Excepcio: "+ se.getMessage());}
```

1.1 Respectant que les sentències de modificació (UPDATE i DELETE) només poden modificar una fila per cada crida (és a dir, no es poden canviar les seves clàusules WHERE), expliqueu les modificacions que caldria fer al codi anterior per seguir els criteris de qualitat de l'assignatura.

- PS1 ha de ser un Statement
- PS3 es pot fer sense la subconsulta, restant nomes les reproduccions de cada cançó esborrada

1.2 Doneu un exemple de contingut de la base de dades i un idList per tal que en executar el codi anterior es mostri per pantalla "La Playlist no té cançons". Justifiqueu breument la resposta.

- No es pot, perquè la tupla actual del bucle sempre existirà.

1.3 Doneu un exemple de contingut de la base de dades i un idList per tal que en executar el codi anterior es mostri per pantalla "La cançó no existeix". Justifiqueu breument la resposta.

- No es pot, perquè hi ha una clau forana que garanteix que la cançó sempre existirà

2. (3 punts) Considereu l'horari següent:

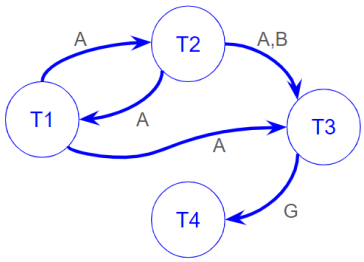
T1	T2	T3	T4
R(E)			
			R(E)
		R(G)	
	R(B)		
RU(A)			
			R(F)
	XXX		
	RU(B)		
	W(B)		
W(A)			
			RU(G)
			W(G)
			R(E)
			commit
	commit		
		RU(A)	
		W(A)	
		R(B)	
		commit	
commit			

2.1 **<u>Per cadascuna</u>** de les substitucions de l'operació **XXX** següents:

a) **XXX** = {RU(A),W(A)}, b) **XXX** = {R(G)}, c) **XXX** = {RU(F),W(F),R(G)}

- Doneu el graf de precedències de l'horari
- Indiqueu quina/es interferència/es entre dues transaccions hi ha a l'horari: transaccions implicades, grànuls implicats
- Hi ha horaris serials equivalents? Si n'hi ha digues quins? Sinó n'hi ha, justifica la resposta.

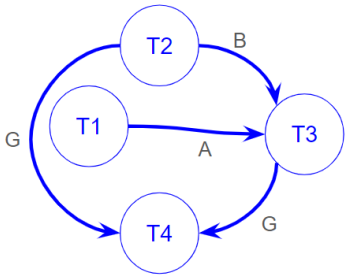
a)



Actualització perduda

No hi ha horaris serials equivalents

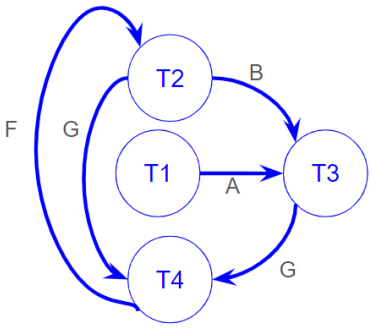
b)



No hi ha interferències

Hi ha horaris serials equivalents: T1;T2;T3;T4 i T2;T1;T3;T4

c)



Anàlisi inconsistent

No hi ha horaris serials equivalents

2.2 Suposeu ara que **XXX** = {RU(E),W(E)}. Doneu l'horari suposant que les transaccions T1 i T4 treballen amb nivell d'aïllament REPETEABLE READ, i T2 i T3 amb nivell d'aïllament READ COMMITTED. L'horari ha d'incloure les operacions que executen les transaccions (R, RU, W, commit), les operacions de petició i alliberament de reserves (LOCK, UNLOCK) i les esperes que siguin necessàries. Quan més d'una transacció espera per un mateix gràNul, considereu que la política de la cua és FIFO (First In, First Out).

T1	T2	T3	T4
L(E,S)			
R(E)			
			L(E,S)
			R(E)
		L(G,S)	
		R(G)	
		U(G)	
	L(B,S)		
	R(B)		
	U(B)		
L(A,X)			
RU(A)			
			L(F,S)
			R(F)
	L(E,X)		
W(A)			
			L(G,X)
			RU(G)
			W(G)
			R(E)
			c+U(G,F,E)
		L(A,X)	
c+U(A,E)			
	RU(E)		
	W(E)		
	L(B,X)		
	RU(B)		
	W(B)		
	c+U(E,B)		
		RU(A)	
		W(A)	
		L(B,S)	
		R(B)	
		U(B)	
		c+U(A)	

2.3 Suposeu ara la base de dades de l'inici de l'examen. Donat l'estat de la base de dades i la transacció següent, indiqueu quin serà l'estat de la base de dades després de l'execució de la transacció. Cal escriure les files concretes que hi haurà a la base de dades.

Usuaris			
nomUsuari	presentació	Tarifa	reproduccions
@toni	Vull anar de viatge	Premium	45000
@maria	Avui fa sol	Premium	200000
@dolors	Anem a caminar	Premium	50000

```
Start transaction;
insert into usuaris values ('@carme','No se','free',5000);
insert into usuaris values ('@ona','Quina calor','premium',8000);
update usuaris set reproduccions = reproduccions + 100
where tarifa = 'premium';
commit;
```

Salta una excepció al fer l'UPDATE per la restricció CHECK de la taula usuaris. Es desfà la transacció deixant l'estat de la base de dades com abans de l'inici de la transacció

usuaris			
nomUsuari	presentació	tarifa	reproduccions
@toni	Vull anar de viatge	premium	45000
@maria	Avui fa sol	premium	200000
@dolors	Anem a caminar	premium	50000

3. (3 pts) Suposeu la taula següent:

```
CREATE TABLE mobles
( idMoble integer PRIMARY KEY,
  descripcio char(50) UNIQUE,
  fusta char(30) NOT NULL,
  preuVenda integer NOT NULL CHECK (preuvenda > 0),
  estocMoble integer NOT NULL CHECK (estocmoble > 0));
```

3.1 Suposeu que hi ha un índex agrupat arbre B+ ja creat per l'atribut idMoble. Per cadascuna de les consultes següents indiqueu quin índex afegiríeu per tal de minimitzar la quantitat d'accessos a pàgines. Per a cada a índex indiqueu l'atribut/s sobre el/s que es definirà, i si haurà de ser agrupat o no agrupat. Justifiqueu la resposta.

a) SELECT * FROM mobles WHERE descripcio = 'Taula de Roure';
b) SELECT * FROM mobles;
c) SELECT * FROM mobles WHERE idMoble = 15;
d) SELECT * FROM mobles WHERE preuVenda<=1000 AND fusta = 'Pi';

a) Afegiríem un índex arbre B+ definit sobre l'atribut "descripció", ja que es tracta d'un accés directe per valor sobre aquest atribut. Per la consulta no té importància si és agrupat o no agrupat ja que només hi ha una única fila per cada valor de l'atribut "descripció", ara bé, com que ja existeix un índex agrupat per idMoble no es pot afegir cap altre índex agrupat. El nou índex ens permet trobar tant sols baixant per l'arbre el RID de l'única fila que compleix la condició.

b) Aquest 'select' correspon a un accés seqüencial per posició per tant no es pot aconseguir menys accessos a pàgines definint cap índex, ja que es farà un accés a totes les pàgines del fitxer de dades.

c) No cal afegir cap índex ja que l'accés més eficient és usant l'índex que ja està definit segons l'enunciat.

d) Afegiríem un índex arbre B+ multi-atribut sobre els atributs "fusta, preuVenda" ja que es tracta d'un accés directe per valor per "fusta" i seqüencial per valor per "preuVenda". Per la consulta seria millor si fos un índex agrupat, ara bé, com que ja existeix un índex agrupat per idMoble no es pot afegir cap altre índex agrupat. El nou índex ens permet trobar seguits tots els RIDs corresponents a files on la fusta és la indicada i entre aquests RIDs el corresponent a la fila del moble amb preu just per sobre o igual a 1000, després recorrent les fulles de l'índex anem trobant seguides els RIDs de les files que ens interessen.

3.2 Suposeu que a la taula Mobles hi ha 50000 mobles, que el factor de bloqueig de les pàgines de dades és de 20 files per pàgina, que l'índex agrupat definit per idMoble té ordre d=50, i tots els nodes excepte l'arrel estan plens en mitjana al 75%. Digueu quin és el cost, en nombre d'accessos a pàgines, de resoldre la consulta següent usant aquest índex. Indiqueu clarament quants accessos són a pàgines d'índex i quants a pàgines de dades, i com s'han calculat.

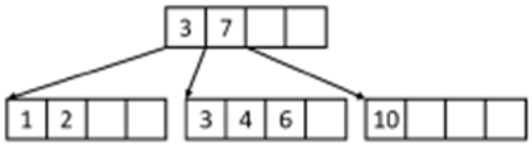
```
SELECT * FROM mobles WHERE idMoble = 30 and estocMoble > 500
```

$n = \lceil 2*d*\text{ocupació} \rceil = \lceil 2*50*0{,}75 \rceil = 75$ valors per node
$app = n + 1 = 76$
$h = \lceil \log_{76} 50000 \rceil = 3$

Cost d'accés a pàgines de l'índex = 3
Cost d'accés a pàgines de dades = 1 (només hi ha una fila amb idMoble 30).

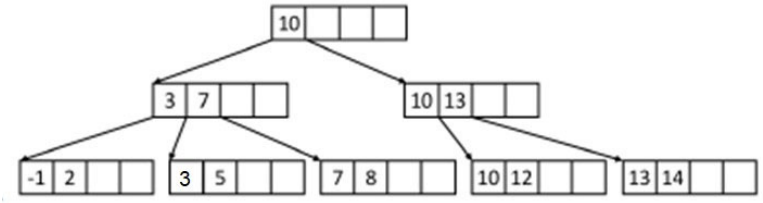
3.3 Per cadascun dels arbres B+ següents, digueu si compleix o no la definició d'arbre B+ vista a classe. Suposeu que els arbres són d'ordre d=2, que totes les fulles estan convenientment encadenades i que les fulles contenen els apuntadors a dades que siguin necessaris. En cas de que un arbre no sigui correcte, expliciteu els nodes erronis i doneu una explicació de l'error.

a)



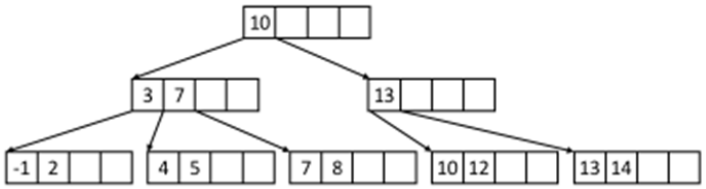
No compleix. La fulla de més a la dreta no té mínim 2 entrades i no té el valor 7.

b)



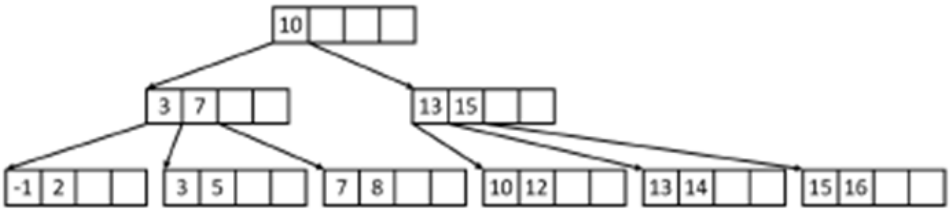
No compleix. El valor 10 està repetit 3.

c)



No compleix. El node (13) està per sota del mínim. El valor 3 hauria d'estar a la fulla (4,5)

d)



Compleix.

4. (2.5 punts) Considereu que a la base de dades de l'inici de l'examen s'afegeix la taula següent:

```
CREATE TABLE auditoria
(instant integer,
nomUsuari char(20),
operacio char(6) check(operacio IN ('INSERT','UPDATE','DELETE')),
presentacio_OLD char(500),
presentacio_NEW char(500),
tarifa_OLD char(7),
tarifa_NEW char(7),
reproduccions_OLD integer,
reproduccions_NEW integer,
PRIMARY KEY (instant));
-- l'instant té una precisió suficient per identificar el moment
individual de l'operació, fins i tot per discriminar les diverses
actualitzacions fetes en una mateixa sentència SQL
```

4.1 Definiu un trigger, amb el seu procediment, que impedeixi fer operacions de modificació i esborrat sobre la taula `auditoria`. En concret, cal que salti una excepció si s'intenta fer una modificació o esborrat sobre aquesta taula.

```
CREATE OR REPLACE FUNCTION evita_modificacions() RETURNS TRIGGER AS $$
BEGIN
  RAISE EXCEPTION 'No es poden fer modificacions ni esborrats';
END;
$$ LANGUAGE PLPGSQL;

CREATE OR REPLACE TRIGGER tg_evitarModificacions
BEFORE UPDATE OR DELETE ON auditoria
FOR EACH STATEMENT EXECUTE PROCEDURE evita_modificacions();
```

4.2 Definiu un trigger, amb el seu procediment, que registri a la taula `auditoria` qualsevol actualització que es produeixi a la taula `usuaris`. Suposeu que l'instant en el que es fa una actualització es pot obtenir mitjançant la funció CURRENT_INSTANT. Suposeu també que el nom de l'usuari no es pot canviar.

```
CREATE OR REPLACE FUNCTION registrarAuditoria() RETURNS TRIGGER AS $$
DECLARE  usuari CHAR(6);
BEGIN
    IF ( TG_OP = 'DELETE' ) THEN usuari = OLD.nomUsuari;
    ELSE  usuari = NEW.nomUsuari;
    END IF;

    INSERT INTO auditoria
     VALUES (CURRENT_INSTANT, usuari, TG_OP,
          OLD.presentacio, NEW.presentacio,
          OLD.tarifa, NEW.tarifa, OLD.reproduccions, NEW.reproduccions);
```

```
    RETURN NULL;
END;
$$ LANGUAGE PLPGSQL;

CREATE OR REPLACE TRIGGER tg_auditoriaUsuaris
AFTER INSERT OR UPDATE OR DELETE ON usuaris
FOR EACH ROW EXECUTE PROCEDURE registrarAuditoria();
```

4.3 Com a incentiu perquè els usuaris contractin la tarifa `premium`, i aprofitant les possibilitats de la intel·ligència artificial, es decideix oferir la creació d'una cançó exclusiva a qualsevol usuari que contracti aquesta tarifa.

Definiu un trigger, i el seu procediment, per tal que en detectar un nou usuari premium, s'insereixi una cançó amb els valors següents:

- idCanço: valor més alt de l'atribut idCanço de la taula cançons, més 1
- títol: el nom de l'usuari
- cantant: el nom de l'usuari
- numTotalReproduccions: 0

Si no es pot inserir la nova cançó perquè ja existeix s'ha de generar una excepció: 'A aquest usuari ja se li ha creat una cançó'. En qualsevol altre cas s'ha de generar l'excepció: 'Error intern'.

```
CREATE OR REPLACE FUNCTION crearNovaCanço() RETURNS TRIGGER AS $$
DECLARE
   maxIdCanço integer;
BEGIN
    IF (NEW.tarifa = 'premium') THEN

        SELECT MAX(idCanço) INTO maxIdCanço FROM Cançons;
        IF (NOT FOUND) THEN
            maxIdCanço := 1;
        ELSE
            maxIdCanço := maxIdCanço + 1;
        END IF;

        INSERT INTO cançons
        VALUES (maxIdCanço, NEW.nomUsuari, NEW.nomUsuari, 0);
    END IF;

    RETURN NULL;

    EXCEPTION
        WHEN unique_violation THEN
        RAISE EXCEPTION '%','Usuari ja se li ha creat una cançó';
        WHEN others THEN
        RAISE EXCEPTION '%','Error intern';
```

```
END;
$$ LANGUAGE PLPGSQL;


CREATE OR REPLACE TRIGGER tg_novaCanço
AFTER INSERT OR UPDATE OF tarifa ON usuaris
FOR EACH ROW EXECUTE PROCEDURE crearNovaCanço();
```

Podeu fer servir la plantilla de triggers següent.

```
CREATE OR REPLACE FUNCTION actualitza_lloger() RETURNS TRIGGER AS $$
DECLARE
   ...  ...
BEGIN
    ...  ...
EXCEPTION
   WHEN { raise_exception | unique_violation | foreign_key_violation | check_violation | others }
   THEN
     RAISE EXCEPTION '%', missatge;
   ...
END;
$$ LANGUAGE PLPGSQL;

CREATE TRIGGER disparador {BEFORE | AFTER}{esdeveniment[OR ...]} ON taula
FOR [EACH] {ROW | STATEMENT} EXECUTE PROCEDURE actualitza_lloger();
```

Temps: 2 hores

Notes 18 novembre tarda **Revisió: 20 novembre tarda** **Cada pregunta en un full separat**

1. **(2.5 punts)** Considereu l'esquema de la base de dades següent:

```sql
CREATE TABLE usuaris (
     mail_usuari VARCHAR(50) PRIMARY KEY,
     nom VARCHAR(50) NOT NULL,
     codi_pais INTEGER,
     instant_ultim_login INTEGER);

CREATE TABLE plansSubscripcio (
     nom_pla VARCHAR(50) PRIMARY KEY,
     preu FLOAT NOT NULL DEFAULT(5));

CREATE TABLE subscripcions (
     mail_usuari VARCHAR(50),
     nom_pla VARCHAR(50),
     instant_inici INTEGER NOT NULL,
     instant_fi INTEGER,
     PRIMARY KEY (mail_usuari,nom_pla,instant_inici),
     FOREIGN KEY (mail_usuari) REFERENCES usuaris,
     FOREIGN KEY (nom_pla) REFERENCES plansSubscripcio
);
-- instant_fi NULL indica que la subscripció és vigent

CREATE TABLE videos (
     titol VARCHAR(100) PRIMARY KEY,
     durada INTEGER NOT NULL,
     puntuacio FLOAT,
     premium char(2) CHECK(premium='SI' OR premium='NO'),
     CHECK (durada > 0));

CREATE TABLE visualitzacions (
     mail_usuari VARCHAR(50),
     titol VARCHAR(100),
     instant_visualitzacio INTEGER,
     PRIMARY KEY (mail_usuari,titol,instant_visualitzacio),
     FOREIGN KEY (mail_usuari) REFERENCES usuaris,
     FOREIGN KEY (titol) REFERENCES videos);
```

1.1. Doneu una sentència SQL per obtenir per cada vídeo del que s'ha fet alguna visualització, la quantitat de visualitzacions que han fet usuaris diferents. Al resultat de la consulta es vol el títol, la durada, la puntuació del vídeo i la quantitat de visualitzacions.

```
SELECT v.titol, v.durada, v.puntuacio, COUNT(DISTINCT vi.mail_usuari)
FROM videos v NATURAL INNER JOIN visualitzacions vi
GROUP BY v.titol
```

1.2. Doneu una sentència SQL per obtenir el nom i el codi de país dels usuaris *inactius*. Els usuaris *inactius* són aquells que no tenen cap subscripció vigent o bé que no han fet mai cap visualització.

```
SELECT u.nom, u.codi_pais
FROM usuaris u
WHERE NOT EXISTS (SELECT *
                  FROM subscripcions s
                  WHERE u.mail_usuari = s.mail_usuari AND
                        s.instant_fi IS NULL)
UNION
SELECT u.nom, u.codi_pais
FROM usuaris u
WHERE NOT EXISTS (SELECT * FROM visualitzacions v
                  WHERE u.mail_usuari = v.mail_usuari)
```

o bé:

```
SELECT DISTINCT u.nom, u.codi_pais
FROM usuaris u
WHERE NOT EXISTS (SELECT *
                  FROM subscripcions s
                  WHERE u.mail_usuari = s.mail_usuari AND
                        s.instant_fi IS NULL)
OR NOT EXISTS (SELECT * FROM visualitzacions v
                  WHERE u.mail_usuari = v.mail_usuari)
```

2. (1.5 punts) Tenint en compte la base de dades de l'exercici 1, suposeu la sentència SQL següent:

```
SELECT vi.titol
FROM videos vi, visualitzacions v
WHERE vi.titol = v.titol AND
      vi.puntuacio > 3
GROUP BY vi.titol
HAVING COUNT(DISTINCT v.mail_usuari) >= 2;
```

2.1. Escriviu una frase que expressi quines són les dades que s'obtindrien a partir de l'execució de la sentència SQL anterior.

S'obté el títol dels vídeos amb una puntuació superior a 3 que els han visualitzat 2 o més usuaris diferents.

2.2. És possible donar una seqüència d'operacions d'àlgebra relacional que doni el mateix resultat que la consulta anterior?

- En cas afirmatiu doneu la seqüència d'operacions.
- En cas negatiu, expliqueu perquè no ho és.

```
A=VIDEOS(puntuacio>3)
B=A*VISUALITZACIONS
C=B[mail_usuari,titol]
D=C{mail_usuari->mail2,titol->titol2}
E=C[mail_usuari<>mail2,titol=titol2]D
R=E[titol]
```

3. (2 punts) Suposeu la base de dades de l'exercici 1. Implementeu amb una asserció la restricció següent: No hi pot haver cap visualització d'un vídeo premium feta per un usuari que no tingui cap subscripció vigent en l'instant de la visualització.

```
CREATE ASSERTION visualitzacions_anti_pirates CHECK (
    NOT EXISTS (SELECT *
        FROM visualitzacions v, videos vi
        WHERE v.titol = vi.titol AND
            vi.premium ='SI' AND
              NOT EXISTS (SELECT * FROM subscripcions s
                  WHERE s.mail_usuari = v.mail_usuari AND
                      v.instant_visualitzacio >= s.instant_inici AND
                      (s.instant_fi IS NULL OR
                      v.instant_visualitzacio <= s.instant_fi))))
```

4. (2 punts) Suposeu la base de dades de l'exercici 1.

4.1. Doneu una vista sobre la base de dades que permeti consultar les dades de les subscripcions vigents. Doneu també una sentència d'actualització de la vista que doni error en cas que la vista estigui definida WITH CHECK OPTION, i que es pugui executar en cas que no s'inclogui aquesta clàusula.

```
CREATE VIEW subscripcionsVigents AS
        SELECT *
        FROM subscripcions s
        WHERE s.instant_fi IS NULL;
INSERT INTO subscripcionsVigents VALUES ('a@b.com', 'gold',124,136);
// qualsevol sentència d'actualització que posi un valor diferent de nul a l'instant_fi
```

4.2. Suposant que la base de dades és propietat de l'Ona, doneu la/es sentència/es SQL necessària/es per tal d'autoritzar a l'usuari Ton per a que pugui consultar (però no modificar) les dades dels vídeos premium que tenen una puntuació per sota de la mitjana de les puntuacions de tots els vídeos premium de la plataforma.

```
CREATE VIEW videosSotaMitjana AS
        SELECT *
        FROM videos v
        WHERE v.premium = 'SI' AND
                v.puntuacio < (SELECT AVG(v1.puntuacio)
                               FROM videos v1
                               WHERE v1.premium = 'SI')
GRANT SELECT ON videosSotaMitjana TO Ton;
```

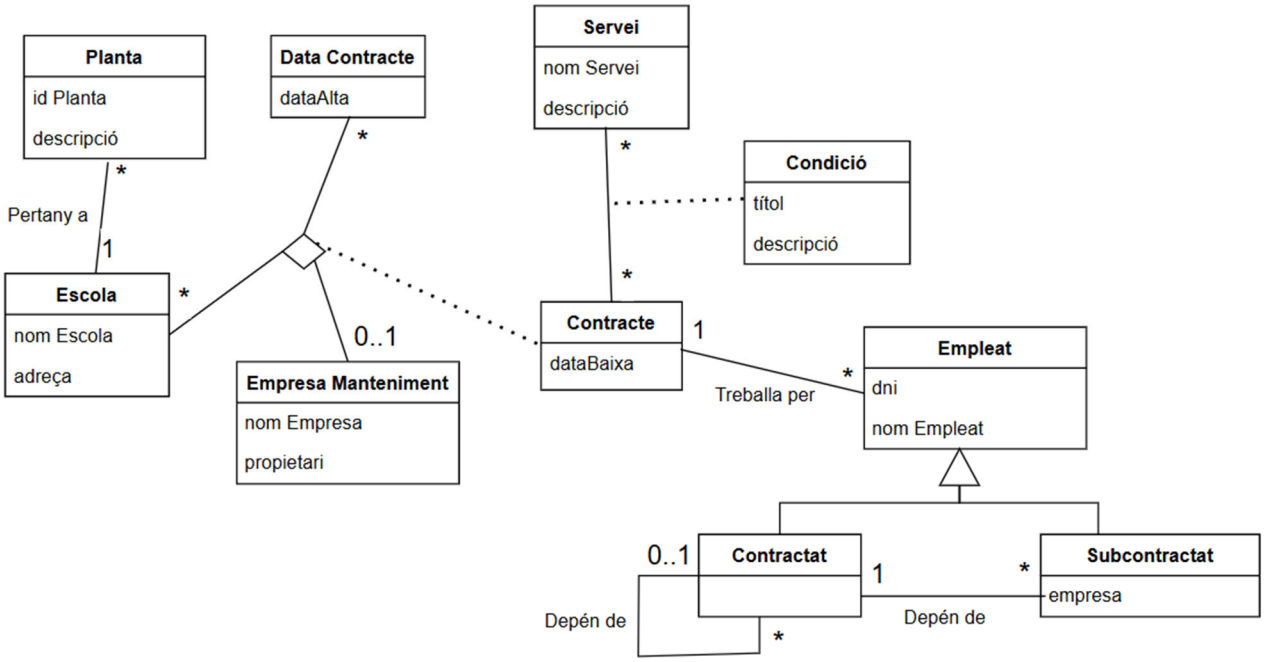
4.3. Suposeu que a la base de dades hi ha l'usuari 'a@b.com' que té 3 subscripcions, de les quals una és vigent, i ha fet 500 visualitzacions. És possible amb una única operació DELETE esborrar totes les dades d'aquest usuari emmagatzemades a la base de dades?

- En cas afirmatiu, doneu la sentència SQL per esborrar totes les dades de l'usuari.
- En cas negatiu, expliqueu perquè no ho és, i justifiqueu si es podria o no fer un canvi en la definició de les taules per tal que fos possible.

  No és possible, perquè la política de FOREIGN KEY per defecte és RESTRICT.

  Per tal que fos possible, caldria definir que les FOREIGN KEY de les taules *subscripcions* i *visualitzacions* fossin ON DELETE CASCADE.

5. (2 punts) Donat el model conceptual següent en UML:



I les restriccions textuals següents:

- Clau externa Escola: nom Escola
- Clau externa Planta: No poden existir dues plantes amb el mateix id Planta en una mateixa escola, però sí en escoles diferents
- Clau externa Empresa Manteniment: nom Empresa
- Clau externa Servei: nom Servei
- Clau externa Empleat: dni

Es demana fer la traducció a model relacional donant el disseny lògic de la base de dades resultant.

Concretament cal indicar:

- taules
- atributs
- claus primàries
- claus foranes
- restriccions NOT NULL per a les claus foranes que ho requereixin
- restriccions UNIQUE en cas d'existir claus alternatives

Solució

```
escoles(nomEscola, adreça)
plantes(nomEscola, idPlanta, descripcio)
  {nomEscola} referencia escoles
empresesManteniment(nomEmpresa, propietari)
contractes(nomEscola, dataAlta, nomEmpresa, dataBaixa)
  {nomEscola} referencia escoles
```

{nomEmpresa} referencia empresesManteniment NOT NULL

serveis(nomServei, descripcio)

condicions(nomEscola, dataAlta, nomServei, titol,
descripcio)

{nomEscola, dataAlta} referencia contractes

{nomServei} referencia serveis

empleats(dni, nomEmpleat, nomEscola, dataAlta)

{nomEscola, dataAlta} referencia contractes NOT NULL

empleatsContractats(dni, dniCap)

{dni} referencia empleats

{dniCap} referencia empleatsContractats

empleatsSubContractats(dni, dniCap, empresa)

{dni} referencia empleats

{dniCap} referencia empleatsContractats NOT NULL

BD 09.01.2025

Temps: 3 hores

Notes 21 gener tarda Revisió: 22 gener tarda Cada pregunta en un full separat

1. **(1 punt)** Considereu l'esquema de la base de dades següent:

```
CREATE TABLE usuaris
(nomUsuari char(20),
presentacio char(500),
numLlistesCreades integer CHECK (numLlistesCreades >= 0) NOT NULL,
tarifa char(7) CHECK (tarifa IN ('premium','free')) NOT NULL,
PRIMARY KEY (nomUsuari),
CHECK(numLlistesCreades<=2 OR tarifa = 'premium'));

CREATE TABLE seguidors
(nomSeguit char(20),
nomSeguidor char(20),
silenciat integer CHECK (silenciat IN (0, 1)) DEFAULT 0,
PRIMARY KEY (nomSeguit,nomSeguidor),
FOREIGN KEY (nomSeguit) REFERENCES usuaris,
FOREIGN KEY (nomSeguidor) REFERENCES usuaris);

CREATE TABLE playlist
(idList integer,
nomList char(100) NOT NULL,
tematica char(50),
creador char(20) NOT NULL,
PRIMARY KEY (idList),
FOREIGN KEY (creador) REFERENCES usuaris);

CREATE TABLE cançons
(idCanço integer,
titol char(100) UNIQUE,
cantant char(100),
numTotalReproduccions integer CHECK (numTotalReproduccions >= 0),
PRIMARY KEY (idCanço));

CREATE TABLE cançonsAPlaylist
(idList integer,
idCanço integer,
numReproduccions integer CHECK (numReproduccions >= 0) NOT NULL,
PRIMARY KEY (idList, idCanço),
FOREIGN KEY (idList) REFERENCES playlist,
FOREIGN KEY (idcanço) REFERENCES cançons);
```

Considereu el fragment següent de codi Java amb JDBC següent:

```
1   try {
2      /* Tenim ja una connexió c establerta amb la base de dades */
3      /* Tenim una variable varTitol que conté el títol d'una cançó */
4
5      PreparedStatement ps1 = c.prepareStatement(
6           "SELECT idCanço " +
7           "FROM cançons "+
8           "WHERE titol = ?"  );
9      ps1.setString(1, varTitol);
10     ResultSet rs = ps1.executeQuery();
11
12     int idCanso, idPlayL, numRepr;
13     boolean totesModificades = true;
14
15     if (rs.next()) {
16          idCanso = rs.getInt("idCanço");
17
18          Statement s1 = c.createStatement();
19          ResultSet rs1;
20          rs1 = s1.executeQuery("SELECT idList,numReproduccions " +
21                                "FROM cançonsAPlaylist " +
22                                "WHERE idCanço = " + idCanso);
23
24          while (rs1.next()) {
25              idPlayL = rs1.getInt("idList");
26              numRepr = rs1.getInt("numReproduccions");
27
28              if ((numRepr - 100) < 0) {
29                   System.out.println("Alguna cançó en playlist passaria "+
30                                      "a reproduccions negatives");
31                   totesModificades = false;
32                   break;   /* surt del while */
33                   }
34
35               PreparedStatement ps2 = c.prepareStatement(
36                             "UPDATE cançonsAPlaylist " +
37                             "SET numReproduccions = ? - 100 " +
38                             "WHERE idCanço = ? and idList = ?");
39               ps2.setInt(1, numRepr);
40               ps2.setInt(2, idCanso);
41               ps2.setInt(3, idPlayL);
42               ps2.executeUpdate();
43               }
44          }
45     if (totesModificades) c.commit();
46     else c.rollback();
47     c.close();
48     } catch (SQLException se) {
49          System.out.println ("Excepcio: "+ se.getMessage());
50     }
```

Donat aquest fragment de codi, identifiqueu quines parts del codi no compleixen els criteris de qualitat de l'assignatura. Per cada part: indiqueu els números de línia que ocupa, expliqueu quin és el criteri de qualitat que no es compleix, i expliqueu quins canvis serien necessaris en la codificació per tal de satisfer aquest criteri.

Criteris i millores:

- PreparedStatement en lloc de Statement
  - Línies 5-10
  - El SELECT sobre la taula `cançons` s'executa només 1 vegada, s'hauria d'executar amb un Statement.
- Ús incorrecte del PreparedStatement
  - Línies 35-38
  - El PreparedStatement s'ha de crear fora del while ja que només s'ha de crear una única vegada.
- Accessos innecessaris a la base de dades
  - Línies 18-42
  - El UPDATE està modificant tots els numReproduccions de les playlists on està una cançó. Es podria fer sense el WHILE amb un únic Statement UPDATE de totes les files de la taula cançonsAPlayList tals que la cançó és la que hi ha a la variable idCanso.
- Codi innecessari
  - Línies 28-33
  - No és necessaria aquesta part. En cas que alguna fila de la taula passi a tenir numReproduccions negatiu en el UPDATE saltarà una excepció degut al CHECK que es pot capturar al CATCH.
- Accessos innecessaris a la base de dades
  - Línies 5-44
  - Es pot fer tot en un únic Statement (amb una sentència UPDATE) que modifiqui les files de la taula cançonsAPlayList tals que el seu identificador és l'identificador de la cançó amb el títol donat a la variable titol. Igualment, si alguna fila de la taula passa a tenir numReproduccions negatiu saltarà una excepció degut al CHECK que es pot capturar al CATCH.

Codi de la millor resposta (no demanat a la resposta):

```
try {
     Statement s2 = c.createStatement();
     s2.executeUpdate("UPDATE cançonsAPlaylist " +
            "SET numReproduccions = numReproduccions - 100 " +
            "WHERE idCanço = (SELECT idCanço FROM cançons "+
                        "WHERE titol = '"+titol+"')");
     c.commit();
     c.close();}
 catch (SQLException se) {
      if (se.getSQLState().contentEquals("23514")) {
            System.out.println("Alguna cançó en playlist passaria "+
                              "a reproduccions negatives");}
      else {System.out.println ("Excepcio: "+ se.getMessage());}
    }
```

2. (1,5 punts) Donada la base de dades de l'inici de l'examen, es volen implementar disparadors per garantir les regles que s'indiquen a continuació.

2.1. Per a cada disparador doneu, tenint en compte els criteris de qualitat de l'assignatura:

- L'esdeveniment que l'activa (esdeveniment, taula i atributs rellevants)
- Si ha de ser BEFORE/AFTER
- Si ha de ser ROW/STATEMENT

Justificar breument la resposta per cada regla.

a) R1 – S'ha de mantenir actualitzat l'atribut derivat `tarifa`. Tarifa ha de ser "free" si es tenen menys de 1000 seguidors i "premium" si se'n tenen 1000 o més.

b) R2 – No es poden fer modificacions de la taula cançons de 2 a 3 de la matinada.

c) R3 – Un seguidor ha de ser usuari. En cas que no estigui donat d'alta com a usuari, cal inserir-lo a la taula usuaris amb tarifa "free" i numLlistesCreades 0.

a) R1
   - insert taula seguidors after/row
   - update nomSeguit taula seguidors after/row
   - delete taula seguidors after/row
   - AFTER: Es tracta d'actualitzar un atribut derivat, per això millor fer-ho un cop se sap que les restriccions d'integritat associades a l'esdeveniment no han fallat.
   - ROW: Millor fer un disparador incremental, que té en compte els usuaris a qui li han augmentat o disminuit els seguidors.

b) R2
   - update taula cançons before/statement.
   - BEFORE: Es tracta d'una restricció d'integritat. Millor comprovar les restriccions d'integritat el més abans possible.
   - STATEMENT: No és necessari saber ni quines cançons es volen modificar, ni quines cançons es volen modificar.

c) R3
   - insert taula seguidors before/row
   - update nomSeguidor taula seguidors before/row
   - BEFORE: Hauria de ser after perquè es tracta de fer accions compensatòries quan no es compleix la restricció d'integritat de clau forana, ara bé, no es pot implementar after perquè mai arribaria a executar el procediment perquè fallaria la restricció d'integritat abans.
   - ROW: No es pot implementar statement perquè l'única manera seria fer-ho after, i la restricció d'integritat fallaria abans.

2.2. El valor de l'atribut `numTotalReproduccions` d'una cançó és igual a la suma de les reproduccions de la cançó en cadascuna de les playlists en les que està.

A continuació us proporcionem una implementació d'un disparador i el seu procediment associat que permeten mantenir actualitzat l'atribut

numTotalReproduccions de la taula canço quan es fan modificacions de l'atribut numReproduccions a la taula cançonsAPlaylist.

Aquesta implementació funciona correctament però no compleix els criteris de qualitat de l'assignatura per a disparadors ja que es tracta d'una implementació no incremental, proposeu una implementació alternativa del disparador i del procediment que sigui incremental.

```
CREATE OR REPLACE FUNCTION actualitza_reproduccions()
RETURNS TRIGGER AS $$
BEGIN
     UPDATE cançons c
     SET numTotalReproduccions =
          (SELECT sum(p.numReproduccions)
          FROM cançonsAPlaylist p
          WHERE p.idCanço = c.idcanço);
     RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER mantenir_reproducccions
AFTER UPDATE OF numReproduccions ON cançonsAPlaylist
FOR EACH STATEMENT
EXECUTE PROCEDURE actualitza_reproduccions();
```

Nota: Podeu assumir que els atributs *idList* i *idCanço* d'una PlayList no es poden modificar.

La solució proposada no és incremental, a continuació es proposa una versió del disparador i procediment incrementals.

```
CREATE OR REPLACE FUNCTION actualitza_reproduccions()
RETURNS TRIGGER AS $$
BEGIN
     UPDATE cançons
     SET numTotalReproduccions = numTotalReproduccions +
               (NEW.numReproduccions-OLD.numReproduccions)
     WHERE idCanço = NEW.idcanço;
     RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER mantenir_reproducccions
AFTER UPDATE OF numReproduccions ON cançonsAPlaylist
FOR EACH ROW EXECUTE PROCEDURE actualitza_reproduccions();
```

3. (3 punts) Considereu la base de dades de l'inici de l'examen, en un SGBD on el gränul és la fila.

3.1. A partir de l'horari següent proposat, es demana:

T1	T2	T3

`select *`
`from usuaris`
`where nomUsuari='@Joan';`		

	`update usuaris`
	`set tarifa='premium'`
	`where nomUsuari='@Maria';`	

`select *`
`from usuaris`
`where nomUsuari='@Maria';`		

		`select *`
		`from usuaris`
		`where nomUsuari='@Maria';`

	`update usuaris`
	`set tarifa='premium'`
	`where nomUsuari='@Joan';`	

		`COMMIT`
	`COMMIT`	
`COMMIT`		

a) Traduïu l'horari a operacions de baix nivell

b) Doneu l'horari suposant que T1 i T3 treballen a nivell d'aïllament de READ COMMITTED i T2 treballa a nivell d'aïllament de REPEATABLE READ. L'horari ha d'incloure les operacions que executen les transaccions (R, RU, W, commit), les operacions de petició i alliberament de reserves (LOCK, UNLOCK) i les esperes que siguin necessàries. Quan més d'una transacció espera per un mateix gränul, considereu que la política de la cua és FIFO (First In, First Out).

c) L'horari resultant és serialitzable? Justifica la resposta.

d) Doneu el graf d'esperes associat a l'horari resultant de l'apartat b) just abans del commit de T2.

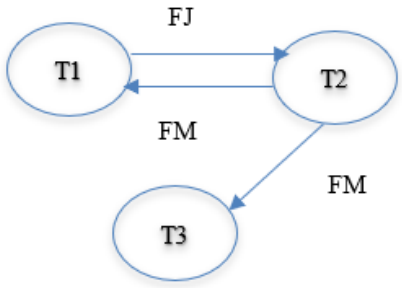
a) Anomenem la fila corresponent a l'usuari '@Joan' FJ i la fila corresponent a la usuària '@Maria' FM.

T1	T2	T3
R(FJ)		
	RU(FM)	
	W(FM)	
R(FM)		
		R(FM)
	RU(FJ)	
	W(FJ)	
		Commit
	Commit	
Commit		

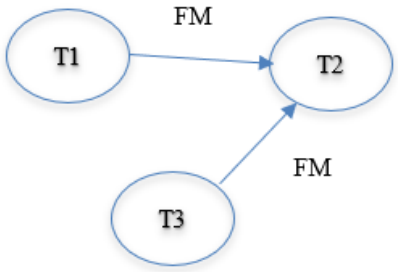
b)

T1	T2	T3
L(FJ, S)		
R(FJ)		
U(FJ)		
	L(FM, X)	
	RU(FM)	
	W(FM)	
L(FM, S)		
		L(FM, S)
	L(FJ,X)	
	RU(FJ)	
	W(FJ)	
	Commit + U(FM,FJ)	
R(FM)		
U(FM)		
		R(FM)
		U(FM)
		Commit
Commit		

c) Per saber si l'horari és serialitzable construim el graf de precedències. Com que el graf té cicles, l'horari té interferències (del tipus anàlisi inconsistent) i per tant no és serialitzable.



d) Graf d'esperes: T1 i T3 estan esperant per adquirir la reserva sobre el gràNul FM.



3.2. Considereu un SGBD amb un mecanisme de control de concurrència amb reserves, i la següent transacció (**T1**) que treballa a nivell d'aïllament SERIALIZABLE i acaba en ROLLBACK. La resta de transaccions que s'executen concurrentment amb T1 són transaccions READ ONLY i treballen a nivell READ COMMITTED, digueu quines interferències es poden produir? Justifica la resposta.

T1	`Select * from usuaris where nomUsuari='@Joan';`	`Update usuaris set tarifa='premium' where nomUsuari='@Joan';`	`ROLLBACK`
Resultat execució	`Obté les dades de l'usuari @Joan: ('@Joan', 'A excursió', 'free',1000)`	`Es canvia la tarifa de l'usuari '@Joan' a 'premium'`	

- No es poden produir actualitzacions perdudes ni lectures no confirmades entre T1 i les transaccions read only perquè el nivell d'aïllament de READ COMMITED les evita.
- Tampoc es poden produir anàlisis inconsistents perquè la transacció T1 només actualitza un gràNul i les altres transaccions són read only.
- Es podrien produir lectures no repetibles i fantasmes, entre T1 i les transaccions read only, però com T1 acaba en rollback aquesta transacció desfà tots els seus canvis i per tant les lectures de les transaccions read only donaran el mateix resultat.

4. (2,5 punts) Considereu la taula cançons que té 25.000 tuples. El factor de bloqueig de les pàgines de dades és de 15 tuples per pàgina. Disposem d'un índex agrupat sobre la columna *cantant,* amb un ordre d=75 i una ocupació del 70%.

Mostreu com heu fet els càlculs que es demanen als apartats següents.

4.1. Determineu el nombre de pàgines necessàries per a emmagatzemar l'índex. Justifiqueu la resposta.

Valors per node (vpn) = ⌈2*d*ocupació⌉ = ⌈2 * 75 * 0,7⌉ = 105

Número d'apuntadors per node (nap) = vpn + 1 = 105 + 1 = 106

Nivell fulla: ⌈25.000 valors a guardar / 105 valors per node⌉ = 239 apuntadors al nivell anterior

Nivell intern 1: ⌈239 apuntadors / 106 apuntadors per node⌉ = 3 apuntadors al nivell anterior

Nivell intern 2: ⌈3 apuntadors / 106 apuntadors per node⌉ = 1 node arrel

Número de pàgines necessàries per emmagatzemar l'índex: 239+3+1 = 243

4.2. Considereu la consulta següent. Sabem que el cantant *Pere Altaveu* té un total de 900 cançons. Calculeu el cost en número de pàgines d'índex i dades a les que cal accedir per resoldre la consulta usant l'índex. Justifiqueu la resposta.

```
SELECT titol
FROM cançons
WHERE cantant = 'Pere Altaveu' AND
      numTotalReproduccions > 150
```

Fórmula: Cost d'accés a l'índex + Cost d'accés a dades

Cost accés a l'índex = h = 3 (calculat abans) – noteu que al ser un índex agrupat no cal accedir a pàgines fulla addicionals

Cost accés a dades = ⌈|cançons(cantant = 'Pere Altaveu')| / fb⌉ = ⌈900/15⌉ = 60

Cost en número de pàgines per resoldre la consulta: 60 + 3 = 63

4.3. Suposeu ara que disposem **a més** d'un índex no agrupat per la columna *numTotalReproduccions* amb ordre d=60 i ocupació del 80%. Tingueu en compte que del total de cançons de la base de dades, n'hi ha 2.250 que tenen més de 150 reproduccions, i d'aquestes 20 són del cantant *Pere Altaveu*. Justifiqueu calculant els costos si és millor des del punt de vista de costos utilitzar un dels dos índexs o usar una estratègia d'intersecció de RIDs per a resoldre la consulta anterior. Justifiqueu la resposta.

Apliquem l'índex per numReproduccions

Fórmula: Cost d'accés a l'índex + Cost d'accés a dades

Calculem el cost d'accés a l'índex *numReproduccions*:

- vpn = ⌈2*d*ocupació⌉ = ⌈2 * 60 * 0,8⌉ = 96
- nap = vpn + 1 = 96 + 1 = 97
- h = ⌈$\log_{97}$(25000) ⌉ = 2
- F = ⌈|cançons(numReproduccions > 150)|/nvpn⌉ - 1 = ⌈2250/96⌉ - 1 = 23

Cost d'accés a l'índex *numReproduccions*: h + F = 2 + 23 = 25

Cost accés a dades = |cançons(numReproduccions>150)| = 2250

Cost en número de pàgines per resoldre la consulta: 25 + 2250

Apliquem estratègia d'intersecció de RIDs.

Fórmula: Cost d'accés a l'índex de *cantant* + Cost d'accés a l'índex de *numTotalReproduccions* + Cost d'accés a dades

1) Cost d'accés a l'índex *cantant*: h + F = h + ⌈|cançons(cantant = 'Pere Altaveu')| / nvpn⌉ - 1 = 3 + ⌈900 / 105⌉ – 1 = 3 + 8 = 11
   (Noteu que en el cas d'estratègia d'intersecció per RIDs sí que cal accedir a les fulles addicionals)

2) Cost d'accés a l'índex numReproduccions (calculat abans) = 25

3) Cost d'accés a dades: |cançons(cantant = 'Pere Altaveu' AND numReproduccions > 150)| = 20

Aplicant l'estratègia d'intersecció de RIDs el cost és: 11 + 25 + 20 = 56.

Aquest cost és menor que usar l'índex de *cantant* i l'índex de *numReproduccions*.

4.4. Considereu que existeixen els dos índexs indicats anteriorment. Expliqueu les diferents alternatives que podria usar el SGBD per executar la consulta següent (no es demana calcular els costos).

```
SELECT titol
FROM cançons
WHERE cantant = 'Pere Altaveu' OR
numTotalReproduccions > 150
```

- No té sentit un únic índex perquè no serveix de res al haver-hi un OR.
- Es pot fer un escaneig seqüencial de les pàgines de dades.
- Es pot fer una estratègia d'unió de RIDs. És a dir obtenir els RIDs que compleixen la primera condició, obtenir els que compleixen la segona, fer la unió i accedir a les files que estan en la unió de RIDs.

5. **(2 punts)** Implementeu un procediment emmagatzemat tal que, donats l'identificador, el nom, la temàtica i l'usuari creador d'una playlist, afegeixi la playlist o, si ja existeix, actualitzi les seves dades. El procediment també ha de mantenir actualitzat l'atribut `numLlistesCreades` de la taula usuaris (aquest atribut manté per cadascun dels usuaris el número de llistes que ha creat). Finalment, el procediment ha de gestionar les possibles excepcions mostrant aquests missatges:
   - La temàtica no pot ser nul·la.
   - El creador no pot ser nul.
   - Un usuari free no pot crear més de 2 playlists
   - El nou creador no existeix.

   Utilitzeu la plantilla següent com a referència:

```
CREATE OR REPLACE FUNCTION actualitzar_playlist(pidList INTEGER,
        pnomList char(100), ptematica CHAR(50), pcreador char(20))
    RETURNS void AS $$
    DECLARE
    ...
    BEGIN
    ...
    EXCEPTION
    WHEN { raise_exception | unique_violation | not_null_violation |
            foreign_key_violation | check_violation | others }
        THEN RAISE EXCEPTION '%', missatge;
    ...
    END;
    $$ LANGUAGE PLPGSQL;
```

```
CREATE FUNCTION actualitzar_playlist(p_idList INTEGER,
           p_nomList char(100), p_tematica CHAR(50),
           p_creador char(20)) RETURNS void AS $$
DECLARE
 var_creador char(20);
BEGIN
  IF p_tematica IS NULL THEN
           RAISE EXCEPTION 'La temàtica no pot ser nul·la';
  END IF;

  SELECT p_creador INTO var_creador
  FROM playlist
  WHERE p.idList = p_idList;

  IF NOT FOUND THEN
           INSERT INTO playlist
           VALUES (p_idList,p_nomList,p_tematica,p_creador);
  ELSE
      UPDATE playlist p
      SET  p.tematica = p_tematica,
           p.creador = p_creador,
           p.nou_nomList = p_nomList
      WHERE p.idList = p_idList;

      UPDATE usuaris
      SET numLlistesCreades = numLlistesCreades - 1
      WHERE nomUsuari = var_creador;
  END IF;

  UPDATE usuaris
  SET numLlistesCreades = numLlistesCreades + 1
  WHERE nomUsuari = p_creador;

  EXCEPTION
  WHEN raise_exception THEN RAISE EXCEPTION '%',SQLERRM;
  WHEN not_null_violation THEN
           RAISE EXCEPTION 'El creador no pot ser nul';
  WHEN foreign_key_violation THEN
           RAISE EXCEPTION 'El nou creador no existeix';
  WHEN check_violation THEN RAISE EXCEPTION 'Un usuari free
  no pot crear més de 2 playlists';
  WHEN OTHERS THEN
           RAISE EXCEPTION 'Altres errors';
END;
$$ LANGUAGE plpgsql;
```

Temps: 1 hora i 45 minuts

Notes 22 abril tarda **Revisió: 23 abril tarda** **Cada pregunta en un full separat**

1. **(3,5 punts)** Considereu l'esquema de la base de dades següent:

```
CREATE TABLE persones (
email varchar(100) PRIMARY KEY,
nomPersona varchar(100) NOT NULL,
telefon varchar(15),
edat integer);

CREATE TABLE edicionsCongressos (
nomCongres varchar(150),
edicio integer CHECK (edicio > 2000 and edicio <= 2030),
descripcio varchar(255),
pressupost float NOT NULL CHECK (pressupost > 0),
PRIMARY KEY (nomCongres, edicio));
-- Una fila per cada congrés i any en que es fa.
-- P.ex: hi haurà una fila pel MWC del 2025, i una pel MWC del 2024

CREATE TABLE participants (
email varchar(100),
nomCongres varchar(150),
edicio integer,
rol varchar(20) CHECK (rol IN ('organitzador', 'basic')),
PRIMARY KEY (email, nomCongres, edicio),
FOREIGN KEY (email) REFERENCES persones,
FOREIGN KEY (nomCongres, edicio) REFERENCES edicionsCongressos);
-- Una fila per cada persona participant en una edició d'un congrés

CREATE TABLE activitats (
nomActivitat varchar(150),
nomCongres varchar(150),
edicio integer,
descripcio varchar(255),
preu float CHECK (preu >= 0),
coordinador varchar(100) NOT NULL,
PRIMARY KEY (nomActivitat, nomCongres, edicio),
FOREIGN KEY (nomCongres, edicio) REFERENCES edicionsCongressos,
FOREIGN KEY (coordinador) REFERENCES persones);
-- Una fila per cada activitat que es fa en una edició d'un congrés
```

```
CREATE TABLE inscripcions (
inscrit varchar(100),
nomActivitat varchar(150),
nomCongres varchar(150),
edicio integer,
pagada date,
PRIMARY KEY (inscrit, nomActivitat, nomCongres, edicio),
FOREIGN KEY (inscrit) REFERENCES persones,
FOREIGN KEY (nomActivitat, nomCongres, edicio) REFERENCES
activitats);
-- Hi ha una fila per cada inscrit en una activitat
-- pagada té valor nul quan encara no s'ha pagat la inscripció
```

1.1. Doneu una sentència SQL per obtenir l'identificador de les activitats amb alguna inscripció no pagada, i amb totes les inscripcions que siguin de persones amb edat superior a la mitjana de les edats de les persones de la base de dades.

```
SELECT DISTINCT i.nomActivitat, i.nomCongres, i.edicio
FROM inscripcions i
WHERE i.pagada is null AND
      NOT EXISTS (SELECT *
            FROM inscripcions ii, persones p
            WHERE ii.inscrit = p.eMail AND
               ii.nomCongres = i.nomCongres AND
               ii.edicio = i.edicio AND
               ii.nomActivitat = i.nomActivitat AND
               p.edat <= (SELECT avg(p1.edat)
                     FROM persones p1));
```

1.2. Doneu una sentència SQL per obtenir els noms dels congressos amb més de 300 participants amb rol 'bàsic' que siguin persones amb edat superior a 30 anys en alguna de les edicions del congrés.

```
SELECT DISTINCT pa.nomCongres
FROM participants pa natural inner join persones p
WHERE pa.rol = 'bàsic' AND p.edat>30
GROUP BY pa.nomCongres, pa.edicio
HAVING COUNT(*)>300;
```

1.3. Doneu una seqüència d'operacions d'àlgebra relacional per obtenir el mail de les persones que han sigut en alguna ocasió coordinadores d'una o més activitats, però mai d'una activitat que tingui inscrits, o han participat en alguna ocasió en una edició d'un congrés però no han coordinat cap activitat d'aquesta edició del congrés.

A = activitats[coordinador]

B = activitats*inscripcions

C = B[coordinador]

D = A-C

E = participants[email,nomCongres,edicio]

F = activitats[coordinador,nomCongres,edicio]

G = E{email -> coordinador}

H = G-F

J = H[coordinador] /* correcció solució abril 2026 */

R = D _u_ H

2. (2 punts)

2.1. El propietari de la base de dades de l'exercici 1 ha executat les sentències següents:

```
CREATE VIEW act1000 AS
   SELECT * FROM activitats
   WHERE preu <= 1000;
GRANT ALL ON act1000 TO Pastor WITH GRANT OPTION;
GRANT SELECT ON activitats TO Pastor WITH GRANT OPTION;
```

Digueu quines sentències ha d'executar a continuació l'usuari Pastor per tal que la usuària Sirena pugui fer el següent. Els privilegis otorgats a la usuària Sirena han de ser els mínims.

- Crear activitats amb un preu que no superi 500 euros.
- Consultar activitats amb un preu menor o igual a 2000 euros.
- Modificar activitats amb un preu menor o igual a 1000 euros, però sense poder modificar el preu ni l'identificador de l'activitat.

```
CREATE VIEW act500 AS
  SELECT * FROM act1000
  WHERE preu <= 500 WITH CHECK OPTION;
GRANT INSERT ON act500 TO Sirena;

CREATE VIEW act2000 AS
  SELECT * FROM activitats
  WHERE preu <= 2000;
GRANT SELECT ON act2000 TO Sirena;

GRANT UPDATE(descripcio, coordinador) ON act1000 TO Sirena;
```

2.2. Suposeu que a la taula edicionsCongressos hi ha 1000 files amb 100 noms de congressos diferents. Considereu les sentències següents i digueu quantes files es llegeixen/afegeixen com a màxim per cadascun dels SELECT S1 i S2 i INSERT I1 i I2. Tingueu en compte que les files que es llegeixen són les que el SGBD ha de llegir del seu emmagatzematge físic. Justifiqueu breument la resposta.

```
CREATE TABLE t
    (nomCongres varchar(150),
    quantitat integer);
INSERT INTO t
    (SELECT nomCongres, COUNT(*)
    FROM edicionsCongressos
    GROUP BY nomCongres);
CREATE VIEW v1 AS
    SELECT nomCongres, COUNT(*)
    FROM edicionsCongressos
    GROUP BY nomCongres;
CREATE VIEW v2 AS
    SELECT * FROM t;

(S1) SELECT * FROM v1;
(S2) SELECT * FROM v2;

(I1) INSERT INTO v1 VALUES ('Maragalliada', 2);
(I2) INSERT INTO v2 VALUES ('Maragalliada', 2);
```

sentència	nombre de files
S1	1000, la vista no està materialitzada, cal calcular el resultat a partir del contingut de la taula congressos
S2	100, la vista està definida sobre t que tindrà emmagatzemades 100 files
I1	0, la vista no és actualitzable, dóna error
I2	1, la vista és actualitzable perquè està definida sobre una única taula, sense funcions, sense group by, i té tots els atributs amb restricció no nul que no tenen valor per defecte.

3. (3 punts)

3.1. Considerant la base de dades donada a l'inici de l'examen. Per cada una de les restriccions o comportaments següents:

- Si es pot implementar com a restricció en la definició de les taules **doneu** el codi SQL que cal afegir a la sentència CREATE TABLE (indicant clarament el nom de la taula on s'afegiria).
- Si no es pot implementar com a restricció de taula, però si com a asserció, **doneu** la definició en SQL estàndard de l'asserció.
- Si no es pot implementar com a restricció de taula i tampoc com a asserció **expliqueu perquè, i indiqueu** com es pot implementar amb altres components que hagueu estudiat fins ara a l'assignatura.

A. Una persona no pot coordinar més d'una activitat.

B. Una persona no es pot inscriure dues vegades a la mateixa activitat d'una mateixa edició d'un congrés.

C. Les persones inscrites a una activitat han de ser obligatòriament participants en la mateixa edició del congrés on es fa l'activitat.

D. Tota persona inscrita a alguna activitat només podrà consultar i modificar les seves inscripcions.

E. No hi pot haver cap persona que organitzi o hagi organitzat més de dues edicions d'un mateix congrés després de 2021.

F. Si el preu d'una activitat és superior a 200 euros, la descripció de l'activitat ha de ser `'prèmium'` o `'avançat'`.

G. A l'esborrar una activitat de la base de dades també caldrà esborrar les inscripcions a aquesta activitat.

A) Afegir **UNIQUE** (coordinador) a Activitats

B) Aquesta restricció ja està implementada a la BD amb PK de la taula inscripcions.

C) Afegir **FOREIGN KEY** (email, nomCongres, edicio) **REFERENCES** Participants a la taula Inscripcions.

D) Aquesta restricció no es pot comprovar amb restriccions de taula, columna ni assercions. Cal definir una vista per seleccionar les inscripcions d'una persona i donar privilegis de select i update sobre aquesta vista a la persona.

E) Aquesta restricció es pot implementar amb una asserció.

```
CREATE ASSERTION A8 CHECK(
        NOT EXISTS( SELECT email, nomCongres
                FROM participants
                WHERE rol = 'organitzador' AND
                        edicio >= 2021
                GROUP BY email, nomCongres
                HAVING COUNT(*) > 2))
```

F) Afegir **CHECK** (preu<=200 OR (descripció IN ('prèmium','avançat')) a activitats.

G) Afegir **FOREIGN KEY** (nomActivitat, nomCongres, edicio)
REFERENCES activitats(nomActivitat, nomCongres, edicio)
ON DELETE CASCADE; a la taula d'inscripcions.

3.2. Donat la seqüència de sentències SQL següents que s'executen d'una en una des del DBeaver, indiqueu quin és el resultat que retorna el SELECT. Justifiqueu breument la resposta.

```
INSERT INTO persones
VALUES('clara.vila@exemple.com', 'Clara Vila', '612345678',29);

INSERT INTO persones
VALUES('david.grau@exemple.com', 'David Grau', '611223344',31);

INSERT INTO edicionsCongressos
VALUES('EduTech', 2024, 'Tecnologia en educació', 120000);

INSERT INTO edicionsCongressos
VALUES ('HealthData', 2025, 'Dades i salut digital', 200000);

INSERT INTO activitats
VALUES ('Realitat Virtual', 'EduTech', 2024, 'Aplicacions en aula', 15.0,
'clara.vila@exemple.com');

INSERT INTO activitats
VALUES ('Plataformes LMS', 'EduTech', 2024, 'Comparativa de plataformes
educatives', 10.0, 'david.grau@exemple.com');

INSERT INTO activitats
VALUES ('Anàlisi predictiva', 'HealthData', 2025, 'Modelització de dades
mèdiques', 25.0, 'clara.vila@exemple.com');

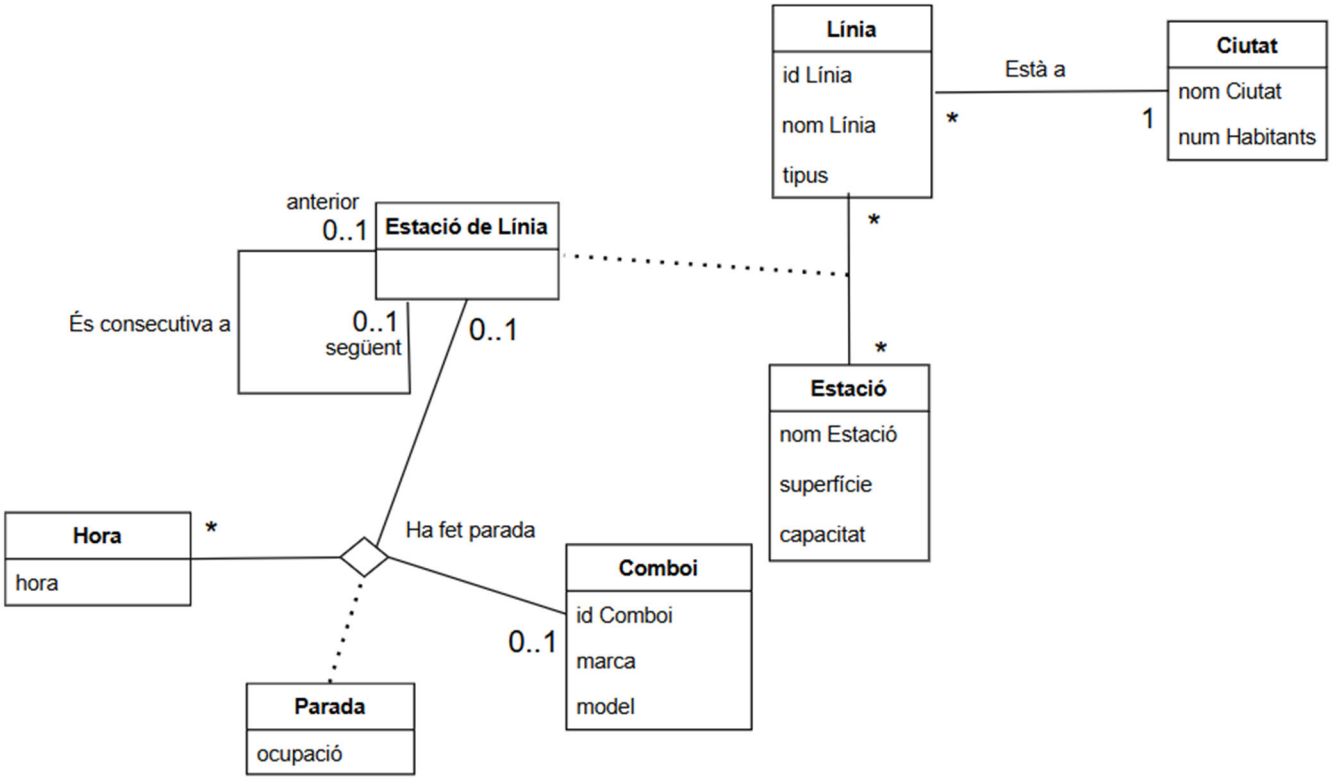
INSERT INTO activitats VALUES
('Privacitat i dades', 'HealthData', 2025, 'Discussió sobre seguretat i
privacitat', 20.0, 'anna.gomez@exemple.com');

SELECT nomCongres, edicio, COUNT(*)
FROM activitats
GROUP BY nomCongres, edicio
ORDER BY nomCongres,edicio;
```

Si les sentències insert s'executen d'una en una, com que l'execució de la darrera sentència falla per un error de FK (perquè la usuària organitzadora no existeix a la BD), el select retornarà:

nomCongres	edició	count(*)
Edutech	2024	2
HealthData	2025	1

4. (1.5 punts) Donat el model conceptual en UML següent:



I les restriccions textuals següents:

- Clau externa Ciutat: nom Ciutat
- Clau externa Línia: Id Línia
- Clau externa Estació: nom Estació
- Clau externa Comboi: id Comboi
- Clau externa Hora: hora
- Restricció addicional: Dues estacions de línia únicament poden ser consecutives si són de la mateixa línia.

4.1. Es demana fer la traducció a model relacional donant el disseny lògic de la base de dades resultant. Concretament cal indicar:

- taules
- atributs
- claus primàries
- claus foranes
- restriccions NOT NULL per a les claus foranes que ho requereixin
- restriccions UNIQUE en cas d'existir claus alternatives
- altres restriccions que es poden definir com a restriccions de columna o taula que es puguin derivar de les restriccions textuals addicionals

```
Solució:
CIUTAT (nomCiutat, numHabitants)
LINIA (idLinia, nomLinia, tipus, nomCiutat)
    on {nomCiutat} referencia CIUTAT  NOT NULL
ESTACIÓ (nomEstació, superficie, capacitat)
ESTACIO_DE_LINIA (idLinia, nomEstacio, idLiniaSeg, nomEstacioSeg)
   on {idLinia} referencia LINIA
   on {nomEstacio} referencia ESTACIO
   on {idLiniaSeg, nomEstacioSeg} referencia ESTACIO_DE_LINIA
   UNIQUE (idLiniaSeg, nomEstacioSeg)
   CHECK (idLinia = idLiniaSeg)
// Per mantenir la restrició addicional també es podria posar un únic
atribut idLinia.
COMBOI(idComboi, marca, model)
HORA (hora)
PARADA (idComboi, hora, idLinia, nomEstacio, ocupacio)
    on {idComboi} referencia COMBOI
    on {hora} referencia HORA  // OPCIONAL
    on {idLinia, nomEstacio} referencia ESTACIO_DE_LINIA
    {idLinia, nomEstacio} NOT NULL
    UNIQUE (hora, idLinia, nomEstacio)

//Implementar HORA com a taula és OPCIONAL. També és vàlid
implementar-ho com un atribut NOT NULL [veure diapo. 306]
```

4.2. Digueu si són certes o falses les afirmacions següents tenint en compte el UML donat a l'enunciat. **Si una afirmació és certa,** doneu un contingut de la base de dades que ho mostri. **Si una afirmació és falsa**, indiqueu què hauria de canviar al UML per tal que fos certa.

a) En dues ciutats diferents hi pot haver dues línies amb un mateix id Línia.

b) Un comboi en una hora determinada pot parar a més d'una estació d'una línia.

c) Hi poden haver línies circulars. És a dir, línies de metro que no tenen inici ni final perquè són circulars.

a) **Fals. Per tal que fos certa caldria canviar la clau externa de Línia, indicant que hi pot haver un mateix id de linia en ciutats diferents però no en una matexa ciutat.**

b) **Fals. Per tal que fos certa caldria canviar la multiplicitat de l'associació ternària per ser molts-molts-un. Pasant a "*" a la banda Estació de Línia.**

c) **Certa.**

estacioDeLinia(idLinia, nomEstacio, idLiniaSeg, nomEstacioSeg)

L20	'ABC'	L20	'DEF'
L20	'DEF'	L20	'GHI'
L20	'GHI'	L20	'JKL'
L20	'JKL'	L20	'ABD'

Temps: 3 hores

Notes 25 juny tarda **Revisió: 26 juny tarda** **Cada pregunta en un full separat**

1. **(1 punt)** Considereu l'esquema de la base de dades següent:

```
CREATE TABLE persones (
email varchar(100) PRIMARY KEY,
nomPersona varchar(100) NOT NULL,
telefon varchar(15),
edat integer);

CREATE TABLE edicionsCongressos (
nomCongres varchar(150),
edicio integer CHECK (edicio > 2000 and edicio <= 2030),
ciutat varchar(150),
descripcio varchar(255),
pressupost integer NOT NULL CHECK (pressupost > 0),
PRIMARY KEY (nomCongres, edicio));
-- Una fila per cada congrés i any en que es fa.
-- P.ex: hi haurà una fila pel MWC del 2025, i una pel MWC del 2024

CREATE TABLE activitats (
nomActivitat varchar(150),
nomCongres varchar(150),
edicio integer,
descripcio varchar(255),
preu float CHECK (preu >= 0),
coordinador varchar(100) NOT NULL,
PRIMARY KEY (nomActivitat, nomCongres, edicio),
FOREIGN KEY (nomCongres, edicio) REFERENCES edicionsCongressos,
FOREIGN KEY (coordinador) REFERENCES persones);
-- Una fila per cada activitat que es fa en una edició d'un congrés

CREATE TABLE inscripcions (
inscrit varchar(100),
nomActivitat varchar(150),
nomCongres varchar(150),
edicio integer,
pagada date,
PRIMARY KEY (inscrit, nomActivitat, nomCongres, edicio),
FOREIGN KEY (inscrit) REFERENCES persones,
FOREIGN KEY (nomActivitat, nomCongres, edicio) REFERENCES
activitats);
-- Hi ha una fila per cada inscrit en una activitat
-- pagada té valor nul quan encara no s'ha pagat la inscripció
```

Considereu el fragment de codi Java/JDBC següent. **Doneu el codi que hi ha d'haver en la part etiquetada com a //codi a implementar** per tal que el programa:

- Modifiqui el preu de cadascuna de les activitats de la base de dades. Després de l'execució del programa cada activitat ha de tenir com a preu el preu mitjà de les activitats fetes en la seva mateixa edició de congrés.
- Mostri per pantalla quantes activitats hi ha en total a la base de dades.

```
try
  {
    /* Tenim ja una connexió c establerta amb la base de dades */

    //codi a implementar

    System.out.println ("Hi ha " + q + " activitats.");
  }
catch (SQLException se)
   {
    System.out.println ("Excepció es: " + se.getMessage());
  }
```

Els mètodes/codis d'excepció de JDBC estudiats a classe són els següents:

Statement

- `Statement createStatement();`
- `ResultSet executeQuery(String sql);`
- `int executeUpdate(String sql);`

PreparedStatement

- `PreparedStatement prepareStatement(String sql);`
- `ResultSet executeQuery();`
- `int executeUpdate();`
- `void setXXX(int posicio, XXX valor);`

ResultSet

- `boolean next();`
- `XXX getXXX(String nomColumna)`

SQLException

- `// 23502 -not_null_violation`
- `// 23503 -foreign_key_violation`
- `// 23505 -unique_violation`
- `// 23514 -check_violation`
- `String getSQLState();`

```
try
   {
   /* Tenim ja una connexió c establerta amb la base de dades */

   Statement s = c.createStatement();
   ResultSet r = s.executeQuery("SELECT nomCongres, edicio, AVG(preu) AS preu "+
                                "FROM activitats "+
                                "GROUP BY nomCongres, edicio");
   System.out.println ("consulta executada");

   PreparedStatement ps = c.prepareStatement("UPDATE activitats "+
                                "SET preu = ? "+
                                "WHERE nomCongres = ? AND edicio = ?");
   int q = 0;
   while (r.next()) {
         ps.setFloat(1, r.getFloat(3));
         ps.setString(2, r.getString(1));
         ps.setInt(3, r.getInt(2));

         q = q + ps.executeUpdate();
   }
   System.out.println ("Hi ha " + q + " activitats.");
   }
catch (SQLException se)
   {
         System.out.println ("Excepció es: " + se.getMessage());
    }
```

Una altra possible solució que es considerarà correcta és:

```
try
   {
   /* Tenim ja una connexió c establerta amb la base de dades */

   Integer q = s.executeUpdate("UPDATE activitats a1"+
                               "SET preu = (SELECT avg(a2.reu) "+
                               "            FROM activitats a2 "+
                               "            WHERE a2.nomCongres = a1.nomCongres AND "+
                               "                  a2.edicio = a1.edicio) ";

   System.out.println ("Hi ha " + q + " activitats.");
   }
catch (SQLException se)
   {
         System.out.println ("Excepció es: " + se.getMessage());
    }
```

2. (3 punts) Transaccions

2.1. Suposeu un SGBD sense cap mecanisme de control de concurrència en el que es produeix l'horari següent (R= Read, RU= Read for Update, W= Write; les accions s'han numerat per facilitar fer-hi referència):

#n	T1	T2	T3	T4
10		R(B)		
20	R(B)			
30			R(A)	
40			RU(C)	
50		RU(C)		
60				RU(E)
70				W(E)
80		W(C)		
90				R(F)
100	R(E)			
110	RU(D)			
120			W(C)	
130	W(D)			
140				R(F)
150	commit			
160				RU(E)
170				W(E)
180		R(E)		
190		commit		
200				commit
210			RU(A)	
220			W(A)	
230			R(D)	
240			commit	

A. Doneu el graf de precedències associat a l'horari donat?

B. Indiqueu quines interferències es produeixen en aquest horari?. Per cada interferència cal que doneu: el nom de la interferència, i les transaccions i grànuls implicats.

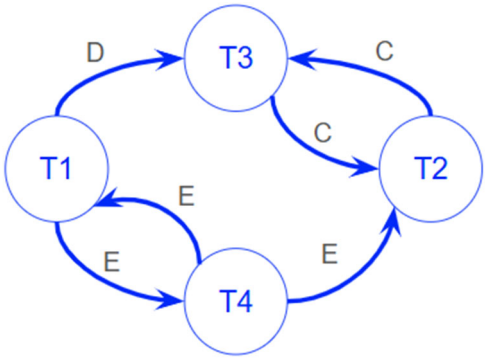
C. L'horari proposat, és recuperable? En cas afirmatiu doneu la definició de quan un horari és recuperable. En cas negatiu indiqueu alguna de les transaccions i operacions de l'horari que mostrin que no ho és.

2.2. Suposeu que el SGBD treballa amb un sistema de control de concurrència per reserves S i X

A. Doneu l'horari resultant, tenint en compte que la transacció T1 treballa amb un nivell d'aïllament de READ COMMITTED i T2, T3 i T4 amb nivell REPETEABLE READ. El nou horari ha d'incloure, a més de les operacions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves (LOCK, UNLOCK), i l'ordre d'execució de totes aquestes operacions. Quan més d'una transacció espera per un mateix gràNul, considereu que la política de la cua és FIFO (First In, First Out).

B. En cas que el nou horari presenti interferències, digueu quines són i entre quines transaccions es produeixen. També digueu quin nivell d'aïllament caldria per eliminar-les. En cas que no n'inclogui cap, doneu els horaris serials equivalents.

SOLUCIÓ 2.1



- Actualització perduda – gràNul C – T2-T3
- Lectura no confirmada – gràNul E - T1-T4

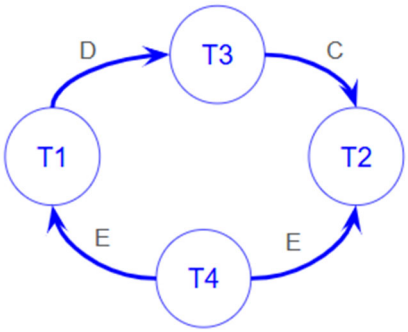
No és recuperable. Per exemple, T2 llegeix el gràNul E que T4 ha actualitzat, i comfirma abans de que ho faci T4.

SOLUCIÓ 2.2.A

#n	T1	T2	T3	T4
		L(B,S)		
		R(B)		
	L(B,S)			
	R(B)			
	U(B)			
			L(A,S)	
			R(A)	
			L(C,X)	
			RU(C)	
		L(C,X)		
				L(E,X)
				RU(E)
				W(E)
				L(F,S)
				R(F)
	L(E,S)			
			W(C)	
				R(F)
				RU(E)
				W(E)
				C+U(E,F)
	R(E)			
	U(E)			
	L(D,X)			
	RU(D)			
	W(D)			
	C+U(D)			
			L(A,X)	
			RU(A)	
			W(A)	
			L(D,S)	
			R(D)	
			C+U(A,C,D)	
		RU(C)		
		W(C)		
		L(E,S)		
		R(E)		
		C+U(B,C,E)		

SOLUCIÓ 2.2.B

No hi ha cap interferència, tal com es pot veure al graf de precedències, i només hi ha un horari serial equivalent que és T4;T1;T3;T2



3. (3 punts)

3.1. Considereu que existeix un índex B+ agrupat sobre la columna **email** de la taula **persones** amb ordre d=15 i alçada h=5. Justifiqueu les vostres respostes.

A. Indiqueu el cost en número mínim i màxim de pàgines a les que s'accedirà per resoldre la consulta següent.

```
SELECT *
FROM persones
WHERE email = 'jordi@mail.cat';
```

B. Indiqueu el cost en número mínim i màxim de pàgines a les que s'accedirà per resoldre la consulta següent. La consulta retorna un 1 per cada fila de la taula persones que compleix la condició del WHERE.

```
SELECT 1
FROM persones
WHERE email = 'jordi@mail.cat';
```

email és Primary Key de la taula persones => màxim una persona pot complir la condició

- Apartat A
  - La consulta Q1 retorna la persona amb un determinat mail, si està a la taula.
  - Cost mínim = h = 5 (en cas que la persona no existeix)
  - Cost màxim = h + 1 = 5 + 1 (en cas que la persona si que existeix).
- Apartat B
  - La consulta Q2 retorna un 1 si la persona amb un determinat mail existeix
  - Cost mínim = 1 (en cas que el mail és un dels valors de l'arrel)
  - Cost màxim = h = 5 (en cas que calgui baixar fins a les fulles per saber si el mail està a l'índex i per tant si la persona existeix).

3.2. Considereu ara que la taula `edicionsCongressos` té 12.000 tuples. El factor de bloqueig de la pàgina de dades és 15 i l'atribut *pressupost* és un enter distribuït uniformement entre 2.301 i 4.700 (per tant l'atribut *pressupost* té 2.400 valors diferents). Hi ha un índex B+ no agrupat definit sobre l'atribut *pressupost*. L'arbre té un ordre d=30 i una ocupació del 75%.

Determineu la quantitat de pàgines necessàries per emmagatzemar l'índex. Justifiqueu la resposta.

Valors per node (vpn) = ⌈2 * d * ocupació⌉ = ⌈2 * 30 * 0,75⌉ = 45

Número d'apuntadors per node (nap) = vpn + 1 = 45 + 1 = 46

Nivell fulla = ⌈12.000 valors / 45 valors per node⌉ = 267 nodes => 267 pàgines

Nodes interns = ⌈267 apuntadors / 46 apuntadors per node⌉ = 6 nodes => 6 pàgines

Node fulla = ⌈6 apuntadors / 46 apuntadors per node⌉ = 1 arrel => 1 pàgines

Número de pàgines necessàries per emmagatzemar l'índex: 1 + 6 + 267

3.3. Indiqueu a quina quantitat de pàgines (d'índex i dades) cal accedir per a resoldre la consulta següent mitjançant l'índex sobre l'atribut pressupost descrit en l'apartat anterior. Justifiqueu la vostra resposta.

```
SELECT DISTINCT ciutat FROM edicionsCongressos
WHERE pressupost >= 4.300
```

Fórmula índex NO agrupat: (h + F) + |edicionsCongressos(pressupost >= 4.300)|

h => alçada de l'arbre

F => nodes fulla addicionals

|edicionsCongressos(pressupost > 4.300)| => número de files que compleix la condició del WHERE

La taula **edicionsCongressos** té 12.000 tuples, i tenim 2.400 valors diferents de pressupost, per tant (donat que el pressupost està distribuït uniformement) cada valor de pressupost està en 12.000/2.400 = 5 files.

La condició del WHERE la compleixen 4.700 – 4.300 = 400 valors de pressupost diferents. Per tant, el número de files que compleix la condició del WHERE és de 400 * 5 = 2.000.

Procedim a calcular el número de pàgines per resoldre la consulta:

h = 3 (calculat anteriorment)

F = ⌈2.000 / 45⌉ - 1 = 45 - 1 = 44

|edicionsCongressos(pressupost > 4.300)| = 2.000

Resultat = 3 + 44 + 2.000 = 2.047 pàgines

4. (3 punts) Considereu la BD de l'exercici 1

4.1. A la BD hi ha també la taula següent:

```
CREATE TABLE auditoria (
    instant timestamp DEFAULT CURRENT_TIMESTAMP,
    usuari text DEFAULT CURRENT_USER,
    inscrit_OLD varchar(100),
    inscrit_NEW varchar(100),
    nomActivitat_OLD varchar(150),
    nomActivitat_NEW varchar(150),
    nomCongres_OLD varchar(150),
    nomCongres_NEW varchar(150),
    edicio_OLD integer,
    edicio_NEW integer,
    pagada_OLD date,
    pagada_NEW date,
    PRIMARY KEY (instant));
```

Indiqueu quins són els triggers necessaris (tenint en compte els criteris de qualitat establerts a l'assignatura) per tal d'implementar els comportament següents:

A. "Registrar a la taula auditoria una fila per cada actualització (INSERT, DELETE, UPDATE) que es faci a la taula inscripcions". A banda dels atributs instant i usuari, que tindran el valor per defecte, la resta d'atributs corresponen als valors abans i després de l'actualització de la taula inscripcions.

B. "El coordinador d'una activitat no pot estar inscrit a l'activitat". En cas que s'intenti no complir la restricció s'ha de rebutjar qualsevol esdeveniment que ho causi, generant una excepció.

Per cada trigger cal indicar:

- Taula per a la que es defineix
- Esdeveniment que l'activa (INSERT, DELETE, UPDATE - en cas d'UPDATE indicar atribut/s rellevants)
- BEFORE/AFTER (justificar la resposta)
- ROW/STATEMENT (justificar la resposta)

- Apartat A
  - taula inscripcions
    - insert, delete, update (de qualsevol atribut)
    - AFTER: Per evitar haver de desfer els inserts sobre la taula Auditoria, en cas que finalment no es faci l'actualització de la taula inscripcions.
    - ROW: Cal accedir al contingut nou i antic de les files. No es pot implementar statement.
- Apartat B
  - taula activitats
    - update of coordinador
    - BEFORE: Per evitar haver de desfer els updates de la taula activitats en cas que la restricció no es compleixi.
    - ROW: Per tal de fer una solució incremental, que només miri les files d'activitats modificades, en lloc de totes les files de les taules implicades.
  - taula inscripcions
    - insert, update of inscrit, nomActivitat, nomCongres, edicio
    - BEFORE: Per evitar haver de desfer els inserts o updates de la taula inscricions en cas que la restricció no es compleixi.
    - ROW: Per tal de fer una solució incremental, que només miri les files d'activitats modificades, en lloc de totes les files de les taules implicades.

4.2. Tingueu en compte la implementació del procediment `nouInscrit` següent:

- Indiqueu quina part de la implementació no compleix els criteris de qualitat:
- Indiqueu quin és el criteri de qualitat que no compleix
- Doneu una nova versió que sí que compleixi el criteri.

```
create or replace function nouInscrit(mail varchar(100),
  nomAct varchar(10), nomCon varchar(150), ed integer)
returns void language plpgsql as $$
begin
if((not exists(select * from persones
        where email = mail) or
  (not exists(select * from activitats
        where nomaActivitat = nomAct AND
              nomCongres = nomCon AND
              edicio = ed)))) then
  raise exception 'E1: La persona o l''activitat no existeixen';
  elseif(exists(select * from inscripcions
             where inscrit = mail and
                   nomaActivitat = nomAct AND
                   nomCongres = nomCon AND
                   edicio = ed)) then
  raise exception 'E2: Ja està inscrit a l''activitat';
  else insert into inscripcions values(mail,nomAct,nomCon,ed,null);
end if;
return;
exception
        when raise_exception then raise exception '%',SQLERRM;
        when others then raise exception 'E3: Altres errors';
end; $$
```

SOLUCIÓ:

- **Les excepcions E1 i E2 no cal fer consultes per saber si es produexin. Simplement es pot mirar en la secció exception si hi ha una PK violation, o una FK violation.**
- **Accessos innecessaris a la base de dades.**

```
create or replace function nouInscrit(mail varchar(100),
  nomAct varchar(10), nomCon varchar(150), ed integer)
returns void language plpgsql as $$
begin
  insert into inscripcions values(mail,nomAct,nomCon,ed,null);
end if;
  return;
  exception
        when raise_exception then raise exception '%',SQLERRM;
      WHEN foreign_key_violation THEN 'La persona o l''activitat no
existeixen';
      WHEN unique_violation THEN 'Ja està inscrit a l''activitat';
        when others then raise exception 'Altres errors';
end; $$
```

4.3. Considereu el procediment emmagatzemat següent. Implementeu en la part etiquetada com a **"//codi a implementar"** el següent:

- Per cadascuna de les activitats fetes durant l'edició del congrés indicada en els paràmetres d'entrada del procediment, retornar les dades de l'activitat (nomActivitat, descripcio, preu) i la quantitat d'inscripcions que hi ha registrades en el moment d'executar el procediment. Només han de sortir les activitats que tenen algun inscrit.
- Generar una excepció en el cas que: "No s'han trobat activitats amb inscrits en l'edició del congrés indicada".

```
CREATE TYPE inscritsPerActivitat AS (
    nomActivitat VARCHAR(150),
    descripcio VARCHAR(150),
    preu float,
    quantitat int
);

CREATE FUNCTION insActivitats(nomCongres VARCHAR(150), anyEdicio
integer)
RETURNS SETOF inscritsPerActivitat AS $$
DECLARE
  inscrit inscritsActivitat;

BEGIN
    //Codi a implementar

    RETURN;
END
$$ LANGUAGE plpgsql;
```

SOLUCIÓ:

```
CREATE TYPE inscritsActivitat AS (
    nomActivitat VARCHAR(150),
    descripcio VARCHAR(150),
    preu float,
    quantitat int
);

CREATE FUNCTION insActivitats(nomCongres VARCHAR(150), anyEdicio integer)
    RETURNS SETOF inscritsActivitat AS $$
DECLARE
   inscrit inscritsActivitat;
BEGIN
    FOR inscrit IN
     SELECT a.nomactivitat, a.descripcio, a.preu, count(*)
           FROM activitats a NATURAL INNER JOIN inscripcions i
           WHERE a.nomcongres = congres and a.edicio = anyEdicio
           GROUP BY a.nomactivitat, a.descripcio, a.preu;
    LOOP
        RETURN NEXT inscrit;
    END LOOP;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'No s''han trobat activitats';
    END IF;

    RETURN;
END
$$ LANGUAGE plpgsql;
```

Si us és necessari podeu fer servir la plantilla de triggers següent.

CREATE OR REPLACE FUNCTION actualitza_lloger() **RETURNS TRIGGER AS** $$
DECLARE
  ... ...
BEGIN
  ... ...
EXCEPTION
  WHEN { raise_exception | unique_violation | foreign_key_violation | check_violation | others }
  THEN
   RAISE EXCEPTION '%', missatge;
  ...
END;
$$ **LANGUAGE PLPGSQL**;

CREATE TRIGGER disparador {**BEFORE** | **AFTER**}{esdeveniment[**OR** ...]} **ON** taula
FOR [**EACH**] {**ROW** | **STATEMENT**} **EXECUTE PROCEDURE** actualitza_lloger();

Temps: 2 hores i 30 minuts

Notes 18 novembre tarda **Revisió: 19 novembre tarda** **Cada pregunta en un full separat**

1. **(3 punts)** Considereu l'esquema de la base de dades següent:

```
CREATE TABLE persones (
email varchar(100) PRIMARY KEY,
nomPersona varchar(100) NOT NULL,
ciutat varchar(250));

CREATE TABLE congressos (
idCongres varchar(10) PRIMARY KEY,
nomCongres varchar(150),
idFundador varchar(100),
FOREIGN KEY (idFundador) REFERENCES persones(email));
-- P.ex: hi haurà una fila amb idCongres MWC, i nom Mobile World Congress
-- i una altra fila amb id CòmicBCN, i amb nom Saló del Còmic Barcelona

CREATE TABLE edicionsCongressos (
idCongres varchar(10),
edicio integer CHECK (edicio > 2000 and edicio <= 2030),
pressupost float NOT NULL CHECK (pressupost > 0),
preuAssistencia float NOT NULL CHECK (preuAssistencia > 0),
PRIMARY KEY (idCongres, edicio),
FOREIGN KEY (idCongres) REFERENCES congressos);
-- P.ex: hi haurà una fila pel MWC del 2025, i una altra pel MWC del 2026
--  i files CòmicBCN del 2024, i CòmicBCN del 2025

CREATE TABLE participants (
idParticipant varchar(100),
idCongres varchar(10),
edicio integer,
rol varchar(20) NOT NULL CHECK (rol IN ('organitzador', 'assistent')),
PRIMARY KEY (idParticipant, idCongres, edicio),
FOREIGN KEY (idParticipant) REFERENCES persones(email),
FOREIGN KEY (idCongres, edicio) REFERENCES edicionsCongressos);
-- Hi ha una fila per cada persona que participa
-- com organitzador o bé assistent a una edició d'un congrés

CREATE TABLE activitats (
nomActivitat varchar(10),
idCongres varchar(10),
edicio integer,
descripcio varchar(255),
preuActivitat float CHECK (preuActivitat >= 0),
coordinador varchar(100) NOT NULL,
PRIMARY KEY (nomActivitat, idCongres, edicio),
FOREIGN KEY (idCongres, edicio) REFERENCES edicionsCongressos,
FOREIGN KEY (coordinador) REFERENCES persones(email));
-- Hi ha una fila per cada activitat que es fa en una edició d'un congrés
```

```sql
CREATE TABLE inscripcions (
inscrit varchar(100),
nomActivitat varchar(10),
idCongres varchar(150),
edicio integer,
PRIMARY KEY (inscrit, nomActivitat, idCongres, edicio),
FOREIGN KEY (inscrit) REFERENCES persones(email),
FOREIGN KEY (nomActivitat, idCongres, edicio) REFERENCES activitats);
-- Hi ha una fila per cada inscrit en una activitat
```

1.1. Tenint en compte l'esquema de la base de dades tal com està en l'enunciat digueu si són certes o no les situacions descrites a continuació. Per cada situació:

- En cas que sigui certa, doneu un contingut de la base de dades que ho mostri.
- En cas què no, expliqueu què caldria afegir o canviar a la base de dades per tal que siguin certes.

A. En dues edicions d'un mateix congrés es poden fer dues activitats (una en cada edició) que tinguin un mateix nom d'activitat.

B. Totes les edicions d'un mateix congrés han de tenir el mateix preu d'assistència.

C. Pot passar que una persona sigui coordinadora d'una activitat i que a l'hora estigui inscrita en aquesta mateixa activitat.

D. Una persona inscrita a una activitat d'una edició d'un congrés ha de ser participant en l'edició del congrés.

A- CERT, perquè la PK és nomActivitats, nomCongres, edicio

persones	**congressos**
'a@b.c','nomABC',...	'MWC','Mobile World Congress',...
'd@e.f','nomDEF',...	
edicionsCongressos	**activitats**
'MWC',2025,...	'activitat1', MWC, 2025, ..., 'a@b.c'
'MWC',2026,...	'activitat1', MWC, 2026, ..., 'd@e.f'

B- FALS: L'atribut preuAssitència està a la taula edicionsCongres, i això permet que hi hagi un preu diferent per cada edició.

- Si el preu és el mateix per totes les edicions d'un congrés, l'atribut s'hauria d'eliminar de la taula edicionsCongressos, i que estigui a la taula congressos.

C- CERT, tant el coordinador d'una activitat com l'inscrit són FK que referencien la taula persones, no hi ha cap restricció que eviti que siguin el mateix.

```
Suposant, el contingut donat a A, es pot afegir:
      inscripcions
      'a@b.c', 'activitat1', 'MWC', 2025
```

D- FALS, pel mateix motiu que l'afirmació anterior, tant el participant com l'inscrit són FK que referencien la taula persones, no hi ha cap restricció que faci que tot inscrit sigui participant o a l'inrevés.

- Cal afegir una clau forana d'inscrits cap a participants a la taula inscripcions. FOREIGN KEY (inscrit, nomCongres, edicio) REFERENCES Participants (idParticipant, idCongres, edicio)
- Les FK d'inscripcions cap a persones seria redundant, i s'hauria d'eliminar.

1.2. Doneu una sentència SQL per obtenir per cada edició de congrés amb un pressupost superior a 90000, el total que s'ha ingressat en l'edició pel pagament que han fet els participants com a assistents (rol='assistent'). Cada participant com a assistent paga el preuAssistencia per participar.

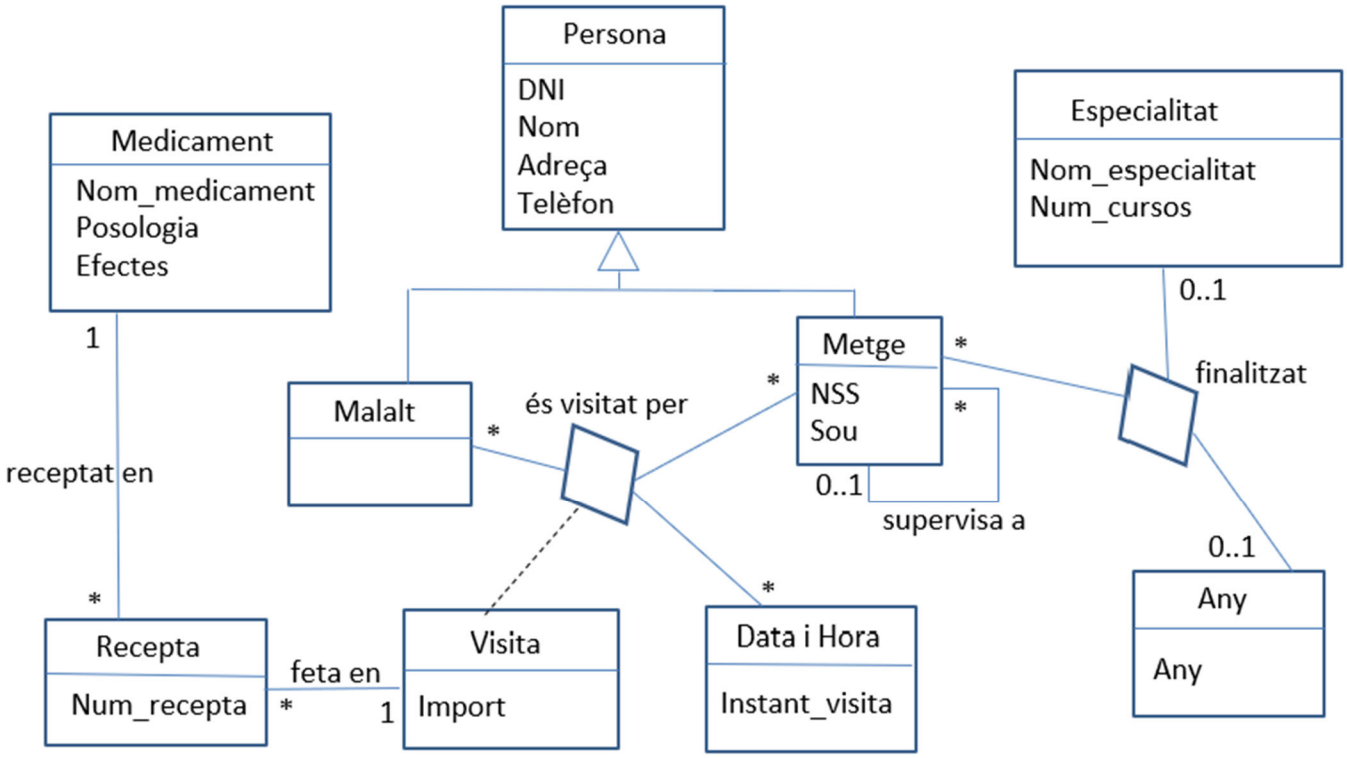
```
SELECT ec.idCongres, ec.edicio,
         ec.preuAssistencia*COUNT(*)
FROM participants p
         natural inner join edicionsCongressos ec
WHERE ec.pressupost > 90000 AND
      p.rol ='assistent'
GROUP BY ec.idCongres, ec.edicio
```

1.3. Doneu una sentència SQL per modificar el preu de les activitats quan hi hagi inscrita una o més persones que es diuen 'Ona' i només si totes les persones inscrites viuen a la mateixa ciutat. Al preu de les activitats indicades se li ha de sumar 100.

```
UPDATE activitats
SET preuActivitat = preuActivitat + 100
WHERE EXISTS (SELECT *
          FROM inscripcions i, persones p
          WHERE i.nomActivitat = activitats.nomActivitat AND
               i.idCongres = activitats.idCongres AND
               i.edicio = activitats.edicio AND
               i.inscrit = p.email AND
               p.nomPersona = 'Ona' AND
               NOT EXISTS (SELECT *
                    FROM inscripcions i2, persones p2
                    WHERE p2.email = i2.inscrit AND
                         i2.nomActivitat = i.nomActivitat AND
                         i2.idCongres = i.idCongres AND
                         i2.edicio = i.edicio AND
                         p.ciutat <> p2.ciutat));

--Solució també correcta però amb 2 taules innecessàries
UPDATE activitats
SET preuActivitat = preuActivitat + 100
WHERE EXISTS (SELECT *
          FROM inscripcions i, persones p
          WHERE i.nomActivitat = activitats.nomActivitat AND
               i.idCongres = activitats.idCongres AND
               i.edicio = activitats.edicio AND
               i.inscrit = p.email AND
               p.nomPersona = 'Ona')
     AND NOT EXISTS (SELECT *
          FROM inscripcions i1, inscripcions i2,
               persones p1, persones p2
          WHERE p1.email = i1.inscrit AND P2.email = i2.inscrit AND
               i1.nomActivitat = activitats.nomActivitat AND
               i1.idCongres = activitats.idCongres AND
               i1.edicio = activitats.edicio AND
               i2.nomActivitat = i1.nomActivitat AND
               i2.idCongres = i1.idCongres AND
               i2.edicio = i1.edicio AND
               p1.ciutat <> p2.ciutat);
```

2. **(2 punts)** Donat el model conceptual en UML següent:



I les restriccions textuals següents:

Clau externa Persona: Dni
Clau externa Especialitat: Nom_especialitat
Clau externa Any: Any
Clau externa Data i Hora: Instant_visita
Clau externa Recepta: Num_recepta
Clau externa Medicament: Nom_medicament
Restricció: No hi pot haver dos metges amb el mateix NSS
Restricció: No hi pot haver dues receptes del mateix medicament fetes a la mateixa visita

Es demana fer la traducció a model relacional donant el disseny lògic de la base de dades resultant. Concretament cal indicar:

- taules
- atributs
- claus primàries
- claus foranes
- restriccions NOT NULL per a les claus foranes que ho requereixin
- restriccions UNIQUE en cas d'existir claus alternatives
- altres restriccions que es poden definir com a restriccions de columna o taula que es puguin derivar de les restriccions textuals addicionals

persona(dni, nom, adreca, telefon)

malalt(dni)

{dni} referencia persona

metge(dni, NSS, sou, dni_supervisor)

{dni} referencia persona

{dni_supervisor} referencia metge

UNIQUE(NSS)

especialitat(nom especialitat, num_cursos)

finalitzacio(dni, nom especialitat, any)

{dni} referencia metge

{nom_especialitat} referencia especialitat

any NOT NULL

UNIQUE(dni, any)

visita(dni metge, dni malalt, instant visita, import)

{dni_metge} referencia metge

{dni_malalt} referencia malalt

medicament(nom medicament, posologia, efectes)

recepta(num recepta, num_met, num_mal,
instant_visita, medicament)

{num_met, num_mal, instant_visita} referencia visita NOT NULL

{medicament} referencia medicament NOT NULL

UNIQUE(num_met, num_mal, instant_visita, medicament)

Es considera correcta una solució on també hi hagi les taules:

data_i_hora (instant_visita)

any(any)

en aquest cas, caldrà posar claus foranes a:

finalitzacio:{any} referencia any

visita: {instant_visita} referencia data_i_hora

3. **(1.5 punts)** Sobre la base de dades de l'exercici 1. Definiu una asserció per garantir que no hi pugui haver més de 5 inscrits que visquin a la mateixa ciutat a cadascuna de les activitats coordinades per una persona de 'Vic'.

```
CREATE ASSERTION max5VeinsMateixaActivitat
   CHECK (
     NOT EXISTS (SELECT i.nomActivitat, i.idCongres, i.edicio
          FROM persones p
               INNER JOIN inscripcions i ON p.email = i.inscrit
               NATURAL INNER JOIN activitats a
               INNER JOIN persones p2 ON a.coordinador = p2.email
          WHERE p2.ciutat ='Vic'
          GROUP BY i.nomActivitat, i.idCongres, i.edicio, p.ciutat
          HAVING COUNT(*) > 5))
```

4. **(1.5 punts)** Sobre la base de dades de l'exercici 1. Doneu una seqüència d'operacions d'àlgebra relacional per obtenir una fila per cada persona i congrés, en el cas que la persona hagi organitzat (rol = 'organitzador') el congrés en les tres edicions 2020, 2021 i 2022 (totes tres). Cal fer atenció que una persona pot aparèixer múltiples vegades en el resultat de la consulta, una per cada congrés que hagi organitzat en les tres edicions indicades. En el resultat de la consulta hi haurà el nom i email d'una persona i el nom d'un congrés.

```
A = participants(rol='organitzador')
B = A(edicio=2020)
C = A(edicio=2021)
D = A(edicio=2022)
E = B[idParticipant, idCongres]
F = C[idParticipant, idCongres]
G = D[idParticipant, idCongres]
H = E _n_ F      /* podria ser una natural join implícita*/
I =  H _n_ G     /* podria ser una natural join implícita*/
J = persones[email*idParticipant]I
K = J*congressos
R = K[nomPersona, email, nomCongres]
```

5. **(2 punts)** Considereu la BD de l'inici de l'examen i les vistes següents:

```
CREATE VIEW edicionsActuals AS
SELECT idCongres, edicio, pressupost, preuAssistencia
FROM edicionsCongressos
WHERE edicio >= 2024;

CREATE VIEW edicionsActualsBarates AS
SELECT idCongres, edicio, pressupost, preuAssistencia
FROM edicionsActuals
WHERE preuAssistencia <= 100 WITH CHECK OPTION;

CREATE VIEW activitatsGratis AS
SELECT a.nomActivitat, a.idCongres, a.edicio,
       a.descripcio, a.coordinador
FROM activitats a
     NATURAL INNER JOIN edicionsActualsBarates eab
WHERE a.preuActivitat = 0;
```

5.1. Suposeu que l'usuari Ot, que és el propietari de totes les taules i vistes de la base de dades, ha executat les sentències següents:

```
GRANT SELECT ON edicionsActualsBarates TO Mar;
GRANT SELECT ON activitats TO Mar WITH GRANT OPTION;
GRANT INSERT ON inscripcions TO Mar;
GRANT SELECT(email, nomPersona) ON persones TO Mar;
```

Amb les autoritzacions actuals, digueu si l'usuari Mar pot executar amb èxit la sentència següent? En cas afirmatiu justifiqueu per què. En cas negatiu, indiqueu quins privilegis li falten i qui li hauria de concedir.

```
INSERT INTO inscripcions
       (SELECT DISTINCT p.email, ag.nomActivitat,
                ag.idCongres, ag.edicio
       FROM persones p, activitatsGratis ag
       WHERE p.email = ag.coordinador  AND
                p.ciutat = 'Barcelona');
```

5.1 L'usuari Mar no té prou privilegis per executar la sentencia perquè té privilegi d'insert sobre la taula inscripcions, però li falta grant de select sobre la vista activitatsGratis i grant de select(ciutat) sobre persones. Aquests privilegis els ha d'atorgar l'usuari Ot.

5.2. Suposeu que la taula Persones té 500 tuples, edicionsCongressos té 200 tuples i la taula activitats té 2000 tuples. Hi ha 300 activitats que s'han organitzat des del 2024, de les quals n'hi ha 80 que tenen preu 0. Per cadascuna de les consultes següents digueu:

- Quantes tuples retorna la consulta. Sinó podeu dir el número exacte, digueu el mínim i el màxim número de tuples resultants que pot donar. Justifiqueu breument la resposta.
- Justifiqueu si la consulta pot donar o no resultats repetits

A.

```
SELECT ag.nomActivitat, ag.descripcio,
          p.email, p.nomPersona, p.ciutat
FROM activitatsGratis ag
     INNER JOIN persones p ON ag.coordinador = p.email;
```

B.

```
SELECT ag.coordinador
FROM activitatsGratis ag
GROUP BY ag.coordinador
HAVING COUNT(*) = 0;
```

5.2.A La consulta dóna exactament 80 files. La consulta donarà resultats repetits si a dues activitats que tenen un mateix nom s'hi han apuntat una mateixa persona.

5.2.B. La consulta dóna exactament 0 files. No pot donar resultats repetits

5.3. Per cadascuna de les vistes:

- Indiqueu si és o no actualitzable segons l'estàndard SQL. Justifiqueu què ens indica en la vista que ho és o que no ho és.
- En cas que sigui actualitzables, doneu una **sentència SQL d'inserció a la vista**, que faci saltar una excepció. Indiqueu de **quin és el contingut de la base de dades abans de la inserció** (només la part rellevant), **i quin serà el motiu de l'excepció que salta.**

- En cas que no siguin actualitzables, doneu un **contingut de la vista** i una **sentència d'actualització de la vista** (delete o update) que mostri el motiu que hi ha darrera el fet de que la vista no sigui actualitzable. Justifiqueu breument la resposta.

5.3 Actualitzables:

- Les vista edicionsActuals i edicionsActualsBarates són actualitzables perquè fan SELECT sobre una única taula (edicionsCongressos) , no tenen funcions d'agregació, no tenen GROUP BY, no tenen joins, i inclouen tots els atributs de la clau primària (idCongres, edicio) i tots els atributs NOT NULL.

Insersions vistes actualitzables:

Partim de que a la BD hi ha el congrés 'MWC'

- ```
  INSERT INTO edicionsActuals
  VALUES ('MWC',2032,20000,200);
  ```
  - Salta una excepció pel check de la taula edicionsCongressos
- ```
  INSERT INTO edicionsActualsBarates
  VALUES('MWC', 2025,200000,200);
  ```
  - Salta una excepció pel with check option de la vista, ja que l'edició que volem inserir té preu superior a 100.

No actualitzables:

- La vista activitatsGra no és actualitzable perquè té joins.

Vistes NO actualitzables: Un exemple pot ser:

- A la taula activitats hi ha {('reunió','MWC',2025,'...',0,'a@b.c')}
- A la taula edicionsCongressos hi ha {('MWC',2025,20000,100)}
- A la vista es pot veure la fila {('reunió','MWC',2025,'....','a@b.c')}
- Si tenim la operació següent, hi ha diferents maneres, d'eliminar la fila de la vista, es podria eliminar l'activitat, o bé es podria canviar el preu de l'activitat, o es podria fer les dues coses. No hi ha una única manera de traduir el canvi de la vista a canvis sobre les taules.

```
DELETE FROM activitatsGratis
WHERE nomActivitat = 'reunió' AND
   idCongres = 'MWC' AND
   Edicio = 2025
```

Temps: 2 hores i 45 minuts

Notes 21 gener tarda **Revisió: 22 gener tarda** **Cada pregunta en un full separat**

1. **(1.5 punts)** Considereu l'esquema de la base de dades següent:

```
CREATE TABLE persones (
email varchar(100) PRIMARY KEY,
nomPersona varchar(100) NOT NULL,
ciutat varchar(250));

CREATE TABLE congressos (
idCongres varchar(10) PRIMARY KEY,
nomCongres varchar(150) UNIQUE,
idFundador varchar(100),
FOREIGN KEY (idFundador) REFERENCES persones(email));
-- P.ex: hi haurà una fila amb idCongres MWC, i nom Mobile World Congress
-- i una altra fila amb id CòmicBCN, i amb nom Saló del Còmic Barcelona

CREATE TABLE edicionsCongressos (
idCongres varchar(10),
edicio integer CHECK (edicio > 2000 and edicio <= 2030),
pressupost float NOT NULL CHECK (pressupost > 0),
preuAssistencia float NOT NULL CHECK (preuAssistencia > 0),
PRIMARY KEY (idCongres, edicio),
FOREIGN KEY (idCongres) REFERENCES congressos);
-- P.ex: hi haurà una fila pel MWC del 2025, i una altra pel MWC del 2026
--  i files CòmicBCN del 2024, i CòmicBCN del 2025

CREATE TABLE participants (
idParticipant varchar(100),
idCongres varchar(10),
edicio integer,
rol varchar(20) NOT NULL CHECK (rol IN ('organitzador', 'assistent')),
PRIMARY KEY (idParticipant, idCongres, edicio),
FOREIGN KEY (idParticipant) REFERENCES persones(email),
FOREIGN KEY (idCongres, edicio) REFERENCES edicionsCongressos);
-- Hi ha una fila per cada persona que participa
-- com organitzador o bé assistent a una edició d'un congrés

CREATE TABLE activitats (
nomActivitat varchar(10),
idCongres varchar(10),
edicio integer,
descripcio varchar(255),
preuActivitat float CHECK (preuActivitat >= 0),
coordinador varchar(100) NOT NULL,
PRIMARY KEY (nomActivitat, idCongres, edicio),
FOREIGN KEY (coordinador,idCongres,edicio)
            REFERENCES participants(idParticipant,idCongres,edicio));
-- Hi ha una fila per cada activitat que es fa en una edició d'un congrés
```

```sql
CREATE TABLE inscripcions (
inscrit varchar(100),
nomActivitat varchar(10),
idCongres varchar(150),
edicio integer,
PRIMARY KEY (inscrit, nomActivitat, idCongres, edicio),
FOREIGN KEY (inscrit) REFERENCES persones(email),
FOREIGN KEY (nomActivitat, idCongres, edicio) REFERENCES activitats);
-- Hi ha una fila per cada inscrit en una activitat
```

1.1. Considereu el fragment de codi Java/JDBC següent. Completeu-lo per implementar que per cada activitat de la base de dades corresponent a l'edició de congrés {'MWC', 2026}, si aquesta activitat no té cap persona inscrita de fora de 'Barcelona', es redueixi el preu de l'activitat en 10€. Es vol que un cop fetes les modificacions de preu, es mostri un missatge amb la quantitat d'activitats modificades. En cas que alguna activitat passi a tenir un preu menor de 0€, no s'ha de modificar cap activitat i s'ha de mostrar un missatge d'error.

```java
try {
   /* Tenim ja una connexió c establerta amb la base de dades */

   c.commit();
   c.close();
}
catch (SQLException se) {
  System.out.println ("Excepció: " + se.getMessage());
}
```

Recordeu que podeu usar els següents mètodes/codis d'excepció de JDBC estudiats a classe:

Statement

- `Statement createStatement();`
- `ResultSet executeQuery(String sql);`
- `int executeUpdate(String sql);`

PreparedStatement

- `PreparedStatement prepareStatement(String sql);`
- `ResultSet executeQuery();`
- `int executeUpdate();`
- `void setXXX(int position, XXX value);`

ResultSet

- `boolean next();`
- `XXX getXXX(String nomColumna);`

SQLException

- `// 23502 not_null_violation`
- `// 23503 foreign_key_violation`
- `// 23505 unique_violation`
- `// 23514 check_violation`
- `String getSQLState();`

SOLUCIÓ:

```
try {
   /* Tenim ja una connexió c establerta amb la base de dades */
   Statement st = c.createStatement();
   int n = st.executeUpdate(
       "UPDATE activitats a " +
       "SET preuActivitat = preuActivitat - 10 " +
       "WHERE a.idCongres = 'MWC' AND a.edicio = 2026 AND " +
            "NOT EXISTS (SELECT * " +
            "FROM inscripcions i INNER JOIN persones p " +
                 "ON i.inscrit = p.email " +
            "WHERE i.nomActivitat = a.nomActivitat AND " +
                 "a.idCongres = 'MWC' AND a.edicio = 2026 AND " +
                 "p.ciutat <> 'Barcelona')");
   System.out.println("S'han actualitzat " + n + " activitats");
   st.close();
}
catch (SQLException se) {
   if (se.getSQLState().equals("23514")) {
       System.out.println("El preu d'una activitat no pot ser negatiu");
   }
   else {System.out.println("Excepció: " + se.getMessage());}
}
```

2. **(3 punts)** Suposeu la base de dades de l'inici de l'examen. El contingut inicial de la BD rellevant per a aquest exercici és:

```
INSERT INTO edicionsCongressos VALUES
  ('MWC', 2024, 40000, 600),
  ('MWC', 2025, 80000, 400);

INSERT INTO activitats VALUES
  ('Taller5', 'MWC', 2024, 'Taller gratuït', 0, 'coordi@a.b');
```

2.1. Per aquest apartat considereu que la base de dades està creada en un SGBD sense cap mecanisme de control de concurrència, i que el gránul és la fila. Suposeu les sentències SQL següents.

(*) Podeu utilitzar l'identificador que s'indica per a cada sentència en els horaris que es demanen en aquest apartat.

S1	`SELECT pressupost`
	`FROM edicionsCongressos`
	`WHERE idCongres = 'MWC' AND edicio = 2024;`

S2	`UPDATE edicionsCongressos`
	`SET pressupost = pressupost - 30000`
	`WHERE idCongres = 'MWC' AND edicio = 2025;`

S3	`DELETE FROM activitats`
	`WHERE idCongres = 'MWC' AND edicio = 2024`
	`AND preuActivitat=0;`

S4	`SELECT *`
	`FROM edicionsCongressos`
	`WHERE pressupost <= 50000;`

S5	`SELECT pressupost`
	`FROM edicionsCongressos`
	`WHERE idCongres = 'MWC' AND edicio = 2025;`

S6	`UPDATE edicionsCongressos`
	`SET pressupost = pressupost - 20000`
	`WHERE idCongres = 'MWC' AND edicio = 2024;`

S7	`SELECT COUNT(*)`
	`FROM edicionsCongressos`
	`WHERE pressupost <= 50000;`

A. Doneu un horari on intervinguin dues transaccions, que poden executar una o més de les sentències SQL donades, i que presenti una única interferència de lectura no repetible. S'ha d'incloure a l'horari el **mínim nombre de sentències**, i **una sentència es pot usar només una vegada**. **En cas que no sigui possible formar un horari amb els requisits indicats**, justifiqueu per què no es pot.

B. Doneu un horari on intervinguin dues transaccions, que poden executar una o més de les sentències SQL donades, i que presenti una única interferència d'anàlisi inconsistent. S'ha d'incloure a l'horari el **mínim nombre de sentències**, i **una sentència es pot usar només una vegada**. **En cas que no sigui possible formar un horari amb els requisits indicats**, justifiqueu per què no es pot.

Solució:

2.1.A. Es pot generar un horari com el següent amb la interferència de lectura no repetible.

T1	T2
(S1) SELECT pressupost FROM edicionsCongressos WHERE idCongres = 'MWC' AND edicio = 2024;	
	(S6) UPDATE edicionsCongressos SET pressupost = pressupost - 20000 WHERE idCongres = 'MWC' AND edicio = 2024;
(S4) SELECT * FROM edicionsCongressos WHERE pressupost <= 50000;	
COMMIT;	COMMIT;

2.1.B Es pot generar el següent horari amb interferència d'anàlisi inconsistent.

T1	T2
(S1) SELECT pressupost FROM edicionsCongressos WHERE idCongres = 'MWC' AND edicio = 2024;	
	(S6) UPDATE edicionsCongressos SET pressupost = pressupost - 20000 WHERE idCongres = 'MWC' AND edicio = 2024;
	(S2) UPDATE edicionsCongressos SET pressupost = pressupost - 30000 WHERE idCongres = 'MWC' AND edicio = 2025;
(S5) SELECT pressupost FROM edicionsCongressos WHERE idCongres = 'MWC' AND edicio = 2025;	
COMMIT;	
	COMMIT;

2.2. Suposeu ara l'horari següent. A l'igual que abans, el gràNul és la fila.

A. Indiqueu quina/es interferència/es presenta l'horari abans d'aplicar cap nivell d'aïllament. Doneu les sentències i el/s gràNul/s implicat/s en cada interferència que hagueu identificat. Expliqueu el motiu de l'error que provoca cada interferència identificada. No cal fer graf de precedències.

B. Suposant que T1 treballa amb nivell d'aïllament READ COMMITTED i T2 amb nivell REPETEABLE READ, que l'SGBD fa servir reserves S, X. **Doneu l'horari de baix nivell resultant d'aplicar les reserves adequades a cada nivell d'aïllament.** L'horari ha d'incloure, a més de les peticions que executen les transaccions (R, RU, W, COMMIT), les operacions de petició i alliberament de reserves (LOCK, UNLOCK), les esperes i l'ordre d'execució de totes aquestes peticions.

	T1	T2
1	SELECT pressupost FROM edicionsCongressos WHERE idCongres = 'MWC' AND edicio = 2025;	
2	SELECT * FROM edicionsCongressos WHERE pressupost <= 50000;	
3		DELETE FROM edicionsCongressos WHERE idCongres = 'MWC' AND edicio = 2025;
4		INSERT edicionsCongressos VALUES ('MWC',2026,45000,700);
5	SELECT pressupost FROM edicionsCongressos WHERE idCongres = 'MWC' AND edicio = 2025;	
6	SELECT COUNT(*) FROM edicionsCongressos WHERE pressupost <= 50000;	
7	COMMIT;	
8		COMMIT;

Solució:

2.2.A.

- La fila A correspon a {'MWC', 2024},
- La fila B a {'MWC', 2025},
- La fila C a {'MWC', 2026}

Interferències:

- lectura no repetible entre les sentències dels instants 1,3 i 5. Gränul B. El problema és que T1 llegeix el gränul B abans del DELETE i quan el vol tornar a llegir no pot perquè ja no existeix.
- fantasmes entre les sentències 2, 4, 6. La sentència 2 llegeix el gränul A, i, amb la mateixa condició de selecció, la sentència 6 llegeix els gränuls A i el C.

2.2.B.

T1	T2	
L(B,S)		
R(B)		
U(B)		
R(IC)		
L(A,S)		
R(A)		
U(A)		(1)
	L(B,X)	
	RU(B)	
	W(B)	
	L(C,X)	
	RU(C)	
	W(C)	
	RU(IC)	
	W(IC)	
L(B,S)		
	Commit + U(B,C)	
R(B)		
U(B)		
R(IC)		
L(A,S)		(2)
R(A)		
U(A)		
L(C,S)		
R(C)		
U(C)		
Commit;		

(1) Només la fila corresponent a {'MWC', 2024} té pressupost <= 50000

(2) Ara les files corresponents a {'MWC', 2024} i {'MWC', 2026} tenen pressupost <= 50000

3. (3 punts) Suposeu la base de dades de l'exercici 1.

3.1. La taula `congressos` té 3 atributs. Suposeu que hi ha tres índexs arbre B+ definits, un índex sobre cada un dels atributs. Indiqueu quants d'aquests 3 índexs, com a màxim, poden ser agrupats i quants no agrupats. Justifiqueu la resposta.

- Com a màxim 1 podria ser agrupat. El motiu és que quan un índex és agrupat les les files a les pàgines de dades estan ordenades com l'índex. No hi pot haver més d'un índex agrupat, perquè és impossible tenir físicament ordenades les files a les pàgines de dades segons dos atributs a l'hora.
- No agrupats ho podrien ser tots 3. Els índexos no agrupats, no impliquen cap ordenació de files a les pàgines de dades.

3.2. Suposeu que existeix un índex arbre B+ definit sobre els atributs `{idCongres, edicio}` de la taula `edicionsCongressos`. L'índex té ordre d=120 i una ocupació del 80%. La taula `edicionsCongressos` té 10.000 files, cada congrés s'ha fet en una mitjana de 15 edicions, i el factor de bloqueig és de 10.

A. Quants valors hi haurà a cada node de l'arbre. Mostreu i justifiqueu els càlculs.

B. Quantes pàgines ocupen en total els nodes fulla de l'arbre. Mostreu i justifiqueu els càlculs.

C. Quants nivells tindrà l'arbre B+. Mostreu i justifiqueu els càlculs.

A. quantitat valors per node = **nvpn** = $\lceil 2*d*oc \rceil = \lceil 2*120*0{,}8 \rceil = 192$ valors

B. quantes pàgines ocupen els nodes fulla = quantitat nodes fulla

$= \lceil$quantitat valor a indexar / quantitat valors per node $\rceil =$

$= \lceil 10.000 / 192 \rceil = 53$ pàgines

C. quantitat apuntadors per node = **napn** = nvpn + 1 = 193 apuntadors

nivells de l'arbre B+ = **h** = $\lceil \log_{193} 10.000 \rceil = 2$ nivells

3.3. Considereu les consultes següents. Doneu el cost de cadascuna utilitzant l'índex indicat a l'apartat anterior **per als casos en que l'índex és agrupat, i quan l'índex és no agrupat**. Mostreu i justifiqueu els càlculs.

A.
```
SELECT * FROM edicionsCongressos
WHERE idCongres = 'MWC' AND edicio = 2025;
```

B.
```
SELECT edicio FROM edicionsCongressos
WHERE idCongres = 'MWC;
```

3.3.A. És una consulta a una taula per la clau primària, per tant s'obté únicament una fila. El cost és doncs el mateix tant si l'índex és AGRUPAT com NO AGRUPAT.

cost = h + 1

3.3.B. AGRUPAT. Com que el què s'està buscant està a l'índex no cal accedir a les pàgines de dades. Per tant la formula usada per al càlcul només es basa en h i F.

cost = h + F + 0

= 2 + ($\lceil$ |edicionsCongressos(idCongres='MWC')|/ nvpn $\rceil$ -1)

= 2 + ($\lceil$ 15 / 192 $\rceil$ -1) = 2 pàgines.

NO AGRUPAT. El cost és el mateix ja que no és necessari accedir a les dades.

3.4. Suposeu que les taules `activitats` i `inscripcions` estan en un espai virtual d'agrupació. A la base de dades hi ha 50.000 activitats. Cada activitat té una mitjana de 5 inscripcions, i en una edició de congrés es fan unes 200 activitats. Les files de les dues taules tenen la mateixa grandària. El factor de bloqueig de les pàgines de dades és 50.

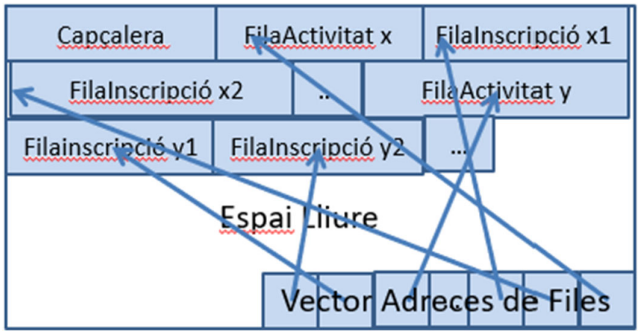
A. Doneu una figura que mostri a nivell físic quina serà l'estructura interna d'una pàgina de dades de les que pertanyen a aquest espai virtual d'agrupació. Expliqueu a què correspon cada element dins de la pàgina.

B. Suposeu que no hi ha cap índex definit sobre activitats i inscripcions. Doneu el cost de resoldre la consulta. Mostreu i justifiqueu els càlculs.

```
SELECT * FROM activitats
WHERE idCongres = 'MWC' AND edicio = 2025;
```

C. Suposeu ara que hi ha un índex **no agrupat** arbre B+ multi-atribut sobre els atributs {idCongres, edició} per accedir a les activitats d'una edició de congrés. L'índex té ordre d=120 i una ocupació del 80%. Doneu el cost de resoldre la mateixa consulta de l'apartat B. Mostreu i justifiqueu els càlculs.

3.4.A. Explicació en el material del LearnSQL



3.4.B. Cal tenir en compte que: les files estan intercalades, no hi ha índex, les files no estan ordenades físicament. Aleshores, cal llegir totes les pàgines de dades, que inclouen totes les files d'activitats i d'inscripcions.

número de files = 50.000 + 5*50.000 = 300.000 files

cost = quantitat de pàgines de dades

= ⌈ número de files / factor de bloqueig ⌉

= ⌈ 300.000 / 50 ⌉ = 6.000 pàgines

3.4.C. El nvpn i nap h són els mateixos que els de l'arbre de l'apartat 3.2

$h = \lceil \log_{193} 50.000 \rceil = 3$

Cost = h + F + D

$= 3 + (\lceil |activitats(idCongres = 'MWC' AND edicio = 2025)| / nvpn \rceil - 1)$

$+ |activitats(idCongres = 'MWC' AND edicio = 2025)|$

$= 3 + ((\lceil 200/192 \rceil - 1) + 200 = 204$ pàgines

4. **(2.5 punts)** Suposeu la base de dades de l'exercici 1. Tingueu en compte que el idCongres i l'edició no es poden modificar en la taula activitats.

4.1. Considereu la implementació del disparador següent.

A. Justifiqueu si compleix o no els criteris de qualitat de l'assignatura.

B. En cas que els compleixi, doneu un contingut de la taula activitats i un exemple de esdeveniment que activa el disparador, indicant l'efecte que tindrà l'activació del disparador degut a l'esdeveniment. En cas contrari, doneu una implementació que sí que compleixi els criteris de qualitat de l'assignatura.

```
CREATE OR REPLACE FUNCTION recalc_pressupost_congres()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE edicionsCongressos ec
    SET pressupost = (
        SELECT SUM(a.preuActivitat)
        FROM activitats a
        WHERE a.idCongres = ec.idCongres
              AND a.edicio = ec.edicio);
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_recalc_pressupost_activitats
AFTER INSERT
      OR UPDATE OF preuActivitat ON activitats
FOR EACH ROW
EXECUTE FUNCTION recalc_pressupost_congres();
```

SOLUCIÓ

A. No s'adequa als criteris de qualitat. Tot i ser una implementació FOR EACH ROW és NO incremental, és a dir, recalcula el pressupost de totes les edicions de congressos i NO només modifica el pressupost d'aquelles edicions per les que hi ha hagut un canvi de preu en alguna de les seves activitats. A més farà aquest recàlcul per cada fila actualitzada a la taula activitats.

B.

```
CREATE OR REPLACE FUNCTION recalcular_pressupost_congres()
RETURNS TRIGGER AS $$
DECLARE
    delta INTEGER;
BEGIN
    IF TG_OP = 'INSERT' THEN
        delta := NEW.preuActivitats;
    ELSIF TG_OP = 'UPDATE' THEN
        delta := NEW.preuActivitat - OLD.preuActivitat;
    END IF;
    UPDATE edicionsCongressos
    SET pressupost = pressupost + delta
    WHERE idCongres = NEW.idCongres
              AND edicio = NEW.edicio;
    RETURN null;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_ajustar_pressupost_activitat
AFTER INSERT OR UPDATE OF preuActivitat ON activitats
FOR EACH ROW
EXECUTE FUNCTION ajustar_pressupost_activitat();
```

4.2. Considereu la implementació del disparador següent.

A. Indiqueu per cadascun dels esdeveniments indicats a continuació quin efecte té l'esdeveniment a la base de dades, i justifiqueu breument el per què.

A.1. Inserir una activitat gratuïta coordinada per un assistent.

A.2. Inserir una activitat de pagament coordinada per un organitzador.

A.3. Modificar el preu d'una activitat gratuïta (coordinada per un assistent) a un valor positiu.

A.4. Modificar el preu d'una activitat gratuïta (coordinada per un assistent) a un valor negatiu.

B. Expliqueu quina és la regla o restricció d'integritat que implementa el procediment

```
CREATE OR REPLACE FUNCTION controlar_coordinador_activitat()
RETURNS TRIGGER AS $$
DECLARE
    rol_coord varchar(20);
BEGIN
    -- Obtenim el rol del coordinador en aquella edició
    SELECT rol INTO rol_coord
    FROM participants
    WHERE idParticipant = NEW.coordinador
      AND idCongres = NEW.idCongres
          AND edicio = NEW.edicio;

    IF rol_coord = 'organitzador' THEN RETURN NEW;
    END IF;

    IF rol_coord = 'assistent' THEN
        IF NEW.preuActivitat = 0 THEN
            RETURN NEW;
        ELSEIF (NEW.preuActivitat > 0)
           THEN RETURN OLD;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_controlar_coordinador_activitat
BEFORE INSERT
       OR UPDATE OF coordinador, preuActivitat ON activitats
FOR EACH ROW
EXECUTE FUNCTION controlar_coordinador_activitat();
```

SOLUCIÓ

A.1 - Es fa la inserció de l'activitat. Es fa RETURN NEW per tant, l'esdeviniment s'acaba executant perquè un return NEW en un insert fa que l'esdeveniment s'executi.

A.2 - Es fa la inserció de l'activitat. Es fa RETURN NEW per tant, l'esdeviniment s'acaba executant perquè un return NEW en un insert fa que l'esdeveniment s'executi.

A.3 - No es fa la modificació de l'activitat. Es fa `RETURN OLD` per tant, l'esdeveniment no s'executa perquè un return OLD en un update fa que l'esdeveniment NO s'executi.

A. 4 - No es fa la modificació de l'activitat. Es fa RETURN NEW perquè no entra a cap de les condicions del procediment, això farà que salti el `CHECK (preuActivitat >= 0)` de la taula `activitats` quan s'executi l'esdeveniment després del procediment.

B. Un participant assistent NO pot coordinar activitats NO gratuïtes

4.3. El disparador de l'apartat 4.2 implementa una regla o restricció d'integritat quan hi ha canvis en la taula activitats. En cas que hi hagi altres esdeveniments que poden afectar a aquesta regla, es vol definir disparadors per fer-la complir.

- En cas que hi hagi altres esdeveniments, per cada esdeveniment pel qual cal definir disparador indiqueu: esdeveniment activador, taula, tipus de disparador (before/after, row/statement), i expliqueu què hauria de fer la implementació del procediment activat pel disparador. No cal escriure tot el codi, només indicar clarament què s'ha de fer.
- En cas que no sigui possible, justificar perquè no ho és.

SOLUCIÓ

S'ha d'afegir un trigger **`BEFORE UPDATE OF rol`** sobre **`participants`** que comprovi si el participant coordina activitats de pagament. Si el nou rol és **`assistent`** i coordina alguna activitat amb **`preuActivitat > 0`**, s'ha de bloquejar l'actualització.

Solució alternativa (donant el codi)

```
CREATE OR REPLACE FUNCTION controlar_canvi_rol_participant()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.rol = 'assistent' AND OLD.rol = 'organitzador'
THEN
        IF EXISTS (
            SELECT * FROM activitats a
            WHERE a.coordinador = NEW.idParticipant
              AND a.idCongres = NEW.idCongres
              AND a.edicio = NEW.edicio
              AND a.preuActivitat > 0
          ) THEN
            RETURN OLD; (*)
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_controlar_canvi_rol_participant
BEFORE UPDATE OF rol ON participants
FOR EACH ROW
EXECUTE FUNCTION controlar_canvi_rol_participant();
```

(*) també seria vàlid llençar una EXCEPTION

BD **09.04.2026**

Temps: 2 hores

Notes: 27 abril tarda **Revisió: 29 abril tarda** **Cada pregunta en fulls separats**

Considereu l'esquema de la base de dades següent:

```
CREATE TABLE cliniques (
   idClinica          INTEGER,
   nomClinica         VARCHAR(100) UNIQUE NOT NULL,
   ciutat             VARCHAR(100),
   pressupostAnual    FLOAT CHECK (pressupostAnual > 0),
   PRIMARY KEY (idClinica));

CREATE TABLE metges (
   idMetge            INTEGER,
   nom                VARCHAR(100) UNIQUE NOT NULL,
   especialitat       VARCHAR(50) NOT NULL,
   sou                FLOAT CHECK (sou > 0),
   idClinicaActual    INTEGER NOT NULL,
   PRIMARY KEY (idMetge),
   FOREIGN KEY (idClinicaActual) REFERENCES cliniques);

--idClinicaActual és on actualment exerceix el metge

CREATE TABLE pacients (
   idPacient          INTEGER,
   nom                VARCHAR(100) NOT NULL,
   edat               INTEGER CHECK (edat BETWEEN 0 AND 120),
   ciutat             VARCHAR(100),
   PRIMARY KEY (idPacient));

CREATE TABLE tractaments (
   idClinica          INTEGER,
   codiTractament     INTEGER,
   descripcio         VARCHAR(200),
   preuBase           FLOAT,
   PRIMARY KEY (idClinica, codiTractament),
   FOREIGN KEY (idClinica) REFERENCES cliniques);

-- Hi ha una fila per cada tractament ofert en la clínica
```

```
CREATE TABLE visites (
   idPacient          INTEGER,
   idClinica          INTEGER,
   codiTractament     INTEGER,
   dataHora           TIMESTAMP,
   idMetgeTractant    INTEGER NOT NULL,
   idMetgeSolicitor   INTEGER,
   PRIMARY KEY (idPacient, idClinica, codiTractament, dataHora),
   FOREIGN KEY (idPacient) REFERENCES pacients,
   FOREIGN KEY (idClinica, codiTractament)
                          REFERENCES tractaments,
   FOREIGN KEY (idMetgeTractant) REFERENCES metges,
   FOREIGN KEY (idMetgeSolicitor) REFERENCES metges);


-- idMetgeTractant: és qui realitza el tractament.

-- El metge tractant exercia a la clínica en el moment del
tractament.

-- idMetgeSolicitor: és qui recomana o envia el pacient a la
visita.

-- Té valor NULL si el pacient no és enviat per cap metge.
```

1. (3 punts)

1.1. Doneu una sentència SQL per obtenir el nom i la ciutat de les clíniques que ofereixen més de tres tractaments amb un preu base superior a 100€, i en les quals cap dels metges que actualment hi exerceixen cobra més del doble de la mitjana de sous de tots els metges de la base de dades.

```
SELECT c.nomClinica, c.ciutat
FROM cliniques c
    NATURAL INNER JOIN tractaments t
WHERE t.preuBase > 100
  AND NOT EXISTS (
        SELECT *
        FROM metges m
        WHERE m.idClinicaActual = c.idClinica
          AND m.sou > 2 * (SELECT AVG(m2.sou) FROM metges m2)
      )
GROUP BY c.nomClinica, c.ciutat
HAVING COUNT(*) > 3;
```

1.2. Doneu una asserció que garanteixi que cap clínica tingui més pacients diferents que s'han visitat a la clínica que el doble del nombre de tractaments que la clínica ofereix.

Possibles solucions:

```
CREATE ASSERTION doble_pacients_tractaments CHECK(
     NOT EXISTS( SELECT v.idClinica
                 FROM visites v
                 GROUP BY v.idClinica
                 HAVING COUNT(DISTINCT v.idPacient) >
                      2 * COUNT(DISTINCT v.codiTractament)));
```

```
CREATE ASSERTION doble_pacients_tractaments CHECK
      (NOT EXISTS (SELECT v.idclinica
           FROM visites v
           GROUP BY v.idclinica
           HAVING COUNT(DISTINCT v.idpacient)>
                    2*(SELECT COUNT(*)
                    FROM tractaments t
                    WHERE t.idclinica = c.idclinica));
```

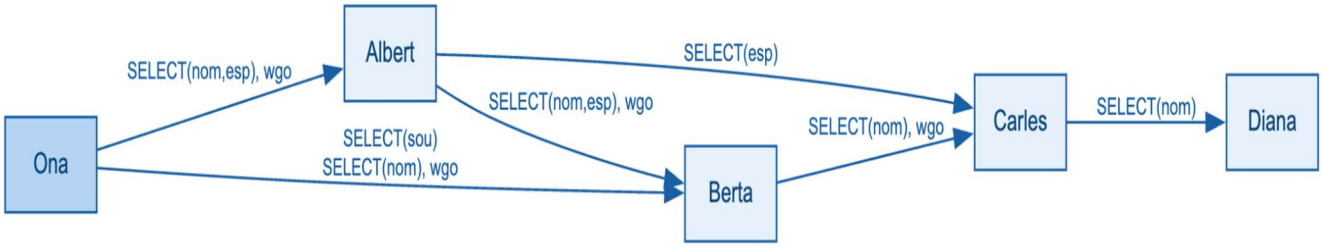
2. **(2 punts**) Doneu una seqüència d'operacions d'àlgebra relacional per obtenir el nom i l'especialitat dels metges que en tota la seva activitat **únicament** han fet visites com a metges tractants de pacients jubilats (de 65 anys o més) i que, a més, **han exercit exclusivament** a la clínica "EL REMEI". En el resultat de la consulta només han de sortir metges que hagin fet alguna visita com a metges tractants.

```
A = pacients(edat<65)                    // Pacients NO jubilats
B = A*visites                            // Visites a pacients NO jubilats
C = B[idMetgeTractant]                   // idMetgeTractant a NO jubilats
E = cliniques(nomClinica <> 'EL REMEI')  // Clíniques que no són 'EL REMEI'
F = E[idClinica]                         // idClinica que no són 'EL REMEI'
G = F * visites                          // Visites a clíniques NO 'EL REMEI'
H = G[idMetgeTractant]                   // idMetgeTractant a clíniques NO 'EL REMEI'
K = visites[idMetgeTractant]             // idMetgeTractant amb visites
L = K - C                                // idMetgeTractant que no han visitat
                                         // NO jubilats
M = L - H                                // idMetgeTractant que no han visitat
                                         // NO jubilats,
                                         // i no han visitat
                                         // a fora de la clínica 'EL REMEI'
N = M[idMetgeTractant*idMetge]metges     // informació completa dels metges
R = N[nomMetgeTractant, especialitat]    // nom i especialitat dels metges
```

3. **(2 punts)** Suposeu que l'Ona és la propietària de la base de dades.

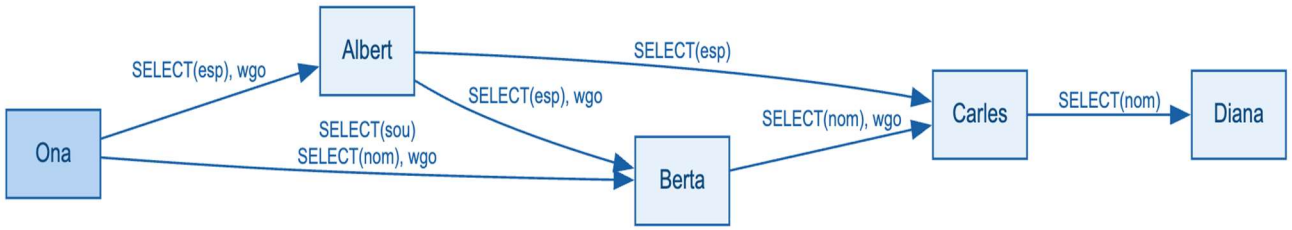
3.1. Doneu el diagrama d'autoritzacions resultant d'executar les sentències següents:

```
Ona: GRANT SELECT(nom,especialitat) ON metges TO Albert WITH GRANT OPTION;
Ona: GRANT SELECT(sou) ON metges TO Berta;
Ona: GRANT SELECT(nom) ON metges TO Berta WITH GRANT OPTION;
Albert: GRANT SELECT(nom,especialitat) ON metges TO Berta WITH GRANT OPTION;
Albert: GRANT SELECT(especialitat) ON metges TO Carles;
Berta:  GRANT SELECT(nom) ON metges TO Carles WITH GRANT OPTION;
Carles: GRANT SELECT(nom) ON metges TO Diana;
```



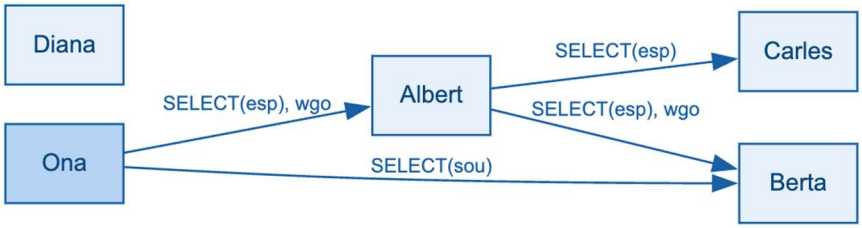
3.2. A continuació s'executen les dues sentències següents (**en l'ordre presentat**). Doneu dos diagrames d'autoritzacions: el resultant després del primer REVOKE i el resultant després del segon. Per a cadascun, justifiqueu quins privilegis guanyen o perden l'Albert, la Berta, en Carles i la Diana i per quin motiu.

```
Ona: REVOKE SELECT(nom) ON metges FROM Albert CASCADE;
Ona: REVOKE SELECT(nom) ON metges FROM Berta CASCADE;
```



Després del primer REVOKE (FROM Albert CASCADE):

- L'Albert perd SELECT(nom). Conserva SELECT(esp) amb wgo.
- La Berta perd l'aresta Albert→Berta per SELECT(nom) via CASCADE. Però conserva SELECT(nom) perquè li ve directament de l'Ona. Conserva també SELECT(esp) amb wgo d'Albert, i SELECT(sou) de l'Ona.
- En Carles conserva SELECT(nom) (li ve de Berta, que el manté) i conserva SELECT(esp) (li ve d'Albert, que el manté).
- La Diana conserva SELECT(nom) (la cadena Ona→Berta→Carles→Diana segueix intacta).



Després del segon REVOKE (FROM Berta CASCADE):

- La Berta perd SELECT(nom) definitivament (l'única font que li quedava era Ona, ara revocada). Conserva SELECT(sou) i SELECT(esp) via Albert.
- En Carles perd SELECT(nom) per CASCADE des de Berta. Conserva SELECT(esp) perquè el rep d'Albert.
- La Diana perd SELECT(nom) per CASCADE des de Carles. Queda desconnectada del graf perquè no li queda cap privilegi.

3.3. L'Ona vol que en Marc pugui fer **totes** les operacions següents **definint la mínima quantitat de vistes** i donant al Marc **els privilegis mínims necessaris sobre les vistes**:

- Inserir visites en què el pacient hi va pel seu compte (idMetgeSolicitor és NULL) i consultar totes les columnes d'aquestes visites. Les insercions han de garantir que les noves files siguin visibles a la vista.
- Consultar el nom i l'especialitat dels metges tractants de les visites on els pacients hi van pel seu compte.

- Modificar únicament l'atribut idMetgeTractant de les visites on els pacients hi van pel seu compte.

Com a resposta cal donar:

- Les sentències CREATE VIEW i GRANT corresponents.
- Per a cada vista, justificar si és actualitzable o no, indicant quins elements de la definició ho determinen.
- Per a cada vista, justificar si cal WITH CHECK OPTION o no.

```
CREATE VIEW visitesComptePropi AS
     SELECT *
     FROM visites
     WHERE idMetgeSolicitor IS NULL
     WITH CHECK OPTION;

 GRANT SELECT, INSERT ON visitesComptePropi TO Marc;
 GRANT UPDATE(idMetgeTractant) ON visitesComptePropi TO Marc;

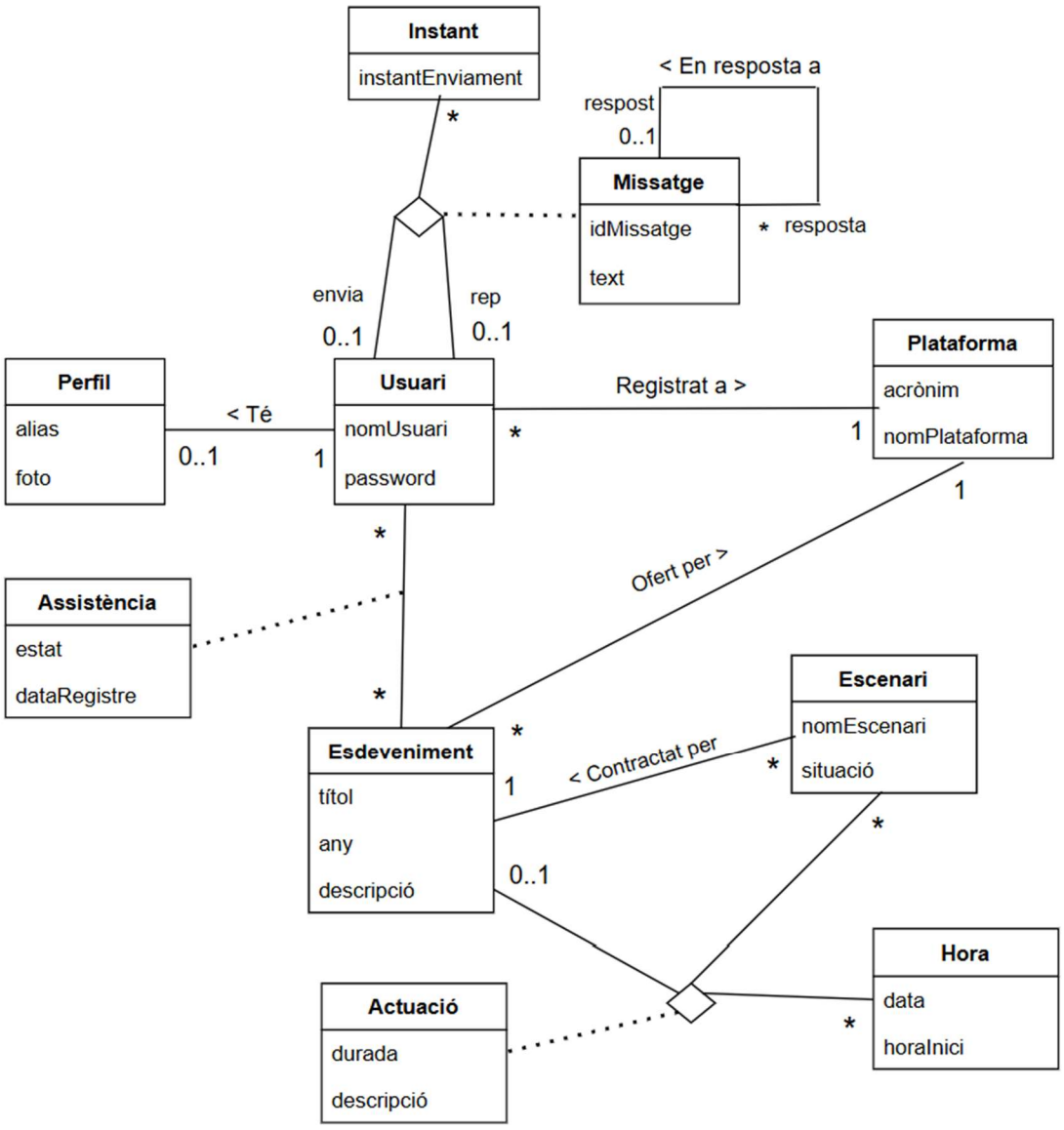
 CREATE VIEW visitesCompteAmbMetge AS
     SELECT m.nom AS nomMetge, m.especialitat
     FROM visitesCompteAmbMetge vcp, metges m
     WHERE vcp.idMetgeTractant = m.idMetge;

 GRANT SELECT ON visitesCompteAmbMetge TO Marc;
```

Justificacions:

- visitesCompteAmbMetge
  - **Actualitzable**: defineix SELECT * sobre una única taula base, sense JOIN, GROUP BY, DISTINCT ni funcions d'agregació. Per tant admet INSERT i UPDATE.
  - **Cal WITH CHECK OPTION**: l'enunciat exigeix explícitament que les noves files inserides siguin visibles a la vista. Com que l'UPDATE només afecta idMetgeTractant, el WCO no entra mai en conflicte amb l'UPDATE: qualsevol valor de idMetgeTractant deixa la fila dins de la vista.
- visitesCompteAmbMetge
  - **No actualitzable**: conté un JOIN entre visites i metges, cosa que viola la condició d'actualitzabilitat (la vista ha de provenir d'una sola taula base sense joins).
  - **No cal WITH CHECK OPTION**: només s'hi fa SELECT, i WCO només té sentit per a vistes sobre les quals s'hi fan modificacions.

4. **(3 punts)** Donat el model conceptual en UML següent, es demana fer la traducció a model relacional donant el disseny lògic de la base de dades resultant. Concretament cal indicar:
   - taules
   - atributs
   - claus primàries
   - claus foranes
   - restriccions NOT NULL per a les claus foranes que ho requereixin
   - restriccions UNIQUE en cas d'existir claus alternatives
   - **Restriccions de columna o de taula per a implementar R1, R2, R3, R4. En cas que alguna d'aquestes restriccions no es pugui implementar com a restricció de columna o de taula, explicar perquè no es pot**.



Restriccions textuals següents:

- Clau externa **Plataforma**: acrònim
- Clau externa **Usuari**: nomUsuari
- Clau externa **Instant**: instantEnviament
- Clau externa **Missatge**: idMissatge (en el disseny d'aquesta BD s'ha decidit afegir una clau externa per facilitar la identificació dels missatges, tot i tractar-se d'una classe associativa).
- Clau externa **Perfil**: No poden existir dos perfils amb el mateix nomUsuari
- Clau externa **Esdeveniment**: títol, any
- Clau externa **Escenari**: No poden existir escenaris amb un mateix nomEscenari en un mateix esdeveniment, però si en esdeveniments diferents.
- Clau externa **Hora**: data, horaInici
- **R1**: L'estat d'una assistència només pot ser 'confirmada' o 'cancel·lada'
- **R2**: Si una assistència està en estat confirmada, aleshores ha de tenir una data de registre diferent de null
- **R3**: Les actuacions en un esdeveniment han de ser en escenaris contractats per aquest esdeveniment
- **R4**: No hi pot haver dues plataformes amb el mateix nom

```
plataformes(acronim, nomPlataforma)
    UNIQUE(nomPlataforma)                                    /*R4*/
usuaris(nomUsuari, password, idPlataforma)
    {idPlataforma} referencia plataformes(acronim)
    NOT NULL (idPlataforma)
perfils(nomUsuari, nom, foto)
    {nomUsuari} referencia usuaris
missatges(idMissatge, idMissatgeRespost,
            instantMissatge, nomUsuarienvia, nomUsuariRep,
            text)
    {idMissatgeRespost} referencia missatges
    UNIQUE(instantMissatge, nomUsuariEnvia)
    UNIQUE(instantMissatge, nomUsuariRep)
    NOT NULL(instantMissatge, nomUsuariEnvia, nomUsuariRep)
```

esdeveniments(titol, any, descripcio, idPlataforma)

{idPlataforma} **referencia** plataformes(acronim)

NOT NULL(idPlataforma)

assistencies(nomUsuari, titol, any, estat, dataRegistre)

{nomUsuari} **referencia** usuaris

{titol, any} **referencia** esdeveniments

CHECK (estat IN ('confirmada', 'cancel·lada')) **/*R1*/**

CHECK (estat!='confirmada OR dataRegistre IS NOT NULL) **/*R2*/**

escenaris(nomEscenari, titol, any, situació)

{titol, any} **referencia** esdeveniments

actuacions(data, horaInici, nomEscenari, titol, any,
titolEsdeveniment, anyEsdeveniment,
durada, descripcio)

{nomEscenari, titol, any} **referencia** escenaris

{titolEsdeveniment, anyEsdeveniment} **referencia** esdeveniments

NOT NULL(titolEsdeveniment, anyEsdeveniment)

CHECK(titol = titolEsdeveniment **AND**
Any = anyEsdeveniment) **/*R3*/**

BD **16.06.2026**

Temps: 3 hores

Notes: 25 juny **Revisió: 26 juny** **Cada pregunta en fulls separats**

Considereu l'esquema de la base de dades següent:

```
CREATE TABLE cliniques (
   idClinica          INTEGER,
   nom                VARCHAR(100) UNIQUE NOT NULL,
   ciutat             VARCHAR(100),
   pressupost         FLOAT CHECK (pressupost > 0),
   quantsMetges       INTEGER CHECK (quantsMetges <= 4),
   PRIMARY KEY (idClinica));
-- quantsMetges indica quants metges exerceixen actualment a la clínica

CREATE TABLE metges (
   idMetge            INTEGER,
   nom                VARCHAR(100) UNIQUE NOT NULL,
   especialitat       VARCHAR(50) NOT NULL,
   sou                FLOAT CHECK (sou > 0 AND sou <= 10),
   idClinicaActual    INTEGER NOT NULL,
   PRIMARY KEY (idMetge),
   FOREIGN KEY (idClinicaActual) REFERENCES cliniques);
-- idClinicaActual és on actualment exerceix el metge

CREATE TABLE pacients (
   idPacient          INTEGER,
   nom                VARCHAR(100) NOT NULL,
   edat               INTEGER CHECK (edat BETWEEN 0 AND 120),
   ciutat             VARCHAR(100),
   PRIMARY KEY (idPacient));

CREATE TABLE tractaments (
   idClinica          INTEGER,
   codiTractament     INTEGER,
   descripcio         VARCHAR(200),
   preuBase           FLOAT CHECK (preuBase > 0 AND preuBase <=10),
   PRIMARY KEY (idClinica, codiTractament),
   FOREIGN KEY (idClinica) REFERENCES cliniques);
-- Hi ha una fila per cada tractament ofert en la clínica

CREATE TABLE visites (
   idPacient          INTEGER,
   idClinica          INTEGER,
   codiTractament     INTEGER,
   dataHora           TIMESTAMP,
   idMetgeTractant    INTEGER NOT NULL,
   idMetgeSolicitor   INTEGER,
   PRIMARY KEY (idPacient, idClinica, codiTractament, dataHora),
   FOREIGN KEY (idPacient) REFERENCES pacients,
   FOREIGN KEY (idClinica, codiTractament) REFERENCES tractaments,
   FOREIGN KEY (idMetgeTractant) REFERENCES metges,
   FOREIGN KEY (idMetgeSolicitor) REFERENCES metges);
-- idMetgeTractant: és qui realitza el tractament.
-- El metge tractant exercia a la clínica en el moment del tractament.
-- idMetgeSolicitor: és qui recomana o envia el pacient a la visita.
-- Té valor NULL si el pacient no és enviat per cap metge.
```

1. (3 punts) Suposeu que l'estat de la base de dades per aquest exercici és el següent:

cliniques	idClinica	nom	ciutat	pressupost	quantsMetges
	1	A	Calella	500	2
	2	B	Manresa	200	1
	3	C	Tremp	800	1

metges	idMetge	nom	especialitat	sou	idClinicaActual
	123	Anna	Interna	7	1
	234	Mercè	Derma	9	2
	345	Pol	Interna	6	1
	456	Biel	Estètica	10	3

1.1. Suposeu la transacció T1 següent:

#n	T1
10	START TRANSACTION;
20	UPDATE metges SET idClinicaActual = 3 WHERE idMetge = 345;
30	UPDATE cliniques SET pressupost = pressupost - 6 WHERE idClinica = 1;
40	UPDATE cliniques SET pressupost = pressupost + 6 WHERE idClinica = 3;
50	COMMIT;

A. Partint de la transacció T1, doneu un horari on hi hagi dues transaccions i una d'elles sigui la transacció T1 i que compleixin les condicions que es demanen a continuació. Si no es possible donar algun d'aquests horaris, indiqueu perquè no. En la transacció a afegir cal utilitzar sentències SELECT, INSERT, DELETE i/o UPDATE.

   A.1. Un horari amb una única interferència de lectura no repetible.

   A.2. Un horari amb una única interferència de tipus fantasma.

B. Les transaccions en bases de dades relacionals han de complir les propietats ACID. Explica perquè la transacció T1 no compleix la propietat corresponent a la lletra C de Consistència.

Una possible solució de l'apartat A.1

T1	T2
START TRANSACTION;	
	START TRANSACTION;
UPDATE metges SET idClinicaActual = 3 WHERE idMetge = 345;	
	SELECT * FROM cliniques WHERE idClinica = 1;
UPDATE cliniques SET pressupost = pressupost - 6 WHERE idClinica = 1;	
	SELECT * FROM cliniques WHERE idClinica = 1;
UPDATE cliniques SET pressupost = pressupost + 6 WHERE idClinica = 3;	
	COMMIT;
COMMIT;	

Una possible solució de l'apartat A.2

T1	T2
START TRANSACTION;	
	START TRANSACTION;
	SELECT * FROM metges WHERE idClinicaActual = 3;
UPDATE metges SET idClinicaActual = 3 WHERE idMetge = 345;	
	SELECT count(*) FROM metges WHERE idClinicaActual = 3;
UPDATE cliniques SET pressupost = pressupost - 6 WHERE idClinica = 1;	
UPDATE cliniques SET pressupost = pressupost + 6 WHERE idClinica = 3;	
	COMMIT;
COMMIT;	

B- La propietat de Consitència demana que els dissenyadors/programadors implementin les transaccions de manera que satisfacin els requisits d'usuari. En aquesta transacció es passa el metge 345 de la clínica 1 a la clínica 3, però no es modifica l'atribut quantsMetges d'aquestes dues clíniques. Això deixa l'estat de la base de dades NO consistent.

1.2. Partint de l'horari que es dona a continuació.

A. Doneu l'horari resultant d'aplicar les reserves S,X adequades, tenint en compte que T1 treballa amb nivell d'aïllament READ COMMITTED, T2 amb nivell REPETEABLE READ, i T3 amb nivell READ UNCOMMITTED. L'horari ha d'incloure, a més de les peticions que executen les transaccions (R, RU, W,

COMMIT), les operacions de petició i alliberament de reserves (LOCK, UNLOCK), les esperes i l'ordre d'execució de totes aquestes peticions.

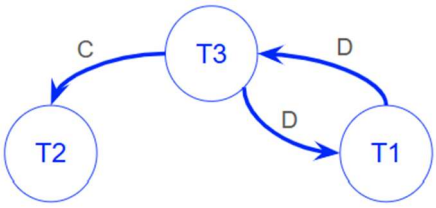
B. Doneu el graf de precedències de l'horari resultant. Indiqueu si hi ha o no interferències en l'horari. Si n'hi ha, indicar entre quines transaccions i grànuls es produeixen, si no, doneu horaris serials equivalents.

#n	T1	T2	T3
10		R(B)	
20	R(B)		
30			R(A)
40			RU(C)
50		RU(C)	
60		W(C)	
70	R(E)		
80	RU(D)		
90			W(C)
100			R(D)
110	W(D)		
120	commit		
130			R(D)
140		R(E)	
150		commit	
160			RU(C)
170			W(C)
180			commit

Solució:

T1	T2	T3
	L(B,S)	
	R(B)	
L(B,S)		
R(B)		
U(B)		
		R(A)
		L(C,X)
		RU(C)
	L(C,X)	
L(E,S)		
R(E)		
U(E)		
L(D,X)		
RU(D)		
		W(C)
		R(D)
W(D)		
C+U(D)		
		R(D)
		RU(C)
		W(C)
		C+U(C)
	RU(C)	
	W(C)	
	L(E,S)	
	R(E)	
	C+U(B,C,E)	

Hi ha una interferència de lectura no repetible entre T1 i T3 pel gràNul D.



1.3. Considereu, **per aquest apartat**, que a la base de dades hi ha definit el disparador següent. **Per cadascuna de les transaccions** donades (partint de l'estat inicial de la bases de dades) indiqueu:

- quins canvis hi ha a l'estat de la base de dades **a conseqüència de cadascuna de les sentències de cada transacció**: Id de les files afectades i quins canvis concrets hi ha hagut en aquestes files.
- contingut de la base de dades després d'executar la transacció.

```
CREATE OR REPLACE FUNCTION actualitzar_comptador_metges()
            RETURNS TRIGGER AS $$
BEGIN
    IF (TG_OP = 'INSERT') THEN
        UPDATE cliniques
        SET quantsMetges = quantsMetges + 1
        WHERE idClinica = NEW.idClinicaActual;
    ELSIF (TG_OP = 'DELETE') THEN
        UPDATE cliniques
        SET quantsMetges = quantsMetges - 1
        WHERE idClinica = OLD.idClinicaActual;
    ELSIF (TG_OP = 'UPDATE') THEN
        IF (OLD.idClinicaActual <> NEW.idClinicaActual) THEN
            UPDATE cliniques
            SET quantsMetges = quantsMetges - 1
            WHERE idClinica = OLD.idClinicaActual;
            UPDATE cliniques
            SET quantsMetges = quantsMetges + 1
            WHERE idClinica = NEW.idClinicaActual;
        END IF;
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_comptador_metges
AFTER INSERT OR DELETE OR UPDATE OF idClinicaActual ON metges
FOR EACH ROW EXECUTE FUNCTION actualitzar_comptador_metges();
```

A.

10	START TRANSACTION;
20	UPDATE cliniques SET pressupost = pressupost + (SELECT pressupost FROM cliniques WHERE idClinica = 1) WHERE idClinica =2;
30	UPDATE metges SET idClinicaActual = 2 WHERE idClinicaActual = 1;
40	UPDATE cliniques SET pressupost = 0 WHERE idClinica = 1;
50	COMMIT;

B.

10	START TRANSACTION;
20	INSERT INTO pacients VALUES (111,’Jon’,50,’Das’);
30	INSERT INTO pacients VALUES (222,’Moi’,35,’Vic’);
40	INSERT INTO pacients VALUES (333,’Lia’,40,’Ger’);
50	ROLLBACK;

C.

10	START TRANSACTION;
20	INSERT INTO metges VALUES (567,'Bea','Palogia',9,1);
30	UPDATE metges SET idClinicaActual = 1 WHERE idClinicaActual = 2;
40	INSERT INTO metges VALUES (678,'Ona','Cirugia',10,1);
50	COMMIT;

A	UPDATE cliniques SET pressupost = pressupost + (SELECT pressupost FROM cliniques WHERE idClinica = 1) WHERE idClinica =2;	-Clínica 2 (update) . pressupost passa de 200 a 700
	UPDATE metges SET idClinicaActual = 2 WHERE idClinicaActual = 1;	-Metge 123 (update) . idClinicaActual passa a 2 -Metge 345 (update) . idClinicaActual passa a 2 -Clínica 2 (disparador) . quantsMetges passa a 3 -Clínica 1 (disparador) . quantsMetges passa a 0
	UPDATE cliniques SET pressupost = 0	-Clínica 1 (update) . pressupost passa de 500 a 0 -Excepció de CHECK, taula cliniques

	WHERE idClinica = 1;	. columna pressupost passa a 0
	Un cop acabada la transacció	El contingut de la BD és exactament el mateix que l'inicial que es dona en l'enunciat (degut a l'excepció)

B	INSERT INTO pacients VALUES (111,'Jon',50,'Das');	-Pacient 111 (insert)
	INSERT INTO pacients VALUES (222,'Moi',35,'Vic');	-Pacient 222 (insert)
	INSERT INTO pacients VALUES (333,'Lia',40,'Ger');	-Pacient 333 (insert)
	Un cop acabada la transacció	El contingut de la BD és exactament el mateix que l'inicial que es dona en l'enunciat (degut al rollback)

C	INSERT INTO metges VALUES (567,'Bea','Palogia',9,1);	-Metge 567 (insert) -Clínica 1 (disparador) . quantsMetges passa a 3
	UPDATE metges SET idClinicaActual = 1 WHERE idClinicaActual = 2;	-Metge 234 (update) . idClinicaActual passa a 1 -Clínica 1 (disparador) . quantsMetges passa a 4 -Clínica 2 (disparador) . quantsMetges passa a 0
	INSERT INTO metges VALUES (678,'Ona','Cirugia',10,1);	-Metge 678 (insert) -Clínica 1 (disparador) . quantsMetges passa a 5 -Excepció de CHECK, taula clíniques . columna quantsMetges
	Un cop acabada la transacció	El contingut de la BD és exactament el mateix que l'inicial que es dona en l'enunciat (degut a l'excepció)

2. **(3 punts)** Suposeu la base de dades de l'exercici 1. La taula visites té 3.000.000 de files. Cada pacient té de mitjana 50 visites, i cada metge tractant apareix de mitjana en 6.000 visites. El factor de bloqueig de les pàgines de dades de visites és f = 50. Justifiqueu totes les respostes.

2.1. Sobre la taula metges existeix un índex B+ agrupat sobre l'atribut idMetge (clau primària), amb ordre d = 100 i alçada h = 3. Indiqueu el cost en nombre mínim i màxim de pàgines a què s'accedirà per resoldre cadascuna de les consultes següents, i expliqueu la diferència entre tots dos casos.

A. `SELECT * FROM metges WHERE idMetge = 42;`

B. `SELECT idMetge FROM metges WHERE idMetge = 42;`

idMetge és la clau primària de la taula metges, per tant només una fila complirà la condició. L'índex és agrupat, però com que la cerca és per igualtat sobre la clau, només es recupera una fila.

A. En la primera consulta cal recuperar la fila sencera de la pàgina de dades.
   - Cost mínim = h = 3 (cas en què el metge no existeix, s'arriba a la fulla i no hi és).
   - Cost màxim = h + 1 = 3 + 1 = 4 (el metge existeix i cal un accés addicional a la pàgina de dades).

B. En la segona consulta, el valor demanat ja és a l'índex per tant no cal accedir a la pàgina de dades
   - Cost mínim = 1 (el valor 42 és un dels valors del node arrel)
   - Cost màxim = h = 3 (cal baixar fins la fulla per saber si el valor hi és).

2.2. Considereu un índex B+ no agrupat sobre l'atribut dataHora de la taula visites. L'índex té ordre d = 120 i una ocupació del 80%. Determineu:

A. Quants valors hi ha a cada node i quants apuntadors.

B. Quantes pàgines ocupen els nodes interns de l'arbre.

C. Quants nivells té l'arbre B+.

A. A. Valors per node (vpn) = ⌈2·d·ocupació⌉ = ⌈2·120·0,8⌉ = 192 valors.
   Apuntadors per node (nap) = vpn + 1 = 193 apuntadors.

B. Nodes fulla = ⌈3.000.000 / 192⌉ = 15.625
   Nodes interns per nivells, dividint successivament per nap = 193:
   Nivell 1 (pares de les fulles) = ⌈15.625 / 193⌉ = ⌈80,96⌉ = 81 pàgines
   Nivell 2 = ⌈81 / 193⌉ = ⌈0,42⌉ = 1 pàgina (arrel)
   Total nodes interns = 81 + 1 = 82 pàgines

C. Nivells = h = ⌈log193 3.000.000⌉ = ⌈ln(3.000.000)/ln(193)⌉ = ⌈14,91/5,26⌉ = ⌈2,83⌉ = 3 nivells.

2.3. Utilitzant l'índex no agrupat sobre dataHora de l'apartat anterior, doneu el cost de resoldre la consulta següent. Mostreu i justifiqueu els càlculs. Considereu que de mitjana es fan 240 visites al dia i que el rang de la consulta cobreix 32 dies (del 31 de desembre de 2025 al 31 de gener de 2026, tots dos inclosos).

```
SELECT *
FROM visites
```

```
WHERE dataHora >= '2025-12-31' AND
      dataHora < '2026-02-01';
```

Es tracta d'una consulta per rang on es fa un accés seqüencial per valor amb un index no agrupat.

- Cost = h + F - 1 + |visites(dataHora >= '2025-12-31' AND dataHora <= '2026-01-31')
- Número de visites dins el rang: 240 visites/dia * 32 dies = 7680 files.
- F = ⌈7.680 / 192⌉ − 1 = ⌈40⌉ − 1 = 40 − 1 = 39 (nodes fulla addicionals a recórrer, excepte el primer que es comptabilitza en l'alçada)
- Cost = 3 + 39 + 7680 = 7.722 pàgines.

2.4. Considereu ara la consulta següent.

```
SELECT idPacient
FROM visites
WHERE idMetgeTractant = 77;
```

Disposeu de dos índexs B+ no agrupats sobre visites: un sobre idMetgeTractant (ordre d = 120, ocupació 80%, alçada h = 3) i un altre multi-atribut sobre [idPacient, idMetgeTractant] (ordre d = 120, ocupació 80%, alçada h = 4).

A. Doneu el cost de resoldre la consulta utilitzant l'índex sobre idMetgeTractant.

B. Es pot resoldre aquesta consulta accedint només a l'índex multi-atribut, sense accedir al fitxer de dades? Justifiqueu la resposta. En cas afirmatiu, indiqueu si el cost seria menor o major que el de l'apartat A i per què.

A. Índex no agrupat sobre idMetgeTractant. La consulta projecta idPacient, que no és a l'índex, per tant cal accedir a les pàgines de dades. És un accés directe per valor amb valors repetits.

- Cost = (h + F) + |visites(idMetgeTractant = 77)|
  - |visites(idMetgeTractant = 77)| = 6.000 files
- F = ⌈6.000 / 192⌉ − 1 = ⌈31,25⌉ − 1 = 32 − 1 = 31
- Cost = 3 + 31 + 6000 = 6034 pàgines

B. Sí, aquesta consulta es pot resoldre accedint només a l'índex multi-atribut. Tots dos atributs implicats en la consulta (idPacient, que es projecta, i idMetgeTractant, que es filtra) són dins de l'índex. Per tant, el què es busca ja és a l'índex i no cal accedir al fitxer de dades. Però el problema és que l'índex està ordenat primer per idPacient i després per idMetgeTractant. La condició del WHERE és sobre idMetgeTractant, que és el segon element d'ordenació. Com que els valors no estan ordenats per aquest segon element, no

es pot fer accés directe per valor: cal recórrer TOTS els nodes fulla de l'índex per trobar les entrades amb idMetgeTractant = 77.

- Cost = h + F - 1 (tots els nodes fulla menys 1 que ja es comptabilitza a l'alçada)
- F = ⌈3.000.000 / 192⌉ = 15.625 (igual que al segon apartat)
- Cost = 4 + 15625 - 1 = 15628 pàgines

3. (3 punts) Considereu la base de dades de l'inici de l'examen. Tingueu en compte que:

- Es disposa de la funció day_of_week(DATE) que retorna el dia de la setmana de la data que rep com a paràmetre. En concret, retorna un valor enter comprès entre 1 i 7 on 1 representa el dilluns i 7 el diumenge.
- Es disposa de la variable CURRENT_DATE que retorna la data actual.

3.1. Indiqueu quin(s) disparador(s) cal definir per implementar el comportament següent:

- Es vol que no es pugui crear, modificar o eliminar visites en cap de setmana (dissabte o diumenge).
- Les visites no es poden fer en cap de setmana (dataHora de la visita).

Per cada disparador cal que indiqueu: l'esdeveniment activador, la taula i el tipus de disparador. En cas de l'esdeveniment UPDATE cal indicar també les columnes rellevants. Per cada disparador, cal també que justifiqueu breument el tipus de disparador escollit.

INSERT	-	visites	BEFORE ROW	BEFORE: Restricció, millor comprovar lo abans possible. --- ROW: cal poder llegir NEW.dataHora per saber si correspon a cap de setmana
UPDATE	dataHora	visites	BEFORE ROW	
INSERT	-	visites	BEFORE STATEMENT	BEFORE: Restricció, millor comprovar lo abans possible. --- STATEMENT: No depen de la quantitat de files, ni de quines files estan afectades.
DELETE	-	visites	BEFORE STATEMENT	
UPDATE	-	visites	BEFORE STATEMENT	

3.2. Implementeu els disparadors que heu indicat a l'apartat anterior:

```
CREATE OR REPLACE FUNCTION caps_de_row() RETURNS TRIGGER AS $$
DECLARE
dia_setmana INTEGER;
BEGIN
  dia_setmana = day_of_week(NEW.dataHora);
  IF (dia_setmana = 6 OR dia_setmana = 7) THEN
      RAISE EXCEPTION '%',
               'No es poden planificar visites en cap de setmana';
  END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_caps_de_setmana_row
   BEFORE INSERT OR UPDATE of dataHora ON visites
   FOR EACH ROW
   EXECUTE PROCEDURE caps_de_row();


CREATE OR REPLACE FUNCTION caps_de_statement() RETURNS TRIGGER AS $$
DECLARE
dia_setmana INTEGER;
BEGIN
    dia_setmana = day_of_week(CURRENT_DATE);
    IF (dia_setmana = 6 or dia_setmana = 7) THEN
       RAISE EXCEPTION '%','No es poden fer canvis en cap de setmana';
    END IF;
RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_caps_de_setmana_statement
   BEFORE INSERT OR UPDATE OR DELETE ON visites
   FOR EACH STATEMENT
   EXECUTE PROCEDURE caps_de_statement();
```

3.3. Suposeu que a la base de dades hi ha també la taula següent. Implementeu un disparador per tal de mantenir el contingut d'aquesta taula.

```
CREATE TABLE historic_assignacions_metges (
   idMetge                INTEGER,
   idClinica         INTEGER,
   dataInici              DATE,
   dataFinal         DATE,
   PRIMARY KEY (idMetge, idClinica, dataInici),
   FOREIGN KEY (idClinica) REFERENCES cliniques,
   FOREIGN KEY (idMetge) REFERENCES metges ON DELETE CASCADE;
);
-- una fila per cada període en que un metge ha estat assignat a una clínica.
-- dataFinal té valor null quan una assignació és vigent.
```

```
CREATE OR REPLACE FUNCTION registrar_historic_metges()
                                    RETURNS TRIGGER AS $$
BEGIN
    IF ( TG_OP = 'UPDATE' ) THEN

       UPDATE historic_assignacions_metges
       SET dataFinal = CURRENT_DATE
       WHERE idMetge = OLD.idMetge
             AND idClinica = OLD.idClinicaActual
             AND dataFinal IS NULL;
      RETURN NULL;

    END IF;

    INSERT INTO historic_professional_metges VALUES (
       NEW.idMetge, NEW.idClinicaActual, CURRENT_DATE, NULL
    );
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;


-- No es necessari el trigger DELETE, donat que "ON DELETE CASCADE"
-- provoca que en esborrar un metge,
-- s'esborri també tot el seu historial
CREATE TRIGGER tg_registrar_historic_metges
    AFTER INSERT OR UPDATE OF idClinicaActual ON metges
    FOR EACH ROW
    EXECUTE PROCEDURE registrar_historic_metges();
```

4. **(1 punt)** Considereu el fragment de codi Java següent, d'una funció JDBC (incentiuMetges) que incrementarà el sou als metges més productius en base a les regles següents:
   - La funció rebrà com a paràmetres: un interval de dates *dataInici*, *dataFi*, i un *bonusPerVisita*. Així com també, l'objecte *Connection c*.
   - El sou s'incrementarà només als metges que han fet visites dintre de l'interval de dates ambdues incloses.
   - Aquests metges han d'haver visitat almenys 3 pacients diferents durant l'interval.
   - Per cada metge al que se li incrementi el sou, aquest increment es igual a multiplicar el *bonusPerVisita* per la quantitat de visites que ha fet el metge durant l'interval.
   - Caldrà imprimir la quantitat de metges als que s'ha incrementat el sou.
   - Cal cancel·lar qualsevol acció si el sou d'algun metge passa a superar el 10. En aquest cas es traurà l'avís: "Incentius incompatibles amb polítiques corporatives".

```
public static void incentiuMetges (Connection c,
                    String dataInici, String dataFi,
                    Float bonusPerVisita)
{
    try {
        .........
        c.commit();
        c.close();
    }
    catch (SQLException se) {
        .........
        System.out.println ("Excepció: " + se.getMessage());
    }
}
```

Recordeu que podeu usar els mètodes/codis d'excepció de JDBC estudiats a classe:

Statement

- Statement createStatement();
- ResultSet executeQuery(String sql);
- int executeUpdate(String sql);

PreparedStatement

- PreparedStatement prepareStatement(String sql);
- ResultSet executeQuery();
- int executeUpdate();
- void setXXX(int position, XXX value);

ResultSet

- boolean next();
- XXX getXXX(String nomColumna);

SQLException

- // 23502 not_null_violation
- // 23503 foreign_key_violation
- // 23505 unique_violation
- // 23514 check_violation
- String getSQLState();

SOLUCIÓ:

```
public static void incentiuMetges(Connection c,
                        String dataInici, String dataFi,
                        Float bonusPerVista)
{
    System.out.println("Inici del proces");
    try {
        Statement s = c.createStatement();
        String sql = "UPDATE metges m "
        + "SET sou=sou + "+bonusPerVista+"*(SELECT count(*) "
        +                  "FROM visites v "
        +                  "WHERE dataHora>='" + dataInici +"' "
        +                  "AND dataHora<='" + dataFi +"' "
        +                  "AND m.idMetge = v.idMetgeTractant) "
        + "WHERE 3 <= (SELECT count(distinct v.idPacient) "
        +                  "FROM visites v "
        +                  "WHERE dataHora>='" + dataInici +"' "
        +                  "AND dataHora<='" + dataFi +"' "
        +                   "AND m.idMetge = v.idMetgeTractant); ";

        int modificats = s.executeUpdate(sql);

        System.out.println("modificats: " + modificats);
```

```
            c.commit();
            c.close();
        }
        catch (SQLException se){
            if(se.getSQLState().equals("23514"))
                System.out.println("Incentius incompatibles amb p...");
            else System.out.println("Excepcio: "+ se.getMessage());
        }
}
```